



System Specifications

3

Management

5

Resources

1

Summary

This document specifies the elements that are available in the network and in the device to the Management Client and that shall be managed by this Management Client.

Version 01.10.01 is a KNX Approved Standard.

Document updates

Version	Date	Modifications
1.0	2001.12.13	Integration of comments from FV, CSG feedback.
1.2	2007.01.05	• Supplement 18 "Network Resources" integration started and completed.
1.2	2007.01.07	• Supplement 15 "Easy Common Parts", requirements on Individual Address, etc. integrated. • Started list of references below.
1.2	2007.01.15	• AN038 "Function Services": PID_APPLICATION
1.2		• AN029 "DPSU on TP1" integrated.
1.2	2007.11.07	• AN031 "Coupler Resources" integrated.
1.2	2007.11.12	• AN033 "Common EMI" integrated.
1.2	2007.11.23	• Added: OptionReg,
1.2	2008.01.16	• AN043 "LTE on RF": PID_SERIAL_NR_TABLE
1.2	2008.01.17	• AN044 "RF Specification Complements": PID_OBJECTADDRESS
1.2	2008.02.13	• Added indication about inverse coding of bit 2 in PID_SERVICE_CONTROL, acc. KSG conclusion of 2007.12.04-05.
1.2	2008.04.28	• AN066 "cEMI adaptations" integrated.
1.2	2008.04.29	• AN032 "Easy Resources" integrated.
1.2	2008.05.05	• AN067 "APCI ServiceInformation Indication Write" integrated.
1.2	2008.05.07	• AN071 "Link services and DMA access" integrated.
1.2	2008.07.14	• AN060 "Bidirectional Battery driven devices" integrated.
1.2	2008.08.01	• AN081 "2-level topology"
1.2	2008.08.05	• AN090 "Discovery of long frame range"
1.2	2008.08.05	• AN091 "Telegram rate limitation for System B" integrated. • Introduction of Resource "Group telegram rate limitation"
1.2	2008.08.05	• AN094 "DD0 for devices with and without direct bus connection" integrated. • Added mask version 0AFDh.
1.2	2008.08.06	• AN095 "Non-selective Data Link Layer acknowledge" integrated
1.2	2008.08.07	• AN078 "cEMI Extensions for RF and BiBat" integrated
1.2	2008.08.11	• AN102 "TP1 Bridges" integrated
1.2		• Merged Group Object Association Table Realisation Type 7 (System 300) IN Realisation Type 6 (System B).
1.2	2008.08.12	• AN103 "Structure of PID_MGT_DESCRIPTOR_01" integrated. • AN104 "Global Property PID_OBJECT_INDEX" integrated. • AN101 "File Transfer Protocol" integrated.
1.2	2008.12.02	• Added PID_DEVICE_DESCRIPTOR in Device Object and in KNXnet/IP Parameter Object.
1.2	2009.01.08	• Preparation of the Approved Standard v1.2.
1.2	2009.01.21	• Removed PID_DEVICE_DESCRIPTOR from the KNXnet/IP Parameter Object.
1.2	2009.01.22	• Added indication that the Properties of the KNXnet/IP Parameter Object are specified in Chapter 3/8/3 "Management" for completeness.
1.2	2009.02.04	• Preparation for inclusion in the KNX Specifications v2.0.
1.2	2009.03.31	• Editorial update of Load- and Run State Machines
1.2	2009.06.23	• Editorial update in view of inclusion in the KNX Specifications v2.0.
1.2.01	2009.11.24	• Editorial updates
1.2.02	2010.10.22	• AN127 "Master Reset" integrated
1.3.00	2010.12.23	• AN115 "Mask 5705h" integrated
01.04.00	2012.09.10	• Revision of the integration of AN127 "Master Reset" according the updated AN127 "Master Reset" v05.
01.05.00	2012.10.22	• AN149 "New Load State LoadCompleting" integrated.

Version	Date	Modifications
01.05.01	2013.03.19	- Accepted all changes in the document. - Removed use restrictions of PID_HARDWARE_TYPE to mask 0705h and 0025h. - PID_PROGRAM_VERSION: Property Name and PDT corrected. - PID_TABLE_REFERENCE: Property Name corrected.
01.05.02	2013.06.13	- PID_HARDWARE_TYPE: removed reference to V06 "Profiles" and eased the reading of the use of MSB 00h. - PID_TABLE_REFERENCE: PDT corrected. - PID_MCB_TABLE: corrected multiple appearances of PDT_GENERIC_07[] to be PDT_GENERIC_08[]. - Multiple editorial corrections.
01.06.00	2013.07.18	AN146 "Ctrl-Mode for KNX RF Multi" integrated.
01.07.00	2013.07.19	AN148 "PB-Mode for KNX RF Multi" integrated.
01.07.01	2013.07.19	AN151 "cEMI AddInfo for KNX RF Multi and new Properties" integrated.
01.07.02	2013.07.22	Editorial review.
01.08.00	2013.09.03	AN137 "Configuration Signature" integrated.
01.08.01	2013.10.18	Editorial review.
01.09.01	2013.10.22	AN153 "Mask 0912h Property based configuration" integrated. - Transfer of Coupler Resources of masks 0910h and 0911h from Part 9/3 "Couplers".
01.09.02	2013.10.28	Editorial updates for the publication of KNX Specifications 2.1.
01.09.03	2013.12.10	Final editorial review in view of publications of the KNX Specifications v2.1.
01.10.01	2019.09.11	- AN134 "Flexible E-Mode Channels" integrated.
01.10.01	2019.12.23	- AN163 "Extended Interface Object addressing" integrated: extension of PID_IO_LIST.
01.10.01	2020.03.02	- AN164 "Handling of reserved EFF-values" integration started and completed: update of PID_LTE_ROUTETABLE (clause 4.6.4).
01.10.01	2020.08.19	- AN168 v02 "KNX RF USB Interface" integrated. - PID_TRANSP_ENABLE removed according AN168 v02 "KNX RF USB interface"
01.10.01	2020.10.14	- AN178 v02 "Specification of services using EFF" integrated.
01.10.01	2020.10.29	- AN183 "KNXnet/IP System Broadcast" integrated started.
01.10.01	2021.04.21	- AN185 "KNXnet/IP Tunnelling v2" integration started
01.10.01	2021.08.19	- AN160 "KNX RF S-Mode device Profiles" integration started.
01.10.01	2021.09.22	- AN161 "Coupler Model 2.0" integration started.
01.10.01	2021.09.29	Preparation for inclusion in the KNX Specification v3.0.

References

- [01] Chapter 3/1/2 “Glossary”
- [02] Chapter 3/2/2 “Communication Medium TP1”
- [03] Chapter 3/2/5 “Radio Frequency”
- [04] Chapter 3/3/2 “Data Link Layer General”
- [05] Chapter 3/3/3 “Network Layer”
- [06] Chapter 3/3/4 “Transport Layer”
- [07] Chapter 3/3/7 “Application Layer”
- [08] Chapter 3/4/1 “Application Interface Layer”
- [09] Chapter 3/5/2 “Management Procedures”
- [10] Chapter 3/5/3 “Configuration Procedures”
- [11] Chapter 3/6/2 “Physical External Interface”
- [12] Chapter 3/6/3 “External Message Interface”
- [13] Chapter 3/7/1 “Interworking Model”
- [14] Chapter 3/7/2 “Datapoint Types”
- [15] Chapter 3/7/3 “Standard Identifier Tables”
- [16] Chapter 3/8/3 “Management” (KNXnet/IP)
- [17] Volume 6 “Profiles”
- [18] Chapter 6/2/1 “Coupler Model 2.0”
- [19] Volume 7 “Application Descriptions”
- [20] Chapter 8/2/2 “TP1 Physical and Link Layer Tests”
- [21] Part 9/3 “Couplers”
- [22] Chapter 9/4/1 “BCUs and BIMs”
- [23] Part 10/1 “Logical Tag Extended”

Filename: 03_05_01 Resources v01.10.01 AS.docx
Version: 01.10.01
Status: Approved Standard
Savedate: 2021.09.29
Number of pages: 362

Contents

1	Introduction	12
1.1	Scope.....	12
1.2	Definition.....	12
1.2.1	Contained information	12
1.2.2	Access rights	12
1.2.3	Access ways	13
1.2.4	Types of Resources	13
1.3	Requirements for the secure storage of Security information	13
1.4	Specifying Resources through their management interface	14
1.5	Abbreviations.....	14
2	Realisation types of Resources	15
2.1	Common elements of Resource definitions	15
2.2	Allowed usage of realisation types	16
2.3	E-Mode Resources.....	16
2.3.1	Differences between S-Mode Resources and E-Mode Resources.....	16
2.3.2	S-Mode Resources	16
2.3.3	E-Mode Resources	17
2.3.4	Resource data integrity in case of dual access by Link Services and memory mapped access.....	17
3	Network Resources.....	18
3.1	Overview.....	18
3.2	Domain Addresses	18
3.2.1	Definition	18
3.2.2	Receiving frames	18
3.2.3	DoA Realisation Type 1 – DoA on 2 octets	19
3.2.4	DoA Realisation Type 2 – DoA on 6 octets	19
3.3	Individual Addresses	20
3.4	Group Addresses.....	21
3.4.1	Definition	21
3.4.2	Group Address space	21
4	Device Resources	24
4.1	Device Descriptors.....	24
4.1.1	Overview	24
4.1.2	Device Descriptor Type 0 (mask version)	24
4.1.3	Device Descriptor Type 2	26
4.1.4	Descriptor Type 1 and 3 to 7.....	29
4.2	Interface Object Type independent Properties	30
4.2.1	PID_OBJECT_TYPE (PID = 1)	30
4.2.2	PID_OBJECT_NAME (PID = 2)	30
4.2.5	PID_LOAD_STATE_CONTROL (PID = 5)	30
4.2.6	PID_RUN_STATE_CONTROL (PID = 6).....	31
4.2.7	PID_TABLE_REFERENCE (PID = 7).....	31
4.2.8	PID_SERVICE_CONTROL (PID = 8).....	31
4.2.9	PID_FIRMWARE_REVISION (PID = 9)	33
4.2.10	PID_SERVICES_SUPPORTED (PID = 10).....	33
4.2.11	PID_SERIAL_NUMBER (PID = 11).....	33
4.2.12	PID_MANUFACTURER_ID (PID = 12)	33

4.2.13	PID_PROGRAM_VERSION (PID = 13).....	33
4.2.14	PID_DEVICE_CONTROL (PID = 14)	34
4.2.15	PID_ORDER_INFO (PID = 15).....	36
4.2.16	PID_PEI_TYPE (PID = 16).....	37
4.2.17	PID_PORT_CONFIGURATION (PID = 17).....	37
4.2.18	PID_POLL_GROUP_SETTINGS (PID = 18)	37
4.2.19	PID_MANUFACTURER_DATA (PID = 19)	38
4.2.21	PID_DESCRIPTION (PID = 21).....	38
4.2.23	PID_TABLE (PID = 23).....	38
4.2.25	PID_VERSION (PID = 25)	39
4.2.27	PID_MCB_TABLE (PID = 27).....	39
4.2.28	PID_ERROR_CODE (PID = 28)	40
4.2.29	PID_OBJECT_INDEX (PID = 29).....	40
4.2.30	PID_DOWNLOAD_COUNTER (PID = 30)	41
4.3	Device Object (Object Type 0).....	43
4.3.1	General requirements.....	43
4.3.2	PID_ROUTING_COUNT (PID = 51).....	44
4.3.3	PID_MAX_RETRY_COUNT (PID = 52)	44
4.3.4	PID_ERROR_FLAGS (PID = 53).....	44
4.3.5	PID_PROGMODE (PID = 54)	45
4.3.6	PID_PRODUCT_ID (PID = 55).....	46
4.3.7	PID_MAX_APDU_LENGTH (PID = 56)	46
4.3.8	PID_SUBNET_ADDR (PID = 57).....	47
4.3.9	PID_DEVICE_ADDR (PID = 58).....	47
4.3.10	PID_PB_CONFIG (PID = 59).....	48
4.3.11	PID_ADDR_REPORT (PID = 60).....	59
4.3.12	PID_ADDR_CHECK (PID = 61).....	59
4.3.13	PID_OBJECT_VALUE (PID = 62)	60
4.3.14	PID_OBJECTLINK (PID = 63)	61
4.3.15	PID_APPLICATION (PID = 64)	66
4.3.16	PID_PARAMETER (PID = 65)	67
4.3.17	PID_OBJECTADDRESS (PID = 66).....	68
4.3.18	PID_PSU_TYPE (PID = 67)	69
4.3.19	PID_PSU_STATUS (PID = 68)	70
4.3.20	PID_PSU_ENABLE (PID = 69).....	71
4.3.21	PID_DOMAIN_ADDRESS (PID = 70).....	71
4.3.22	PID_IO_LIST (PID = 71)	72
4.3.23	PID_MGT_DESCRIPTOR_01 (PID = 72)	72
4.3.24	PID_PL110_PARAM (PID = 73).....	75
4.3.25	PID_RECEIVE_BLOCK_TABLE (PID = 75)	76
4.3.26	PID_RANDOM_PAUSE_TABLE (PID = 76)	77
4.3.27	PID_RECEIVE_BLOCK_NR (PID = 77).....	78
4.3.28	PID_HARDWARE_TYPE (PID = 78).....	78
4.3.29	PID_RETRANSMITTER_NUMBER (PID = 79)	79
4.3.30	PID_SERIAL_NR_TABLE (PID = 80)	79
4.3.31	PID_BIBAT_MASTER_ADDRESS (PID = 81)	82
4.3.32	PID_RF_DOMAIN_ADDRESS (PID = 82)	82
4.3.33	PID_DEVICE_DESCRIPTOR (PID = 83).....	82
4.3.34	PID_GROUP_TELEGR_RATE_LIMIT_TIME_BASE (PID = 85)	82
4.3.35	PID_GROUP_TELEGR_RATE_LIMIT_NO_OF_TELEGR (PID = 86).....	83

4.3.36	PID_FEATURES_SUPPORTED (PID: 89).....	84
4.3.37	PID_CHANNEL_01_PARAM (PID = 101) to PID_CHANNEL_32_PARAM (PID = 132)	85
4.4	Applicationprogram Object (Object Type 3).....	86
4.4.1	PID_OPERATION_MODE (PID: 52)	86
4.5	Router Object (Object Type 6)	90
4.5.1	General requirements	90
4.5.2	PID_OBJECT_TYPE (PID = 1)	90
4.5.3	PID_LOAD_STATE_CONTROL (PID = 5)	91
4.5.4	PID_MEDIUM_STATUS (PID = 51).....	91
4.5.5	PID_MAIN_LCCONFIG (PID = 52) PID_SUB_LCCONFIG (PID = 53)	92
4.5.6	PID_MAIN_LCGRPCONFIG (PID = 54) PID_SUB_LCGRPCONFIG (PID = 55)	93
4.5.7	PID_ROUTETABLE_CONTROL (PID = 56).....	94
4.5.8	PID_COUPL_SERV_CONTROL (PID = 57)	99
4.5.9	PID_MAX_APDU_LENGTH (PID = 58)	100
4.5.10	PID_L2_COUPLER_TYPE (PID = 59)	100
4.5.11	PID_HOP_COUNT (PID = 61).....	102
4.5.12	PID_MEDIUM (PID = 63)	102
4.5.13	PID_FILTER_TABLE_USE (PID = 67).....	102
4.5.14	KNX PL110 specific Coupler Properties.....	104
4.5.15	KNX RF specific Coupler Properties	104
4.5.16	KNX IP Specific Coupler Parameters.....	107
4.6	LTE Address Routing Table Object (Object Type 7).....	110
4.6.1	General requirements	110
4.6.2	PID_LOAD_STATE_CONTROL (PID = 5)	110
4.6.3	PID_LTE_ROUTESELECT (PID = 51)	111
4.6.4	PID_LTE_ROUTETABLE (PID = 52)	111
4.7	cEMI Server Object (Object Type 8).....	112
4.7.1	PID_MEDIUM_TYPE (PID = 51)	112
4.7.2	PID_COMM_MODE (PID = 52)	112
4.7.3	PID_MEDIUM_AVAILABILITY (PID = 53).....	113
4.7.4	PID_ADD_INFO_TYPES (PID = 54)	113
4.7.5	PID_TIME_BASE (PID = 55).....	113
4.7.6	PID_CEMI_SERVER_SNA (PID = 57)	114
4.7.7	PID_CEMI_SERVER_DEVICE_ADDRESS (PID = 58)	116
4.7.8	PID_BIBAT_NEXTBLOCK (PID = 59)	117
4.7.9	PID_RF_MODE_SELECT (PID = 60)	117
4.7.10	PID_RF_MODE_SUPPORT (PID = 61)	118
4.7.11	PID_RF_FILTERING_MODE_SELECT (PID = 62).....	118
4.7.12	PID_RF_FILTERING_MODE_SUPPORT (PID = 63).....	118
4.7.13	PID_COMM_MODES_SUPPORTED (PID = 64)	119
4.7.14	PID_FILTERING_MODE_SUPPORT (PID = 65).....	120
4.7.15	PID_FILTERING_MODE_SELECT (PID = 66).....	121
4.7.16	PID_MAX_INTERFACE_APDU_LENGTH (PID: 68).....	123
4.7.17	PID_MAX_LOCAL_APDU_LENGTH (PID: 69)	124
4.8	Group Object Table Object (Object Type 9)	124
4.8.1	PID_GO_DIAGNOSTICS (PID: 66)	124
4.9	KNXnet/IP Parameter Object (Object Type 11).....	144
4.10	File Server Object (Object Type 13)	145

4.10.1	Overview	145
4.10.2	Property MAX. APDU-Length Out (PID = 51)	145
4.10.3	Property File Command (PID = 52)	145
4.10.4	Property File Path (PID = 53)	161
4.10.5	Property MAX. APDU-Length (PID = 56)	162
4.11	E-Mode Channel Object (Object Type 14).....	162
4.11.1	General.....	162
4.11.2	E-Mode Channel Object – PID_OBJECT_TYPE (PID = 1).....	163
4.11.3	E-Mode Channel Object – PID_OBJECT_NAME (PID = 2)	163
4.11.4	E-Mode Channel Object – PID_DESCRIPTION (PID = 21).....	164
4.11.5	E-Mode Channel Object – PID_VERSION (PID = 25)	164
4.11.6	E-Mode Channel Object – PID_CHAN_NUMBER (PID = 51)	165
4.11.7	E-Mode Channel Object – PID_CHAN_CODE (PID = 52)	165
4.11.8	E-Mode Channel Object – PID_CHAN_FLAGS (PID = 53).....	165
4.11.9	E-Mode Channel Object – PID_CHAN_FB_LIST (PID = 54).....	167
4.11.10	E-Mode Channel Object – PID_CHAN_ADJ_LISTS (PID = 55)	167
4.11.11	E-Mode Channel Object – PID_GO_CCODES_LIST (PID = 61).....	168
4.11.12	E-Mode Channel Object – PID_GO_CFLAGS_LIST (PID = 62)	169
4.11.13	E-Mode Channel Object – PID_OBJECTLINK (PID = 63)	170
4.11.14	E-Mode Channel Object – PID_GO_SUBUNIT (PID = 64)	170
4.11.15	E-Mode Channel Object – PID_GO_NAME_LIST (PID = 65).....	170
4.11.16	E-Mode Channel Object - PID_GO_DIAGNOSTICS (PID: 66).....	171
4.11.17	E-Mode Channel Object – PID_PARAM_TYPES (PID = 70)	172
4.11.18	E-Mode Channel Object – PID_PARAM_FLAGS (PID = 71).....	176
4.11.19	E-Mode Channel Object – PID_PARAM_NAMES (PID = 72)	178
4.11.20	E-Mode Channel Object – PID_PARAM_UNITS (PID = 73).....	178
4.11.21	E-Mode Channel Object – PID_PARAM_VALUES (PID = 79).....	180
4.12	Adjusted E-Mode Channel Object (Object Type 15)	181
4.12.1	General.....	181
4.12.2	Adjusted E-Mode Channel Object – PID_OBJECT_TYPE (PID = 1)	182
4.12.3	Adjusted E-Mode Channel Object – PID_OBJECT_NAME (PID = 2).....	182
4.12.4	Adjusted E-Mode Channel Object – PID_GO_CCODES_LIST (PID = 61) .	182
4.12.5	Adjusted E-Mode Channel Object – PID_GO_CFLAGS_LIST (PID = 62)..	182
4.12.6	Adjusted E-Mode Channel Object – PID_OBJECTLINK (PID = 63).....	182
4.12.7	Adjusted E-Mode Channel Object – PID_GO_DIAGNOSTICS (PID: 66)...	182
4.12.8	Adjusted E-Mode Channel Object – PID_GO_NAME_LIST (PID = 65)	182
4.12.9	Adjusted E-Mode Channel Object – PID_PARAM_TYPES (PID = 70).....	183
4.12.10	Adjusted E-Mode Channel Object – PID_PARAM_FLAGS (PID = 71)	183
4.12.11	Adjusted E-Mode Channel Object – PID_PARAM_NAMES (PID = 72).....	183
4.12.12	Adjusted E-Mode Channel Object – PID_PARAM_UNITS (PID = 73)	183
4.12.13	Adjusted E-Mode Channel Object – PID_PARAM_VALUES (PID = 79) ...	183
4.13	Text Catalogue Object (Object Type 16).....	183
4.13.1	General requirements	183
4.13.2	PID_OBJECT_TYPE (PID = 1)	183
4.13.3	PID_VERSION (PID = 25)	184
4.13.4	PID_LOCALE_LIST (PID = 51)	184
4.13.5	PID_LOCALE_SELECTION (PID = 52)	185
4.13.6	PID_ACTIVE_LOCALE (PID = 53)	186
4.13.7	PID_STRING_001 (PID = 60) to PID_STRING_141 (PID = 200).....	186
4.14	E-Mode Device Object (Object Type 18).....	187

4.14.1	Overview	187
4.14.2	General requirements	187
4.14.3	E-Mode Device Object – PID_OBJECT_TYPE (PID = 1)	187
4.14.4	E-Mode Device Object – PID_LOCALISATION_MODE (PID = 60)	188
4.14.5	E-Mode Device Object – PID_LOCALISATION_REPORT (PID = 61)	189
4.14.6	E-Mode Device Object – PID_LOCALISATION_COMMAND (PID = 62)	190
4.15	RF Medium Object (Object Type 19)	193
4.15.1	Overview	193
4.15.2	RF Medium Object for cEMI	193
4.15.3	PID_RF_MULTI_TYPE (PID = 51)	194
4.15.4	PID_RF_MULTI_PHYSICAL_FEATURES (PID = 52)	195
4.15.5	PID_RF_MULTI_CALL_CHANNEL (PID = 53)	196
4.15.6	PID_RF_MULTI_OBJECT_LINK (PID = 54)	197
4.15.7	PID_RF_MULTI_EXT_GA_REPEATED (PID = 55)	204
4.15.8	PID_RF_DOMAIN_ADDRESS (PID = 56)	206
4.15.9	PID_RF_RETRANSMITTER (PID: 57)	206
4.15.10	PID_RF_BIDIR_TIMEOUT (PID: 60)	207
4.15.11	PID_RF_DIAG_SA_FILTER_TABLE[] (PID: 61)	208
4.15.12	PID_RF_DIAG_QUALITY_TABLE[] (PID: 62)	210
4.15.13	PID_RF_DIAG_PROBE (PID: 63)	211
4.15.14	PID_TRANSMISSION_MODE ((PID = 70)	213
4.15.15	PID_RECEPTION_MODE (PID = 71)	214
4.15.16	PID_TEST_SIGNAL (PID = 72)	216
4.15.17	PID_FAST_ACK (PID = 73)	217
4.15.18	PID_FAST_ACK_ACTIVATE (PID = 74)	218
4.15.19	PID_RF_TYPES_SUPPORTED (PID = 75)	220
4.16	Group Address Table (GrAT)	222
4.16.1	Abstract Resource definition	222
4.16.2	Access Control to the Group Address Table	222
4.16.3	Group Address Table - Realisation Type 1	223
4.16.4	Group Address Table - Realisation Type 2	226
4.16.5	Group Address Table - Realisation Type 4	227
4.16.6	Group Address Table - Realisation Type 5	230
4.16.7	Group Address Table – Realisation Type 6	234
4.16.8	Group Address Table – Realisation Type 7	237
4.16.9	Group Address Table – Realisation Type 8	240
4.16.10	Group Address Table - E-Mode Realisation Type 1 (GrAT – Easy 1)	240
4.16.11	Group Address Table - E-Mode Realisation Type 2 (GrAT – Easy 2)	242
4.17	Group Object Association Table (GrOAT)	245
4.17.1	Abstract Resource definition	245
4.17.2	Access Control to the Group Object Association Table	245
4.17.3	Group Object Association Table - Realisation Type 1	245
4.17.4	Group Object Association Table - Realisation Type 2	248
4.17.5	Group Object Association Table - Realisation Type 6	248
4.17.6	Group Object Association Table - Realisation Type 8	252
4.17.7	Group Object Association Table - E-Mode Realisation Type 1 (GrOAT – Easy 1)	253
4.17.8	Group Object Association Table – E-Mode Realisation Type 2 (GrOAT – Easy 2)	255

4.17.9	Group Object Association Table – E-Mode Realisation Type 3 (GrOAT – Easy 3)	258
4.18	Group Object Table	261
4.18.1	Abstract Resource Definition.....	261
4.18.2	Access Control to the Group Object Table	261
4.18.3	Group Object Table - Realisation Type 1	261
4.18.4	Group Object Table - Realisation Type 2	265
4.18.5	Group Object Table - Realisation Type 6	266
4.18.6	Group Object Table - Realisation Type 7	269
4.18.7	Group Object Table – E-Mode Realisation Type 1 (GrOT – Easy 1)	273
4.18.8	Group Object Table – E-Mode Realisation Type 2 (GrOT – Easy 2)	275
4.18.9	Group Object Table – E-Mode Realisation Type 3 (GrOT – Easy 3)	276
4.19	Easy Parameter Block Table (PaBT).....	278
4.19.1	Abstract Resource definition.....	278
4.19.2	Parameter Block Table – E-Mode Realisation Type 1 (PaBT – Easy 1).....	278
4.19.3	Parameter Block Table – E-Mode Realisation Type 2 (PaBT – Easy 2).....	281
4.19.4	Parameter Block Table – E-Mode Realisation Type 3 (PaBT – Easy 3).....	284
4.20	Application Program.....	286
4.20.1	Abstract Resource Definition.....	286
4.20.2	Application Program – Realisation Type 6.....	286
4.20.3	Application Program – Realisation Type 7.....	288
4.21	Application Program 2.....	289
4.21.2	Application Program 2 – Realisation Type 7.....	289
4.22	KNX Serial Number	291
4.22.1	Abstract Resource Definition.....	291
4.22.2	KNX Serial Number - Realisation Type 1	292
4.22.3	KNX Serial Number - Realisation Type 2	293
4.23	Load State Machine	293
4.23.1	Abstract Resource definition.....	293
4.23.2	Load State Machine – Realisation Type 1 (Property based).....	293
4.23.3	Load State Machine – Realisation Type 2 (Memory mapped).....	297
4.24	Run State Machine.....	298
4.24.1	Abstract Resource definition.....	298
4.24.2	Run State Machine – Realisation Type 1 (Property based).....	298
4.24.3	Run State Machine – Realisation Type 2 (Memory mapped).....	302
4.25	OptionReg.....	302
4.25.1	Abstract Resource definition.....	302
4.25.2	OptionReg – Realisation Type 1	302
4.26	Programming Mode (prog_mode)	303
4.26.1	Abstract Resource definition.....	303
4.26.2	Programming Mode – Realisation Type 1	303
4.26.3	Programming Mode – Realisation Type 2	303
4.27	Group telegram rate limitation.....	305
4.27.1	Abstract Resource definition.....	305
4.27.3	Telegram Rate Limitation – Realization Type 2.....	306
5	Resources for Couplers.....	307
5.1	Resources for mask 0910h and mask 0911h.....	307
5.2	Resources for mask 0912h.....	311
5.2.1	Filter Table Realisation Type 2.....	311
5.2.2	Coupler Parameters.....	312

5.2.3	Load State Machine – Realisation Type 1 (Property based).....	316
5.3	Resources for Coupler Model 2.0.....	317
5.3.1	Filter Table Realization Type 3	317
5.3.2	Device Object.....	319
5.3.3	Router Object.....	321
6	Security Resources	324
6.1	Factory Default Setup Key (FDSK)	324
6.1.1	Abstract Resource definition.....	324
6.1.2	Security	324
6.1.3	Location and format.....	324
6.1.4	Usage by the MaS	324
6.1.5	Usage by the MaC.....	325
6.2	Sequence Number for Tool Access	325
6.3	Security Interface Object	325
6.3.1	Introduction and overview	325
6.3.2	PID_OBJECT_TYPE (PID: 1)	326
6.3.3	PID_OBJECT_NAME (PID: 2)	327
6.3.4	PID_LOAD_STATE_CONTROL (PID: 5)	327
6.3.5	PID_SECURITY_MODE (PID: 51).....	327
6.3.6	PID_P2P_KEY_TABLE (PID: 52)	330
6.3.7	PID_GRP_KEY_TABLE (PID: 53)	333
6.3.8	PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE (PID: 54)	334
6.3.9	PID_SECURITY_FAILURES_LOG (PID: 55).....	336
6.3.10	PID_TOOL_KEY (PID: 56).....	342
6.3.11	PID_SECURITY_REPORT (PID: 57).....	344
6.3.12	PID_SECURITY_REPORT_CONTROL (PID: 58).....	345
6.3.13	PID_SEQUENCE_NUMBER_SENDING (PID: 59)	346
6.3.14	PID_ZONE_KEY_TABLE (PID: 60).....	347
6.3.15	PID_GO_SECURITY_FLAGS (PID: 61).....	348
Annex A (informative)	Use cases for Group Object Diagnostics.....	351
A.1	Use case 1: set the value of a GO to diagnose its proper working	351
A.2	Use case 2: get the value of a GO to diagnose its proper working.....	351
A.3	Use case 3: path check.....	352
A.4	Use case 4: web services (and visualisation).....	354
A.5	Use case 5: write group message from ETS	355
A.6	Use case 6: read group message from ETS	356
Annex B (informative)	Examples of table implementations.....	358
B.1	Introduction.....	358
B.2	Extension of KNX to enable use of Extended Addresses.....	358
B.2.1	Existing resources from KNX.....	358
B.2.2	New resources	358
B.3	Association Table Object.....	360
B.3.1	Existing object	360
B.3.2	New Property : Association Table format 1	360
B.4	Examples of usage	361
B.4.1	Use of tables in case of reception	361
B.4.2	Use of tables for sending.....	362

1 Introduction

1.1 Scope

To allow for KNX devices to function properly in the network in which they are installed, all KNX devices shall allow for being configured, from the minimum level up to a more or less advanced configuration functionality. This document targets on specifying data structures that shall be used for configuring KNX devices. These data structures are called *Resources*.

The standard KNX system communication stack already provides the necessary functionality for accessing Resources. This functionality consists either of dedicated services in the Application Layer (see [07]) or the generic architecture of the Interface Objects.

This document specifies the existing standard Resources of the KNX system. For every Resource it specifies the configuration functionality that can be managed by it, the data structure of the Resource, how the Resource shall be managed by a Management Client and how the Management Server – typically the device – shall interpret the Resource's data.

The Management Procedures shall target on managing these Resources.

1.2 Definition

1.2.1 Contained information

Resources are these data in a device that contain information relevant to the device configuration.

The information contained in Resources serves for

- device identification and discovery
EXAMPLES serial number, device descriptor, manufacturer, device type, ...
- addressing and binding
EXAMPLES Individual Address, Group Address Table, Group Object Association Table, Group Object Table, ...
- application configuration
EXAMPLES application code, application parameters, ...

This list is not exclusive.

1.2.2 Access rights

This data can be

- read-only, this is, provided by the device for information to a Management Client, but not changeable by a Management Client, or
- read-write, this is, a Management Client can set the Resource data in the device in order to influence its behaviour.

Next to the read-only or read-and-write access to a Resource, the access can be limited to secure communication, using at least authentication. For the KNX standard Resources, this is indicated further in this paper.

1.2.3 Access ways

A Resource can be accessed by

- a Management Client, remote

The Resource is accessed (read and/or write) from the bus by a Management Client, using dedicated services.

- a Management Client, local

The Resource is accessed via the PEI (see [11]) and the EMI (see [12]).

This method is relevant when a Management Client, typically the S-Mode Tool, configures the device to which it is directly connected via the PEI.

It is however also relevant in case of a dual-processor device. These devices are based on standard BAU serving as host for the bus access for an external processor. The external processor shall use the same access methods to the Resources hosted by the BAU as specified for the local Management Client.

- a Management Server, internal

This specifies how the device shall access and interpret the information contained by the Resource.

This differentiation in access ways is reflected in the individual Resource descriptions in the following subclauses.

1.2.4 Types of Resources

In this document a differentiation is made between *Network Resources* and *Device Resources*.

Network Resources are these Resources of which the possible data range is shared with other devices. The data set in the Network Resources of different devices shall either

- match, to make these devices work together

EXAMPLES Domain Address, Subnetwork Address...

- differ, to avoid communication conflicts

EXAMPLES Individual Address (Subnetwork Address + Device Address), KNX Serial Number ...

Device Resources are these Resources of which the possible data solely serves for the configuration of the device in which they are contained. The contained data has no relation to the data of Resources in other devices; these Resources do not (directly) influence the relationship between this device and any other device.

1.3 Requirements for the secure storage of Security information

Devices must protect critical security information like keys, permission tables etc. from any access. Even when having physical access to the device an attacker should not be able to gain any critical information by for example reading out flash or using a JTAG interface.

1.4 Specifying Resources through their management interface

The specification of Resources given below is actually the specification of their management interface. This means that the identifiers (maybe even location identifiers like “memory addresses”), encoding and formats given, specify the content of the management messages to control (i.e. read, write or provide) the (elements of) a Resource. *The way this is mapped to internal memory locations (including bit order etc.) is not defined and left entirely to the implementation.*

In other words: the specification lays down how these Resources are seen and handled by a Management Client (either local or remote). As such, they are essentially platform-independent.

This specification lays no requirements on the internal storage and modelling in the Management Client. The Management Client shall be able to map between its internal storage style and the modelling required by the Resource specifications below.

1.5 Abbreviations

Please refer to [01] for the abbreviations used in this document.

2 Realisation types of Resources

2.1 Common elements of Resource definitions

A Realisation Type fixes a particular standard implementation for a Resource (interface). This implies certain choices are made, for example concerning:

- the length of identifiers (e.g. SAPs), which influences the required storage space as well as the allowable range;

NOTE 1- The term Service Access Point (SAP) is defined in ISO/IEC 74981-1 "The Basic Model" as "The point at which (N)-services are provided by the (N)-entity to an (N+1)-entity".

- sorting of “lookup” Resources, to speed up their evaluation, etc.
- (location) identifiers, usually corresponding to a (historical) memory mapped implementation.

In the clauses of this document, a *common structure* is used to describe a realisation type of a Resource.

1. *Format*

This clause specifies the image of the Resource realisation type as it is visible by its interfaces: this can be a memory mapped structure or an Interface Object or a Property of an Interface Object. It is up to the implementer of the Realisation Type to make this image match with his data.

2. *Location*

This clause specifies the way to access the realisation type: this can be an address for memory mapped Resource types or Object Types and Property Identifiers for Resources implemented as (part of) Interface Objects.

3. *Usage by the Management Server*

This clause specifies how the Resource shall be evaluated by the services that base on it.

4. *Usage by the Management Client*

This clause specifies how any Management Client shall manage the realisation type of the Resource. A differentiation is made in function of the access way to the actual Resource:

- | | |
|----------------------------|--|
| 4.1 Remote access | This specifies how the Resources shall be managed when being accessed via the bus. |
| 4.2 Local access | This specifies how the Resources shall be managed when being accessed via the PEI. |
| 4.3 Internal Access | This specifies how an internal application shall access this Resource if it takes direct access to it. |

2.2 Allowed usage of realisation types

Some realisation types may be restricted for future use, as “not recommended” or “not allowed” in the Resource container (server). In this case, they are specified to allow the implementation of backward compatible Management Clients.

To accommodate a limited, well-defined set of implementations, for several Resources there may be more than one realisation type defined.

One complete system implementation should provide a sufficiently homogenous set of Resource management interfaces. For this reason, realisation types of different Resources shall not be combined at will. Rather, the allowable combinations are laid down in the Profile specifications (see [17]).

2.3 E-Mode Resources

2.3.1 Differences between S-Mode Resources and E-Mode Resources

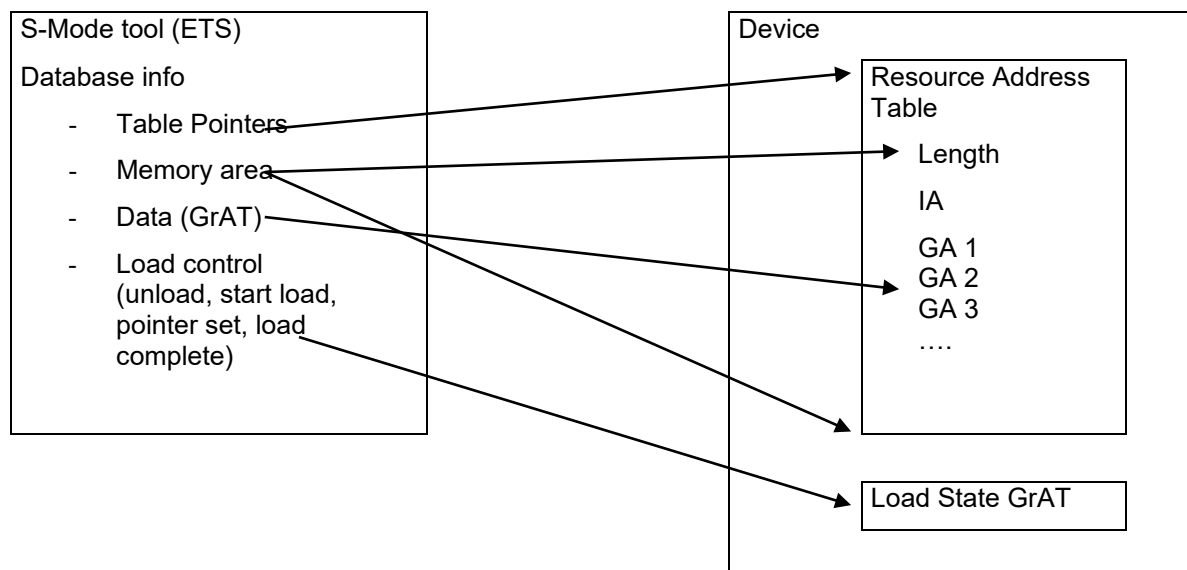
The S-Mode Resources and the E-Mode Resources are very similar, but there is a distinction in handling of table data, pointers and procedures. For the access inside the device through the device management there is no difference, in fact the device itself can be both an E-Mode device as well an S-Mode device.

2.3.2 S-Mode Resources

The S-Mode Resources are designed for a download into an S-Mode device by the S-Mode Management Client, typically a computer based software. This procedure requires the information for table data, for the pointers, the available memory Resource inside the device in the tools’s database.

With this information the Resource can be handled e.g. a Group Address Table can be created, filled in and started.

EXAMPLE Group Address Table System mode

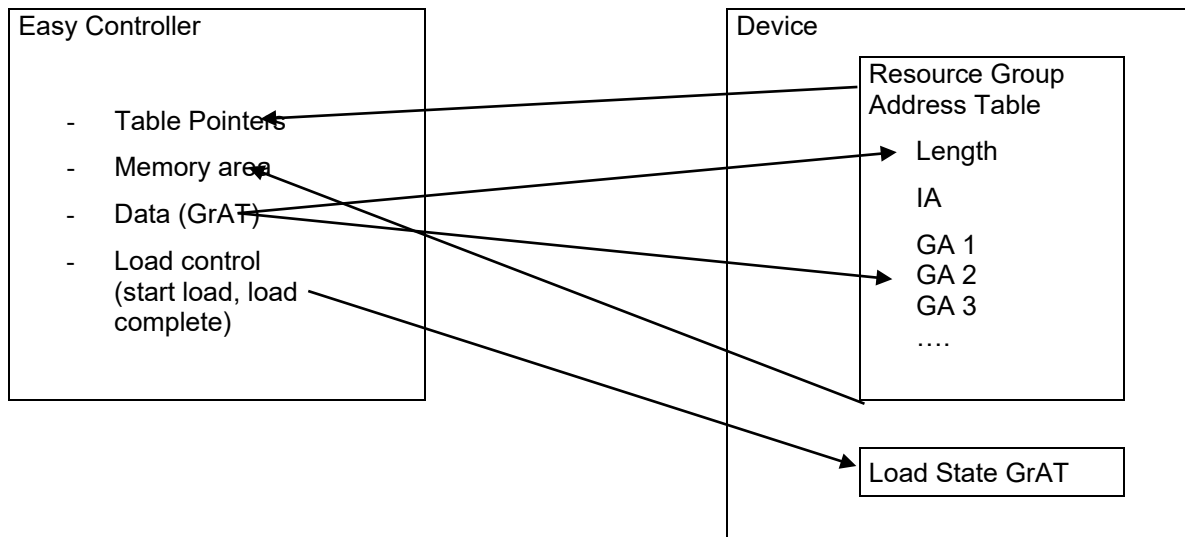


2.3.3 E-Mode Resources

The E-Mode Resources for the Ctrl-Mode are designed for writing and reading in a preloaded Ctrl-Mode device. The Resource can be read and written like a handling read - modify - write back.

EXAMPLE The content of an existing Group Address Table can be changed. The Group Addresses are generated by the controller; the pointers and memory information are fetched from the E-Mode device. The (existing) GrAT is stopped, changed and then restarted.

EXAMPLE Group Address Table System mode



2.3.4 Resource data integrity in case of dual access by Link Services and memory mapped access

In devices providing memory mapped access and Link Services, the same Resources shall be shared (Group Address Table, Group Association Table and Group Object table). A device may provide both mechanisms but must ensure Resources' consistency. For this, Resources shall not be accessed with both mechanisms at one given time.

In this condition, an E-Mode Controller may use both mechanisms for link management. It does not have to check the device state before downloading via memory mapped access or Link Services.

3 Network Resources

3.1 Overview

In the KNX System, the network addresses are encoded on a 16 bit field, providing an addressable range of 65 535 addresses.

The usage of the address space for the Individual Addresses is subject to the structured topology that is designed for the network. The format of the Individual Address is specified in [04]. The *Subnetwork Address* reflects the position of the Subnetwork to which the device is connected in the topology.

The address space available for devices - the *Device Address space* - allows for 256 different Device Addresses on a Subnetwork. Table 2 indicates the allocation of Device Addresses per Subnetwork.

The address space available for groups of objects or devices - the *Group Address space* - is 16 bit wide, allowing for up to 65 535 of addresses on the whole network. It is recommended to assign the Group Addresses only once per network to avoid potential inconsistent network behaviour and to simplify the integrity tests for the Group Address assignments that are made.

Table 4 indicates the recommended partitioning of the Group Address space. The partitioning is particularly important when KNX installations make use of the different Configuration Modes.

3.2 Domain Addresses

3.2.1 Definition

If the medium is an open medium and if this has to be shared with instances of other Subnetworks, a Domain Address (DoA) is required as an identifier to logically separate the Subnetworks from each other.

Each Domain Address shall identify a logical Subnetwork on an open medium. *On some open media the address space may be reduced or practically avoided (e.g. direct line-of-sight links with IR).*

Alternative solutions than the Domain Address are possible to limit communication on open media to one or more Subnetworks of a given KNX installation. For instance, communication can be limited to devices with a known KNX Serial Number. Please refer to the specification of the various communication media in Part 3/2 "Communication Media" for the options per medium.

3.2.2 Receiving frames

A device shall always receive frames that are sent using the Broadcast Domain Address, whether it has gained an Individual Address for its runtime operation (through installation procedures, registration or contention) or not. The Destination Address – Group – or Individual - shall be evaluated and be considered for acceptance.

To avoid abuse of the broadcast Domain Address it is recommended to use this address only in a controlled (guarded) situation. Therefore, the device's management processes will decide on the acceptance and execution of the message.

3.2.3 DoA Realisation Type 1 – DoA on 2 octets

The Domain Address shall be encoded on a 16 bit field, providing over 65 535 possible Domain Addresses,

Table 1 - Domain Address space

Domain Address	Usage
FFFFh	DoA for use as non-divided open medium
FFFEh	reserved Domain Addresses
FFF0h	
FFEFh	
	free Domain Address space (e.g. boundary of a reduced DoA space)
00FFh	
0001h	Broadcast Domain Address
0000h	

The Domain Addresses space is subdivided in four segments with a specific or recommended use.

1. A first segment is the single Domain Address 0000h that is exclusively reserved as a Broadcast Domain Address. This address shall be supported by devices that use the open medium for Domain Address management.
2. The second segment of Domain Addresses ranges from 0001h to FFEFh. The usage of the Domain Address in this range shall be subject to prior Domain Address Acquisition procedures to create logical Subnetworks on an open medium.
3. The third segment of Domain Addresses, ranging from FFF0h to FFFEh, is reserved for future extensions. These Domain Addresses shall not be used.

The fourth segment is the single Domain Address FFFFh that shall be reserved for simple or small configurations and environments where the division of the open medium in logical Subnetworks is not required. This Domain Address shall not be used by the Domain Address Acquisition procedures.

NOTE 2 The support of this Domain Address will actually enable a system to operate without any assignment of a Domain Address by a central Management Client, or self acquisition, or a Domain Address Server

In case the Domain Address space is smaller than the indicated 65 535, the Domain Address space shall start from the broadcast Domain Address and consecutively to higher Domain Addresses. The Broadcast Domain Address shall always be the first Domain Address in the available address space.

EXAMPLE In Table 1 the address 00FFh is the upper value of a reduced Domain Address space, used to reduce the hardware demands on devices using E-Mode, or open media with a rather low density of Domain Addresses.

3.2.4 DoA Realisation Type 2 – DoA on 6 octets

If unconfigured (ex-factory) the Domain Address is void and shall have the default value FFFFFFFFh.

The value 000000000000h is not a valid DoA. The MaC shall not assign this value to any MaS.

LTE devices shall not use the default Domain Address for runtime communication.

3.3 Individual Addresses

The allocation of the Device Addresses - as part of the Individual Addresses - is given in Table 2.

The Subnetwork Address is subject to the hierarchical network topology and shall as such be assigned by a tool, the installer (end-user) or one of the Network Management procedures.

The table of the Device Addresses, and in particular the pre-assigned addresses, shall be applied to every Subnetwork.

The addresses 00h and FFh are reserved Device Addresses.

- Address 00h shall be used by the Router on any given Subnetwork.
- Address FFh shall be used as a default Device Address for any device that has not got assigned a Device Address.

The reserved Device Addresses shall be respected by any Configuration Mode.

Table 2 - Device Address space

Device Address	Usage
FFh	Unregistered Devices
FEh	free
FDh	free
05h	free
04h	free
03h	free
02h	free
01h	free
00h	Router

If there are no means to determine the local Subnetwork Address, the medium dependent default Subnetwork Address shall be used.

Table 3 - Medium dependent default Subnetwork Addresses

Configuration Mode	Default Subnetwork Address	Medium	Resulting medium dependent default IA
• S-Mode	FFh	All media	FFFFh
• All E-Modes	01h	reserved	01FFh
	02h	TP 1	02FFh
	03h	reserved	03FFh
	04h	PL 110	04FFh
	05h	RF	05FFh
	06h to FFh	reserved	not applicable

The procedure for assigning the Individual Address to a device depends on the Configuration Mode(s) supported by the device. Please refer to [10]. These Configuration Procedures use the Management Procedures NM_DomainAndIndividualAddress_Write2, NM_IndividualAddress_Write, NM_IndividualAddress_SerialNumber_Write, NM_IndividualAddress_SerialNumber_Write2 or other as specified in [09].

3.4 Group Addresses

3.4.1 Definition

If Group Addresses used in one Subnetwork would be re-used for different purposes in one or more other Subnetworks, then this can lead to consistency problems and loss of integrity of the applications. Therefore, Group Addresses shall be unique on the whole network.

The Group Addresses are assigned as 16 bit wide identifiers, allowing for 65 535 Group Addresses ¹⁾. This is specified in [04].

3.4.2 Group Address space

3.4.2.1 Ranges

The Group Address space is subdivided in four ranges with a specific or recommended use. Table 4 reflects this subdivision.

A first range is the single Group Address 0000h that is exclusively reserved for the Broadcast Address that shall be exclusively used in broadcast communication mode.

The further ranges are made only to reduce the risk of conflicts in multicast communication mode with Group Addresses when the KNX system comprises more than one Configuration Mode. It is recommended to respect these ranges; however, per installation it may be decided to extend the range, foreseen that the end user does not introduce inconsistencies.

The range of 0001h to 6FFFh is preferably used for the System Configuration mode (S-Mode).

The range of 7000h to 7FFFh is preferably used for automatic configuration of the Group Addresses.

The range of 8000h to FFFFh is preferably used for the Easy Configuration mode (E-Mode).

Table 4 - Group Address space

Group Address	Usage
FFFFh	Range preferably to be used by Easy Configuration Mode (E-Mode)
8000h	
7FFFh	Range preferably to for automatic configuration of the Group Addresses.
7000h	
6FFFh	Range preferably to be used by System Configuration Mode (S-Mode)
0001h	
0000h	Reserved for broadcast communication

¹⁾ The Extended Frame Format allows for a larger range. Please refer to Part 10/1 “LTE Specification”.

3.4.2.2 Range 8000h to FFFFh - Range preferably used by E-Mode

Range 8000h to FFFFh is further subdivided, defining ranges to be used for Functions of Common Interest, Group Address Ex-factory, Predefined addresses ... This is specified in Table 5 below.

Table 5 - Subdivision of Group Address segment to be used by E-Mode

Start address	End address	Number of addresses	Usage
FF00h	FFFFh	256	Reserved for specific system usage
F400h	FEFFh	2 816	Reserved for future extensions
F000h	F3FFh	1 024	- Group address ex-factory - Group Addresses for Functions of Common Interest See 3.4.2.3 below.
E000h	EFFFh	4 096	Reserved for future extensions
C000h	DFFFh	8 192	Free area with Group Address checking See 3.4.2.4 below.
8000h	BFFFh	16 384	reserved See 3.4.2.5 below.

The KNX System specifies a specific procedure to check if a Group Address is already in use or not. This is specified as the procedure NM_GroupAddress_Scan in [09]. It is mandatory to execute this procedure with any Group Address one wants to assign in the range C000h to DFFFh.

In the segment for Auto Configuration Mode, the Group Addresses 7000h to 700Fh are reserved for network management specific use.

3.4.2.3 Range F000h to F3FFh

Predefined Group Addresses for functions of common interest and Group Address ex-factory share the same address range F000h to F3FFh] according the following assignment scheme.

Group address	Usage
F000h	Group addresses ex-factory
	↓
	↑
F3FFh	Group address for Functions of Common Interest

Table 6 specifies the reserved Group Addresses that have been assigned already.

Table 6 – Assigned Group Addresses for functions of common interest and Group Addresses ex-factory

Group Address	Type	Usage
...		
F3F3h	FOCI	current electricity tariff
...		
F3F8h	FOCI	ChangeOverStatusWater (DPT_Heat/Cool, 1.100)
F3F9h	FOCI	Outside Temperature (DPT_Value_Temp, 9.001)
F3FAh	FOCI	Technical Alarm "AlarmInfo"
F3FBh	FOCI	Technical Alarm "AlarmText"
F3FCh	reserved	reserved
F3FDh	reserved	reserved
F3FEh	FOCI	current date and time
F3FFh	reserved	reserved

Other Group Addresses than the above in this range, which have not yet been assigned, shall not be used and are under the control of KNX Association.

3.4.2.4 Range C000h to DFFFh

Before using a Group Address in the range C000h to DFFFh a Management Client shall ensure that this Group Address is free before assigning it (either by check, or by systematic scanning of links in devices, assuming it is possible for all devices)

Warning

Once some change has been made with the tool in this address space, the tool has to be reused when making subsequent changes in the installation.

3.4.2.5 Range 8000h to BFFFh

This range is not used.

4 Device Resources

4.1 Device Descriptors

4.1.1 Overview

The following Descriptor Types are specified

- Device Descriptor Type 0
Abbreviation: DD0
Synonym: “mask version”

and

- Device Descriptor Type 2
Abbreviation: DD2
This Device Descriptor Type contains also an application description.

Any KNX device shall support at least one Descriptor Type.

The types 1 and 3 to 63 are reserved for future use by KNX Association and shall not be used in implementations.

NOTE 3 Device Descriptor Type 0 is designed for use in S-Mode. Existing S-Mode devices may respond to an A_DeviceDescriptor_Read in which the Descriptor Type differs from 0 with an A_DeviceDescriptor_Response-PDU containing the Device Descriptor formatted according DD0 but the field Descriptor Type as in the request, thus different from 0. The conclusion on the real contained Descriptor Type can then only be based on the length of the field device_descriptor. Therefore, no new Descriptor Type should have a Device Descriptor format of 2 octets.

4.1.2 Device Descriptor Type 0 (mask version)

4.1.2.1 Device Descriptor Type 0 Format

The Device Descriptor Type 0 shall have the format as specified in Figure 1.

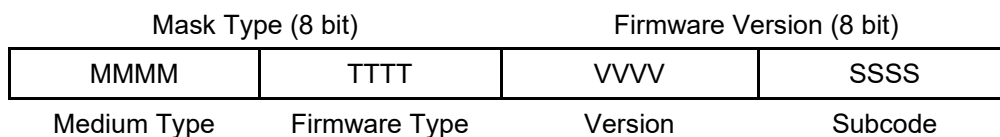


Figure 1 – Device Descriptor Type 0 format

4.1.2.2 Device Descriptor Type 0 assigned values

Table 7 – KNX Association approved mask versions for devices

Mask version	Mask Type Medium Type	Mask Type Firmware Type	Firmware Version	Subcode	Medium	Device Profile
0010h	0h	0h		0	TP1	BCU 1, System 1
0011h	0h	0h		1	TP1	BCU 1, System 1
0012h	0h	0h		2	TP1	BCU 1, System 1
0013h	0h	0h	1h	3	TP1	BCU 1, System 1
0020h	0h	0h		0	TP1	BCU 2, System 2
0021h	0h	0h		1	TP1	BCU 2, System 2
0025h	0h	0h	2h	5h	TP1	BCU 2, System 2
0300h	0h	3h	0h	0	TP1	System 300
0310h	0	3	1	0	TP1	TP1 USB interface v1
0311h	0	3	1	1	TP1	TP1 USB interface v2
0700h	0h	7h		0	TP1	BIM M112
0701h	0h	7h		1	TP1	BIM M112
0705h	0h	7h	0h	5	TP1	BIM M112
07B0h	0h	7h	Bh	0h	TP1	System B
0810h	0h	8h		0	TP1	IR-Decoder
0811h	0h	8h		1	TP1	IR-Decoder
0910h	0h	9h		0	TP1	Coupler 1.0
0911h	0h	9h		1	TP1	Coupler 1.1
0912h	0h	9h		2	TP1	Coupler 1.2
091Ah	0h	9h	1h	tbd	IP (prim) TP1 (sec)	KNXnet/IP Router
0AFDh	0h	Ah	Fh	Dh	TP1	none (see V06 "Profiles")
0AFEh	0h	Ah	Fh	Eh	TP1	none (see V06 "Profiles")
1012h	1h	0h		2	PL110	BCU 1
1013h	1h	0h		3	PL110	BCU 1
1310h	1	3	1	0	PL110	PL110 USB interface v1
1311h	1	3	1	1	PL110	PL110 USB interface v2
17B0h	1h	7h	Bh	0	PL110	System B
1900h	1h	9h		0	TP1-PL110	Media Coupler PL-TP
2010h	2h	0h		0	RF	Bi-directional devices
2110h	2h	1h		0	RF	Uni-directional devices

Mask version	Mask Type Medium Type	Mask Type Firmware Type	Firmware Version	Subcode	Medium	Device Profile
2311h	2	3	1	1	RF	RF USB interface v2
3012h	3h	0h		2	TP0	BCU 1
4012h	4h	0h		2	PL132	BCU 1
5705h	5h	7h	0h	5h	KNX IP	System 7
57B0h	5h	7h	Bh	0h	KNX IP	System B

4.1.3 Device Descriptor Type 2

The Device Descriptor Type 2 shall have the format as specified in Figure 2.

Octet 0 (MSB)	Octet 1	Octet 2	Octet 3	Octet 4	Octet 5	Octet 6	Octet 7	Octet 8	Octet 9	Octet 10	Octet 11	Octet 12	Octet 13 (LSB)
Application Manufacturer		Application Identification		Appli- cation Version	Mgmt Prof. + reserved	Channel Info 1		Channel Info 2		Channel Info 3		Channel Info 4	

Figure 2 – Device Descriptor Type 2 format

All fields shall always be provided, even if unused.

- **Application Manufacturer**
 position: octets 0 (MSB) and 1
 description: This shall be the manufacturer code of the manufacturer of the E-Mode device.
 format, encoding: U_{16} . The value shall be binary encoded.
- **Application Identification ²⁾**
 position: octets 2 and 3
 description: This field shall encode the manufacturer specific Application Identification.
 format, encoding: 16 bit value. The coding is manufacturer specific.
- **Application Version**
 position: octet 4
 description: This field shall contain the version of the manufacturer specific version of the application.
 format, encoding: U_8 . The value shall be binary encoded.
- **Management Profile**
 position: octet 5 bits 7 (msb) to 4
 description: This field shall contain the Management Profile of the E-Mode device.
 format, encoding: N_4

²⁾ This field may be named “Device Type” in other publications.

Table 8 – Management Profile and reserved field - overview

Management Profile								reserved	Link services shall be supported	Flexible E-Mode Channels possible	Master Reset	Requirements
7	6	5	4	3	2	1	0					
0	0	0	0	0	0	0	0	No	No	No		These are the Profiles: – Easy Ctrl fixed DMA, and – Easy Ctrl reloc DMA as specified in [17]. This is not allowed for future use.
0	0	1	1	1	1	1	1	No	No	No		Reserved Management Profile.
0	1	0	0	0	0	0	0	Yes	No	No		These are the Profiles: – Easy Ctrl fixed DMA, and – Easy Ctrl reloc DMA as specified in [17] with the support of Link management Services This is not allowed for future use.
1	0	0	0	0	0	0	0	No	Yes	Yes		This denotes a Ctrl FEC or PB FEC device. For this Management Profile, the fields Channel Info 1 to Channel Info 4 shall be 0000h.

Other values of Management Profile are reserved and shall not be used by the E-Mode device.

The Management Client shall verify that the reserved bits are 0 or 1 as specified in Table 8. If these bits do not have the specified value then the Management Client should not change the device.

NOTE 4 The value 1111b for the reserved bits shall not be used for any future Management Profile.

NOTE 5 A FEC Profile implementation may be combined with a S-Mode Profile to allow for ETS access to the device. As it is not the intention that any such combination be a new Profile, there are no provisions for such combinations in the above encoding of the Management Profile.

- Channel Info 1

position: octet 6 and 7

description: Number and type of channels, first type

format, encoding: U_3U_{13}

bit nr	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
name:	number			Channel Code												

- **number**
 - position: bit 15 to bit 13
 - description: The field *number* shall contain the number of available E-Mode Channels of the following E-Mode Channel Code. The possible number of E-Mode Channels for a given E-Mode Channel Code implemented in a device varies from 1 to 8. Value 0 shall mean 1 such E-Mode Channel.
 - format, encoding: U_3
The real available number of E-Mode Channels of this E-Mode Channel Code is $\text{number} + 1$.
If the field *Management Profile* denotes a FEC device (value 1000b) then this field *Number* shall be 0.
- **Channel Code**
 - position: bit 12 to bit 0
 - description: E-Mode Channel Code
 - format, encoding: See below.

This field Channel Code shall identify the type of the E-Mode Channel. The E-Mode Channels and the corresponding E-Mode Channels are specified for every application in the appropriate Chapters in [19]. The overview of the E-Mode Channels Codes is located in [15].

The E-Mode Channels Codes shall be arranged as follows.

For Management Profiles 0000b and 0100b

If a channel does not exist the Channel Code shall be 0000h. For unknown channels the Channel Code shall be set to 0001h.

Some Channel Code values are reserved as specified in Table 9.

Table 9 – Reserved Channel Code values

Channel Code	Use
0000h	unused (then no more valid data afterwards)
0001h	descriptor not provided through this service This value is reserved by KNX Association for future extensions. It shall not be used.
1FF0h	Localisation Channel for 1 to 8 Channels
1FF1h	Localisation Channel for 1 to 16 Channels
1FF2h	Localisation Channel for 1 to 24 Channels
1FF3h	This Channel Code shall not be used.
1FF4h	This channel code shall be used by LTE-Mode devices
1FF5h to 1FFFh	system reserved

For Management Profile 1000b

If the field *Management Profile* denotes a FEC device (value 1000b) then this field *Channel Code* shall be 0.

-
- Channel Info 2
 - position: octet 8 and 9
 - description: Number and type of channels, second type. See Channel Info 1.
 - format, encoding: U₃U₁₃
 - Channel Info 3
 - position: octet 10 and 11
 - description: Number and type of channels, third type. See Channel Info 1.
 - format, encoding: U₃U₁₃
 - Channel Info 4
 - position: octet 12 and 13 (LSB)
 - description: Number and type of channels, fourth type. See Channel Info 1.
 - format, encoding: U₃U₁₃

4.1.4 Descriptor Type 1 and 3 to 7

These Device Descriptor Types are reserved by KNX Association for future extensions and shall not be used.

4.2 Interface Object Type independent Properties

4.2.1 PID_OBJECT_TYPE (PID = 1)

- Property name: Interface Object Type
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: DPT_PropDataType (DPT_ID = 7.010)

This Property shall have the value of the Object Type of the Interface Object. This Property is always mandatory and shall always be readable.

EXAMPLE In the Device Object, this Property shall have the value 0000h.

4.2.2 PID_OBJECT_NAME (PID = 2)

- Property name: Interface Object Name
- Property Datatype:
 - PDT_UNSIGNED_CHAR[]
 - PDT_REFERENCE
- Datapoint Type: DPT_UTF_8 (DPT_ID: 28.001)
- Access: Data Property: 3/1

This Property shall include the name of the Interface Object.

In case this Property is implemented in the Device Object (Object Type = 0), then this Property shall be interpreted as the name of the device.

This Property is always optional.

This is only a descriptive value. The string shall be encoded in ASCII / ISO 8859-1 as an array of characters (unsigned char[]; the max. number of elements shall be fixed in the device).

The access rights of this Property depend on the implementation.

4.2.5 PID_LOAD_STATE_CONTROL (PID = 5)

- Property name: Load Control
- Property Datatype: PDT_CONTROL
- Datapoint Type: None.

This Property shall control the Load State Machine of the hosting Interface Object. A write-access to this Property shall be equal to an event for the Load State Machine and a read-access to this Property shall provide the current state of the Load State Machine. The response value shall always be one octet and include the current state of the Load State Machine. The write value shall always be 10 octets. Its first octet shall be the event for the Load State Machine. The following octets shall be additional information for the Load State Machine.

The Load State Machine is specified in clause 4.23.

This Property is mandatory if the application is loadable. If this Property does not exist the Application Program is not loadable.

4.2.6 PID_RUN_STATE_CONTROL (PID = 6)

- Property name: Run Control
- Property Datatype: PDT_CONTROL ⁹⁴⁾
- Datapoint Type: None.

This Property shall control the execution of the executable part of the hosting Interface Object. A write access to this Property shall be equal to an external event for the Run State Machine and a read-access to this Property shall provide the current state of the Run State Machine.

The Run State Machine is specified in clause 4.24.

4.2.7 PID_TABLE_REFERENCE (PID = 7)

- Property name: Table Reference
- Property Datatype: PDT_UNSIGNED_LONG
- Datapoint Type: None.

A Management Client shall use the value of this Property as reference for memory mapped access to this Resource. The Management Server (device) shall set this reference when the Management Client allocates the memory space for this Resource. This shall be a read-only Property, which shall be used by the Management Client for memory mapped access (using A_Memory_Write and A_Memory_Read) to the Resource.

The address shall be a 4 octet pointer to an absolute memory space location within the device. Due to the fact that A_UserMemory_Write and A_UserMemory_Read services are limited to a 20 bit address range the reference pointer shall not exceed this range. The most significant 12 bit shall be reserved for future use. A_Memory_Read; A_Memory_Write, A_UserMemory_Read and A_UserMemory_Write shall operate on the same physical address space (except that A_Memory_Read and A_Memory_Write shall be restricted to the lower 64 kb, while A_UserMemory_Read and A_UserMemory_Write shall be able to address the full range of 1 Mb).

If the Load State of the related Resource changes to 'unloaded' then the Management Server shall set the value to zero. When the Management Client has successfully allocated memory then the value shall be set to the absolute memory location from where the Resource will be downloaded. When memory allocation is not successful then the value shall be set to zero.

4.2.8 PID_SERVICE_CONTROL (PID = 8)

- Property name: Service Control
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

The difference between which Profile has to support which bits is made in V06 "Profiles".

PID_SERVICE_CONTROL shall be a permanent control field for the device. It shall be structured as specified in Table 10.

Table 10 – PID_SERVICE_CONTROL

Octet	Bit		Encoding (d = default value)
Low Octet	0	Reserved. Shall be 0 ³⁾ .	0
	1	Reserved. Shall be 0 ³⁾ .	0
	2	Individual Address Write Enable	0 = disable (d) 1 = enable
	3	Reserved	-
	4	Reserved	-
	5	Reserved	-
	6	Reserved	-
	7	Reserved	-
High octet	8	Application Interface Layer Services on EMI Disable	0 = enable 1 = disable
	9	Link Layer Services on EMI Disable	0 = enable 1 = disable
	10	Network Layer Services on EMI Disable	0 = enable 1 = disable
	11	Transport Layer Group Services on EMI Disable	0 = enable 1 = disable
	12	Switch Service-Services on EMI Disable	0 = enable 1 = disable
	13	Transport Layer Connection Oriented Services on EMI Disable	0 = enable 1 = disable
	14	Application Layer Services on EMI Disable	0 = enable 1 = disable
	15	Management Services on EMI Disable	0 = enable 1 = disable

Octet 1 of the Property Service Control shall only be valid for devices with an EMI. For devices without EMI all services shall always be disabled.

- Bit 2 IndividualAddress_Write:

This bit shall control the possibility to set the Individual Address via programming mode or KNX Serial Number services. If this bit is cleared, it shall not be possible to change the Individual Address of the device.

Mask 0021h implementations shall have inverse coding for this bit:

2	Individual Address Write Enable	0 = enable 1 = disable
---	---------------------------------	---------------------------

NOTE 6 The specification of which KNX Profile shall support which fields in PID_SERVICE_CONTROL is given in Volume 6 "Profiles".

³⁾ In PID_SERVICE_CONTROL, bit 0 and bit 1 shall be cleared. They are used in legacy implementations for the control of the abandoned service A ServiceInformation Indication-Write, as follows:

Bit		Encoding
0	User Stopped_ServiceInfo Enable Enables the generation of an A_ServiceInformation_Indication_Write report of the device in case of unexpected stop of the user program.	0 = disable 1 = enable
1	Received own Individual Address Service Info enable Enables the generation of a ServiceInformation Report of the device in case of reception of a frame with the own Individual Address as source address.	0 = disable 1 = enable

4.2.9 PID_FIRMWARE_REVISION (PID = 9)

- Property name: Firmware Revision
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

This Property shall contain the revision number of the firmware of the object in which it is contained. Values shall start with 00h.

This Property is encoded as a single octet unsigned character.

This information can also be stored in PID_VERSION (PID = 25), which is encoded as DPT_Version. Volume 6 “Profiles” specifies which of the given Profile(s) shall be implemented.

4.2.10 PID_SERVICES_SUPPORTED (PID = 10)

- Property name: Services Supported
- Property Datatype: None.
- Datapoint Type: None.

This Property shall contain information about the services that are supported by this device.

4.2.11 PID_SERIAL_NUMBER (PID = 11)

- Property name: Serial Number
- Property Datatype: PDT_GENERIC_06
- Datapoint Type: DPT_SerNum (DPT_ID = 221.001)

This Property shall contain the KNX Serial Number of the device in which it is contained. PID_SERIAL_NUMBER is the Realisation Type 2 of the Resource KNX Serial Number as specified in 4.22.3.

4.2.12 PID_MANUFACTURER_ID (PID = 12)

- Property name: Manufacturer Identifier
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

Code assigned by the KNX Association to identify the manufacturer of the device.

4.2.13 PID_PROGRAM_VERSION (PID = 13)

- Property name: Program Version
- Property Datatype: PDT_GENERIC_05
- Datapoint Type: None.

4.2.13.1.1 Abstract Resource definition

This Property shall contain the application version of the object in which it is contained.

NOTE 7 The Property Identifier of this Property was formerly named PID_APPLICATION_VERSION.

4.2.13.1.2 Format

Program Version shall be a 5 octet identifier that shall be formatted as follows.

Octet	1 (MSB)	2	3	4	5 (LSB)
Meaning	Manufacturer ID		Device Type		Application Program Version
Size	2 octets		2 octets		1 octet

Figure 3 – Format of PID_PROGRAM_VERSION

4.2.13.1.2.1 Manufacturer ID

This field shall contain the manufacturer code of the manufacturer that produces the device. This manufacturer code is assigned by KNX Association. In case of OEM, this shall be the manufacturer code of the OEM customer, not the one of the OEM provider.

4.2.13.1.2.2 Device Type

This field shall contain a manufacturer specific Device Type.

4.2.13.1.2.3 Application Program Version

This field shall contain the manufacturer specific and Device Type dependent version of the application program.

4.2.13.1.3 Usage by the MaS

The use of *Program Version* is Profile dependent: please refer to the Configuration Procedures in [10] for the use of Program Version for each Profile.

The mandatory - or optional access rights to Program Version are as well Profile dependent and are specified in [17].

4.2.13.1.4 Use by the MaC

The MaC may either only read-out Program Version or also modify it during configuration. This is Profile dependent and is specified in the Configuration Procedures in [10].

4.2.14 PID_DEVICE_CONTROL (PID = 14)

- Property name: Device Control
- Property Datatype: PDT_BITSET8 (alt.: PDT_GENERIC_01)
- Datapoint Type: DPT_Device_Control (DPT_ID = 21.002)

4.2.14.1 Abstract Resource definition

Property Device Control shall be a temporary control field for the device.

4.2.14.2 Format

Table 11 – Device Control

Bit	Usage
0	User stopped
1	Individual Address duplication
2	Verify Mode On
3	Safe State On
4 ... 7	Reserved

4.2.14.3 Usage by the Management Client

A Management Client can reset the value by writing 0 to this Property.

4.2.14.4 Usage by the Management Server (device)

The device shall set and clear the fields as specified in the clauses below.

The device shall set the value of this Property to 00h on start-up.

4.2.14.5 User stopped

4.2.14.5.1 Encoding

0	The user program is running.
1	The user program is stopped.

4.2.14.5.2 Usage by the Management Server (device)

This bit shall be set by the device if the user program stopped for internal reason (e.g. wrong PEI-Type).

4.2.14.6 Individual Address Duplication

4.2.14.6.1 Encoding

0	No frame has been received with the own Individual Address as Source Address
1	At least one frame has been received with the own Individual Address as Source Address

4.2.14.6.2 Usage by the Management Server (device)

This bit shall be set by the device if a frame has been received with the own Individual Address as Source Address of the received frame.

4.2.14.7 Verify Mode Control

4.2.14.7.1 Abstract Resource Definition

Verify Mode controls the reaction of the Management Server for handling of the following Application Layer services primitives as specified in [07]:

- an A_Memory_Write.ind primitive, as specified for the A_Memory_Write-service, or
- an A_MemoryBit_Write.ind primitive, as specified for the A_MemoryBit_Write-service, or
- an A_Memory_Write.ind primitive, as specified for the A_Memory_Write-service, or
- an A_UserMemoryBit_Write.ind primitive, as specified for the A_UserMemoryBit_Write-service, or
- an A_FilterTable_Write.ind primitive, as specified for the A_FilterTable_Write-service, or
- an A_RouterMemory_Write.ind primitive, as specified for the A_RouterMemory_Write-service.

Verify Mode shall per default be disabled in the Management Server. A Management Client can enable Verify Mode via the Verify Mode Control. Verify Mode can only be enabled by the Management Client if there is a Transport Layer connection to the Management Server. Verify Mode shall automatically be disabled by the Management Client when the Transport Layer connection is closed.

The Verify Mode Control is a device resource to control the Verify Mode of the Management Client functionality in the device.

4.2.14.7.2 Encoding

Used by: mask 0020h, 0021h, 0701h, 0705h
 System B, System 300
 mask 091Ah

1 bit

0	Verify mode is disabled
1	Verify mode is enabled

4.2.14.7.3 Usage by the Management Server (device)

The value of Verify Mode Control shall per default be 0 ("disabled"). When a Transport Layer connection to the device is closed, the Management Server shall immediately and automatically disable Verify Mode and clear the Verify Mode Control bit.

4.2.14.7.4 Usage by the Management Client

When opening a Transport Layer connection to the Management Server, the Management Client shall assume the Verify Mode Control to be cleared; it shall actively set the Verify Mode Control.

When closing a Transport Layer connection, the Management Client may assume the Verify Mode Control to be automatically cleared and thus not have to clear it explicitly.

4.2.14.8 Safe State Control

4.2.14.8.1 Abstract Resource definition

Safe State denotes a state of the device in which no user-supplied part shall be executed.

The implementation of the Safe State is optional.

4.2.14.8.2 Encoding

1 bit

0	The device is not in Safe State.
1	The device is in Safe State.

4.2.14.8.3 Usage by the Management Server (device)

The device may enter in Safe State through manufacturer specific procedures.

The change back to normal mode shall only be possible through reset.

4.2.14.8.4 Usage by the Management Client

The Safe State Control shall be used to switch to Safe State via an A_PropertyValue_Write. If the Safe State is not implemented bit 3 shall be reserved and set to 0.

4.2.15 PID_ORDER_INFO (PID = 15)

- Property name: Order Info
- Property Datatype: PDT_GENERIC_10
- Datapoint Type: None.

This Property shall contain manufacturer specific Order Information (PDT_GENERIC_10).

4.2.16 PID_PEI_TYPE (PID = 16)

- Property name: PEI Type
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

This Property shall contain the currently connected PEI-Type (“Hardware Type”). This value shall regularly be updated, e.g. on every cycle through the system stack. It shall be evaluated as specified in [11]. In devices without PEI the value shall be zero.

4.2.17 PID_PORT_CONFIGURATION (PID = 17)

- Property name: PortADDR
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

This Property shall include the settings of the hardware configuration of a device (e.g. on BCU PortA direction control register 0 = input; 1 = output). The value of this Property is manufacturer specific.

A Management Client shall read this value and compare it to a value stored for this device in its repository (database). Only if both values equal, the Management Client may continue with the configuration of the device’s application, such as download of the user application code or its parameters. The manufacturer shall take care setting the value of this Property to the value required by the hardware.

This Property is not usable for LTE-Mode devices.

4.2.18 PID_POLL_GROUP_SETTINGS (PID = 18)

- Property name: Pollgroup Settings
- Property Datatype: PDT_POLL_GROUP_SETTINGS
- Datapoint Type: None.

This Property value shall be composed of:

- the Polling Group Address: 2 octets
- the Polling Disable bit: 1 bit

0	Polling shall be enabled.
1	Poling shall be disabled.

- the Polling Slot Number: 4 bit
- This field shall contain the polling slot number in the range 0 to 15.

Polling Group Address																Disable	Reserved				Polling Slot Nr.			
A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	D	0	0	0	0	S	S	S	S

4.2.19 PID_MANUFACTURER_DATA (PID = 19)

- **Property name:** Manufacturer Data
- **Property Datatype:** Variable
- **Datapoint Type:** not applicable: implementation specific encoding

This shall be a Property with manufacturer specific device information (e.g., manufacture date ...). The Property Datatype of this Property is not defined. It depends on the data a manufacturer wants to be handled there. Therefore, the Property Datatype of the Property must be read out via A_Property-Description_Read before data can be read. This requires the implementation of full Interface Objects.

4.2.21 PID_DESCRIPTION (PID = 21)

4.2.21.1 Interface Object Type independent specification

- **Property name:** Description
- **Property Datatype:**
 - PDT_UNSIGNED_CHAR[]
 - PDT_REFERENCE
- **Datapoint Type:** Every character: DPT_UTF_8 (DPT_ID: 28.001)
- **Access:** Data Property: 3/1

This Property shall include additional descriptive information to the functionality represented by the Interface Object in which this Property is located.

EXAMPLE 1 In the Device Object, this Property shall be interpreted as additional descriptive information to the device.

The descriptive information shall be encoded in ASCII / ISO 8859-1 as an array of characters (unsigned char[]; the maximum number of elements shall be fixed in the device). The access rights of this Property depend on the implementation.

4.2.21.2 Resource definition

For the Interface Object Type independent specification of PID_DESCRIPTION, see above.

In a FEC device, this Property

- is mandatory in the Device Object, and
- shall have a minimal size of 10 characters; no maximal size is specified.

4.2.21.3 Usage by the Management Client

The Management Client may use PID_DESCRIPTION to store human readable textual information, entered by the installer, to help localising, identifying and describing the device.

This shall help for reverse engineering E-Mode installations.

4.2.23 PID_TABLE (PID = 23)

- **Property name:** Table
- **Property Datatype:** None.
- **Datapoint Type:** None.

The Property Table is used in various Interface Objects for the storage of data in table format. The format (2 octets, 4 octets...) and contents (Group Addresses, Associations...) depend on the host object. Please refer therefore to the use of PID_TABLE in the Interface Object specifications further in this document.

4.2.25 PID_VERSION (PID = 25)

- Property name: Version
- Property Datatype: PDT_VERSION (alt.: PDT_GENERIC_02)
- Datapoint Type: DPT_Version (DPT_ID = 217.001)

This Property shall specify version information related to the Interface Object in which it resides. Version information is encoded using the new standard DPT_Version.

In case of the Device Object PID_VERSION shall be the version information for the device itself.

EXAMPLE Revision number of the firmware.

The interpretation of the device version information is product specific. The entry point for interpretation of any data with versioning is the Device Object Property PID_PRODUCT_ID (PID = 55).

4.2.27 PID_MCB_TABLE (PID = 27)

- Property name: Memory Control Table
- Property Datatype: PDT_GENERIC_08[]
- Datapoint Type: None

This Property shall define and control the subsegmentation of a relating memory mapped Resource and its checksum.

Writing the Memory Control Block Table shall only have effect in the Load State 'Loading'.

Table 12 - Memory Control Block (MCB) Table

Nr. of elements				
Segment Size 1 [4 octets]	CRC Control Byte [1 octet]	Read Access 1 [4 bit]	Write Access 1 [4 bit]	CRC [2 octets]

4.2.27.1.1 CRC Control Byte

Table 13 – CRC Control Byte

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
reserved	reserved	reserved	reserved	reserved	reserved	reserved	0 : CRC is always valid 1 : Contents of protected memory area may change

If the CRC Control Byte, Bit 0 is set to zero this shall indicate that the protected memory area will never be changed after download. So the CRC shall always be the calculated checksum of the protected memory area.

If the CRC Control Byte, Bit 0 is set to 1 this shall indicate that the protected memory area can be changed after the load process by an application.

4.2.27.1.2 CRC

The CRC checksum calculation shall provide a higher consistency while using differential download.

The CRC shall be calculated properly by the Management Server (device) on the transition from the load state 'Loading' to the load state 'Loaded' according to the segment's memory content right from the beginning. Its value shall be valid in the load state 'Loaded' only.

The 16 bit CRC shall be calculated according to the CRC16-CCITT specification.

The CRC of the Memory Control Block Table Property is a CRC16-CCITT with the following parameters:

Width = 16 bit

Truncated polynomial = 1021h

Initial value = FFFFh

Input data is NOT reflected.

Output CRC is NOT reflected.

No XOR is performed on the output CRC.

EXAMPLE The correct CRC16-CCITT of the string '123456789' is E5CCh.

4.2.28 PID_ERROR_CODE (PID = 28)

- Property name: Error code
- Property Datatype: PDT_ENUM8 (alt. PDT_UNSIGNED_CHAR)
- Datapoint Type: DPT_ErrorClass_System (20.011)

This Property shall indicate the last error that occurred before the load state 'Error' is set. When the load state changes from 'Error' to a different state then the Error Code shall be set to '0' (no error).

For the error code description see DPT_ErrorClass_System in [14]. This list defines all specified error codes. A manufacturer may implement a subset of these. The definition of new error codes is an ongoing process.

4.2.29 PID_OBJECT_INDEX (PID = 29)

- Property name: Object Index
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: DPT_Value_1_Ucount

This Property shall contain the local Object Index of the Interface Object in which it is located.

The Property PID_OBJECT_INDEX is optional in any Profile.

The Property PID_OBJECT_INDEX is optional in any Interface Object.

The Property PID_OBJECT_INDEX shall always be read-only.

Usage by the Management Server (device)

The Property PID_OBJECT_INDEX shall be readable via LTE Read/Response Services (A_GroupPropValue_Read-PDU and A_GroupPropValue_Response-PDU). If the Property is implemented in the Interface Object, then the Property shall be readable through all LTE zones used by the Interface Object.

The other LTE-services (A_GroupPropValue_InfoReport and A_GroupPropValue_Write) are not supported for this Property.

The Property PID_OBJECT_INDEX shall be readable, as every Property, also through the point-to-point communication mode Property Services (A_PropertyValue_Read), even if this usage makes no sense.

4.2.30 PID_DOWNLOAD_COUNTER (PID = 30)

- Property name: Download Counter
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: DPT_Value_2_Ucount (DPT_ID = 7.010)

4.2.30.1 Abstract Resource definition

The Download Counter shall be a global, Interface Object Type independent read-only Property. A device that has a Download Counter shall at least have PID_DOWNLOAD_COUNTER in the Device Object and may have additional further Interface Objects with PID_DOWNLOAD_COUNTER.

PID_DOWNLOAD_COUNTER in other Interface Objects than the Device Object

Each Download Counter shall be uniquely related to one downloadable part of the device. There may be downloadable parts in the device that are not related to a Download Counter.

PID_DOWNLOAD_COUNTER in the Device Object

If the device has any Download Counter, then it shall at least have PID_DOWNLOAD_COUNTER in the Device Object.

This instance of PID_DOWNLOAD_COUNTER shall signal:

- a change of any further PID_DOWNLOAD_COUNTER in another Interface Object
If any other PID_DOWNLOAD_COUNTER changes, then also the PID_DOWNLOAD_COUNTER in the Device Object shall change.
- a manipulation of the memory not protected by another PID_DOWNLOAD_COUNTER.

4.2.30.2 Usage by the Management Server (device)

The Management Server (device) shall autonomously conclude on incrementing the value of the Download Counter when it is unambiguously registered (see below) one or more modifications of one or more bits or octets of the related downloadable part. It shall respond with an incremented value of the Download Counter if the contents of the related downloadable part have changed compared to the preceding read out of the Download Counter.

NOTE 1 Please note that this counter is thus incremented each time, regardless of whether the modification has been done by an authorised person (electrical installer, planner, ETS user, etc.) or not.

The default value of the download counter, this is, the value ex-factory, should be 0000h.

NOTE 2 This is only a recommendation. It is very well possible that ex-factory, the Download Counter differs from 0000h due to

- loading a default application in the device, or
- due to loading a calibration program in the device, or
- due to loading a test application in the device, etc.

If the Download Counter reaches its maximal value (FFFFh) then it shall not be reset to 0000h!

Unambiguous registration of manipulations (informative)

1. In downloadable parts controlled through a Load State Machine, the transition of the Load State from *Loading* (during which the memory is altered) to *Loaded* unambiguously marks an altered memory contents.
2. In downloadable parts without Load State machine, a simple memory modification through a write-access could be sufficient.

When is the Download Counter incremented and by which value? (informative)

It is not required that the Download Counter increment exactly equals the number of modified bits or octets, or the number of write operations (e.g. A_Memory_Write-PDUs, etc.)

EXAMPLE 2 The Download Counter has an initial value C_1 at the end of a Configuration Procedure and is read out by the Management Client. Afterwards, a first octet is changed in the related downloadable part. This increments the Download Counter to the value C_2 ($C_2 > C_1$). Further modified bits or octets will no longer increment the Download Counter. Now, the Management Client again reads the Download Counter; the device replies with C_2 . If now again the related downloadable part is modified, the Download Counter increments to C_3 ($C_3 > C_2$) and will again remain unchanged by further modifications until it is read, by the Management Client.

EXAMPLE 3 One solution for the above can be to work with an odd and an even counter value: the counter is incremented to an even value internally on a change and then does not change anymore with further write accesses. When the counter is read, this even counter will be incremented to an odd counter, sent back, and can from then again be incremented with the first next change.

It is not required that the Download Counter increments immediately if the downloadable part changes. It is only required that a different; incremented value is reported to the Management Client on its next read out of the Download Counter.

EXAMPLE 4 The Download Counter has an initial value C_1 at the end of a Configuration Procedure and is read out by the Management Client. Afterwards one or more bits or octets are changed in the related downloadable part. The Management Server internally notes this situation, but does not yet increment the Download Counter. Only when later the Download Counter is read out, by the `A_PropertyValue_Read.ind`, the Management Server considers this situation and increments the Download Counter firstly to the value C_2 ($C_2 > C_1$) and then only responds with the `A_PropertyValue_Read.res` with the value C_2 .

What Downloadable Parts shall be governed by the Download Counter?

It is implementation dependent what data is governed by a Download Counter. The application developer concludes and controls what data modification will or will not modify the Download Counter.

A device may have both data that is supervised by a Download Counter as well as data that is not.

Master Reset

For the definition of Master Reset, please refer to [09] clause 3.7.1.2 “Master Reset”.

In case of a “Reset to default”, the MaS shall increment the Download Counter of those parts that change; at least the Download Counter in the Device Object shall increment.

EXAMPLE 5 If only the Network Configuration of the secure device is completed, but no Group Address Table or Application Program has been loaded and these Interface Objects feature a Download Counter, which is still at 0, then these Download Counters may but do not have to increment. If these Download Counters however differ from 0, then they shall increment. In any case, at least the Download Counter in the Device Object shall increment.

This Download Counter in the Device Object shall be incremented for a Master Reset, regardless of how this is initiated (over the bus via an `A_Restart`, via a local interface or via the HMI on the device as required in the Configuration Procedures for KNX Secure devices in [10]).

Erase Code	Effect on PID_DOWNLOAD_COUNTER
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	recalculate: increment.
05h ResetParam	recalculate: increment.
06h ResetLinks	recalculate: increment.
07h Reset to default without IA	recalculate: increment.
n.a. Local Reset to default state	recalculate: increment.

4.2.30.3 Usage by the Management Client

The Management Client should firstly read `PID_DOWNLOAD_COUNTER` in the Device Object. If this `PID_DOWNLOAD_COUNTER` has not changed, then it may conclude that no further instance of `PID_DOWNLOAD_COUNTER` in the Management Server has changed value. If `PID_DOWNLOAD_COUNTER` in the Device Object has changed, it should discover and read possible further instances of `PID_DOWNLOAD_COUNTER` in the other Interface Objects to possibly discover the changed memory contents.

The Management Client may read any Download Counter at any time t_1 (DC1) and compare it with any other value of the Download Counter t_2 (DC2) (t_2 can be before or after) and conclude on the possible difference between the read values ($DC2 < DC1$ or $DC2 > DC1$) on whether the downloadable part has been changed or not.

4.3 Device Object (Object Type 0)

4.3.1 General requirements

NOTE 8 - In the clauses below, indications are given for each Resource in which existing mask version(s) it is used. This is only to help the reader locating and recognizing the Resource. In future versions, these references will be moved to Volume 9 of the KNX Specifications, where the various BAUs will be specified.

The Device Object shall include information about the device. Properties in the Device Object above 201 shall be manufacturer specific. Except for Easy Parameter blocks and manufacturer specific private parts in E-Mode, application parameters shall be in other Interface Objects than the device Interface Object.

Interface Object: Device Object (object index 0)				
	Property	PDT (DPT)	Input	Output
	PID_OBJECT_VALUE	PDT_Function	Write: Group Object Handle (2 octets) Value (n octets, $n \leq 10$)	ReturnCode (1 octet)
			Read: Group Object Handle (2 octets)	ReturnCode (1 octet) Value (n octets, $n \leq 10$)
	PID_OBJECTLINK	PDT_Function	Write: Set/Delete Extended GA (8 octets) Group Object Handle (2 octets)	ReturnCode (1 octet)
			Read: Iterator (1 octet)	ReturnCode (1 octet) Iterator (1 octet) Extended GA (8 octets) Group Object Handle (2 octets)
	PID_APPLICATION	PDT_Function	channel number (1 octet) channel code (2 octets)	ErrorCode (1 octet) Or Channel number (1 octet) channel code (2 octets)
	PID_PARAMETER	PDT_Function	Write: channel number (1 octet) parameter number (1 octet) value (n octets, $n \leq 10$)	Return Code (1 octet)
			Read: channel number (1 octet) parameter number (1 octet)	Return Code (1 octet) Value (n octets, $n \leq 10$)
	PID_OBJECTADDRESS	PDT_Function	Write: Iterator (1 octet)	Return Code (1 octet): always FFh
			Read: Iterator (1 octet)	Return Code (1 octet) Iterator (1 octet) Ch number (1 octet) Ch. Code (2 octets) Grp. Obj. Number (1 octet) Grp. Obj. Handle (2 octets)

4.3.2 PID_ROUTING_COUNT (PID = 51)

- Property name: Routing Count
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

This Property shall include the default value for the hop count. (This is a parameter for the Network Layer of the device.) With Property based management it shall be used to adjust the default value of the hop count parameter of the Network Layer (BCU normally presents this parameter memory mapped at 10Eh in bit 4-6). Values from 0 to 7 are possible. For details, please refer to the Network Layer parameter specification in [05].

Table 14 - Routing Count

Bit	7	6	5	4	3	2	1	0
Meaning	reserved	Routing Counter (0...7)			reserved	reserved	reserved	reserved

In Couplers, PID_ROUTING_COUNT shall only be the standard value of the hop count in telegrams of which the Coupler is the initiating transmitter, not the routed telegrams.

4.3.3 PID_MAX_RETRY_COUNT (PID = 52)

- Property name: MaxRetryCount
- Property Datatype: PDT_GENERIC_01
- Datapoint Type: None.

This Property shall include the information of the Data Link Layer parameters nak_retry and busy_retry. This parameter is not adjustable on every device. It depends on the used medium and the implementation. It shall be specified in the Profiles ([17]) whether this parameter shall be adjustable or which fixed value shall be implemented. This Property shall be used in Property based Management to adjust the default value of these parameters of the Data Link Layer (BCU normally presents this parameter memory mapped at 10Fh in bit 2-0 and 7-5).

The default value shall be 3 busy repetitions and 3 Nack repetitions (33h).

Format

Bit	7	6	5	4	3	2	1	0
Meaning	0	busy_retry			0	nak_retry		

4.3.4 PID_ERROR_FLAGS (PID = 53)

- Property name: Error Flags
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

This Property shall contain device specific error flags. Error flag value 0 is set.

Bit	Name	Description	Encoding (d = default value)
0	System 1	Internal System Error EXAMPLE Message system corrupted The message shall be deleted. The system shall be reset. The application shall continue or shall be restarted.	0 = error 1 = ok (d)
1	App	Illegal System State EXAMPLE Invalid value of the Special Function register (SFR)	0 = error

Bit	Name	Description	Encoding (d = default value)
		The system shall be reset. The application shall be restarted.	1 = ok (d)
2	MEMORY_ERROR	Checksum / CRC-Error in internal non volatile memory EXAMPLE Verify error after flash write operation. The state machine (if present in the device) shall go to the state <i>Error</i> . The application shall be stopped.	0 = error 1 = ok (d)
3	Stack	Stack Overflow error The system shall reset. The application shall be restarted.	0 = error 1 = ok (d)
4	Table Error	Inconsistence in System Tables The application shall be stopped.	0 = error 1 = ok (d)
5	Trans	Physical Transceiver error (stuck / overheat detection) The transmission of frames shall be stopped until reset. The application shall continue without the transmission of frames.	0 = error 1 = ok (d)
6	System 2	Internal System error. EXAMPLE Corrupt Memory Table This is only a warning. The application shall continue.	0 = error 1 = ok (d)
7	System 3	Internal System error. This is only a warning. The application shall continue.	0 = error 1 = ok (d)

Please refer to [17] for the specification of which fields (bits) shall be supported by which Profile. Fields that are not supported shall have the default value.

This Property shall be used in Property based Management. This Property corresponds to the error flags of a BCU (located at 10Dh).

4.3.5 PID_PROGMODE (PID = 54)

- Property name: Programming Mode
- Property Datatype: PDT_BITSET8
- Datapoint Type: None.

This Property shall include a binary value that shall be the Programming Mode of the device.

Bit	Name	Description	Encoding (d = default value)
0	PROGMODE	The value of this bit shall reflect the state of the Programming Mode. The value shall be set to 0 on reset. The Management Client can set the device to Programming Mode by setting this bit to 1. This Property shall be used in Property based management and is equal to bit 0 of address 60h of a BCU.	0 = device is not in Programming Mode (d) 1 = device is in Programming Mode
1 to 7	reserved	These bits are reserved and shall always be 0.	not applicable

4.3.6 PID_PRODUCT_ID (PID = 55)

- Property name: Product Identification
- Property Datatype: PDT_GENERIC_10
- Datapoint Type: None.

This Property shall be the manufacturer specific device type. This Property shall enable in LTE-Mode an exact identification of the device (e.g. used by a service tool or for remote operation by a management station).

Name	Offset (Octets)	Size	
Application Manufacturer	0	2 octets	
Device Type	2	8 octets	manufacturer specific device Type

4.3.7 PID_MAX_APDU_LENGTH (PID = 56)

- Property name: MAX. APDU-Length
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

4.3.7.1 Usage by the Management Server

The Management Server shall hold in PID_MAX_APDU_LENGTH the maximal supported APDU-length for Management of the device.

The value of PID_MAX_APDU_LENGTH may be in the range between 15 and 254 ⁴⁾.

4.3.7.2 Usage by the Management Client

4.3.7.2.1 End devices

This Property shall allow a Property based Management Client to use the maximal APDU-Length to manage the device. A Management Client supporting the L_Data_Extended-frame has to check this value before starting download.

If the PID_MAX_APDU_LENGTH is not present in the Device Object, then the Management Client shall manage the device with L_Data_Standard-frames with an APDU-length of maximal 15 octets.

4.3.7.2.2 Couplers (Routers)

This shall specify the maximal APDU-length that shall be supported for Management of the Coupler.

If this Property is not present in the Device Object, then the device shall be managed with L_Data_Standard frames.

If PID_MAX_APDU_LENGTH is solely in the Router Object (see 4.5.9), then the device shall only support L_Data_Extended-frames for Routing and only L_Data_Standard-frames for Management.

4.3.7.2.3 cEMI Server

PID_MAX_APDU_LENGTH in the Device Object of a cEMI server

- shall hold the maximum length (of APDU within the frames) that the device shall be able to route from bus to the cEMI client and vice versa, and
- shall be the maximum length (of APDU within the frames) that the device itself shall be able to be managed with (management of the local device).

⁴⁾ 255 as value for PID_MAX_APDU_LENGTH is an ESCape Code (see [02] clause “Length” of the L_Data_Extended-frame).

4.3.8 PID_SUBNET_ADDR (PID = 57)

- Property name: Subnetwork Address
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

This Property shall be the Subnetwork Address part (high octet) of the Individual Address.

4.3.8.1 Usage by the Management Server

In a Router, this Property can be used to provide the Subnetwork Address from the Subnetwork attached to its secondary side as specified in “SNA Read” in [10].

4.3.8.2 Usage by the Management Client

This Property can be used by a Management Client (tool or device) for acquiring the Subnetwork Address from the Subnetwork to which it is connected from the Router to that Subnetwork, as specified in “SNA Read” in [10].

4.3.9 PID_DEVICE_ADDR (PID = 58)

- Property name: Device Address
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

This Property shall be the Device Address part (low octet) of the Individual Address.

4.3.10 PID_PB_CONFIG (PID = 59)

- Property name: PID_CONFIG_LINK
- Property Datatype: PDT_GENERIC_04
- Datapoint Type: Not applicable.

4.3.10.1 Encoding

The encoding shall always as follows: 4 octets: MSB: Command/Flags; MSB + 1...MSB + 3: data.

Table 15 specifies the coding of the actions that can be performed with PID_CONFIG_LINK.

Table 15 – Overview of actions encoded on PID_CONFIG_LINK

Pos.	Action	Value																																					
		MSB																								LSB													
		Command				Flags				Data								Data								Data													
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0						
1	Enter_Config_Mode	0	0	0	1	0	0	0	0	00h								00h								00h													
2	Start_Link	0	0	1	0	Flags		Sub function	Manufacturer Code																Number of Group Objects to link														
3	Channel_Function_Actuator	0	0	1	1				0	0	0	0	0	0	0	Channel Code ¹								00h															
4	Channel_Function_Sensor	0	1	0	0				0	0	0	0	0	0	0	Channel Code ¹								00h															
5	Set_Channel_Param	0	1	0	1				Flags				Parameter Index = 1								Value												Value						
6	Channel_Param_Response	0	1	1	0	Flags				Parameter Index								Value								Value													
7	Begin_Connection	0	1	1	1	0	0	0	0	00h								00 h								00h													
8	Set_Delete_Link	1	0	0	0	Sub-function			- Connection Code ² or - scene number value, or - AckSlotNb ³								Group Address																						
9	Link_Response	1	0	0	1																												Flags				- Connection Code ² or - AckSlotNb		
10	Stop_Link	1	0	1	0	Flags				00h								00h								00h													
11	Quit_Config_Mode	1	0	1	1	0	0	0	0	00h								00h								00h													
12	Reset_Installation	1	1	0	0	0	0	0	0	00h								00h								00h													
13	Features	1	1	0	1	Sub-function			Physical requirements								reserved								reserved														

¹ E-Mode Channel Code = 13 bit

The E-Mode Channel Code shall be part of the E-Mode Channel information as part of the Device Descriptor Type 2, as specified in clause 4.1.3.

² Connection Code = 8 bit

The values Connection Codes are for each E-Mode Channel Specification given in [19]. An overview is given in [15].

³ AckSlotNb = 6 bit (lsb)

Reserved values shall be set to 0 and shall be checked on reception; in case of discrepancy the message shall be discarded.

Unknown command values (14 and 15) are reserved values. Messages received with these command values shall be discarded and shall not abort the current link-procedure.

4.3.10.2 Action specifications

4.3.10.2.1 Overview

This clause 4.3.10.2 specifies in detail the different actions. Table 16 summarizes all the specified actions.

Table 16 – Actions encoded on PID_CONFIG_LINK - overview

Pos.	Action	Message direction	Service
1	Enter_Config_Mode	Actuator to all	A_NetworkParameter_Write
2	Start_Link	Sensor to all	A_NetworkParameter_Write
3	Channel_Function_Actuator	Actuator to sensor	A_NetworkParameter_Write
4	Channel_Function_Sensor	Sensor to actuator	A_NetworkParameter_Write
5	Set_Channel_Param	Actuator to sensor	A_NetworkParameter_Write
6	Channel_Param_Response	Sensor to actuator	A_NetworkParameter_Write
7	Begin_Connection	Actuator to sensor	A_NetworkParameter_Write
8	Set_Delete_Link	Sensor to actuator	A_NetworkParameter_Write
9	Link_Response	Actuator to sensor	A_NetworkParameter_Write
10	Stop_Link	Sensor to all	A_NetworkParameter_Write
11	Quit_Config_Mode	Actuator to all	A_NetworkParameter_Write
12	Reset_Installation	To all	A_NetworkParameter_Write
13	Features	Sensor to actuator or actuator to sensor	A_NetworkParameter_Write

After the transmission of each message, the expected response shall be received before the 3 s timeout. If the timeout elapses then the procedure shall be aborted (except if the actuator waits for the Features message from the sensor). See Features message for details.

4.3.10.2.2 PID_CONFIG_LINK (Enter_Config_Mode)

Pos.	Action	Value															
		Octet 11								Octet 12							
		Command				Flags				Data				Data			
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
1	Enter_Config_Mode	0	0	0	1	0	0	0	0	00h				00h			

If the electrical installer wants to link two or more devices, then he at first has to activate Config Mode in a first device. The way how Config Mode has to be activated in any device is implementation specific.

This command shall be sent by a device to be linked when Config Mode becomes active in the device. (The way how Config Mode is activated in a device is implementation specific.)

This action shall activate Config Mode in all sensors and optionally locks all actuators not concerned in this link step.

Open media

This action is optional on open media.

KNX RF Multi

The devices should implement a timeout (same as in KNX RF 1.1) that shall be started when activating the Config Mode. If the Config Mode is not deactivated before the timeout, the devices shall deactivate the Config Mode autonomously. A short timeout is especially useful for the devices that are not in permanent reception mode in runtime. The timeout shall be retriggered each time a configuration message is received in order to enable a long Configuration Procedure.

After the devices have activated the Config Mode, they shall remain in permanent reception mode and the configuration process shall be performed entirely in the F1r RF channels.

4.3.10.2.3 PID_CONFIG_LINK (Start_Link, Flags, Subfunction, Manufacturer Code, Number of Group Objects to link)

Pos.	Action	Value																															
		Octet 11								Octet 12								Octet 13								Octet 14							
		Command				Flags				Data								Data								Data							
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
2	Start_Link	0	0	1	0	d	p	Sub function	Manufacturer Code (16 bit)																Number of Group Objects to link								
						Flags																											

Flags: d = bit 3: shall indicate to actuators how the link procedure shall be handled.

0 = bidirectional device

1 = unidirectional device

This shall allow systems mixing uni- and bidirectional sensors.

p = bit 2: “parameter indicator” shall indicate whether parameters will be sent in additional messages or not

0 = no additional messages.

1 = messages concerning parameters will be sent after DD2.

Subfunction: 00b: basic mechanism.

Sensors shall send this action after activation (e.g. press push button). It shall indicate to the selected actuator that the link process between this sensor and the actuator that initiated the configuration mode has started. All other sensors are optionally locked after receiving this action.

01b: optional extension.

This action shall be sent by sensors. It shall indicate the activation of Config Mode to link one sensor to several actuators. All others sensors are optionally locked after receiving this action and consider configuration going on (prevents from opening a new session).

Manufacturer Code (16 bit): This shall be sent for information only. No standard action is defined.

- Number of Group Objects to link** This field shall inform the receiver (actuator) how many Set_Delete_Link actions will follow. If any are lost, for instance due to interferences, then the actuator shall discard any other links received during this process in order to avoid “half-linked” E-Mode Channels.
- 00h shall mean that the link is valid for all Datapoints common to both devices ,the sender and the receiver. Only one Set_Delete_Link action is necessary.
- This is allowed if there is no possible ambiguity in the Datapoints links between these two devices.
- Condition:** To be secure, this action is usually processed if Config Mode is active. It may however be directly used to start configuration to link one sensor to several actuators. In this case the device needs some specific HMI to initiate the service.
- Requirements:** Support of Subfunction 00h is mandatory. Others are optional.

4.3.10.2.4 PID_CONFIG_LINK (Channel_Function_Actuator, Channel Code of the actuator)

Pos.	Action	Value																															
		Octet 11								Octet 12								Octet 13								Octet 14							
		Command				Flags				Data								Data								Data							
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
3	Channel_Function_Actuator	0	0	1	1	0	0	0	0	0	0	0	Channel Code								00h												

After receiving the Start_Link action the actuator shall send the Channel Code of its selected channel to the sensor. If the selected sensor channel is of an appropriate type then the sensor shall select the function according to the received Channel Code.

Expected response: Channel_Function_Sensor

Condition: To be secure, this action shall only be processed in config mode.

4.3.10.2.5 PID_CONFIG_LINK (Channel_Function_Sensor, Channel Code of the sensor)

Pos.	Action	Value																															
		Octet 11								Octet 12								Octet 13								Octet 14							
		Command				Flags				Data								Data								Data							
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
4	Channel_Function_Sensor	0	1	0	0	0	0	0	0	0	0	0	Channel Code								00h												

This action shall be the reaction from the sensor after receiving the Channel_Function_Actuator action.

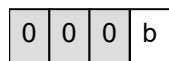
Condition: To be secure, this action shall only be processed in config mode.

4.3.10.2.6 PID_CONFIG_LINK (Set_Channel_Param, Flags, Parameter index, Value)

Pos.	Action	Value															
		Octet 11				Octet 12				Octet 13				Octet 14			
		Command				Flags				Data				Data			
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
5	Set_Channel_Param	0	1	0	1	Flags				Parameter Index = 1				Value			

With this action it is possible for actuator to set parameters in sensors.

Flags:



0 = do not override
 1 = override
 (possibility to change an already set parameter)

Parameter Index: 1 ... 255 (see Channel Code definition)

Value: Parameter value according channel definition (right adjusted)

Expected response: Channel_Param_Response within a delay time ⁵⁾

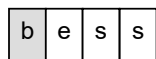
Condition: To be secure, this action is only processed in config mode

4.3.10.2.7 PID_CONFIG_LINK (Channel_Param_Response, Flags, Parameter Index, Value)

Pos.	Action	Value															
		Octet 11				Octet 12				Octet 13				Octet 14			
		Command				Flags				Data				Data			
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
6	Channel_Param_Response	0	1	1	0	Flags				Parameter Index				Value			

This action shall be the response from the sensor after receiving the Set_Channel_Param action. It shall include information for the parameter setting.

Flags:



00b = OK: Parameter has been set successfully
 01b = Parameter has already been set before:
 override depends on application
 specification or product features
 10b = Parameter is set locally on the device:
 override depends on application
 specification or product features
 11b = Parameter override not possible

0 = no error
 1 = parameter error (incorrect index, incorrect value)

0 = parameter sent
 1 = parameter block sent

⁵⁾ Depending of medium, a suitable delay time should be selected (typically 1 s for TP1).

- **If b = 0: Parameter is sent**

- Parameter Index: = 0

The field *Value* shall be two octets long and contain the channel number and the number of parameters.

It shall serve to inform the actuator on how many parameters for the current channel number will be sent. If any telegram is lost due to (RF) interferences, the actuator shall discard any other links received during this process in order to avoid “half-linked” channels.

Value: MSB: channel number

LSB: number of parameters

- Parameter Index: N = 1 ... 255 (see Channel Code definition)

Value: Value of parameter N according channel definition (right adjusted).

In case of “parameter locally set” the value shall be the locally set value.

Conditions: To be secure, this action shall only be processed in config mode.

- **If b = 1: Parameter Block is sent**

- Parameter Index: = 0: Shall enable a device to send its parameters in one shot after a manufacturer specific interaction, and a receiver to check whether all parameters are received.

Only parameter blocks with 2 octets can be sent.

Value: MSB: The value shall be fixed to 0.

LSB: Shall contain the number of parameter blocks

- Parameter Index: N = 1 ... 32: gives the channel number in DD2 order declaration.

Value: Parameter block value of the given channel number. It is sent most significant byte (MSB) and most significant bit (msb) first.

Unidirectional devices may send the current value of parameters.

The length of the field Value can be extended to 11 to handle parameters longer than two octets.

Conditions: None.

EXAMPLE Parameter block of channel CH_Generic_PB_1/2_Info_2 (scene function)

0								8				15				
Adjustable selection = 8									Scene Number							
0	0	0	0	1	0	0	0									
msb								lsb								
MSB								LSB								

4.3.10.2.8 PID_CONFIG_LINK (Begin_Connection)

Pos.	Action	Value															
		Octet 11								Octet 12				Octet 13			
		Command				Flags				Data				Data			
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
7	Begin_Connection	0	1	1	1	0	0	0	0	00h				00h			

This action shall be sent by the actuator to indicate that the sensor can start the link procedure.

Expected response: Set_Delete_Link within a delay time ⁶⁾

Condition: To be secure, this action shall only be processed in config mode.

4.3.10.2.9 PID_CONFIG_LINK (Set_Delete_Link, Sub Function, Connection Code, Group Address)

Pos.	Action	Value															
		Octet 11								Octet 12							
		Command				Flags				Data							
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
8	Set_Delete_Link	1	0	0	0	Sub-function											
						0				Connection Code							
						1				scene number value							
						2				AckSlotNb							

This PDU shall be sent for each individual Group Object.

- Subfunction:
- 0: The sensor shall use this Subfunction to link the Group Objects of the selected E-Mode Channel with the Group Objects of the actuator. Octet 12 shall contain a Connection Code.
 - 1: The sensor shall use this Subfunction to associate a scene number Group Object to one GA, indicating at the same time the scene number value to use. It may be repeated for several scene number values. Octet 12 shall contain a scene number value. The Connection Code shall implicitly be the one of a scene number Group Object.
 - 2: This function is designed for the configuration of KNX RF Multi devices.

The sensor device shall use this Subfunction to associate an ack slot number in KNX to one GA and implicitly to a Group Object.

- If the Datapoint is an output Datapoint (O Flag), the sensor shall set the value of the ack slot.
- If the Datapoint is an input Datapoint (I Flag), the value shall have no meaning and the sensor shall wait for the actuator's Link_Response service to get the ack slot number.

If no ack is required in runtime for that Extended Group Address then the device shall set the AckSlotNb to the value FFh. The GA field shall not be interpreted in this case and shall be set to 0. The used GA for the RF Multi link shall be taken from the previous Link_Response message corresponding to Set_Delete_Link request (Subfunction 0 or 1).

Requirements: For subfunctions 0 and 1, the RF Multi frequency (Sx or Fx) that is defined for runtime communication shall be deducted from the capabilities of sensor and actuator exchanged with the Features command. If both devices have at least one compatible frequency, this frequency shall be used for runtime (Fx shall be used if the two frequencies are possible). If no frequency is compatible, then the link shall be refused.

⁶⁾ Dependent of the used medium, a suitable delay time should be selected (typically 1 s for TP1).

Error handling: Discard in case of unknown values.

Expected response: Link_Response within a delay time.

Condition: To be secure, this action shall only be processed while Config Mode is active.

4.3.10.2.10 PID_CONFIG_LINK (Link_Response, Flags, Group Address)

Pos.	Action	Value																													
		Octet 11								Octet 12								Octet 13								Octet 14					
		Command				Flags				Data								Data								Data					
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2
9	Link_Response	1	0	0	1	Flags				Connection Code								Group Address													
						AckSlotNb																									

This action shall be the response from the actuator after receiving the Set_Delete_Link and shall include information about the Link result.

Flags:

3	2	1	0
0	e	s	s

- ss: 00b: Link or AckSlotNb added
 01b: use existing Group Address or existing AckSlotNb
 10b: link deleted
 11b: link or AckSlotNb not added. (The link procedure can be continued.)
- e: 0: No error.
 1: Error. The link procedure shall be stopped.

Comments:

If using the Set_Delete_Link service with subfunction 0 or 1, the flags in the Link_Response service shall apply to the Group Object addition or deletion and shall be relative to the Group Address.

- The required user interaction and HMI for adding or deleting link management is implementation specific.

EXAMPLE 6 A switch can indicate which operation to perform or a push-button can act as a toggle with indication lamp.

- If another Group Address is already linked to a Group Object, this Group Address shall be used by the sensor device.

EXAMPLE 7 For a toggle function.

In this case, the actuator device shall send this action with Flags set to 01b (link already used) and Group Address set with the already linked Group Address).

If using the Set_Delete_Link service with subfunction 2, the flags in the Link_Response action shall apply to the allocation of the ack slot number to the Group Object having the specified Connection Code. The Group Address field is not used in this case and shall be set to 0. The used Group Address for the RF Multi link shall be taken from the previous Link_Response service corresponding to Set_Delete_Link request (Subfunction 0 or 1).

From the actuator's point of view

- If using the Set_Delete_Link action with subfunction 2 for an input Datapoint (I Flag), the flags in the Link_Response action shall apply to the allocation of the ack slot number by the sensor. The sensor shall set the value of the ack slot number and the actuator shall store the ack slot number for this Group Address in the E-Mode Channel having the specified Connection Code.
- If using the Set_Delete_Link service with subfunction 2 for an output Datapoint (O Flag), the flags in the Link_Response action shall apply to the allocation of the ack slot number by the actuator to the sensor. The actuator shall set the value of the ack slot number and the sensor shall store the ack slot number for this Group Address in the E-Mode Channel having the specified Connection Code.

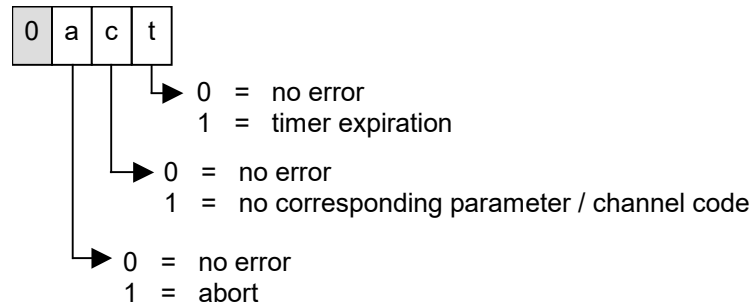
4.3.10.2.11 PID_CONFIG_LINK (Stop_Link)

Pos.	Action	Value															
		Octet 11								Octet 12							
		Command				Flags				Data				Data			
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
10	Stop_Link	1	0	1	0	Flags				00h				00h			

This action shall be sent by the sensor and shall indicate to the actuator that the link process for the selected channel shall be finished. All sensors shall be unlocked.

In case of abort, all devices shall immediately return to normal mode.

Flags:



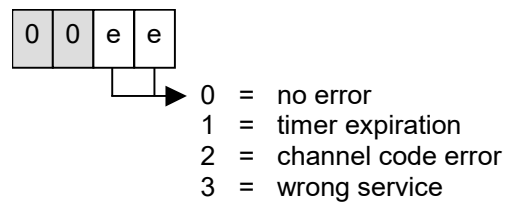
4.3.10.2.12 PID_CONFIG_LINK (Quit_Config_Mode)

Pos.	Action	Value															
		Octet 11								Octet 12							
		Command				Flags				Data							
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
11	Quit_Config_Mode	1	0	1	1	0	0	0	0	00h				00h			

After deactivation of the config mode the actuator shall send the Quit_Config_Mode action.

The sensors shall be set in normal mode after receiving this action and all other actuators shall be unlocked

Flags:



4.3.10.2.13 PID_CONFIG_LINK (Reset Installation)

Pos.	Action	Value															
		Octet 11								Octet 12							
		Command				Flags				Data							
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
12	Reset Installation	1	1	0	0	0	0	0	0	00h				00h			

With this action it shall be possible to reconfigure a device to factory setting (RESET).

Condition: To be secure, this action shall only be processed in config mode.

This action shall not be implemented in KNX-RF devices as no separation from the neighbouring installation is possible.

4.3.10.2.14 PID_CONFIG_LINK (Features, Subfunction, Physical requirements)

Pos.	Action	Value																													
		Octet 11								Octet 12								Octet 13								Octet 14					
		Command				Flags				Data								Data								Data					
		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2
13	Features	1	1	0	1	Sub-function																									
						0			Physical requirements								reserved														

This action shall be sent in both directions.

Subfunction: - 0: The device shall use this action with the subfunction 0 to send its physical requirements to the other devices. Octet 12 shall contain the physical requirements of the device.

Octet 12

Physical requirements

7	6	5	4	3	2	1	0
r	r	r	b4	b3	b2	b1	b0

r: 0: These bits are reserved and shall be 0.

NOTE 9 These may be used for supporting KNX Data Security in the future.

b4: Management (Tx and Rx) of the physical acknowledge (fast Ack)

0b: The device shall not be able to manage a physical acknowledge if requested in runtime.

1b: The device shall be able to manage a physical acknowledge if requested in runtime.

b3: Transmission on the slow Sx RF channels

0b: The device shall not be able to transmit Frames on the slow Sx RF channels.

1b: The device shall be able to transmit Frames on the slow Sx RF channels.

b2: Transmission on the fast Fx RF channels.

0b: The device shall not be able to transmit Frames on the fast Fx RF channels.

1b: The device shall be able to transmit Frames on the fast Fx RF channels.

b1: Scan of the slow Sx RF channels.

0b: The device shall not scan the slow Sx RF channels in runtime and thus shall not be able to receive Frames transmitted on a slow Sx RF channel.

1b: The device shall scan the slow Sx RF channels in runtime and thus shall be able to receive Frames transmitted on a slow Sx RF channel.

b0: scan of the fast Fx RF channels

0b: The device shall not scan the fast Fx RF channels in runtime and shall thus not be able to receive Frames transmitted on a fast Fx RF channel.

1b: The device shall scan the fast Fx RF channels in runtime and shall thus be able to receive Frames transmitted on a fast Fx RF channel.

Expected response: After the transmission of the Features (KNX RF Multi Actuator) message, the Features (KNX RF Multi Sensor) message shall be expected within a delay time. If no Features action is received within this delay, the actuator shall assume that the sensor is a KNX RF Ready sensor. This delay shall have the same value as the delay for all other exchanges of actions, but in this special case, the PB procedure shall continue.

4.3.11 PID_ADDR_REPORT (PID = 60)

- Property name: Address report
- Property Datatype: PDT_GENERIC_06
- Datapoint Type: None.

This Property shall include the KNX Serial Number of the device (6 octets).

If the device has no valid KNX Serial Number then this value shall in datagrams be set to 000000000000h.

4.3.11.1 Usage by the Management Server (device)

The usage by the device shall comply with the Management Server side support of the Management Procedure NM_IndividualAddress_SerialNumber_Report.

4.3.12 PID_ADDR_CHECK (PID = 61)

- Property name: Address Check
- Property Datatype: PDT_GENERIC_01
- Datapoint Type: None.

This Property shall be accessed by a Management Client in the Network Management Procedure NM_IndividualAddress_Check_LocalSubnetwork as specified in [09].

The Management Server (receiving device) shall not interpret the telegram during this procedure, neither give any error message. The telegram shall only be used to receive a L2-acknowledge in that procedure if the tested Individual Address is occupied.

The Property PID_ADDR_CHECK shall not contain data, but only serves for method invocation. As parameter in the procedure NM_IndividualAddress_Check_LocalSubnetwork the value shall consist of a single octet 00h !

4.3.13 PID_OBJECT_VALUE (PID = 62)

- Property name: Object Value
- Property Datatype: PDT_Function
- Datapoint Type: This Property is a Function Property. The coding of the data depends on whether data is written to the function or responded by the function. No single DPT can be given.

4.3.13.1 Abstract Resource Definition

This Property Object Value shall serve to realise the Group Object Indirection as required in [08].

Any access to this Property shall modify the Group Object Flags of the addressed Group Object.

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12	octet...
handle (hi)	handle (lo)	value	value

Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

The function Write ObjectValue shall be used by the client to set the value of a Group Object in the device. Input parameters shall be the Group Object Handle to identify the Group Object in the device and the value to be written to this Group Object.

NOTE 10 The Group Object Handle is specified in [08].

The response shall contain the ReturnCode:

Return Codes: (00h) = SUCCESS (value successfully written)
 (FFh) = ERROR (invalid Group Object Handle)

b) Read (A_FunctionPropertyState_Read-PDU)

octet 10	octet 11
handle (hi)	handle (lo)

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet...
Return Code	value	value	value

The function Read ObjectValue shall be used by the client to read the value of a Group Object in the device. The input parameter is the Group Object Handle to identify the Group Object. The response shall contain the return code and the read value.

Return Codes: (00h) = SUCCESS (value was successfully read)
 (FFh) = ERROR (invalid Group Object Handle)

4.3.13.2 Group Object Handle – Realisation Type 1 – RF bidirectional end device

For sending Group Objects this Group Object Handle shall be identical to the pre-assigned sending Extended Group Address.

For receiving Group Objects the Group Object Handle shall be purely regarded as a local object index and shall have the format of a Group Address.

4.3.13.3 Group Object Handle – Realisation Type 2 –RF unidirectional device

For RF unidirectional devices, which only have sending Group Objects, the following rule is defined, how Group Addresses are assigned. The order of Group Objects is defined by

- a) the E-Mode Channel definitions and
- b) the order in which E-Mode Channels appear in the Device Descriptor Type 2.

In the order defined in this way, Group Addresses shall be assigned to the Group Objects sequentially starting from 0001h.

4.3.13.4 Management Clients

A client (e.g. for visualisation), in order to use the point-to-point access, must know the values of the Group Addresses and Group Object Handles of the Group Objects present in the device.

For an E-Mode Management Server, the client can derive the active Group Objects in the device from the Device Descriptor Type 2 (DD2) in combination with the knowledge contained in the Channel database.

A central Management Client shall have knowledge of the object handle values from its product database.

However, in the general case, it is not possible to know all Group Objects in the device, because some could be hidden, this is, do not appear in DD2 in the current hardware configuration. This is the case e.g. if a generic communication module can be attached to several actuators.

For such Profiles, PID_OBJECT_ADDRESS is required. This Property shall enable reading of the Group Addresses and object handles by the Management Client.

4.3.14 PID_OBJECTLINK (PID = 63)

- Property name: Object Link
- Property Datatype: PDT_Function
- Datapoint Type: None.

Used by: Ctrl FEC
PB FEC

4.3.14.1 Function *Write Object Link*

4.3.14.1.1 Format

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16	octet 17	octet 18	octet 19	octet 20	octet 21
Flags	00h	SN (high)	SN	SN	SN	SN	SN (low)	GA (high)	GA (low)	group object index (hi)	group object index (lo)
Extended Group Address											

**Figure 4 – Function *Write Object Link*
A_FunctionPropertyCommand-PDU (example)**

4.3.14.1.2 Usage by the Management Client

The Management Client shall use the Function *Write Object Link* to establish and to break a single link between a Group Object and a Group Address or an Extended Group Address.

1. Flags

Description: This field shall indicate whether a link to the referred Group Object shall be established or shall be broken. In case the link shall be added, it shall also indicate whether the link shall be a sending Group Address or not.

Encoding:

bit nr	7	6	5	4	3	2	1	0
name:	r	r	r	r	r	aet	d	s

- **Sending (s)**

position: bit 0

description: In case the contained (Extended) GA is added, then this field *Sending* (s) shall indicate whether it shall be the sending GA for the referred GO or not.

The flag s shall only be interpreted in case the flag d equals 0; in case the flag d equals 1, the value of the flag s shall be don't care.

format, encoding: 0: not sending: The contained GA shall not be the sending GA for the referred GO.
1: sending: The contained GA shall be the sending GA for the referred GO.

Error and exception handling

- If a link is Added using PID_OBJECTLINK to a GO to which the contained GA is already linked, but with a different value of the field S, then this new value of the flag s shall be used.
- A GA set as "sending" shall take precedence on other previous sending GA for that GO.

- **Add/Delete (d)**

position: bit 1

description: The field *Add/Delete* (d) shall indicate whether the contained (Extended) GA shall be added to or removed from the list of GA assigned to the referred GO.

format, encoding: 0: add: Add the contained GA to the list of GAs assigned to the referred GO.
1: delete: Remove the contained GA from the list of GAs assigned to the referred GO.

- **Address extension type (aet)**

position: bit 2

description: This field (aet) shall indicate whether an Extended Group Address or a Group Address is used in the link.

format, encoding: 0: use an Extended Group Address based on SN (useful for RF with SN)
1: use a Group Address (useful on TP, on PL with DoA and on RF with DoA)

- **reserved (r)**

position: bits 7 to 2

description: These bits are reserved.

format, encoding: The E-Mode Management Client shall clear (0) these bits.
The E-Mode Management Server (device) shall ignore the value of these bits.

2. KNX Serial Number (SN)

Description: If the field AET equals 0, then this field shall contain the KNX Serial Number of the extended GA.
If the field AET equals 1 (use of a Group Address) then this field shall be 000000000000h.
If this Function request does not contain an Extended GA but only a basic GA, then this field shall be 000000000000h.

Error and exception handling

- If a link is Added using PID_OBJECTLINK with the field AET = 1 and the field SN different from 0, then the links shall be refused with an explicit error code.

Encoding: DPT_SerNum (DPT_ID = 221.001) or 000000000000h.

3. Group Address (GA)

Description: This field shall be the basic GA or the GA part of the Extended GA, to which the Link shall be established or from which the Link shall be broken.

Encoding: U₁₆ value

4. Group Object Index

Description: This shall be the Group Object number. This shall refer to the same value as used to refer Group Object in the Property Value arrays PID_GO_CCODES_LIST, PID_GO_SUBUNIT and PID_GO_NAME_LIST.
This shall be the GO number of the GO within the E-Mode Channel. This shall start with 1.

Encoding: U₁₆ value

4.3.14.1.3 Usage by the Management Server (device)

The Management Server shall execute the Function *Write Object Link*. It shall respond to the Management Client with the response as given in Figure 5 that shall contain the *Return Code* of the Function.

octet 10
Return Code

**Figure 5 – Response to the Function *Write Object Link*
A_FunctionPropertyState_Response-PDU (example)**

1. Return Code

Description: The Return Code shall give information on the success or possible errors that occur in the execution of the Function *Write Object Link*.

Encoding:

00h:	SUCCESS:	Link established or broken or already established or broken before.
FFh:	ERR_TABLEFULL:	The device does not have memory space anymore for establishing the requested Link.
FEh:	ERR_GRP_OBJECT:	Group Object does not exist or is out of range (0 for example).
FDh:	ERR_AET	The value of field SN (different from 0) is not correct with the field AET (equal to 1)
FCh:	ERR_GENERIC	Any other error than previously defined.

Error and exception handling

- If the Extended GA contains the own KNX Serial Number of the Management Server (device), then this Function shall be interpreted as the request to establish or break an internal Link.

4.3.14.2 Function *Read Object Link*

4.3.14.2.1 Format

octet 10	octet 11	octet 12	octet 13
Code	Group object index (hi)	Group object index (lo)	Iterator

**Figure 6 – Function *Read Object Link*
A_FunctionPropertyState_Read-PDU (example)**

Octet 10: Code

The octet 10 shall indicate whether the read command is used to go through the list of links or to know the number of free associations.

Code:

00h:	Read the links through an iterator and a group object index.
01h:	Read free number of associations: in this case, the response contains a return code together with two bytes for the number of free associations to existing GAs and two bytes for the number of associations with new GAs. The octets 11 to 13 are not used in this case and shall be set to 0.

4.3.14.2.2 Read through iterator (Code = 00h)

4.3.14.2.2.1 Usage by the Management Client

The Management Client shall use the Function *Read Object Link* to read a single Group Addresses or Extended Group Address linked to a single Group Object in a Management Server (device). Through subsequent calls of the Function *Read Object Link* with values of the *Iterator* from 0 incremented each call by 1, it can read through all links of the Group Object in the Management Server.

1. Group object index

Description: This shall be the Group Object number. This shall refer to the same value as used to refer Group Object in the Property Value arrays PID_GO_CCOCODES_LIST, PID_GO_SUBUNIT and PID_GO_NAME_LIST.
This shall be the GO number of the GO within the E-Mode Channel. This shall start with 1.

Encoding: U₁₆ value

2. Iterator

Description: This Parameter shall serve as an iterator: it shall be an index in the list with the Group Object links to the Group Object identified by the *Group Object Index* in the Management Server. (This list is an implementation specific storage that is not further standardised.)
The Management Client can read the linked Group Addresses or Extended Group Addresses one by one, starting with Iterator value 0 and incremented by one for each next link to be read, until the Management Server responds that there are no more links for this Group Object for the requested Iterator value and above.

Encoding: U₈ value

4.3.14.2.2.2 Usage by the Management Server (device)

The Management Server shall execute the Function *Read Object Link*. It shall respond to the Management Client with the response as given in Figure 7.

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16	octet 17	octet 18	octet 19	octet 20	octet 21
Return Code	Flags	SN (high)	SN	SN	SN	SN	SN (low)	GA (high)	GA (low)	GO index (hi)	GO index (lo)

**Figure 7 – Response to the Function *Read Object Link* (case Code =00h)
A_FunctionPropertyState_Response-PDU (example)**

NOTE 11 The Iterator from the request in the A_FunctionPropertyState_Read-PDU is not repeated in this response in the A_FunctionPropertyState_Response-PDU. The Management Client can only link this response to the iterator used in the preceding request. Therefore, the Management Client should not issue a new request before the response to a previous request has been received.

1. Return Code

Description: The Return Code shall give information on the success or possible errors that occur in the execution of the Function *Read Object Link* (code = 00h).

Encoding: 00h: This shall signal that for the requested value of the Iterator, the contained link information is valid
 FFh: This shall signal that for the requested value of the Iterator the Function Read Object Link does not find a link.
 In this case, the A_FunctionPropertyState_Response-PDU shall only contain the Return Code and shall not contain the fields Flags, Serial Number, GA or GO index.

2. Flags

Description: This field shall indicate whether the contained GA is the sending GA for the referred GO or not.

Encoding:

bit nr	7	6	5	4	3	2	1	0
name:	r	r	r	r	r	r	aet	s

- **Sending (s)**

position: bit 0

description: This field *Sending* (s) shall indicate whether the contained (extended) GA is the sending GA for the referred GO or not.

format, encoding: 0: not sending: The contained GA is not be the sending GA for the referred GO.
 1: sending: The contained GA is the sending GA for the referred GO.

- **Address extension type (aet)**

position: bit 1

description: This field (aet) shall indicate whether an Extended Group Address or a Group Address is used in the link.

format, encoding: 0: use an Extended Group Address based on SN (useful for RF with SN)
 1: use a Group Address (useful on TP1, on PL110 with DoA and on RF with DoA)

- **reserved (r)**

position: bits 7 to 2

description: These bits are reserved.

format, encoding: The E-Mode Management Client shall clear (0) these bits.
 The E-Mode Management Client shall ignore the value of these bits.

3. KNX Serial Number (SN)

Description: If the field AET equals 0, then this field shall contain the KNX Serial Number of the extended GA. If the field AET equals 1, then this field shall be 000000000000h.

If the Group Address is not an Extended Group Address, then the field KNX Serial Number shall be 000000000000h.

Encoding: DPT_SerNum (DPT_ID = 221.001) or 000000000000h

4. Group Address (GA)

Description: This field shall be the basic Group Address or the Group Address part of the Extended Group Address, for which the contained Link information is responded.

Encoding: U₁₆ value

5. GO index

Description: This shall be the GO index that shall identify the Group Object within the Channel (same value as used to refer Group Object in the Property Value arrays PID_GO_CCODES_LIST, PID_GO_SUBUNIT and PID_GO_NAME_LIST.

This shall be the GO number of the GO within the E-Mode Channel. This shall start with 1.

Encoding: U₁₆ value

4.3.14.2.3 Read number of free associations (Code = 01h)

4.3.14.2.3.1 Usage by the Management Client

The Management Client shall use the Function *Read Object Link* to read the number of free associations (with existing GAs or with new GAs) in a Management Server (device).

Use Case 1: a MaC may want to know, before adding several links that must be added together, whether he can add or not all the associations. Moreover, he already knows the GA that have already been used by the MaS. So if the MaS sends him the information on the number of free associations with existing and with new Gas, then he is able to decide to add or not all associations.

Use Case 2: for diagnostic purpose, an installer may want to know whether a device (MaS) is close to its limits in term of associations, in order to decide on an extension of the installation. If the number of free associations is below a certain level, then the MaC may warn the user (installer) that the present device may not support this extension and the installer should use a new device or modify its installation.

4.3.14.2.3.2 Usage by the Management Server (device)

The Management Server shall execute the Function *Read Object Link*. It shall respond to the Management Client with the response as given in Figure 8.

octet 10	octet 11	octet 12	octet 13	octet 14
Return Code	NFAEGA (high)	NFAEGA (low)	NFANGA (high)	NFANGA (low)

**Figure 8 – Response to the Function *Read Object Link* (case code = 01h)
A_FunctionPropertyState_Response-PDU (example)**

1. Return Code

Description: The Return Code shall give information on the success or possible errors that occur in the execution of the Function *Read Object Link* (code = 01h).

Encoding: 00h: This shall signal a successful answer.
FFh: This shall signal an error.

2. Number of Free Associations to Existing Group Addresses (NFAEGA)

Description: The octets 11 and 12 contain the high and low octets of the number of free associations to existing Group Addresses (GA).

Encoding: U₁₆ value

3. Number of Free Associations with New Group Addresses (NFANGA)

Description: The octets 13 and 14 contain the high and low octets of the number of free associations with new Group Addresses (GA).

Encoding: U₁₆ value

4.3.15 PID_APPLICATION (PID = 64)

- Property name: Application
- Property Datatype: PDT_Function
- Datapoint Type: None.

Used by: This Property is not mandatory for any of the existing KNX Profiles.

a) Write (A_FunctionPropertyCommand_PDU)

octet 10	octet 11	octet 12	octet 13
00h	E-Mode Channel Number	Connection Code	

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13
Return Code	E-Mode Channel Number	Connection Code	

The function Write Application shall be used by a Management Client to change the running application in a device, provided that the device supports multiple applications.

EXAMPLE A push button may support the applications “switching”, “dimming” and “blinds”.

Input parameters shall be the Channel Number of the E-Mode Channel of which the application is to be changed, and the E-Mode Channel Code of the new application. The response shall repeat the E-Mode Channel Number and the new channel code. In case the switch to the desired channel code is not possible, the previously used channel code shall be returned.

Return Codes: (00h) = SUCCESS (application changed)
 (FFh) = ERROR (invalid channel number or channel code)

b) Read (A_FunctionPropertyState_Read-PDU)

octet 10	octet 11
00h	E-Mode Channel Number

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13
Return Code	E-Mode Channel Number	Connection Code	

The function Read Application shall be used by a Management Client to check which application is running in a device.

Return Codes: (00h) = SUCCESS (channel code successfully read)
 (FFh) = ERROR (invalid channel number)

4.3.16 PID_PARAMETER (PID = 65)

- Property name: Parameter
- Property Datatype: PDT_Function
- Datapoint Type: None

Used by: mask 2010h
 RF bidirectional devices

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12	octet ...
E-Mode Channel Number	Parameter Number	Value	Value

Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

The function Write Parameter shall be used by a Management Client to change an application parameter in a device. Input parameters shall be the E-Mode Channel Number of the E-Mode Channel to which the parameter belongs, the parameter number (according to E-Mode Channel definitions), and the parameter value. The output parameter shall be a return code.

Return Codes: (00h) = SUCCESS (value successfully written)
 (FFh) = ERROR (invalid parameter)

b) Read (A_FunctionPropertyState_Read-PDU)

octet 10	octet 11
E-Mode Channel Number	Parameter Number

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet ...
Return Code	00h	Value	Value

The function Read Parameters shall be used by a Management Client to read an application parameter from a device. Input parameters shall be the channel number of the channel to which the parameter belongs, and the parameter number (according to channel definitions). Output parameter shall be a return code and the value read.

Return Codes: (00h) = SUCCESS (value successfully written)
 (FFh) = ERROR (invalid parameter)

Return Codes: (00h) = SUCCESS (value successfully written)
 (FFh) = ERROR (invalid parameter)

In this case, the A_FunctionPropertyState_Response-PDU shall end at octet 10 "Return Code". There shall be no octet 11 and no value (no octet 12 or beyond).

4.3.17 PID_OBJECTADDRESS (PID = 66)

- Property name: Object Address
- Property Datatype: PDT_Function
- Datapoint Type: None.

Used by: mask 2010h
 RF bidirectional devices

a) Read (A_FunctionPropertyState_Read-PDU)

octet 10	octet 11
00h	Iterator

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16	octet 17
Return Code	Iterator	E-Mode Channel Number	E-Mode Channel Code		Group Object Number	Group Object Handle	

The function Read ObjectAddress shall be used by a Management Client to read the Group Object Handle of the objects from a device. Every response shall contain the Group Object Handle of one Group Object.

The Read shall contain an Iterator that shall count the Group Objects in the device. The Management Client shall issue Reads with the Iterator starting from 0 in a continuous order.

The Response shall contain the same Iterator value in case of a valid Group Object Handle. In this case the E-Mode Channel Number (corresponding to the order in DD2), the E-Mode Channel Code to which the Group Object belongs, the Group Object number in the E-Mode Channel (according to the E-Mode Channel definitions, see there), and the Group Object Handle shall follow. The Return Code shall in this case be 00h.

In case there is no corresponding Group Object value present in the device for an Iterator value, the Return Code shall be FFh. The E-Mode Channel Number and Group Object number shall be counted from 0.

b) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11
00h	Iterator

Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

The function Write ObjectAddress has no functionality. The Return Code shall in any case be FFh.

4.3.18 PID_PSU_TYPE (PID = 67)

- Property name: PSU Type
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: DPT_UICurrentmA (7.012)
- Memory type: Non-volatile memory.

4.3.18.1 Abstract Resource definition

This Property shall contain the nominal supply current [mA] of the bus power supply in the device.

4.3.18.2 Format

This supply current value shall be binary encoded using a 16 bit unsigned integer with 1 mA resolution. The value 0 is reserved (no bus power supply functionality available).

4.3.18.3 Usage by the Management Client

This Property can be used by a Management Client to scan presence of PSUs/DPSUs in a system in order to calculate the total available supply current or to activate/deactivate individual PSU/DPSU.

To this, the Management Client shall use:

- the General Procedure for accessing Properties with A_NetworkParameter_Read

```
/* Scan for the presence of PSU and DPSU in the network */
/* hop_count_type = 0 only scans the Subnetwork in which the Management Client is located. */
/* hop_count_type = standard scans the entire installation. */
NM_NetworkParameter_Read_R(hop_count_type, parameter_type = [object_type = Device Object,
Property_id = PID_PSU_TYPE], priority = system, test_info = 00h)
/* Every DPSU or PSU with communication capabilities now reports individually */
/* the nominal current of its PSU or DPSU. */
```

- DMP_InterfaceObject_Read_R with A_PropertyValue_Read

Both procedures are specified in [09].

4.3.18.4 Usage by the Management Server (device)

This Property shall be read only.

This Property shall be readable through A_NetworkParameter_Read and A_PropertyValue_Read.

The implementation of this Property is mandatory if the bus power supply feature is implemented in a communicating device supporting Property based management. For other realisations, as e.g. stand alone bus power supply units, the implementation of this Property is conditional on the DPSU Profile in [17].

4.3.19 PID_PSU_STATUS (PID = 68)

- Property name: PSU Status
- Property Datatype: PDT_BINARY_Information
- Datapoint Type: DPT_Switch (1.001)
- Memory type: Volatile memory.

4.3.19.1 Abstract Resource definition

This Property shall indicate whether the bus power supply in the device is currently switched on or off. This information is mainly used for diagnostics or visualisation.

4.3.19.2 Format

The Property Value shall be encoded according DPT_Switch (1.001) as specified in [14].

NOTE 12 "0": "the power supply in the device is switched off";
"1": "the power supply in the device is switched on"

4.3.19.3 Usage by the Management Client

This Property can be used by a Management Client to retrieve the status of all PSUs and DPSUs in a system with communication capabilities to check the state of the bus power supply feeding functionality.

To this, the Management Client shall use:

- the General Procedure for accessing Properties with A_NetworkParameter_Read

```
/* Scan for the status of PSUs and DPSUs */
/* hop_count_type = 0 only scans the Subnetwork in which the Management Client is located. */
/* hop_count_type = standard scans the entire installation. */
NM_NetworkParameter_Read_R(hop_count_type, parameter_type = [object_type = Device Object,
Property_id = PID_PSU_STATUS], priority = system, test_info = 00h)
/* Every DPSU or PSU with communication capabilities now reports individually */
/* the state of its PSU or DPSU. */
```

- DMP_InterfaceObject_Read_R with A_PropertyValue_Read, or

Both Management Procedures are specified in [09].

4.3.19.4 Usage by the Management Server (device)

This Property shall be read only.

The implementation of this Property is mandatory if the bus power supply feature is implemented in a communicating device supporting Property based management. For other realisations, as e.g. stand alone bus power supply units, the implementation of this Property is conditional on the DPSU profile in [17].

4.3.20 PID_PSU_ENABLE (PID = 69)

- Property name: PSU Enable
- Property Datatype: PDT_ENUM8
- Datapoint Type: DPT_PSUMode (20.008)
- Memory type: Non-volatile memory.

4.3.20.1 Abstract Resource definition

This Property shall be used to control operation of the bus power supply from a Management Client.

4.3.20.2 Format

For the encoding of the Property Value, please refer to DPT_PSUMode (20.008) in [14]. The supported range is product specific; e.g. disabled/enabled only or disabled/auto only.

4.3.20.3 Usage by the Management Client

Access to this Property using A_NetworkParameter_Read and A_NetworkParameter_Write services is not allowed. Broadcast write of PID_PSU_ENABLE could activate more than the allowed 8 DPSU and/or 2 central PSU or could deactivate all PSU/DPSU and communication would break down.

4.3.20.4 Usage by the Management Server (device)

Implementation of this Property is optional. PSU/DPSU can be activated/deactivated also by mechanical means (e.g. a jumper or a switch).

4.3.21 PID_DOMAIN_ADDRESS (PID = 70)

- Property name: Domain Address
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: DPT_Value_2_Ucount

This Property shall include the Domain Address of the Device

The Domain Address shall be managed through the appropriate Management Procedures as specified in [09].

EXAMPLES NM_DomainAndIndividualAddress_Read, NM_DomainAndIndividualAddress_Write

The Property based access to the Domain Address is interesting for local access, e.g. through cEMI. The value of the Property shall be the Domain Address of the cEMI server device itself, this is if the cEMI server device should be in only one Domain as a Management Server, seen from the bus medium.

4.3.22 PID_IO_LIST (PID = 71)

- Property name: Interface Object List
- Property Datatype: PDT_UNSIGNED_INT[]
- Datapoint Type: DPT_Value_2_Ucount (7.001)

4.3.22.1 Resource definition

This Property is mandatory if the Extended Data Property Services and/or the Extended Function Property Services are implemented.

PID_IO_LIST shall be a read-only, array structured Property with as much elements as Interface Objects are located (or “active”) in the device. The array element with index 0 shall contain the number of array elements (this is, Interface Objects) implemented in the device. The Property shall contain the Interface Object Types in ascending order of the object indexes, starting with the array field with index 1. For Interface Objects with more than 1 instance, the 1st instance shall be at the position with the lower/lowest index, the Interface Object Type of the 2nd instance shall be placed at the next index and so on.

4.3.22.2 Usage by the Management Client

4.3.22.2.1 Remote access

The Management Client shall access this Property through

- DMP_InterfaceObject_Read_RCI, or
- DMP_InterfaceObject_Read_RCo if the Management Server supports connection-oriented communication mode and a Transport Layer connection to the Management Server exists.

```
/* Read PID_IO_LIST of the Device Object */
DMP_InterfaceObject_Read_RCI( /* [IN] */ object_index = 0; /* [IN] */ PID = PID_IO_LIST,
/* [OUT] */ PID_IO_LIST = data)
```

4.3.23 PID_MGT_DESCRIPTOR_01 (PID = 72)

- Property name: Management Descriptor 1
- Property Datatype: PDT_GENERIC_10
- Datapoint Type: None

4.3.23.1 Use

This Property shall be read-only and shall provide an identification for the management of the device.

4.3.23.2 Format

The Property Value shall be a 10 octet value, of which octet 1 (MSB) shall contain the field “Structure indication” and shall specify the structure of the further contents of this Management Descriptor 1. The coding and possible values of octets 2 to 10 shall depend on the value given in the field “Structure indication” in octet 1.

Octet	1 (MSB)	2	3	4	5	6	7	8	9	10 (LSB)
Field	Structure indication	structure contents								

Field “Structure indication” (octet 1)

The datatype of the field “Structure indication” shall be enumeration (datatype N₈).

Value	Management Model	Remark
0	reserved	
1	Property-based implementation independent management on TP1	System 300, mask 0300h
2	Property-based implementation independent management on RF for BiBat devices	System 300, mask 2300h
3 to 255	reserved	Reserved for future extensions

4.3.23.3 Structure indication 01h: devices supporting System 300

Octet	1 (MSB)	2	3	4	5	6	7	8	9	10 (LSB)
Field	Structure indication	Reserved		Table Management	Reserved					
Value	01h	00h	00h		00h	00h	00h	00h	00h	00h

For devices supporting System 300 on TP1 the field “Structure indication” (octet 1) shall have the value 01h.

Octets 2 and 3 shall be reserved and shall have the value 00h.

The field “Table Management” (octet 4) shall contain indications to the management models to be used by a Management Client for Group Address Table management and Group Object Association Table management.

Octets 5 to 10 shall be reserved and shall have the value 00h.

Field “Table Management” (octet 4)

This field shall be divided in two subfields of each one nibble.

Bit	7 (MSB)	6	5	4	3	2	1	0
Field	Group Address Table Management				Association Table Management			
Value	0h				0h or 1h			

The datatype of each of the two fields shall be an enumeration (Datatype N₄).

The higher nibble is foreseen to distinguish between different Group Address Table management models. Since currently no different such management models are needed for System 300, this nibble is still reserved and shall have the value 0h.

The lower nibble shall be an indication to the Group Object Association Table management model to be used by a Management Client.

The PDT indicated by this field shall be according the format of the Property based Group Object Association Table as specified in PID_TABLE (PID = 23).

Value	PDT for PID_TABLE
0	PDT_GENERIC_02[] ⁷⁾
1	PDT_GENERIC_02[] or PDT_GENERIC_04[], to be discovered from the Property description of PID_TABLE ⁸⁾
2 to 15	reserved values

4.3.23.4 Structure indication 02h: RF BiBat devices supporting Property-based management

For BiBat devices the field “Structure Indication” (octet 1) shall have the value 02h.

The field “Management Model” (octet 2) shall contain the supported basic Management Model.

The field “BiBat Feature” (octet 3) shall contain the supported BiBat features.

The field “Table Management” (octet 4) shall contain indications to the Management Models to be used by a Management Client for Group Address Table management and Group Object Association Table management.

Octets 4 to 10 shall be reserved and shall have the value 00h.

Octet	1	2	3	4	5	6	7	8	9	10
Field	Structure Indication	Management Model	BiBat Feature	Table Management	Reserved					
Value	02h				00h	00h	00h	00h	00h	00h

Field “Management Model” (octet 2)

The datatype of this field shall be an enumeration (Datatype N₈).

value	Management Model	Remark
0	reserved	
1	Property-based implementation independent management	System 300, LTE Device
2	RF S-Mode	S-Mode Management for bidirectional RF device according [10], clause 2.4 “RF bidirectional devices”
3 to 255	reserved	

Field “BiBat Feature” (octet 3)

The datatype of this field shall be a bitset (Datatype B₈).

b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
reserved 0b	reserved 0b	reserved 0b	BiBat FastACK	BiBat LongHeader	BiBat Retransmitter	BiBat BiBat Slave	BiBat Master

Combined BiBat features are possible. Table 17 shows the allowed combinations. Other combinations are not allowed.

⁷⁾ ETS: application programs created for use with the system 300 system plug-in.

⁸⁾ ETS: application programs created for use with the native ETS-built-in support for system 300 devices.

Table 17 – Allowed values of the field BiBat Feature

Field name						Description
reserved b ₇ to b ₅	FastACK b ₄	LongHeader b ₃	Retransm b ₂	BiBat Slave b ₁	BiBat Master b ₀	
000b	0	0	0	0	1	BiBat Master, no support of fast ACK
000b	1	0	0	0	1	BiBat Master, support of fast ACK
000b	0	0	0	1	0	pure BiBat Slave
000b	1	0	0	1	0	BiBat Slave expecting feedback to own action
000b	0	1	0	0	0	BiBat Alarm device, long header, asynchronous only
000b	0	1	0	1	0	BiBat Alarm device, long header, supporting synchronous comm. as BiBat Slave
000b	0	0	1	0	0	BiBat Retransmitter

Field “Table Management” (octet 4)

The encoding shall be identical as the encoding for [Structure indication 01h](#).

4.3.16.4 Other formats/structures

Other formats shall not be used and are reserved by KNX Association for future extensions.

4.3.24 PID_PL110_PARAM (PID = 73)

- Property name: PL110 Parameters
- Property Datatype: PDT_GENERIC_01
- Datapoint Type: None.

This is a PL110 specific parameter for network management.

Table 18 - Base Configuration for PL110

Bit	7	6	5	4	3	2	1	0
Field name	reserved	Frequency definition		Repeater installed	reserved	Repeater mode	reserved	reserved
Coding or Value	1	11b: channel A 10b: reserved 01b: reserved 00b: channel B		0: enabled 1: disabled	1	0: device is Repeater 1: device is no Repeater	1	1

For frequency definition ‘channel’ means a set of two frequencies for the FSK (frequency shift keying) transmission of the PL110 devices.

When the Management Client changes the frequency definition of a device it will immediately loose connection when the new channel is set.

For the setting of the repeater flags the following table has to be applied:

Table 19 - Setting of repeater flags

Function	Flag “Repeater installed”	Flag “Repeater Mode”
- no repeater present in the installation - device itself is not repeater	1	1
not allowed	1	0
- repeater present in the installation - device itself is not repeater	0	1
- repeater present in the installation - device itself is repeater	0	0

4.3.25 PID_RECEIVE_BLOCK_TABLE (PID = 75)

- Property name: BiBat Receive Block Table
- Property Datatype: PDT_UNSIGNED_CHAR[16]
- Datapoint Type: None.

The Property BiBat Receive Block Table shall contain the activated receive blocks in the BiBat Slave, which are assigned by the BiBat Master. The BiBat Slave shall activate its receiver in the corresponding receive blocks.

The BiBat Master shall assign receive blocks to each BiBat Slave via a 128 bit table at configuration time and may alter this receive block bit table as long as the average number of receive blocks signalled by the BiBat Slave is not exceeded. The BiBat Master shall assign each such additional receive block to a single BiBat Slave, a group of BiBat Slaves or to all BiBat Slaves.

Any BiBat Slave shall be able to receive sync or data frames in the first time slot of block #0 and block #64 of each section, independent of the value of the Property BiBat Receive Block Table.

In the standard system the data length of a Property element is restricted to 10 octets. Therefore the table shall be encoded as an array Property with 16 elements (16 x 1 octet). The BiBat Master shall use two or more messages to write the complete Property BiBat Receive Block Table. Because of the segmented write access to the table, the BiBat Slave cannot check, whether the table data is consistent. The BiBat Master shall ensure complete download of the table.

Table 20 – Receive block Table**Data format**

Element 16	Element 15	Element 14	Element 13	Element 12	Element 11	Element 10	Element 9
Block # 127-120	Block # 119-112	Block # 111-104	Block # 103-96	Block # 95-88	Block # 87-80	Block # 79-72	Block # 71-64

Element 8	Element 7	Element 6	Element 5	Element 4	Element 3	Element 2	Octet 1
Block # 63-56	Block # 55-48	Block # 47-40	Block # 39-32	Block # 31-24	Block # 23-16	Block # 15-8	Block # 7-0

Element 16							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Block # 127	Block # 126	Block # 125	Block # 124	Block #123	Block #122	Block #121	Block #120
0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
.....							
Element 1							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Block # 7	Block # 6	Block # 5	Block # 4	Block #3	Block #2	Block #1	Block #0
0/1	0/1	0/1	0/1	0/1	0/1	0/1	1

Each of the 128 bits of the table shall represent a block. The mapping of the block# to the corresponding bit in the table is shown above.

If the bit is set to '1', the corresponding receive-block shall be active.

If the bit is set to '0', the corresponding receive-block shall be inactive.

The table fields for block #0 and blocks #64 shall always contain the value '1'.

The table shall be stored by the BiBat Slave in non-volatile memory.

4.3.26 PID_RANDOM_PAUSE_TABLE (PID = 76)

- Property name: BiBat Random Pause Table
- Property Datatype: PDT_UNSIGNED_CHAR[12]
- Datapoint Type: None.

Every BiBat Master shall maintain a unique table of 12 values for the length of 12 subsequent random pauses. It shall distribute this random pause table to all its BiBat Slaves during configuration. The table shall be derived from the unique 48 bit KNX Serial Number of the BiBat Master.

Each table element shall be encoded on 8 bit (transport format).

- The allowed range of each value is 0 to 15 (lower 4 bit) corresponding to a random pause length of 0 s to 0,937 5 s.
- The upper 4 bit of each table element are fixed 0000b.

In the BiBat Slave the table may be stored in a packed format (e.g. two 4 bit table values in one octet).

In the standard system the data length of a Property element is restricted to 10 octets. Therefore the table shall be encoded as an array Property with 12 elements (12 x 1 octet). The BiBat Master shall use two or more messages to write the complete Property BiBat Random Pause Table. Because of the segmented write access to the table, the BiBat Slave cannot check, whether the table data is consistent. The BiBat Master shall ensure complete download of the table.

The BiBat Slave shall store the table in non-volatile memory

To allow the BiBat Slave to resynchronise fully to the BiBat Master, the ‘Help Call Response’ message shall contain a random pause pointer value (0 to 11).

- Random pause pointer value 0 to 11: points to the corresponding element 1 to 12 of the random pause table (n+1)
- 13th random pause, fixed length 1 s (value 10h)

Datapoint format

Element 12	Element 11	Element 10	Element 9	Element 8	Element 7	Element 6	Element 5	Element 4	Element 3	Element 2	Element 1
1 octet	1 octet	1 octet	1 octet	1 octet	1 octet	1 octet	1 octet	1 octet	1 octet	1 octet	1 octet
0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx	0000xxxx

↑ Random Pause Pointer = 11 (example) Random Pause Pointer = 0 (example) ↑

4.3.27 PID RECEIVE BLOCK NR (PID = 77)

- Property name: BiBat Receive Block Number
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

A synchronous BiBat Slave may signal a higher receive frequency (= smaller reception period) to the BiBat Master at configuration time. The Property BiBat Receive Block Number shall contain the (average) number of additional receive blocks (0 to 126) within a 128 block section in addition to the minimum (default). The BiBat Master shall check the number of additional receive blocks during configuration and shall then assign receive blocks to each BiBat Slave via the 128 bit Property BiBat Receive Block Number. The BiBat Master may alter this Property BiBat Receive Block Number as long as the average number of receive blocks signalled by the BiBat Slave is not exceeded.

The Property is mandatory and shall be read-only.

Value 0 is allowed and shall mean “no additional receive-blocks”.

4.3.28 PID HARDWARE TYPE (PID = 78)

- Property name: Hardware Type
- Property Datatype: PDT_GENERIC_06
- Datapoint Type: None.

The Property PID `HARDWARE TYPE` in the Device Object shall identify the hardware.

PID_HARDWARE_TYPE shall contain a single value, encoded as specified below, and shall not use the array aspect of Property value (single array element Property). One hardware shall thus have only one value for PID_HARDWARE_TYPE.

Coding of PID_HARDWARE_TYPE

The high octet of this Property shall specify the format of the other five octets:

High octet	Meaning of following 5 octets	Usage
00h	Manufacturer specific device identification.	See Volume 6 ([17]) for the mandatory and optional use of PID_HARDWARE_TYPE..
Other values	To be defined	Reserved for future standard use.

This Property shall be an identifier for the device hardware. It shall specify the hardware of a device in the sense of a product group, i.e. the Management Client shall allow the download of one or more applications into the device with a specific value for Hardware Type.

The MSB shall be 00h. Other values for the MSB shall not be used. The definition of the MSB is under the supervision of the KNX Association System Group.

4.3.29 PID_RETRANSMITTER_NUMBER (PID = 79)

- Property name: BiBat Retransmitter Number
- Property Datatype: PDT_UNSIGNED_CHAR
- Datapoint Type: None.

The Property BiBat Retransmitter Number shall be assigned by the BiBat Master during configuration and shall define time delay of the Retransmitter.

Range: values 1 to 3 are allowed values that shall be assigned by the BiBat Master.

The default value in the Retransmitter at manufacturing time shall be 0. The value 0 shall indicate that the Retransmitter is unconfigured, this is, it is not active for synchronous communication

The Retransmitter shall store the value of the Property BiBat Retransmitter Number in non-volatile memory.

4.3.30 PID_SERIAL_NR_TABLE (PID = 80)

- Property name: KNX Serial Number Table
- Property Datatype: PDT_GENERIC_06[]
- Datapoint Type: DPT_SerNum (221.001)

4.3.30.1 Use

The KNX Serial Number Table Property shall contain the KNX Serial Numbers of the associated RF transmit-only devices.

Implementation of the KNX Serial Number Table is mandatory in RF bidirectional devices if the product has the ability to interwork with transmit-only devices.

Reading out of the KNX Serial Number Table by a Management Client, tool, like ETS, shall be possible. Setting of the KNX Serial Number Table by a Management Client, like ETS, is optional.

4.3.30.2 Format

The KNX Serial Number Table Property shall be an array of 6 octet KNX Serial Numbers. The table shall contain the KNX Serial Numbers of linked devices or void table elements as specified below.

Array Index	Value
1	1 st KNX Serial Number (6 Octets)
2	2 nd KNX Serial Number (6 Octets)
3	
...	...
N	...

Figure 9 – Structure of the KNX Serial Number Table

4.3.30.3 Usage by the Management Server (device)

4.3.30.3.1 Usage by LTE RF BD devices

The KNX Serial Number Table shall be generated by the device itself during linking with RF transmit-only devices.

- For each additional RF transmit-only partner device that is linked to this Management Server (device) the corresponding KNX Serial Number shall be added in the list (last element + 1). The table elements need not to be sorted.
- The maximal size of the table shall be fixed and product specific and the number of devices that can be associated is therefore limited.
- Ex-factory, the table shall be empty and void table elements shall contain the KNX Serial Number 000000000000h.
- There may be gaps in the table with void table elements.
- Handling of full table
 - In case of insufficient table size the handling can be manufacturer specific; it is recommended to generate an error indication and not to overwrite existing table elements.
- Deletion of individual table elements
 - The deletion of individual links is not foreseen in this standard but it shall be possible to reset the whole table by manufacturer specific means.
- Write access to the table shall be optional (e.g. by ETS)
- Individual table elements can be written. There is no table load state machine.
- The current length N of the table Property shall be readable by means of an A_PropertyDescription_Read-service.

4.3.30.3.2 Usage by BiBat Retransmitter

The Property in the BiBat Retransmitter shall at least contain the KNX Serial Number of the BiBat Master of which the BiBat Retransmitter shall retransmit the synchronous and asynchronous telegrams.

The KNX Serial Number of the BiBat Master shall be assigned to element 1 of the table.

NOTE 13 BiBat Retransmitters shall retransmit synchronous telegrams only with the Domain Address of the BiBat System or the KNX Serial Number of the BiBat Master. Selective retransmission/filtering (according to KNX Serial Numbers) of asynchronous telegrams from other devices is an optional feature of the BiBat Retransmitter.

In addition to the BiBat Master KNX Serial Number, the table may contain the KNX Serial Numbers of other devices for selective retransmission of asynchronous telegrams.

Rule for selective retransmission and filtering

- a) If the Property KNX Serial Number Table contains only one element (i.e. the KNX Serial Number of the BiBat Master) then
- asynchronous telegrams containing any KNX Serial Number shall systematically be retransmitted; and
 - asynchronous telegrams containing the Domain Address of the BiBat System shall be retransmitted; and
 - synchronous telegrams with the corresponding KNX Serial Number of the BiBat Master shall be retransmitted, all other synchronous telegrams shall be discarded.
- b) If the Property KNX Serial Number table contains more than one element then
- asynchronous telegrams containing the KNX Serial Number of the sender shall be filtered according to the Property KNX Serial Number Table; and
 - asynchronous telegrams containing the Domain Address of the BiBat System shall be retransmitted; and
 - synchronous telegrams with the KNX Serial Number of the BiBat Master shall be retransmitted, all other synchronous telegrams shall be discarded.

The Property KNX Serial Number table shall be set by the BiBat Master at configuration-time and shall be stored in non-volatile memory.

The Property KNX Serial Number Table shall be an array of 6 octet KNX Serial Numbers. The table shall contain KNX Serial Numbers used by the retransmitter for filtering or void table elements as specified below.

- The size of the table shall be fixed and product specific and the number of devices that can be associated shall therefore be limited.
- The length of the table Property shall be readable by means of A_PropertyDescription_Read.
- The BiBat Master shall check the maximum size of the table using A_PropertyDescription_Read before writing the table.
- The table shall have no load state machine. Therefore all elements shall be written by the BiBat Master.
- The table elements need not to be sorted. Element 1 shall however contain the KNX Serial Number of the BiBat Master.
- Ex-factory, the table shall be empty and void table elements shall contain the KNX Serial Number 000000000000h.
- There may be “gaps” in the table with void table elements.
 - Erasing individual table elements is possible by writing the KNX Serial Number 000000000000h to the corresponding element.
- Write access to the table is mandatory in the BiBat retransmitter.

Array Index	Value
1	1 st KNX Serial Number (6 octets): of the BiBat Master
2	2 nd KNX Serial Number (6 octets): of any other device
3	3 rd KNX Serial Number (6 octets): of any other device
...	...
N	...

Figure 10 – Encoding of the KNX Serial Number Table

4.3.31 PID_BIBAT_MASTER_ADDRESS (PID = 81)

- Property name: BiBat Master Individual Address
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

The Property BiBat Master Individual Address shall contain the Individual Address of the BiBat Master. The BiBat Slave needs the BiBat Master's Individual Address as Destination Address in the help call.

This Property shall be set by the BiBat Master at configuration-time.

The BiBat Slave shall store the value in non-volatile memory.

4.3.32 PID_RF_DOMAIN_ADDRESS (PID = 82)

- Property name: RF Domain Address
- Property Datatype: PDT_GENERIC_06
- Datapoint Type: None.

4.3.32.1.1 PID_RF_DOMAIN_ADDRESS in the Device Object of the cEMI Server

The Property PID_RF_DOMAIN_ADDRESS shall contain the value of the Domain Address of the cEMI Server that shall be used for filtering or that shall be inserted in the transmitted RF frame if the L_Data.req contains the Domain Address 000000000000h; see [12] clause 4.1.4.3.2 "RF medium information".

NOTE 14 This is a different Property than PID_DOMAIN_ADDRESS (PID = 70) that is used on the Powerline communication medium and that has a size of only two octets.

PID_RF_DOMAIN_ADDRESS shall be stored in non-volatile memory.

4.3.33 PID_DEVICE_DESCRIPTOR (PID = 83)

- Property name: Device Descriptor
- Property Datatype: PDT_GENERIC_02
- Datapoint Type: None.

This Property shall contain the Device Descriptor Type 0 of the device.

The format and coding shall be identical to what is specified in clause 4.1.2.

The value of this Property shall be identical to the value of the device's Device Descriptor Type 0 value as accessed though any other method, such as (not exclusive)

- the Device Descriptor Type 0 value provided using the dedicated Application Layer service A_DeviceDescriptor_Read. For use please refer to [09], e.g. Management Procedures NM_IndividualAddress_Write, NM_IndividualAddress_SerialNumber_Write2, NM_DomainAndIndividualAddress_Write, NM_DomainAndIndividualAddress_Write2, NM_IndividualAddress_Check, DMP_Connect_RCo, DMP_Connect_RCl, etc.

4.3.34 PID_GROUP_TELEGR_RATE_LIMIT_TIME_BASE (PID = 85)

- Property name: group telegram rate limitation time base
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

The "group telegram rate limitation time base" shall be a 2 octet unsigned value. Its unit shall be 1 ms. So time bases from 1 ms to 65 535 ms can be encoded.

EXAMPLE A time base value of 10 000 shall indicate a time base of 10 s.

4.3.34.1 Usage by the Management Server (device)

Within “group telegram rate limitation time base” the device shall at maximum transmit a number of frames equal to the “group telegram rate limitation number of telegrams”. If the “group telegram rate limitation number of telegrams” is set to 1 the time base shall indicate the minimum interframe time between any two group telegrams.

If the time base value is set to zero then the telegram rate limitation shall be disabled independently of the value of “group telegram rate limitation number of telegrams”.

Table 21 - Group telegram rate limitation time base values

Value	Meaning
0	Group telegram rate limitation shall be disabled. The system shall not limit the number of telegrams that may be transmitted within any period.
1 to 65 535	Time Base Value multiplied with 1 ms

PID_GROUP_TELEGR_RATE_LIMIT_TIME_BASE shall always be implemented together with PID_GROUP_TELEGR_RATE_LIMIT_NO_OF_TELEGR. For further specifications and requirements, please refer to clause 4.3.35.

4.3.34.2 Usage by the Management Client

Please refer to clause 4.3.35.

4.3.35 PID_GROUP_TELEGR_RATE_LIMIT_NO_OF_TELEGR (PID = 86)

- Property name: group telegram rate limitation number of telegrams
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

4.3.35.1 Usage by the Management Server (device)

The “telegram rate limitation number of telegrams” shall be a 2 octet unsigned value for the number of telegrams that shall be sent at maximum by a device within “telegram rate limitation time base”.

Valid data for “telegram rate limitation number of telegrams” are 0 to 65 535.

If the number of telegrams value is set to zero telegram rate limitation shall be disabled independently from the “telegram rate limitation time base” value.

Table 22 - Group telegram rate limitation number of telegrams values

Value	Meaning
0	Telegram rate limitation is disabled. The system shall not limit the number of telegrams that may be transmitted within any period
1 to 65 535	Number of group telegrams to be transmitted at maximum within “telegram rate limitation time base”.

Group telegram rate limitation functionality via Properties is an optional feature for devices. But if implemented both Properties “group telegram rate limitation time base” and “group telegram rate limitation number of telegrams” shall be implemented.

Both Properties shall be readable and at least one of them shall be writeable.

If “group telegram rate limitation number of telegrams” is set to 1 the interframe time mechanism shall be active. If “group telegram rate limitation number of telegrams” is greater than 1 the number of group telegrams within the time base shall be counted and limited.

4.3.35.2 Usage by the Management Client

Normal conditions

The Properties “group telegram rate limitation time base” and “group telegram rate limitation number of telegrams” shall be accessed through the Property mechanisms DMP_InterfaceObjectWrite_R, DMP_InterfaceObjectVerify_R and DMP_InterfaceObjectRead_R.

Management telegrams are not limited by the group telegram rate limitation functionality.

Error and exception handling

If the Management Server does not support the group telegram rate limitation Properties, Property description read access will fail.

In case the write access to the Properties “group telegram rate limitation time base” or “group telegram rate limitation number of telegrams” fails, then the Management Client shall skip these accesses and proceed with the rest of the download procedure.

4.3.36 PID_FEATURES_SUPPORTED (PID: 89)

- **Property name:** Features Supported
- **Property Datatype:** PDT_GENERIC_10
- **Datapoint Type:** None.
- **Access:**

Service:		Role:	Security:	Access level:
Data:	Read:	any	none	3
	Write:	any	none	1
NwPar:	R	any	none	n/a

List: This Property shall be on the Security White List

4.3.36.1 Abstract Resource definition

PID_FEATURES_SUPPORTED shall be used to indicate the functionality of a device.

4.3.36.2 Format and encoding

PID_FEATURES_SUPPORTED shall be of Property Datatype PDT_GENERIC_10 and each octet shall be a bit field.

Each bit shall define a group of features or services. If a bit is set all services shall be supported by the device.

Unused bits shall be cleared (set to 0).

The following Groups are defined: 10 octets grouping the APCIs.

Octet / BitNr.	Bit Nr.	Service Group	Supported Services / Resource	Alternative discovery
0/0	0	Property Multicast services	A_GroupPropValue_.. / T-Data-Tag	Read Device ID on LTE-Broadcast (Mandatory?)
0/1	1	Extended Properties	A_PropertyExtValue... A_PropertyExtDescription... A_FunctionPropertyExt...	Only with time Out.
0/2	2	Confirmed Restart	A_Restart (Restart Type 1)	no
0/3	3	Extended Memory Services	A_Memory_Extended...	
0/4	4	Verify Mode	PID_SERVICE_CONTROL	Try to set bit in PID_-SERVICE_CONTROL

Octet / BitNr.	Bit Nr.	Service Group	Supported Services / Resource	Alternative discovery
0/5	5	File Stream	A_FileStream_InfoReport	
0/6	6	Authorize (Real support)	A_Authorize_Request A_Key_Write	
0/7	7	Link Services	A_Link_Read, A_Link_Write	
1/0	8	Secure AL	A_Secure	
1/1	9	(any) E-Mode	A_DeviceDescriptor_Read (Type 2)	Further discovery via Device Descriptor Type 2
1/2	10	Configuration Signature	PID_DOWNLOAD_-COUNTER	
1/3	11	Ext. frames (Long)	PID_MAX_APDU_LENGTH is present and value is >15	Read PID_MAX_APDU_LENGTH
1/4	12	TL Connection Oriented	T_Data_Connected...	Try T_Connect and read the Device Descriptor
1/5	13	TL New Style (NOA)	T_..._Noa..	
1/6	14	Group object diagnostic	PID_GO_DIAGNOSTICS	
1/7	15	Local Download Support (IP)	Local TL via IP	Discovery via IP
2/0	16	Local Download Support (USB)	Local TL via USB	

NOTE 15 The following features have been considered for future extension, but are not yet included.

- Data processing
- Firmware Update
- Bus voltage measuring

NOTE 16 The following features have been considered as well, but are not included as there is no use case seen.

- Full Property Support
- System Network parameter
- Network parameter
- Hardware Type

4.3.37 PID_CHANNEL_01_PARAM (PID = 101) to PID_CHANNEL_32_PARAM (PID = 132)

- Property name: “E-Mode parameters for channel 01” to “E-Mode parameters for channel 32”
- Property Datatype: PDT_GENERIC_01[]
- Datapoint Type: None.

These Properties of the Device Object shall provide the parameter blocks for Channel 1 to 32 of the E-Mode Device. The value shall be an array of octets. Its content is defined in the specification of the Channel by the Application Groups. The actual length is static, and depends on the corresponding Channel Code.

Error handling

A negative response (no data) shall be sent each time one of following events appears in the request:

- unknown channel (unknown Property_identifier, channel_number ≥ number of channels), or
- unknown parameter (start_index ≥ param_block size), or
- incompatible size (start_index+nb_of_elements > param_block size +1).

4.4 Applicationprogram Object (Object Type 3)

4.4.1 PID_OPERATION_MODE (PID: 52)

- **Property name:** Operation Mode
- **Property Datatype:** PDT_FUNCTION (alt.: PDT_VARIABLE_LENGTH)
- **Datapoint Type:** None.
- **Access policy:** 15F/00C
- **Access level:** 3/3

4.4.1.1 Abstract Resource Definition

The Operation Mode shall serve to set and read the behaviour of the application program. This shall mainly serve diagnostic purposes. The default Operation Mode shall be “Normal Mode”.

The Operation Mode shall only have effect if the Load State is “Loaded” and the run state is “Running”. The Operation Mode shall not be an alternative to the Run State.

Definition of the Operation Modes

1. Normal Mode

- a. The application shall be running unchanged; the application shall have access to the Group Object value and the Group Object runtime flags.
- b. The AL-service primitives shall be handled by the Group Object server, this is, the GO can be read - and written by the A_GroupValue_Read and –Write-services.
- c. Some Group Object Diagnostic commands shall not be accepted, as listed in Table 34.

2. Diagnostic Mode

This shall allow the MaC to make the application is explicitly aware of its influence.

- a. The application may be running unchanged or may be halted; but the application shall have no access to the Group Object value or the Group Object runtime flags.

NOTE 17 This shall prevent that a GO-value that is set by the MaC is immediately changed by a (coincidental) subsequent access by the application, or that a request by the MaC to send a certain GO-value is immediately overwritten by a coincidental spontaneous immediate next transmission by the application. It shall also prevent that the application reads a received updated value of a GO before the MaC has had the time and possibility to read the GO runtime flags.

- b. The Group Objects shall not be updated by the communication stack by any sender other than the one(s) possibly filtered by the MaC.

NOTE 18 This shall prevent that, if the MaC forces a device 1 to send a value on a GA 1, received by a device 2, that this value is overwritten by a coincidental next spontaneous transmission by a device 3; if this would not be the case, then the MaC has no possibility to conclude whether the communication between device 1 and device 2 would be working properly or not.

Constraints

- Only one single Operation Mode is foreseen. There exist KNX devices with a second Application Program (Application Program 2), which is however controlled (started and stopped) by the first Application Program and which shall follow the Operation Mode of the Application Program 1. If it is needed later, an Operation Mode can be specified for the Application Program 2 as well. This is under control by KNX Association.
- In combined devices, the Operation Mode shall be restricted to the executable part that is controlled by the Application Program Object and not to any other applicative or other layer functionality.

EXAMPLE 8 In a device that would combine an Application Program with the functionality of a (USB) Interface or a Coupler, that Interface functionality or routing functionality shall not be influenced by the Operation Mode of the Application Program.

- The MaS does not announce in any way to its communication partners on its current or changed Operation Mode.

4.4.1.2 Format and encoding

The *Operation Mode* shall be a Function Property that shall be encoded as follows.

4.4.1.2.1 Write (A_FunctionPropertyCommand-PDU)

Overview Write ServiceIDs

Table 23 – Overview PID_OPERATION_MODE WriteServiceIDs

WriteServiceID	Definition
00h	Write the Operation Mode

WriteServiceID 00h: Write the Operation Mode

octet 10	octet 11	octet 12
Reserved	WriteServiceID	ServiceInfo
	Write Operation Mode	Operation Mode
00h	00h	See below.

Figure 11 – A_FunctionPropertyCommand-PDU for WriteServiceID 00h for PID_OPERATION_MODE

The field *Operation Mode* shall contain the Operation Mode requested by the MaC and shall be encoded as an 8 bit enumeration (N_8) as specified in Figure 12.

The MaC shall use this command to set the Operation Mode.

ServiceID: 00h Command “Write the Operation Mode”.

ServiceInfo: Operation Mode

Value	Operation Mode
0	Normal Mode
1	Diagnostic Mode

Figure 12 – Encoding of Operation Mode

If the MaS has any problem then it shall respond as specified in 4.4.1.3 below.

If the MaS can successfully handle the command then it shall set the Operation Mode as requested and respond with an A_FunctionPropertyState_Response-PDU as in Figure 13. Only the Specific Positive Return Code E_OM_-CURRENT_-OPERATION_-MODE is allowed. The Return Code E_SUCCESS shall not be used.

octet 10	octet 11	octet 12	octet 13
Return Code	ServiceID	ServiceResult	
E_OM_- CURRENT_- OPERATION_- MODE	WriteServiceID	Operation Mode	Time Left
20h	00h	See below.	See below.

Figure 13 - A_FunctionPropertyState_Response-PDU for WriteServiceID 00h for PID_OPERATION_MODE

Operation Mode: This shall be the current Operation Mode of the MaS after the evaluation of the command. This value may differ from the value in the request.

Time Left: This shall be the remaining time of the current Operation Mode. This shall be a value in the range from 0 s to 254 s.

If the Operation Mode does not time-out, then the MaS shall return the value 255 s (FFh). This is typically the case for the *Normal Mode*.

4.4.1.2.2 Read (A_FunctionPropertyState_Read-PDU)

Overview ReadServiceIDs

Table 24 – Overview PID_OPERATION_MODE ReadServiceIDs

ReadServiceID	Definition
00h	Read the Operation Mode

ReadServiceID 00h: Read the Operation Mode

octet 10	octet 11
Reserved	ReadServiceID
	Read Operation Mode
00h	00h

Figure 14 - A_FunctionPropertyState_Read-PDU for ReadServiceID 00h for PID_OPERATION_MODE

The MaC shall use this ReadServiceID 00h to read the current *Operation Mode*. There is no field ServiceInfo for this ReadServiceID 00h.

If the MaS has any problem then it shall respond as specified in 4.4.1.3 below.

If the MaS can successfully handle the command then it shall respond with an A_FunctionPropertyState_Response-PDU as in Figure 13 containing its current *Operation Mode* and the *Time Left*. Only the Specific Positive Return Code E_OM_-CURRENT_-OPERATION_-MODE is allowed. The Return Code E_SUCCESS shall not be used.

4.4.1.3 Common error – and exception handling for PID_OPERATION_MODE

Only Specific Negative Return Codes are allowed. E_ERRORS (FFh) shall not be used.

A0h: E_OM_ERROR

octet 10	octet 11	octet 12	octet 13
Return Code	ServiceID	ServiceResult	
		Operation Mode	Time Left
A0h	Read- or WriteServiceID as in the request		

Figure 15 – A_FunctionPropertyState_Response-APDU for PID_OPERATION_MODE in case of E_OM_-CURRENT_-OPERATION_-MODE

The fields Operation Mode and Time Left shall have the same meaning as in E_OM_-CURRENT_OPERATION_MODE.

4.4.1.4 Usage by the MaS (device)

- The MaS shall support the *Operation Mode* as specified in 4.4.1.2.
- The MaS may autonomously reset its *Operation Mode* back to “Normal Mode”. If the *Operation Mode* times out, then the following shall apply.
 - After each write access to the *Operation Mode*, the time-out shall at least be 30 s.
 - When the *Operation Mode* times-out, then the new *Operation Mode* after this time-out shall be *Normal Mode*.

Communication Mode and Transport Layer connections

The MaS shall accept access to *Operation Mode* both connectionless as well as connection-oriented. The *Operation Mode* shall be independent of the Transport Layer connection and shall not be influenced when the Transport Layer connection is opened or closed. Access to the *Operation Mode* shall not be limited to the MaC with which there may be an open Transport Layer connection; it may only be limited through Permissions and Roles.

4.4.1.5 Usage by the MaC

- The MaC shall access the *Operation Mode* as specified in 4.4.1.2.
- The MaC may at any time read the current *Operation Mode* and its *Time Left* using an A_Function-PropertyState_Read-PDU.
- If the MaS returns a *Time Left* different from 255 s and the MaC needs the requested *Operation Mode* for a longer time, then the MaC may retrigger the same *Operation Mode* as many times as necessary each time before it expires.

Communication Modes and Transport Layer connections

The MaC may address *Operation Mode* both connectionless as well as connection-oriented.

4.5 Router Object (Object Type 6)

4.5.1 General requirements

For the Coupler, all Properties of this Interface Object shall be used both in Line Coupler mode as well as in Backbone Coupler mode and as well in TP1 Bridge/Repeater Mode.

4.5.2 PID_OBJECT_TYPE (PID = 1)

4.5.2.1 General requirements

Please refer to clause 4.2.1 for the general requirements of PID_OBJECT_TYPE.

This Property shall contain Object Type of the Interface Object. The Router Object shall have Object Type 0006h.

4.5.2.2 Usage by the Management Server (device)

PID_OBJECT_TYPE in the Router Object shall support the Data Property services as specified in clause 3.4.3 “Data Property services” of [07] without restrictions.

The Coupler shall additionally support the NM_NetworkParameter_Read_R as follows.

On reception of an A_NetworkParameter_Read-PDU with Object Type = Router Object and PID = PID_OBJECT_TYPE and test_info = 00h, the Coupler shall respond with an A_NetworkParameter_Response-PDU with the same service parameter values and with test_result equal to the Property Value, this is, the Interface Object Type.

This response shall be sent in broadcast communication mode.

The Coupler is in this case addressed itself. Therefore, its handling of the hop_count_type shall be as specified in [05] clause 2.4.1 “State Machine of Network Layer for normal devices”, this is, when handling a service indication primitive with hop_count_type equal to 7, respectively different from 7, it shall in the response primitive use the hop_count_type 7 respectively “Network Layer Parameter”.

- The response shall be transmitted on the Medium Interface on which the request has arrived.
- Additionally, the response may be transmitted also on other Medium Interfaces of the Coupler.

On reception of an A_NetworkParameter_Read-PDU with Object Type = Router Object and PID = PID_OBJECT_TYPE and test_info ≠ 00h, the Coupler shall not react.

Error and exception handling

- The above applies for any Coupler regardless of the Coupler model (see clause 6.3 in [04]: KNX Router, KNX TP1 Bridge or KNX TP1 Repeater). The Management Client thus needs to interpret the Individual Addresses of the responses to conclude on the use of the present Couplers.

4.5.2.3 Usage by the Management Client

The Management Client may read PID_OBJECT_TYPE with A_NetworkParameter_Read with hop_count value 0 –to find the nearest Couplers – as well as with value 7.

Please refer to the specification of NM_NetworkParameter_Read in [09] concerning the use of hop_count value 7.

4.5.3 PID_LOAD_STATE_CONTROL (PID = 5)

The Load State Machine in the Router Object shall control the access to the standard Routing Table.

PID_LOAD_STATE_CONTROL in the Router Object shall support the Load Events as listed in Table 25. For the specification and numerical values of these Load Events, please refer to clause 4.23.2 “Load State Machine – Realisation Type 1 (Property based)”.

Table 25 – Load Events for PID_LOAD_STATE_CONTROL in the Router Object

Load Events
No operation
Start Loading
Load Completed
Unload

The Load States as specified in Table 73 may be returned. For the specification and numerical values of these Load Events, please refer to clause 4.23.2 “Load State Machine – Realisation Type 1 (Property based)”.

Table 26 – Load States returned by PID_LOAD_STATE_CONTROL in the Router Object

Load State
Unloaded
Loaded
Loading
Error

If the Group Address Routing Table is to be checked on received frames and the Load State is not *Loaded* at the same time, no frames shall be routed.

Upon the event *Unload* the State Machine shall transit to the state *Unloaded*. It shall not be relied on the fact that the Routing Table is erased in memory. PID_ROUTETABLE_CONTROL serves to erase the Routing Table in memory.

The default Load State shall be *Loaded* and the Routing Table cleared.

4.5.4 PID_MEDIUM_STATUS (PID = 51)

- **Property name:** Medium Status
- **Property Datatype:** PDT_GENERIC_01
- **Datapoint Type:** None.
- **Access levels:** 3/X
- **Access Policy:** 3FF/1FF

NOTE 19 This Property was named PID_LINE_STATUS before.

This Property shall include the status of the communication medium to which this medium interface communicates.

NOTE 20 Bit 0 was named POWER_DOWN_SUBLINE before. This is now named COMMUNICATION_IMPOSSIBLE.

Bit	Name	Description	Coding
0	COMMUNICATION_IMPOSSIBLE	Indicates whether or not communication is possible using the medium connection represented by this Router Object.	0: FALSE: communication is possible 1: TRUE: communication is impossible
1 to 7		reserved	Shall be 0.

The Coupler shall distribute the value of PID_MEDIUM_STATUS of one Medium Interface to all other Medium Interfaces with the Management Procedure NM_NetworkParameter_Write_R under the control of the field EN_SUBLINE_STATUS of PID_COUPL_SERV_CONTROL, with Object Type= Router Object and PID = PID_MEDIUM_STATUS in broadcast communication mode; this single transmission shall be triggered by a change of value of this Property.

```

/* Announce the status of the medium on one Medium Interface through the remaining Medium Interface. */
For each Medium Interface other than the initiating Medium Interface {
    if PID_COUPL_SERV_CONTROL.EN_SUBLINE_STATUS = enable then
        NM_NetworkParameter_Write_R(ASAP = void, comm_mode = point-to-all-points connectionless,
            hop_count_type_req = 6, object_type = Router Object, PID = PID_MEDIUM_STATUS,
            value = PID_MEDIUM_STATUS.Value)
}

```

It shall be noted that this message does not say what Medium Interface has a problem: the Primary Side or the Secondary Side. This would even become more imprecise if ever multiple Secondary Sides were re-introduced again. However, even the use of the Object Instances or Object Indexes would not help, as there are no requirements to these! In broadcast; this message thus only says that one Medium Interface of this Coupler with IA x.y.z has a problem. (In point-to-point, reading out by the MaC, knowing the relation between Medium Interface and Object Index or – Type, this does become clear.)

4.5.5 PID_MAIN_LCCONFIG (PID = 52) PID_SUB_LCCONFIG (PID = 53)

Two Properties (PID_MAIN_LCCONFIG for the Subnetwork on the primary side and PID_SUB_LCCONFIG for the Subnetwork on the secondary side) shall affect the handling of frames in point-to-point (connectionless and connection oriented) or broadcast communication mode.

Bit	Name	Description	Encoding (d = default value)
0-1	PHYS_FRAME	Specifies the static handling for receiving frames in point-to-point connectionless or connection-oriented communication mode	0 = not used 1 = PHYS_UNLOCK: all frames in point-to-point connectionless and point-to-point connection-oriented communication mode shall be routed, independent of the Coupler Mode! 2 = PHYS_LOCK: no frames in point-to-point connectionless and point-to-point connection-oriented communication mode shall be routed, independent of the Coupler Mode! 3 = PHYS_ROUT: routing of frames in point-to-point connectionless and point-to-point connection-oriented communication mode shall depend on Destination Address, the Individual Address of the Coupler and the Coupler Mode (normal operation mode) (d)

Bit	Name	Description	Encoding (d = default value)
2	PHYS_REPEAT	Repetition of frames in point-to-point connectionless or connection-oriented communication mode in case of transmission errors.	Independent of the Coupler Mode! 0 = no repetitions 1 = frames in point-to-point connectionless or connection-oriented communication mode will be repeated in case of transmission errors (up to 6 times for BUSY acknowledged frames and up to 3 times for other acknowledged or not acknowledged frames) (d)
3	BROADCAST_LOCK	switch ON or OFF the routing of frames in broadcast communication mode	independent of the Coupler Mode: 0 = normal: frames in broadcast communication mode will be routed (d) 1 = frames in broadcast communication mode will be blocked (useable during system configuration)
4	BROADCAST_REPEAT	repetition of frames in broadcast communication mode in case of transmission errors	independent of the Coupler Mode: 0 = no repetition 1 = frames in broadcast communication mode will be repeated in case of transmission errors (up to 6 times for BUSY acknowledged frames and up to 3 times for other acknowledged or not acknowledged frames) (d)
5	GROUP_IACK_ROUT	specifies the Layer-2 acknowledge of received frames in multicast communication mode. Switch ON or OFF the Layer-2 acknowledge.	independent of the Coupler Mode: 0 = all multicast frames will be acknowledged, independently of the routing! (useful only to avoid the repetitions of misrouted frames) 1 = normal mode (all multicast frames that will be routed will also be acknowledged) (d)
6-7	PHYS_IACK	specifies the Layer-2 acknowledge of received frames in point-to-point connectionless or – connection-oriented communication mode. Switch ON or OFF the Layer-2 acknowledge.	independent of the Coupler Mode: 0 = not used 1 = normal mode (all frames that will be routed or that are addressed to the Coupler itself will be acknowledged) (d) 2 = all frames will be acknowledged (useful only to avoid the repetitions of misrouted frames) all frames on point-to-point connectionless – or connection-oriented communication mode shall be negatively acknowledge (NACK). This shall serve for protection purposes. (It is useful to prevent all parameterisation in one Subnetwork; the Coupler shall be protected too. A typical use case is the protection of a Subnetwork that is located outside a building) 3 =

4.5.6 PID_MAIN_LCGRPCONFIG (PID = 54) PID_SUB_LCGRPCONFIG (PID = 55)

Two Properties (PID_MAIN_LCGRPCONFIG for the Subnetwork on the primary side and PID_SUB_LCGRPCONFIG for the Subnetwork on the secondary side) shall affect the handling of frames in multicast communication mode.

These values shall be interpreted independently of the Coupler Mode.

Bit	Name	Description	Encoding (d = default value)
0-1	GROUP_6FFF	specify the handling of standard Group Addressed frames with Group Address ≤ 6FFFh	0 = not used 1 = GROUP_UNLOCK6FFF All standard Group Addresses frames shall be routed. (Typical for TP1 Bridge/Repeater Mode.) 2 = GROUP_LOCK6FFF No standard Group Addressed frames shall be routed. 3 = GROUP_ROUT6FFF The Routing of the standard Group Addressed frames shall depend on the Routing Table (d).

Bit	Name	Description	Encoding (d = default value)
2-3	GROUP_7000	specify the handling of group addressed frames $\geq 7000h$	0 = not used 1 = GROUP_UNLOCK7000 All frames shall be routed. (Typical for TP1 Bridge/Repeater Mode.) (d) 2 = GROUP_LOCK7000 No frames shall be routed. 3 = GROUP_ROUT7000 Routing shall depend on the Routing Table.
4	GROUP_REPEAT	repetition of group addressed frames in case of transmission errors	0 = No repetition. 1 = Frames shall be repeated in case of transmission errors (up to 6 times for BUSY acknowledged frames and up to 3 times for other acknowledged or not acknowledged frames). (d)
5-7		not used	Shall be 0.

4.5.7 PID_ROUTETABLE_CONTROL (PID = 56)

4.5.7.1 Structure

This Property shall be used for the Management of the Routing Table for standard KNX Group Addresses. Due to the Property Datatype PDT_Function the configuration is much faster and more flexible than the standard access to a fixed routing address field. The structure of the Property is defined in Figure 16.

octet 10	octet 11	octet 12	octet 13 to octet 21
00h or return_code	ServiceID	ServiceInfo1	ServiceInfo2 ... ServiceInfo11

**Figure 16 - Structure of the Function Property for reading or writing
(octet numbering for TP1)**

The ServiceID shall specify the selected method. For some methods additional ServiceInfos shall be necessary.

4.5.7.2 ServiceID: 1 (SRVID_CLEAR_FILTERTABLE)

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11
00h	01h

For SRVID_CLEAR_FILTERTABLE no additional ServiceInfo fields shall be used.

The Management Client shall apply SRVID_CLEAR_FILTERTABLE with A_FunctionPropertyCommand-PDU to clear all standard Group Addresses in the Routing Table.

The Coupler shall on reception of SRVID_CLEAR_FILTERTABLE clear all standard Group Addresses in the Routing Table and shall respond with an A_FunctionPropertyState_Response-PDU as follows.

Response (A_PropertyPropertyState_Response-PDU)

octet 10	octet 11
return_code	01h

The Coupler shall use the following values as return_code.

return_code: 00h: clearing of the Routing Table has been successful.
 FFh: an error has occurred during the clearing of the Routing Table (verify error)

b) Read (A_FunctionPropertyState_Read-PDU)

The Management Client shall apply the SRVID_CLEAR_FILTERTABLE with A_FunctionPropertyState_Read-PDU to check if all standard Group Addresses are cleared in the Routing Table.

The Coupler shall use the following values as return_code.

return_code: 00h: all standard Group Addresses in the Routing Table are cleared.
 FFh: not all standard Group Addresses in the Routing Table are cleared.

4.5.7.3 ServiceID: 2 (SRVID_SET_FILTERTABLE)

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11
00h	02h

For SRVID_SET_FILTERTABLE no additional ServiceInfo fields shall be used.

The Management Client shall apply SRVID_SET_FILTERTABLE with A_FunctionPropertyCommand-PDU to set all standard Group Addresses in the Routing Table.

The Coupler shall on reception of SRVID_SET_FILTERTABLE set all standard Group Addresses in the Routing Table and shall respond with an A_FunctionPropertyState_Response_PDU as follows.

Response (A_PropertyPropertyState_Response-PDU)

octet 10	octet 11
return_code	02h

The Coupler shall use the following values as return_code.

return_code: 00h: setting of the Routing Table has been successful.
 FFh: an error has occurred during the setting of the Routing Table (verify error)

b) Read (A_FunctionPropertyState_Read-PDU)

The Management Client shall apply the SRVID_SET_FILTERTABLE with A_FunctionPropertyState_Read-PDU to check if all standard Group Addresses are set in the Routing Table.

The Coupler shall use the following values as return_code.

return_code: 00h: all standard Group Addresses in the Routing Table set.
 FFh: not all standard Group Addresses in the Routing Table are set.

4.5.7.4 ServiceID: 3 (SRVID_CLEAR_GROUPADDRESS)

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
00h	03h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Management Client shall apply SRVID_CLEAR_GROUPADDRESS to clear one or more standard Group Addresses in the Routing Table of the Coupler.

START ADDRESS This shall be the first standard Group Address that shall be cleared.

END ADDRESS This shall be the last standard Group Address that shall be cleared.

If only one standard Group Address is to be modified then START ADDRESS and ENDADDRESS shall be equal!

If START ADDRESS is greater than END ADDRESS no address will be modified (error response!)

The Coupler shall on reception of SRVID_CLEAR_GROUPADDRESS clear all Standard Group Addresses in the Routing Table starting from START ADDRESS up to and including END ADDRESS. It shall respond with an A_FunctionPropertyState_Response-PDU as follows.

Response (A_PropertyPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
return_code	03h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Coupler shall use the following values as return_code.

return_code: 00h: The standard Group Addresses are cleared successfully.

FFh: One or more errors have occurred during the clearing of the standard Group Addresses (verify error).

b) Read (A_FunctionPropertyState_Read-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
00h	03h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Management Client shall apply the SRVID_CLEAR_GROUPADDRESS with A_FunctionPropertyState_Read-PDU to check if all standard Group Addresses in the range from START ADDRESS to END ADDRESS are cleared.

START ADDRESS This shall be the first standard Group Address that shall be checked.

END ADDRESS This shall be the last standard Group Address that shall be checked.

If only one standard Group Address shall be checked then START ADDRESS and END ADDRESS shall be equal!

If START ADDRESS is greater than END ADDRESS then no standard Group Address shall be checked.

Response (A_PropertyPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
return_code	03h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Coupler shall use the following values as return_code.

return_code: 00h: all checked standard Group Addresses are cleared.

FFh: not all checked standard Group Addresses are cleared.

4.5.7.5 ServiceID: 4 (SRVID_SET_GROUPADDRESS)

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
00h	04h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Management Client shall apply SRVID_SET_GROUPADDRESS to set one or more standard Group Addresses in the Routing Table of the Coupler.

START ADDRESS This shall be the first standard Group Address that shall be set.

END ADDRESS This shall be the last standard Group Address that shall be set.

If only one standard Group Address is to be modified then START ADDRESS and END ADDRESS shall be equal!

If START ADDRESS is greater than END ADDRESS no standard Group Address shall be modified (error response!).

The Coupler shall on reception of SRVID_SET_GROUPADDRESS set all Standard Group Addresses in the Routing Table starting from START ADDRESS up to and including END ADDRESS. It shall respond with an A_FunctionPropertyState_Response-PDU as follows.

Response (A_PropertyPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
return_code	04h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Coupler shall use the following values as return_code.

return_code: 00h: The standard Group Addresses are set successfully.

FFh: One or more errors have occurred during the setting of the standard Group Addresses (verify error).

b) Read (A_FunctionPropertyState_Read-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
00h	04h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Management Client shall apply the SRVID_SET_GROUPADDRESS with A_FunctionProperty-State_Read-PDU to check if all standard Group Addresses in the range from START ADDRESS to END ADDRESS are set.

START ADDRESS This shall be the first standard Group Address that shall be checked.

END ADDRESS This shall be the last standard Group Address that shall be checked.

If only one standard Group Address shall be checked then START ADDRESS and END ADDRESS shall be equal!

If START ADDRESS is greater than END ADDRESS then no standard Group Address shall be checked.

Response (A_PropertyPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15
return_code	04h	START ADDRESS		END ADDRESS	
		high octet	low octet	high octet	low octet

The Coupler shall use the following values as return_code.

return_code: 00h: all checked standard Group Addresses are set.

 FFh: not all checked standard Group Addresses are set.

4.5.7.6 Error handling

The reaction to an unknown ServiceID shall be as follows

- a) Write (A_FunctionPropertyCommand-PDU), A_PropertyValue_Read-PDU)

octet 10	octet 11	...
00h	undefined ServiceID	...

- b) response A_PropertyValue_Response-PDU

octet 10	octet 11
return_code	undefined ServiceID

The Coupler shall use the following value as return_code.

return_code: FFh: error.

4.5.8 PID_COUPL_SERV_CONTROL (PID = 57)

The Property PID_COUPL_SERV_CONTROL in the Router Object (object_type = 6) shall be used to control SNA management in a Coupler.

Bit	Name	Description	Encoding (d = default value)
0	EN_SNA_INCONSISTENCY_CHECK	This bit shall enable and disable the inconsistency-check of frames in point-to-point connectionless and connection-oriented communication mode. This feature shall only be executed if the Coupler is not in TP1 Bridge/Repeater Mode! See further details below.	0 = disable (d) 1 = enable
1	EN_SNA_HEARTBEAT	This bit shall enable and disable "SNA heartbeat" (see [10] clause 1.3.6 "SNA heartbeat") in the Subnetwork connected to the Coupler's secondary side. This feature shall only be executed if the Coupler is not in TP1 Bridge/Repeater Mode!	0 = disable (d) 1 = enable
2	EN_SNA_UPDATE_WRITE	This bit shall enable and disable the update of the Subnetwork Address by the Coupler in the Subnetwork Address connected to its secondary side by an NM_SubnetworkAddress_Write_Local-Subnetwork (see [09]) after its own Individual Address has changed via programming mode or KNX Serial Number. This feature shall only be executed if the Coupler is not in TP1 Bridge/Repeater Mode!	1 = enable 0 = disable (d)
3	EN_SNA_READ	This bit shall enable and disable the server side support of the SNA read procedure in the Coupler (see [10]). If this feature is enabled, the Coupler shall accept the A_NetworkParameter_Read-service in that Management Procedure and respond as specified in [10]. This feature shall only be executed if the Coupler is not in TP1 Bridge/Repeater Mode! If the Coupler is in TP1 Bridge/Repeater Mode then this procedure shall always be ignored, independently of the Property setting. See further details below.	0 = disable 1 = enable (d)
4	EN_SUBLINE_STATUS	This bit shall enable or disable the distribution of PID_LINE_STATUS in the Subnetwork connected to the primary side on power failure and power return in the Subnetwork Connected to the secondary side. This feature shall be executed independent of TP1 Bridge/Repeater Mode!	0 = disable (d) 1 = enable
5 to 7	reserved		Shall be 0.

EN_SNA_INCONSISTENCY_CHECK

A Router may detect topologically incorrect Individual Addresses on its primary side and on its secondary side and perform the procedure "SNA update on SNA inconsistency". The field EN_SNA_INCONSISTENCY_CHECK shall control whether (enable) or not (disable) the Router shall detect and correct incorrect Subnetwork Addresses. Please refer to [10] for the specifications.

EN_SNA_READ (indication)

Newly installed devices may perform an “SNA Read” (see [10]) to get the current SNA. The field EN_SNA_READ shall control whether (enable) or not (disable) the Router shall react to this procedure or not. Please refer to [10] for the specifications.

4.5.9 PID_MAX_APDU_LENGTH (PID = 58)

- Property name: MAX. APDU-Length
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

4.5.9.1 Usage by the Management Client

In the Router Object, this Property shall contain the maximal APDU-length of the messages that it can route from onto the network connection that is controlled through this Router Object instance.

If this Property is not present in the Router Object, the Management Client shall evaluate the same property PID_MAX_APDU_LENGTH (PID = 56) in the Device Object (see clause 4.3.7).

EXAMPLE If a Management Client needs to find out the maximal APDU-length that it can use to manage (download) a target device (Management Server) then it also needs to know the maximal APDU-length that is supported by any possible in-between Router. To this, it shall read PID_MAX_APDU_LENGTH in the Router Object of these Routers. This is specified in [10] under “Discovery of maximal frame length” Step 3.

4.5.10 PID_L2_COUPLER_TYPE (PID = 59)

- Property name: L2 Coupler Type
- Property Datatype: PDT_BITSET8
- Datapoint Type: None.

4.5.10.1 Format

The Property *L2 Coupler Type* shall be an 8 bit bitset.

Bit	Name	Value	Meaning
0	IS_REPEATER	0	Device is a TP1 Bridge
		1	Device is a TP1 Repeater
1 to 7	Reserved	0	These bits are reserved for future extensions.

The default value shall be “TP1 Repeater” (01h).

4.5.10.2 Usage by the Management Server (device)

Bit 0

The Property *L2 Coupler Type* shall not be evaluated if the device is a Router, this is, if the Device Address equals 0.

If the Device Address differs from 0 and Property *L2 Coupler Type* is not implemented, then the Coupler shall be a TP1 Repeater.

If the Device Address differs from 0 and Property *L2 Coupler Type* is implemented then the Coupler shall be a TP1 Bridge if bit 0 equals 0 and a TP1 Repeater if bit 0 equals 1.

This is summarized in the following pseudo-code.

```
if Device Address = 0
    then Device is Router
else
    if PID_L2_COUPLER_TYPE.BIT0 = 0
        then Device is TP1 Bridge
        else Device is TP1 Repeater
    endif
endif
```

Bit 1 to bit 7

The value of the reserved bits (bit 1 to bit 7) shall be ignored by the Management Server, even if they are set to 1. In the A_PropertyValue_Response-PDU, confirming the A_PropertyValue_Write-PDU back to the Management Client, the Management Server (device) shall clear these bits.

4.5.10.3 Usage by the Management Client

4.5.10.3.1 Read access

Bit 0

The Management Client shall interpret this Property Value, in combination with the Device Address of the Management Server, identically as the Management Server, as specified in 4.5.10.2 above. This shall allow the Management Client to determine whether the device is a TP1 Repeater or a TP1 Bridge. If this Property is not present, then the device shall be considered as a TP1 Repeater.

Bit 1 to bit 7

The Management Client shall ignore the value of these bits.

4.5.10.3.2 Write access

Inputs

- Constants
Bit 1 to bit 7: The Management Client shall clear these bits.
- Application specific
None
- From user
Bit 0 The value of PID_L2_COUPLER_TYPE.BIT0 shall be provided by the user of the Management Client.
- From the Management Server (device)
None.

Access

The Management Client shall set the value of Property PID_L2_COUPLER_TYPE using the Procedure DMP_InterfaceObject_Write_R.

4.5.11 PID_HOP_COUNT (PID = 61)

- **Property name:** Hop Count
- **Property Datatype:** PDT_UNSIGNED_INT
- **Datapoint Type:** None.
- **Access levels:** 3/2
- **Access Policy:** 3FF/0CC

This Property shall contain the default hop count that the Coupler shall use if it forwards a Frame to the Medium Interface that is represented by this Router Object.

This Property shall specifically be used if the Frame is forwarded from a KNX Medium that does not support any hop count, like KNX RF.

If the Coupler acts in the topology as a Line Coupler, the meaningful value for this Property on the Primary Side is five; if the Coupler in the topology acts as a Backbone Coupler, the meaningful value for this Property on the Primary Side is four. This way, Frames forwarded by this Coupler on the Primary Side have the same default hop count as if the Frame would originate from a TP1 Subnetwork.

Please refer to the Profile definitions for the indications for which Primary – or Secondary Sides for which KNX Communication Media this Property is mandatory.

4.5.12 PID_MEDIUM (PID = 63)

- **Property name:** Medium
- **Property Datatype:** PDT_ENUM8
- **Datapoint Type:** DPT_Medium (20.1004) (see [14])
- **Access levels:** 3/X
- **Access Policy:** 3FF/0CC

The Property Medium shall indicate the KNX Communication Medium that is used for the Subnetwork Connected that is represented by this Router Object.

This Property is mandatory for each Router Object, both on the Primary Side as on the Secondary Side.

4.5.13 PID_FILTER_TABLE_USE (PID = 67)

- **Property name:** Filter Table Use
- **Property Datatype:** PDT_BINARY_INFORMATION
- **Datapoint Type:** DPT_Bool (1.002)
- **Access levels:** 3/2
- **Access Policy:** 3FF/0CC

4.5.13.1 Abstract Resource definition

This parameter shall indicate whether the Filter Table shall be used or not.

4.5.13.2 Format and encoding

The parameter *Filter Table Use* shall be encoded as DPT_Bool (DPT_ID: 1.002).

Value	Specification
0	The Filter Table shall not be used.
1	The Filter Table shall be used.

4.5.13.3 Usage by the Management Server (Coupler)

Please refer to the Abstract Resource definition.

The MaS shall actively interpret this Parameter, even if the implementation specific Coupler Parameters would allow concluding on whether the Filter Table is used or not.

The MaS shall interpret this parameter *Filter Table Use* in the routing and filtering functionality as follows.

Value	Interpretation by the MaS
0	The Filter Table shall not be evaluated. If the standard - or implementation specific Coupler Parameter values conclude on the evaluation of Filter Table, then, this evaluation shall return negative for all Group Addresses.
1	The MaS shall evaluate the Filter Table as specified in 5.3.1. If the standard - or implementation specific Coupler Parameter values conclude on the evaluation of Filter Table, then, this evaluation shall be as in 5.3.1.1, and may return positive or negative accordingly.

This Parameter Filter Table Use may be part of the calculation of the routing condition, as indicated in [18].

4.5.13.4 Usage by the Management Client

Writing

The MaC shall always set this parameter, on any modification of the Filter Table contents or the (implementation specific) Coupler parameters.

Value	Interpretation by the MaC - writing
0	The FT contents shall be considered as invalid. The FT shall not be downloaded.
1	The FT contents shall be considered as valid. The FT shall be downloaded.

Reading and interpretation

The MaC shall write this Property parameter during any Configuration (Full download, Partial Download, Parameters only or any other).

The MaC shall read out this Parameter in status information, device information or in case of reverse engineering or other.

Value	Interpretation by the MaC – reading
0	The FT contents shall be considered as invalid. The FT may either be: – read out but marked invalid, or – not be read out.
1	The FT contents shall be considered as valid. The FT may be read out.

4.5.14 KNX PL110 specific Coupler Properties

4.5.14.1 PID_PL110_SBC_CONTROL (PID = 104)

- **Property name:** PL110 System Broadcast routing control
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None
- **Access level:** 3/2
- **Access Policy:** 3FF/0CC

This Property shall control the Routing of System Broadcast services and shall be evaluated as specified in [05] clause 2.4.2.4.5.7 “Routing of system broadcast services”. The encoding shall be as follows.

The Functionality of PID_PL110_SBC_CONTROL for PL110 shall be identical to the functionality of PID_RF_SBC_CONTROL (PID = 112) including the possible deviations through implementation specific Parameters. Please refer to clause 4.5.15.1 for the specification.

System broadcast routing Parameters for KNX RF and for KNX PL110

The parameter for System Broadcast handling for a KNX PL110 connection PID_PL110_SBC_CONTROL (PID = 104) is not shared with the system broadcast handling parameters for a KNX RF connection (PID_RF_SBC_CONTROL; PID = 112).

This allows that there may be different parameter sets, - values and – handling between KNX PL110 and KNX RF.

This could also later on allow for an easy introduction of a PL110/RF Media Coupler.

4.5.15 KNX RF specific Coupler Properties

4.5.15.1 PID_RF_SBC_CONTROL (PID = 112)

- **Property name:** RF System Broadcast routing control
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None
- **Access levels:** 3/2
- **Access Policy:** 3FF/0CC

This Property shall control the Routing of System Broadcast services and shall be evaluated as specified in [05] clause 2.4.2.4.5.7 “Routing of system broadcast services”. This interpretation shall be independent of the Load State Machine. The encoding shall be as follows.

Table 27 – Encoding of Mode in PID_RF_SBC_CONTROL

Mode	Value	Routing of system broadcast services
0	Disabled	System broadcast services shall not be routed ^b .
1	Broadcast	If according to the routing algorithm a system broadcast service is routed, then broadcast communication mode shall be used ^c .
2	System broadcast	If according to the routing algorithm a system broadcast service is routed, then system broadcast communication mode shall be used ^c .
^a This concerns the transmission of system broadcast services when routed onto KNX RF. In the KNX TP1/RF Media Coupler, if vice versa these services are routed onto KNX TP1, there is only broadcast communication mode and this shall be used. ^b If the MaC sets the Property to “Disable”, then, during the first 10 minutes, there shall be no exceptions. After 10 minutes, as noted in clause 1.4.2 “Network Layer, Filter Table and Coupler Parameters” in 06/02/01 “Coupler Model 2.0”, there may be exceptions to this through implementation specific Parameters. ^c To the settings “Broadcast” and “System Broadcast”, there shall be no exceptions (restrictions) through any implementation specific Parameter at any time.		

The default value shall be 0: Disable. Implementation specific exceptions may apply.

System broadcast routing Parameters for KNX RF and for KNX PL110

The parameters for System Broadcast handling for a KNX RF connection PID_RF_SBC_CONTROL (PID = 112) are not shared with the system broadcast handling parameters for KNX PL110 (PID_PL110_SBC_CONTROL; PID = 104).

This allows that there may be different parameter sets, - values and – handling between KNX PL110 and KNX RF.

This could also later on allow for an easy introduction of a PL110/RF Media Coupler.

1) Write (A_FunctionPropertyCommand-PDU)

Octet 10
Mode

The command shall be encoded as specified in Table 27.

- If the MaC sets PID_RF_SBC_CONTROL to “Disable” then the MaS shall immediately deactivate the routing of system broadcast services. During the first 10 minutes, there shall be no exceptions to this. After 10 minutes, the value shall remain “Disable” but exceptions through implementation specific Parameters may apply.
- If the MaC sets PID_RF_SBC_CONTROL to “Broadcast”, then the MaS shall immediately activate the routing of system broadcast services but shall use broadcast communication mode if routing these services.
- If the MaC sets PID_RF_SBC_CONTROL to “System Broadcast” then the MaS shall immediately activate the routing of system broadcast services but shall use system broadcast communication mode of routing these services.

For both cases “Broadcast” and “System Broadcast”, the MaS shall additionally start a timeout of 10 minutes. If the time-out expires, then the MaS shall autonomously set the value of PID_RF_SBC_CONTROL to “Disable” again and behave accordingly.

Response (A_FunctionPropertyState_Response-PDU)

Octet 10	Octet 11
Return Code	Mode

Return Code	00h: SUCCESS
	01h: Unspecified error.
	02h: Invalid command or command not supported.
	03h: Command not accepted because of implementation specific Parameters or local settings
	Other return codes are reserved and shall not be used.
Mode:	This shall be the current mode of the routing of RF System Broadcast Telegrams, according Table 27.

2) Read (A_FunctionPropertyState-Read-PDU)

No Data.

The MaC shall read the state of the Function Property PID_RF_SBC_CONTROL to obtain the current mode of the routing of RF System Broadcast Telegrams.

The format, contents and interpretation of the response (A_FunctionPropertyState_Response-PDU) shall be identical as to the above response to the A_FunctionPropertyCommand.

3) Error – and exception handling**Implementation specific Parameters**

According to Chapter 6/2/1 “Coupler Model 2.0” clause 1.4.3 “Implementation specific Coupler Parameters” in it is possible that the functioning of PID_RF_SBC_CONTROL and PID_PL110_SBC_CONTROL be influenced by implementation specific Parameters. The return code “success” shall however in any case mean that the returned mode is active, but the MaC shall take into account for the value “Disable” that implementation specific exceptions may exist.

EXAMPLE 9 If it is returned that the routing is enabled on system broadcast communication mode, then

- this shall not be restricted to any communication direction (Primary to Secondary or secondary to Primary), or
- this shall not be limited to any subset of system broadcast services or - service primitives, or
- this shall not be limited to any source Individual Address, or
- ...

If such limitation is active, then the return code shall be “command not accepted”.

4.5.16 KNX IP Specific Coupler Parameters

4.5.16.1 PID_IP_SBC_CONTROL (PID: 120)

- **Property name:** IP System Broadcast Routing Control
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** This Property shall be a Function Property. The coding of the data depends on whether data is written to the function or responded by the function. No single DPT can be given.
- **Access policy:** 3FF/00C
- **Access level:** 2/2

4.5.16.1.1 Abstract Resource definition

This Property shall be used to enable and disable the *IP System Broadcast Routing Mode* on KNXnet/IP Routers.

For Coupler 2.0-based implementations, the Properties shall be present in the first (Primary) Router object.

This Property shall be a Function Property.

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

If *Security Mode* is enabled in the KNXnet/IP Router then this Property shall only be accessible using Secure Communication. If *Security Mode* is disabled in the KNXnet/IP Router then this Property shall be accessible using secure - or plain communication.

The *IP System Broadcast Routing Mode* shall initially be “Disable”, but can be switched to “Enable” by a MaC using the Function Property PID_IP_SBC_CONTROL.

The *IP System Broadcast Routing Mode* shall automatically fall back to “Disable” 20 seconds after it has been set to “Enable”. It shall be possible for the MaC to prolong the *IP System Broadcast Routing Mode* by sending another A_FunctionPropertyCommand-PDU or A_FunctionPropertyExtCommand-PDU with IP SBC Mode = “Enable” within this timeout. It shall be possible for the MaC to set the *IP System Broadcast Routing Mode* to “Disable” before the timeout elapses by sending another A_FunctionPropertyCommand-PDU or A_FunctionPropertyExtCommand-PDU with IP SBC Mode = “Disable”.

If the MaS provides any facility to block the routing of broadcasts, successfully enabling the *IP System Broadcast Routing Mode* shall also enable the routing of broadcasts. It is implementation-specific if a configuration blocking broadcasts prevents enabling the *IP System Broadcast Routing Mode* with error code E_COMMAND_IMPOSSIBLE or if the blocking of broadcasts is overruled while the *IP System Broadcast Routing Mode* is enabled.

4.5.16.1.2 Write (A_FunctionPropertyCommand-PDU)

NOTE 21 The corresponding A_FunctionPropertyExtCommand-PDU formats are identical, except for the offset within the message.

Basic format and common handling

octet 10	octet 11	octet 12 ... octet n
Reserved	ServiceID	ServiceInfo
	WriteServiceID	
00h	See below.	See below.

Overview Write ServiceIDs

Table 28 – Overview PID_IP_SBC_CONTROL WriteServiceIDs

WriteServiceID	Description
00h	Write the <i>IP System Broadcast Routing Mode</i>

WriteServiceID 00h: Write the *IP System Broadcast Routing Mode*

octet 10	octet 11	octet 12
	ServiceID	ServiceInfo
reserved	Write IP SBC Mode	IP SBC Mode
00h	00h	

Figure 17 - A_FunctionProperty_Write-PDU for Write ServiceID 00h for PID_IP_SBC_CONTROL

The MaC shall use this command to disable or enable the *IP System Broadcast Routing Mode*.

ServiceID: 00h Command *IP System Broadcast Routing Mode*
 ServiceInfo: IP SBC Mode 00h: Disable the *IP System Broadcast Routing Mode*.
 01h: Enable the *IP System Broadcast Routing Mode*.

If the MaS can successfully handle the command then it shall set the *IP System Broadcast Routing Mode* as requested and respond with an A_FunctionPropertyState_Response-PDU as in Figure 70.

If the MaS has any problem then it shall respond as specified in 6.3.5.3.

The MaS shall respond with an A_FunctionPropertyState_Response-PDU with the appropriate positive or negative Return Code as listed 6.3.5.3.

NOTE 22 This Response-PDU repeats the WriteServiceID but does not indicate the current resulting *IP System Broadcast Routing Mode*. This shall be clear from the Return Code.

octet 10	octet 11
Return Code	ServiceID
	Write IP SBC Mode
	00h

Figure 18 - A_FunctionPropertyState_Response-PDU for WriteServiceID 00h for PID_IP_SBC_CONTROL

4.5.16.1.3 Read (A_FunctionPropertyState_Read-PDU)

Basic format and common handling

octet 10	octet 11	octet 12 ... octet n
Reserved	ReadServiceID	ServiceInfo
00h	See below.	See below.

If the MaS can respond to the command then it shall respond with an A_FunctionPropertyState_Response-PDU with a positive Return Code, the ReadServiceID and the ServiceInfo as from the request (if available) and the command result as specified below.

Overview ReadServiceIDs

Table 29 – Overview PID_IP_SBC_CONTROL ReadServiceIDs

ReadServiceID	Description
00h	Read the current <i>IP System Broadcast Routing Mode</i>

ReadServiceID 00h: Read PID_IP_SBC_CONTROL

This command shall be used to read PID_IP_SBC_CONTROL information.

octet 10	octet 11
Reserved	ReadServiceID
00h	00h

If the MaS has any problem, then it shall respond as specified in 6.3.5.3.

If the MaS can successfully respond to the request then it shall respond with an A_FunctionPropertyState_Response-PDU as in Figure 71.

octet 10	octet 11	octet 12
Return Code	ReadServiceID	ServiceInfo
00h	00h	IP SBC Mode

Figure 19 - A_FunctionPropertyState_Response-PDU for ReadServiceID 01h for PID_IP_SBC_CONTROL

ServiceInfo: IP SBC Mode The encoding values shall be the same as for writing the *IP System Broadcast Routing Mode*.

4.5.16.1.4 Common error – and exception handling for PID_IP_SBC_CONTROL

In case of any error or exception, the MaS shall return the specified A_FunctionPropertyState_Response-PDU, but with the field Return Code as listed in Table 33 and repeating the ServiceID – ReadServiceID or WriteServiceID - as appropriate.

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11
Return Code	ServiceID
> 7Fh	

Next to the Basic (FFh) – and Generic Negative Return Codes (FEh and F8h), the following Function Property specific Return Codes may be returned as appropriate.

Table 30 – PID_IP_SBC_CONTROL Return Codes

Return Code	ErrorName	ErrorType
F2h	E_COMMAND_INVALID	Invalid command
	An invalid value is requested in the field <i>ServiceID</i> (ReadServiceID or WriteServiceID).	
F3h	E_COMMAND_IMPOSSIBLE	Command impossible
	The <i>IP System Broadcast Routing Mode</i> could not be activated, e.g. because broadcasts and/or system broadcasts are blocked by any means.	

Return Code	ErrorName	ErrorType
F8h	E_DATA_VOID	Void data
	The field <i>ServiceInfo</i> contains a value that is not supported.	

4.6 LTE Address Routing Table Object (Object Type 7)

4.6.1 General requirements

This Interface Object shall exclusively control the handling of LTE-HEE extended address type frames.

Please refer to [17] for the specification of which Properties are mandatory or optional in the LTE Address Routing Table Object.

4.6.2 PID_LOAD_STATE_CONTROL (PID = 5)

The Load State Machine in the LTE Address Routing Table Object shall control the access to the LTE Address Routing Table.

PID_LOAD_STATE_CONTROL in the LTE Address Routing Table Object shall support the Load Events as listed in Table 31. For the specification and numerical values of these Load Events, please refer to clause 4.23.2 “Load State Machine – Realisation Type 1 (Property based)”.

Table 31 – Load Events for PID_LOAD_STATE_CONTROL in the LTE Address Routing Table Object

Load Events
No operation
Start Loading
Load Completed
Unload

The Load States as specified in Table 32 may be returned. For the specification and numerical values of these Load Events, please refer to clause 4.23.2 “Load State Machine – Realisation Type 1 (Property based)”.

Table 32 – Load States returned by PID_LOAD_STATE_CONTROL in the LTE Address Routing Table Object

Load State
Unloaded
Loaded
Loading
Error

If the LTE Address Routing Table is to be checked on received frames and the Load State of the LTE Address Routing Table Object is not *Loaded* at the same time, no LTE-HEE extended address type frames shall be routed.

The default Load State shall be *Loaded*. The LTE Address Routing Table shall be cleared. That means the current length shall be zero and all LTE-HEE extended address type frames shall be routed.

4.6.3 PID_LTE_ROUTESELECT (PID = 51)

Bit	Name	Description	Encoding (d = default value)
0-1	LTESUB	Shall specify the general handling for LTE-HEE extended address type frames from the Subnetwork on the secondary side	0 = not used 1 = LTESUB_PASS All frames shall be routed. 2 = LTESUB_BLOCK No frames shall be routed. 3 = LTESUB_NORMAL Routing shall depend on the LTE Address Routing Table (d).
2-3		reserved	Shall be 0.
4-5	LTEMAIN	Shall specify the general handling for LTE-HEE extended address type frames from the Subnetwork on the primary side.	0 = not used 1 = LTEMAIN_PASS All frames shall be routed. 2 = LTEMAIN_BLOCK No frames shall be routed. 3 = LTEMAIN_NORMAL Routing shall depend on the LTE Address Routing Table (d).
6-7		reserved	Shall be 0.

4.6.4 PID_LTE_ROUTETABLE (PID =52)

- Property name: LTE Address Routing Table Object
- Property Datatype: PDT_GENERIC_05
- Datapoint Type: None

The LTE Address Routing Table shall only be checked if the Property PID_LTE_ROUTESELECT is set to LTESUB_NORMAL and LTEMAIN_NORMAL. The table shall be a PDT_GENERIC_05 array. The implementation shall foresee memory space for at least 32 entries. If the current length is zero all for LTE-HEE extended address type frames shall be routed.

Attention

It shall not be forgotten to handle the LTE state machine during the modification of the Routing Table!

octet 1								octet 2	octet 3	octet 4	octet 5
Direction		EFF						Base		Mask	
b7	b6	b5	b4	b3	b2	b1	b0	high octet	low octet	high octet	low octet
0	0	0	0	0	1	x	x				
0	0	0	0	0	1	x	x				

- Octet 1:
 - b0 – b3: EFF (Extended Frame Format)
 - b4 and b5: reserved (fixed to 0)
 - b6: data direction: secondary side to primary side free (0) / blocked (1)
 - b7: data direction: primary side to secondary side free (0) / blocked (1)

- Octet 2 und 3: „Base address“
- Octet 4 und 5: „Mask“

Calculation of the routing condition

The routing condition shall be calculated as follows.

The LTE Group Address (ZA = Zone Address) in the received Frame shall be compared with the entries in the LTE Address Routing Table.

```

routing condition = FALSE;
REPEAT
    IF ((ZA AND Mask) = Base) AND (data direction free) THEN routing condition := TRUE
/* Note that the EFF is part of LTE Zone Address (Destination Address) and must be included */
/* in the address compare check */
    GOTO next table entry
UNTIL (routing condition = TRUE) OR all table entries checked

```

Error – and exception handling

- MaC
The MaC shall not set reserved values of the EFF-field in this Property.
- MaS
In the Network Layer of the MaS, in the routing of LTE-Frames, there are no requirements concerning the exception handling of reserved values of the EFF-field.

In the Data Link Layer of the MaS, messages with reserved values of the EFF-field will not be passed to the Network Layer, even if such entries are contained in the *LTE Address Routing Table Object* of a Coupler or Router, so that the preceding situation will normally not occur.

4.7 cEMI Server Object (Object Type 8)

4.7.1 PID_MEDIUM_TYPE (PID = 51)

- Property name: Medium Type
- Property Datatype: PDT_BITSET16
- Datapoint Type: DPT_Media

This Property shall contain the Media Type(s) supported by cEMI server.

The format complies with the encoding of DPT_Media as specified in [14].

NOTE 23 A cEMI server could be connected to more than one medium⁹⁾; e.g. when the cEMI server device is also a Media Coupler.

4.7.2 PID_COMM_MODE (PID = 52)

- Property name: Communication Mode
- Property Datatype: PDT_ENUM8
- Datapoint Type: DPT_CommMode (DPT_ID = 20.1000)

Please refer to [12] for the handling and interpretation of PID_COMM_MODE by the cEMI Server.

⁹⁾ Such a device does not exist yet (May-2002). For separation to which medium a *L_Data.req* message shall be sent, respectively from which medium a *L_Data.ind* message was received, a ‘channel number’ could be defined as a further ‘additional information’.

4.7.3 PID_MEDIUM_AVAILABILITY (PID = 53)

- Property name: Medium Availability
- Property Datatype: PDT_BITSET16
- Datapoint Type: DPT_Media (DPT_ID = 22.1000)

The Property PID_MEDIUM_AVAILABILITY shall indicate whether the connected medium is active (availability is true = 1) or not (availability is false = 0).

The same Datapoint Type shall be used as for the Property PID_MEDIUM_TYPE. It shall be a 2 octet long bit set B₁₆, in which each bit shall indicate if the corresponding medium is active or not. Please also refer also to 3.2.3.2 “Supported Media Type”.

4.7.4 PID_ADD_INFO_TYPES (PID = 54)

- Property name: Additional Information Types
- Property Datatype: PDT_ENUM8[]
- Datapoint Type: DPT_AddInfoTypes (DPT_ID = 20.1001)

The Property PID_ADD_INFO_TYPES shall indicate which Additional Information Types the cEMI server supports. If the Property is not present, the cEMI server shall not support any Additional Information Types.

An array-structured Property shall be used to represent the supported Additional Information Types. Each (used) array field shall represent one supported Additional Information Type. The Property shall contain as much array fields as different Additional Information Types are supported. The number of used array fields (= number of supported Additional Information Types) can be read (read only) as array-element with start index 0. Within the array, the type IDs shall be sorted in ascending order.

The format of PID_ADD_INFO_TYPES (N_s) shall comply with the encoding of DPT_AddInfoTypes as specified in [14].

The codes *DPT_AddInfoType* shall be the same as specified in clause “Additional information” in [12].

4.7.5 PID_TIME_BASE (PID = 55)

- Property name: Time Base
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: DPT_Value_2_Ucount (DPT-ID = 7.001)

The Property PID_TIME_BASE shall contain the time base used by the cEMI Server for the extended relative timestamp. The value shall be the length of one tick of the free running counter used in the extended relative timestamp in nanoseconds.

Please refer to [12] for the requirements concerning the use of PID_TIME_BASE in the cEMI Server Object.

4.7.6 PID_CEMI_SERVER_SNA (PID = 57)

- **Property name:** cEMI Server Subnetwork Address
- **Property Datatype:** PDT_UNSIGNED_CHAR
- **Datapoint Type:** None.
- **Access:**
 - cEMI Services for local management (M_Prop...) ¹⁰⁾

Service:	M/O
Read:	M
Write:	M

- Data Property services (A_PropertyValue...) ¹¹⁾

Service:	M/O	Role:	Security:	Access level:
Read:	O	any	plain	3
Write:	M	MaC	plain	2

4.7.6.1 Abstract Resource definition

If there is a cEMI Server Individual Address, then this Property shall contain the Subnetwork Address part.

4.7.6.2 Format

This field shall be formatted as the Subnetwork Address part of an Individual Address; see clause “Individual Address” in [04].

4.7.6.3 Usage by the Management Server (device)

4.7.6.3.1 General requirements

This Property shall be the Subnetwork Address part of the Individual Address of the cEMI Server.

- In transmission direction, the cEMI Server shall, unless noted differently in the cEMI message, use this Individual Address as Source Address.
- In reception direction, unless any other filters are set (e.g. the field “IA” in PID_FILTERING - MODE_SELECT, the cEMI Server shall only forward to the cEMI Client L_Data-messages in point-to-point connectionless – or connection-oriented communication mode that contain this Individual Address as Destination Address.

The value of this Property shall have immediate effect, this is, it shall be evaluated for the message received or sent immediately after the setting of this Property.

The Property PID_CEMI_SERVER_SNA shall be stored in non-volatile memory.

4.7.6.3.2 Defending the cEMI Server IA

On USB, the cEMI Client does not explicitly connect to the cEMI Server. The KNX USB interface can only detect whether the USB cable is connected or not. However, if it is detected that the cable is connected, then the USB interface shall assume that the cEMI Client is connected.

¹⁰⁾ The cEMI services for local device management do not exhibit any security feature. This thus has to be supported by the cEMI host protocol, like KNXnet/IP Tunnelling or USB.

¹¹⁾ This concerns access via the bus, over the L_Data-service, as well as access through the cEMI Transport Layer using the cEMI T_Data_Individual or T_Data_Connected to transfer an A_PropertyValue_nnn,-PDU.

- If the cEMI Client **is not connected** to the cEMI Server, then the cEMI Server shall defend the cEMI Server Individual Address: if from the bus a T_Connect-PDU is received addressed to this IA, then the cEMI Server shall respond with a T_Disconnect-PDU to the sender.
- If the cEMI Client **is connected** to the cEMI Server then the handling of this T_Connect-PDU shall depend on the cEMI Communication Mode.

- If the Communication Mode requires the T_Connect-PDU to be passed to the cEMI Client, then the cEMI Client shall react to the T_Connect-PDU.

EXAMPLE 3 If the Communication Mode is set to "Data Link Layer" then the T_Connect-PDU shall be passed to the cEMI Client, who is then responsible to send the T_Disconnect-PDU.

NOTE 24 This could further be blocked by filtering in the Data Link Layer. However, no Data Link Layer filter settings are defined that allow refusing a point-to-point message to the IA of the cEMI Server. A T_Connect-PDU addressed to the IA of the cEMI Server will thus by the Data Link Layer always be accepted and passed through Network Layer to the cEMI Client.

- If the cEMI Communication Mode does not allow or require that that T_Connect-PDU be passed to the cEMI Client, then the cEMI Server shall react by responding with a T_Disconnect-PDU to the sender, which shall contain the cEMI Server IA (PID_CEMI_SERVER_SNA and PID_CEMI_SERVER_DEVICE_ADDRESS) and not the IA of an end device that may possible be combined in the USB interface device.

NOTE 25 This is only possible in case the cEMI Communication Mode is set to "cEMI Transport Layer". In that case, it is already required in [21] clause 3.5.4.2.2 that "further requests for opening a Transport Layer connection shall be refused".

Accepted exceptions

3. If the USB cable is connected but no cEMI Client is active and the cEMI Server tries to pass the T_Connect-PDU to the cEMI Client, then this will not be handled. The IA of the cEMI Server is in this case not defended.
4. Also, if the cEMI Server operates in Busmonitor Mode, it shall pass the T_Connect-PDU to the cEMI Client; as however Busmonitor Mode does not allow transmission to the bus, the cEMI Client cannot send a T_Disconnect-PDU. In this case as well, the cEMI Server IA will not be defended.
5. USB interfaces that are solely powered over USB, and that are for instance not powered over the bus, can, if the USB cable is disconnected, not defend the IA.

4.7.6.3.3 Accessibility of PID_CEMI_SERVER_SNA

Over the bus, PID_CEMI_SERVER_SNA shall be accessible by Frames addressed to the "mask n IA". It shall under no condition – neither in a stand-alone interface nor in combination with a mask n - be available by Frames addressed to the cEMI Server IA".

NOTE 26 This is not possible, because the cEMI server IA only allows access to a minimal stack, which does not foresee a Network Layer or Transport Layer to pass the message to any Management. Frames addressed to the cEMI Server IA will either be ignored or will be passed to the cEMI Client.

Over the cEMI connection, PID_CEMI_SERVER_SNA shall be accessible using the cEMI services for local device management (see clause 4.1.7 in [12]).

4.7.6.4 Usage by the Management Client

The Client shall set this Property according the Individual Address that it wants to use for sending and receiving cEMI L_Data-messages without any further filtering.

It is the responsibility of the cEMI Client to properly set the value of the cEMI Server Individual Address according the position in the logical topology of the cEMI Server. There are no requirements towards the cEMI Server to check for correctness or uniqueness of the cEMI Server Individual Address.

NOTE 27 This shall apply for all cEMI Clients: ETS, visualisation tools, etc.

cEMI Server IA shall differ IA of combined Profile(s)

If in a single device the KNX RF USB Interface is hosted in combination with one or more other KNX Profiles, then the MaC shall not use the same value for the IA of any of these implemented Profiles as for the cEMI Server (PID_CEMI_SERVER_SNA and PID_CEMI_SERVER_DEVICE_ADDRESS in the cEMI Server Object).

EXAMPLE 10 The KNX RF USB Interface may in one device be combined with an RF end device Profile or with a KNX TP1/RF Media Coupler. The MaC shall in this case not assign the same numerical IA value to the KNX RF USB Interface (PID_CEMI_SERVER_SNA and PID_CEMI_SERVER_DEVICE_ADDRESS) as to the end device part, e.g. PID_SUBNET_ADDR and PID_DEVICE_ADDR in the Device Object.

There are no requirements towards the cEMI Server in this case.

Defending the cEMI Server IA

Please refer to 4.7.6.3.2.

4.7.7 PID_CEMI_SERVER_DEVICE_ADDRESS (PID = 58)

- **Property name:** cEMI Server Device Address
- **Property Datatype:** PDT_UNSIGNED_CHAR
- **Datapoint Type:** None.
- **Access:**
 - cEMI Services for local management (M_Prop...) ¹²⁾

Service:	M/O
Read:	M
Write:	M

- Data Property services (A_PropertyValue...) ¹³⁾

Service:	M/O	Role:	Security:	Access level:
Read:	O	any	plain	3
Write:	M	MaC	plain	2

4.7.7.1 Abstract Resource definition

This Property shall contain the Device Address of the cEMI Client.

4.7.7.2 Format

This field shall be formatted as the Device Address part of an Individual Address; see clause “Individual Address” in [04].

4.7.7.3 Usage by the Management Server (device)

4.7.7.3.1 General requirements

This Property shall be the Subnetwork Address part of the Individual Address of the cEMI Server.

- In transmission direction, the cEMI Server shall, unless noted differently in the cEMI message, use this Individual Address as Source Address.
- In reception direction, unless any other filters are set (e.g. the field “Pass all IA” in PID_FILTERING_MODE_SELECT, the cEMI Server shall only forward to the cEMI Client L_Data-messages

¹²⁾ The cEMI services for local device management do not exhibit any security feature. This thus has to be supported by the cEMI host protocol, like KNXnet/IP Tunnelling or USB.

¹³⁾ This concerns access via the bus, over the L_Data-service, as well as access through the cEMI Transport Layer using the cEMI T_Data_Individual or T_Data_Connected to transfer an A_PropertyValue_nnn,-PDU.

in point-to-point connectionless – or connection-oriented communication mode that contain this Individual Address as Destination Address.

The value of this Property shall have immediate effect, this is, it shall be evaluated for the message received or sent immediately after the setting of this Property.

The Property `PID_CEMI_SERVER_DEVICE_ADDRESS` shall be stored in non-volatile memory.

4.7.7.3.2 Accessibility of `PID_CEMI_SERVER_DEVICE_ADDRESS`

The same requirements apply as for `PID_CEMI_SERVER_SNA`: please refer to 4.7.6.3.3.

`PID_CEMI_SERVER_DEVICE_ADDRESS` shall not be accessible from the bus by Frames addressed to the “cEMI Server IA”.

4.7.7.4 Usage by the Management Client

The Client shall set this Property according the Individual Address that it wants to use for sending and receiving cEMI L_Data-messages without any further filtering.

Please refer to 4.7.6.4 (`PID_CEMI_SERVER_SNA` (`PID = 57`)) for the requirements on the following.

- cEMI Server IA shall differ from device IA
- Correctness of the cEMI Server IA
- Defending the cEMI Server IA

4.7.8 `PID_BIBAT_NEXTBLOCK` (`PID = 59`)

- Property name: BiBat Next Block
- Property Datatype: `PDT_UNSIGNED_CHAR`
- Datapoint Type: `DPT_Value_1_Ucount` (5.010)

This Property is mandatory if the cEMI Server is a BiBat Master.

The cEMI Server that has BiBat Master functionality shall spontaneously send an `M_PropInfo.ind` message to the cEMI client with the value of `PID_BIBAT_NEXT_BLOCK`. It shall contain the number of the next BiBat block in the Master timeslot scheme. The allowed range is from 0 to 127.

Refer to [12] clause 4.1.5.5.2 “L_Data.req for BiBat synchronous data frames” as well.

4.7.9 `PID_RF_MODE_SELECT` (`PID = 60`)

- Property name: RF Communication Mode
- Property Datatype: `PDT_ENUM8` (alt: `PDT_UNSIGNED_CHAR`)
- Datapoint Type: None.

With this Property the RF communication mode of the cEMI Server can be selected.

- 0: asynchronous (mandatory)
- 1: asynchronous + BiBat Master (optional)
- 2: asynchronous + BiBat Slave (optional)

Write access to this Property is optional if only asynchronous RF communication mode is supported.

4.7.10 PID_RF_MODE_SUPPORT (PID = 61)

- Property name: Supported RF Communication Modes
- Property Datatype: PDT_BITSET8 (alt.: PDT_GENERIC_01)
- Datapoint Type: None.

With this Property the supported RF communication mode(s) of the cEMI Server can be identified by the cEMI Client and accordingly the mode to be used can be selected.

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved	BiBat Slave	BiBat Master	Asynch
Values	0	0	0	0	0	0, 1	0, 1	1

Bit	Data fields	Description	Encoding	Unit	Range
b ₀	Asynchronous	asynchronous mode support	0 = value not allowed 1 = true	none	{0,1}
b ₁	BiBat Master	BiBat Master mode supported	0 = false 1 = true	none	{0,1}
b ₂	BiBat Slave	BiBat Slave mode supported	0 = false 1 = true	none	{0,1}
b ₃ ...b ₇	reserved	reserved, shall be 0	not applicable	n.a.	n.a.

This Property shall be read only.

4.7.11 PID_RF_FILTERING_MODE_SELECT (PID = 62)

- Property name: RF Filtering Mode Selection
- Property Datatype: PDT_ENUM8 (alt.: PDT_UNSIGNED_CHAR)
- Datapoint Type: None.

With this Property the RF filtering mode for received RF frames in the cEMI Server shall be selected.

- 0: no filtering, all supported received frames shall be passed to the cEMI Client using L_Data.ind
- 1: filtering by Domain Address (optional)
- 2: filtering by KNX Serial Number table (optional)
- 3: filtering by Domain Address and by KNX Serial number table (optional)

4.7.12 PID_RF_FILTERING_MODE_SUPPORT (PID = 63)

- Property name: Supported RF Filtering Modes
- Property Datatype: PDT_BITSET8 (alt.: PDT_GENERIC_01)
- Datapoint Type: None.

With this Property the supported RF filtering mode(s) of the cEMI Server can be identified by the cEMI Client and accordingly the filtering mode to be used can be selected.

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved	Filter by DoA & SerialNr	Filter by SerialNr	Filter by DoA
Value	0	0	0	0	0	0/1	0/1	0/1

This Property shall be read only.

4.7.13 PID_COMM_MODES_SUPPORTED (PID = 64)

- **Property name:** cEMI Supported Communication Modes
- **Property Datatype:** PDT_BITSET16
- **Datapoint Type:** None.
- **Access:**
 - cEMI Services for local management (M_Prop...) ¹⁴⁾

Service:	M/O
Read:	M
Write:	X

- Data Property services (A_PropertyValue...) ¹⁵⁾

Service:	M/O	Role:	Security:	Access level:
Read:	O	any	plain	3
Write:	X	none	n/a	n/a

4.7.13.1 Abstract Resource definition

This Property shall allow discovering the Communication Modes that are supported by the cEMI Server.

4.7.13.2 Format

This Property shall be read-only and shall be formatted as a 16 bit field (B₁₆) as follows.

Field	b ₁₅	b ₁₄	b ₁₃	b ₁₂	b ₁₁	b ₁₀	b ₉	b ₈	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved	reserved	reserved	reserved	reserved	reserved	reserved	reserved	TLL	RAW	BM	DLL
Value	0	0	0	0	0	0	0	0	0	0	0	0				

b ₀	DLL: Data Link Layer normal operation	
	This shall indicate whether or not the cEMI Server supports access to the Data Link Layer, this is, whether this cEMI Server can be used for sending and receiving Frames to and from the KNX bus using L_Data.req and L_Data.ind.	
	0:	Data Link Layer is not supported.
	1:	Data Link Layer is supported.
b ₁	BM: Data Link Layer Busmonitor Mode	
	This shall indicate whether or not the cEMI Server supports access to the Data Link Layer, this is, whether this cEMI Server can be used for receiving Frames from the KNX bus using L_Busmon.ind.	
	0:	Busmonitor Mode is not supported.
	1:	Busmonitor Mode is supported.

¹⁴⁾ The cEMI services for local device management do not exhibit any security feature. This thus has to be supported by the cEMI host protocol, like KNXnet/IP Tunnelling or USB.

¹⁵⁾ This concerns access via the bus, over the L_Data-service, as well as access through the cEMI Transport Layer using the cEMI T_Data_Individual or T_Data_Connected to transfer an A_PropertyValue_nnn,-PDU.

b₂	RAW: Data Link Layer Raw mode	
	This shall indicate whether or not the cEMI Server supports access to the Data Link Layer Raw Mode, this is, whether this cEMI Server can be used for receiving Frames from the KNX bus using L_Raw.req and L_Raw.ind.	
	0:	Raw Mode is not supported.
	1:	Raw Mode is supported.
b₃	TLL: Transport Layer Local	
	This shall indicate whether or not the cEMI Server supports the cEMI Local Transport Layer, thus allowing Local Device Management over the cEMI interface.	
	0:	cEMI Local Transport Layer is not supported.
	1:	cEMI Local Transport Layer is supported.

The fields b₁₅ to b₄ are reserved for future KNX standard extensions and shall be 0.

4.7.13.3 Usage by the Management Server

The cEMI Server shall in this Property indicate the cEMI Communication Modes that it supports, also if this can be concluded through different means.

EXAMPLE 11 So far (November 2013), each cEMI Server shall support the normal Data Link Layer. Still, this shall be properly marked here.

EXAMPLE 12 It is required for the mask versions 0310h and 2311h that the Busmonitor Mode is supported. For cEMI on USB, this can already be concluded reading the Device Descriptor Type 0 using the USB Device Feature services. Yet, this information shall be repeated here explicitly.

4.7.13.4 Usage by the Management Client

If the cEMI Client does not know the Communication Modes that are supported by the cEMI Server, for instance because these cannot be concluded based on the DD0, then the cEMI Client shall firstly read this Property before trying to activate any Communication Mode through PID_COMM_MODE.

4.7.14 PID_FILTERING_MODE_SUPPORT (PID = 65)

- **Property name:** Supported Filtering Modes
- **Property Datatype:** PDT_BITSET16 (alt.: PDT_GENERIC_02)
- **Datapoint Type:** None
- **Access:**
 - cEMI Services for local management (M_Prop...) ¹⁴⁾

Service:	M/O
Read:	M
Write:	X

- Data Property services (A_PropertyValue...) ¹⁵⁾

Service:	M/O	Role:	Security:	Access level:
Read:	O	any	plain	3
Write:	X	any	n/a	n/a

4.7.14.1 Abstract Resource definition

With this Property the supported filtering mode(s) of the cEMI Data Link Layer can be identified by the cEMI Client and accordingly the filtering mode to be used can be selected.

This Property shall be read only.

4.7.14.2 Format

PID_FILTERING_MODE_SUPPORT shall be a 2 octet bit field formatted as follows.

Bit	Name	Controlled Filter
b ₀	IA	Filter on own IA can be disabled 0: False 1: True
b ₁	Repetitions	Filter on repeated Frames 0: False 1: True
b ₂	DoA	Filter on DoA 0: False 1: True
b ₃	Extended GA	Filter on Extended GA 0: False 1: True
b ₄ to b ₁₅	reserved	These bits are reserved and shall be 0. The cEMI Server shall not evaluate these fields.

4.7.14.3 Usage by the Management Server (cEMI Server)

The MaS shall in this Property signal which filters can be disabled.

4.7.15 PID_FILTERING_MODE_SELECT (PID = 66)

- **Property name:** Filtering Mode Selection
- **Property Datatype:** PDT_BITSET16 (alt.: PDT_GENERIC_02)
- **Datapoint Type:** None
- **Access:**
 - cEMI Services for local management (M_Prop...) ¹⁴⁾

Service:	M/O
Read:	M
Write:	M

- Data Property services (A_PropertyValue...) ¹⁵⁾

Service:	M/O	Role:	Security:	Access level:
Read:	O	any	plain	3
Write:	X	none	n/a	n/a

4.7.15.1 Abstract Resource definition

This Property shall allow the control of the filtering of received Frames by the cEMI Data Link Layer.

4.7.15.2 Format and encoding

General

The Data Link Layer of the KNX Communication Media evaluates multiple dependent and subsequent filters when deciding whether or not to pass a received Frame to the Data Link Layer user or not. With *Filtering Mode Selection* it shall be possible to enable (0) or disable (1) each of these Filters. If a filter is disabled, then this shall mean that the evaluation of this Filter shall be seen as positive for the acceptance of the Frame.

EXAMPLE 13 The Data Link Layer will filter out incoming Frames in point-to-point connectionless or connection-oriented that have a Destination Address different from the own IA. If this filter is disabled, then the cEMI server will pass all point-to-point messages to the cEMI Client, regardless of the destination IA (and if the other filter criteria are positive).

Without any filtering switched off, this is, with all fields cleared, the cEMI Data Link Layer shall comply with the standard KNX Data Link Layer protocol of an end device for the host communication medium.

EXAMPLE 14 It shall only accept Broadcast messages or point-to-point messages if these contain the own Domain Address (RF medium Object) or respectively the own Individual Address of the cEMI Server. Through this Property, this filtering can be switched off.

Default value

The default value shall be 0, this is, all filters shall be enabled, as for the Data Link Layer of an end device.

This value shall be assumed

after reset of the cEMI Server, e.g. after restart of the device in which the cEMI Server is hosted, or on M_Reset.req by the cEMI Client, or

if the cEMI Client sets PID_COMM_MODE to “Data Link Layer”.

The Property PID_FILTERING_MODE_SELECT shall be stored in volatile memory.

Dependency of filters

The filters are not independent. An incoming Frame may be accepted by one disabled Filter, but may be ignored because of another Filter that is still enabled.

EXAMPLE 15 The filter on the Destination IA may be disabled. If an incoming Frame is handled with a DA different than the own IA, but that it a repeated Frame, then this Frame will still be ignored if not also the filter on repeated Frames is disabled.

Format

PID_FILTERING_MODE_SELECT shall be a 2 octet bit field formatted as follows.

Bit	Name	Controlled Filter
b ₀	IA	Filter on own IA as Destination Address for Frames in point-to-point connectionless – or connection-oriented communication mode 0: This filter is not disabled. Only Frames with the own IA as contained in PID_CEMI_SERVER_SNA and PID_CEMI_SERVER_DEVICE_ADDRESS in the cEMI Server Object as Destination Address shall be accepted. 1: This filter is disabled. These Frames are accepted regardless of the contained DA.
b ₁	Repetitions	Filter on repeated Frames 0: This filter is not disabled. If the interface has successfully received a preceding transmission of the same Frame, then this repetition shall not be passed to the cEMI Client. In KNX RF, this shall mean that the History List shall be evaluated. This On KNX RF, it means that L2-repetition shall not be accepted. 1: This filter is disabled. Repeated Frames shall be passed if they meet the further filter criteria. This field shall signal whether or not repeated Frames shall be passed. This field shall be evaluated only in an AND-condition with the other fields in this Property. EXAMPLE 16 A repeated Frame in ptp communication mode, with a DA different from the cEMI Server IA, shall only be passed if this bit is set AND if the bit IA is set.
b ₂	DoA	Filter on DoA 0: This filter is not disabled. Only Frames with the own DoA as contained in the Device Object shall be passed to the cEMI Client. 1: This filter is disabled. All Frames with any DoA shall be passed to the Data Link Layer user.

Bit	Name	Controlled Filter
b ₃	Extended GA	Filter on Extended GA
		0: This filter is not disabled. Frames with Extended GA shall be filtered according the LTE-Mode Group Address Table as specified in [23]. 1: This filter is disabled. All Frames with any Extended GA shall be passed to the Data Link Layer user.
b ₄ to b ₁₅	reserved	These bits are reserved and shall be 0. The cEMI Server shall not evaluate these fields.

Figure 20 – PID_FILTERING_MODE_SELECT

4.7.15.3 Usage by the Management Server (cEMI Server)

This is specified in the clause 4.7.15.2 above.

Default value if the cEMI Client connects to Data Link Layer

If the cEMI Client sets PID_COMM_MODE of the cEMI Server to Data Link Layer, then the cEMI Server shall assume the default value (as specified above) for PID_FILTERING_MODE_SELECT.

NOTE 28 As PID_FILTERING_MODE_SELECT only influences the behaviour of the cEMI Server in Data Link Layer mode, this resetting of the value may be done also when the cEMI Client inactivates Data Link Layer Mode.

The cEMI Server shall also assume the default value for PID_FILTERING_MODE_SELECT if the cEMI Server is reset in any way.

Layer-2 acknowledge

The disabling of a filter shall not influence the Layer-2 acknowledge behaviour of the device in which the cEMI Server is hosted.

Error – and exception handling

1. If a field is set for a filter criterion that is not supported, then the cEMI Server shall behave according the value zero for that field in the above definition.

4.7.15.4 Usage by the Management Client (cEMI Client)

PID_FILTERING_MODE_SELECT does not foresee mask bits to allow the cEMI Client to selectively set or clear individual fields without modifying other fields. Therefore, the MaC shall always modify fields in this Property in a read – modify - write back procedure.

4.7.16 PID_MAX_INTERFACE_APDU_LENGTH (PID: 68)

- **Property name:** APDU-length for bus access
- **Property Datatype:** PDT_UNSIGNED_INT
- **Datapoint Type:** None
- **Access Policy:** 3FF/1FF
- **Access level:** 3/X

4.7.16.1 Abstract Resource definition

This Management Server shall hold in PID_MAX_INTERFACE_APDU_LENGTH the maximal length of the APDU that it supports for conveying a cEMI message from a cEMI Client over the KNXnet/IP Tunnelling interface to the field level medium.

4.7.17 PID_MAX_LOCAL_APDU_LENGTH (PID: 69)

- **Property name:** APDU-length for local management
- **Property Datatype:** PDT_UNSIGNED_INT
- **Datapoint Type:** None
- **Access Policy:** 3FF/OCC
- **Access level:** 3/X

4.7.17.1 Abstract Resource definition

This Management Server shall hold in PID_MAX_LOCAL_APDU_LENGTH the maximal length of the APDU that it supports for conveying a cEMI message from a cEMI Client over the KNXnet/IP Tunnelling interface to the own management in the cEMI interface device.

4.8 Group Object Table Object (Object Type 9)

4.8.1 PID_GO_DIAGNOSTICS (PID: 66)

- **Property name:** Group Object Diagnostics
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None
- **Access Policy:** See below.
- **Access Level:** 3/3

The Function Property *Group Object Diagnostics* shall be used by the MaC to set or read the value of a GO in the MaS and to request the transmission or reading of the GO-value from the bus.

In the commands and responses, the field Group Object Number shall refer to the ASAP, this is, the Group Object Number in the Group Object Table.

NOTE 29 The *Group Object Number* is typical S-Mode and is an absolute number of the Group Object for the entire application. This differs to the *Group Object Index*, which is typically used in E-Mode and is the number of the Group Object relative to its E-Mode Channel. See 4.11.16.

The size and contents shall depend on the specific command.

NOTE 30 The functionality of the Property overlaps with the Property PID_OBJECT_VALUE (specified in 4.3.13), used for the "Group Object Indirection" as required in [08]. That Property however does not support the reading or writing of GO-values from the bus.

Access

The access to PID_GO_DIAGNOSTICS may be implementation specific: it may be accessible in plain (without KNX Data Security), with authentication or with confidentiality. Additionally, it may be possible that the access conditions depend on the GO that is being accessed or on the command that is requested. This can be solved through the implementation of Roles. In any case, Group Object Diagnostics shall not have lower security access conditions to a GO than the access through the group services.

EXAMPLE 17 If a GO is only accessible with A+C, then the implementation of GO Diagnostics shall guarantee that that GO shall also only be accessible at least with A+C.

4.8.1.1 Common requirements

4.8.1.1.1 Interaction of PID_GO_DIAGNOSTICS with the GO-server

PID_GO_DIAGNOSTICS shall access to the Group Objects via the Group Object Server, just like the standard Group AL-services. Consequently, the commands given through PID_GO_DIAGNOSTICS may set the communication flags of the Group Object and may be subject to evaluation of the Config flags.

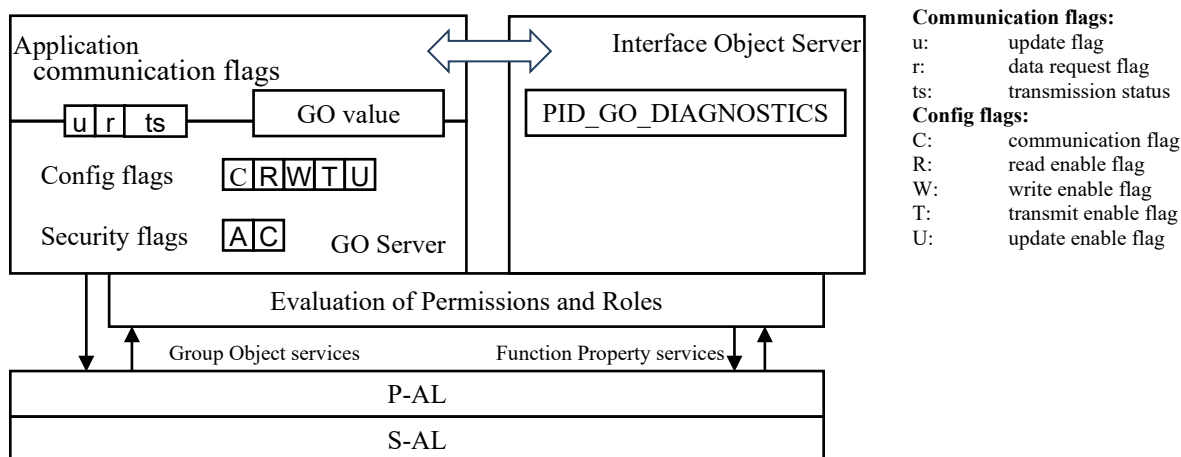


Figure 21 – Interaction of PID_GO_DIAGNOSTICS with the GO-server

4.8.1.1.2 Communication flags

The below requirements are specified by - amongst other - referring to the communication flags. The presence and implementation of these flags is however implementation specific; there are no standard Resources for these flags. The property PID_GO_DIAGNOSTICS shall not directly access the communication flags: this shall be done by the standard Group Object Server, as a consequence of the access by PID_GO_DIAGNOSTICS. In the below commands specification, the indication on the communication flags is only a possibility and not a requirement.

For the specification of the communication flags, please refer to [08].

4.8.1.1.3 Size of the GO-value

Alignment

In the commands and in the responses that carry data, if the data does not fit exactly within the octet boundary, then it shall be right aligned on the least significant bit positions. Unused bits shall thus be at the most significant positions. The MaC shall not set unused bits. The MaS shall ignore the value of these bits.

Size verification

The commands foresee a return code for the case when data is written to a GO-value that is too large or too small.

This is however limited to values written with at least one octet too many or too few.

4.8.1.1.4 Communication Mode and Transport Layer connections

The MaS shall accept access to *Group Object Diagnostics* both connectionless as well as connection-oriented.

4.8.1.1.5 Operation Mode dependency

The dependency of the Group Object Diagnostics functionality on the Operation Mode is specified for each command below.

The MaC shall always set the *Operation Mode* to “Diagnostic Mode” where required. This is indicated in Table 34 and for each of the below commands. If the MaC changes the Operation Mode, then it shall set it back to “Normal Mode” when it finishes its GO Diagnostics.

If the MaS does not accept a GO Diagnostics command because of an incorrect Operation Mode, then it shall respond with the negative Return Code E_COMMAND_IMPOSSIBLE (see below).

4.8.1.1.6 Common Positive Return Codes

00h E_SUCCESS

octet 10	octet 11
Return Code	ServiceID
1 octet	1 octet
00h	Read- or WriteServiceID as in the request

20h E_GD_CONFIG

(Function specific positive Return Code)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16
Return Code	ServiceID	ServiceResult				
		Group Object Number		GO Config		Size
1 octet	1 octet	2 octets		2 octets		1 octet
20h	Read- or WriteServiceID as in the request					

octet 17	octet 18	octet 19	octet 20
ServiceResult			
DPT_ID			
main number		subnumber	

Figure 22 – A_FunctionPropertyState_Response-APDU for PID_GO_DIAGNOSTICS in case of E_GD_CONFIGThe field *GO Config* shall be encoded as follows.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	r	r	r	r	r	L	conf	auth	U	T	I	W	R	C	Priority	
	0	0	0	0	0											

Figure 23 – Encoding of the field GO Config in E_GD_CONFIG

The MaS shall set reserved field (r) to 0. The MaC shall ignore the value of these fields.

Bits 0 to 7 shall be encoded as the higher octet of the Group Object Descriptor for System B (Group Object Table Realisation Type 7, clause 4.18.6.2.4.1.1 - Table 87).

The fields *conf* and *auth* in bits 9 and 8 shall be encoded as PID_GO_SECURITY_FLAGS in 6.3.15.

The *Linked* field (L) shall indicate whether or not at least one Group Address is linked to this Group Object or not.

0: This GO is not linked to any GA.

1: At least one GA is linked to this GO.

NOTE 31 This bit is not part of the Group Object Descriptor in the Group Object Table. The MaS may calculate and conclude upon the value of this bit only when executing this command.

NOTE 32 This missing link may be a cause for the MaS to return E_ERROR for the command Send local GO value on bus.

The field *Size* shall be encoded as the *Value Field Type* again as in the Group Object Descriptor of System B (Group Object Table Realisation Type 7, clause 4.18.6.2.4.1.1 - Table 87).

The field *DPT_ID* shall contain the Datapoint Type Identifier of the Datapoint Type according to which the value of the Group Object is encoded. This shall be composed of the main number and the subnumber as specified in [14].

- ▲ The DPT_ID may vary, for instance under the control of a Parameter. The MaS shall however always return the current DPT_ID.
- ▲ The MaS may return the value 00000000h for this field. This shall denote that this DPT information is invalid, for instance because no DPT_ID is available for this GO or because the DPT_ID information is not supported.

21h E_GD_GO_STATUS_VALUE

(Function specific positive Return Code)

octet 10	octet 11	octet 12	octet 13	octet 14	octets 15 to 15+n-1
Return Code	Service-ID	Service Result			
		Group Object Number	GO Status	Data: Object Value	
1 octet	1 octet	2 octets	1 octet	1 to n octets	
21h	Read- or Write-ServiceID as in the request				

Figure 24 – A_FunctionPropertyState_Response-APDU for PID_GO_DIAGNOSTICS in case of E_GD_GO_STATUS_VALUE

- The field *GO Status* shall be encoded as follows.

Bit	7	6	5	4	3	2	1	0
Field	r	r	r	r	update	data request	Transmission Status	
	0	0	0	0				

Figure 25 – Encoding of the field GO Status in E_GD_GO_STATUS_VALUE

The MaS shall set reserved field (r) to 0. The MaC shall ignore the value of these fields.

- The field *Transmission Status* shall be encoded as follows.

Table 33 – Encoding of Transmission Status

Code	Name	Description
00h	Idle; OK.	No transmission is going on. The preceding transmission was successful.
01h	Idle; Error.	No transmission is going on. The preceding transmission failed. The AL confirms the service negatively. This may be due to a security problem (e.g. transmission request on a GO for which there is no entry in the Security Link Table), but may have any other existing ground (e.g. a negative confirmation by Layer 2).
02h	Transmitting	There is a transmission going on for this GO-value (Write – or read-request).
03h	Transmit request	There is a transmission requested for this GO-value (Write – or read-request) but it has not yet started.

The Transmission Status “Transmitting” and “Transmit request” are intermediate states. It is not required that the MaS supports these states. If the MaS returns any of these states, then the MaC shall leave the MaS and the KNX network sufficient time to complete request, typically 6 s and read the final result by a subsequent “ReadServiceID 01h – Get local GO value” (see 4.8.1.4.3), until either “Idle; OK” or “Idle; Error” is returned.

The encoding of these fields *Update* and *Data Request* shall be identical as in the Communication Flags in the RAM-Flags-Table in (4.18.3.1.1.1).

4.8.1.1.7 Common Negative Return Codes

Several Negative Return Codes for the Group Object Diagnostic are meaningful for more than one command. Therefore, the Negative Return Codes are specified here. In the specification of the command, the possible Negative Return Codes are only listed.

A1h E_GD_GO_VOID

(Function Specific Negative Return Code).

The addressed data is a GO with a number that does not exist in this MaS.

octet 10	octet 11
Return Code	ServiceID
1 octet	1 octet
A1h	Read- or Write-ServicelD as in the request

Figure 26 - E_GD_GO_VOID

NOTE 33 The generic Return Code E_ADDRESS_VOID is not allowed.

A2h E_GD_CONFIG_FLAGS

(Function Specific Negative Return Code).

GO Config flags error:

The GO Config flags do not allow handling this request.

octet 10	octet 11
Return Code	ServiceID
1 octet	1 octet
A2h	Read- or Write-ServicelD as in the request

Figure 27 - E_GD_CONFIG_FLAGS

A3h E_GD_GO_SIZE_MISMATCH

(Function Specific negative Return Code.)

The provided data is too short or too long for the value of the referred GO.

octet 10	octet 11
Return Code	ServiceID
1 octet	1 octet
A3h	Read- or Write-ServicelD as in the request

Figure 28 - E_GD_GO_SIZE_MISMATCH

NOTE 34 The Generic Return Code E_DATA_TYPE_CONFLICT is not allowed.

F2h E_COMMAND_INVALID

(Generic negative Return Code.)

The server does not support the requested command. The Interface Object and the Property are supported, but the requested ReadServiceID or WriteServiceID is not supported.

octet 10	octet 11
Return Code	ServiceID
1 octet	1 octet
F2h	Read- or Write-ServicelD as in the request

Figure 29 - E_COMMAND_INVALID

In case both an invalid command is requested and also the GO is invalid, then this Return Code E_COMMAND_INVALID shall be returned. In the request, the first octet shall always be the Command; the MaS shall not assume that for an unknown command, the next two octets would identify the GO number; it shall thus not interpret these two octets and shall return E_COMMAND_INVALID only.

F3h E_COMMAND_IMPOSSIBLE

(Generic negative Return Code.)

The MaS does support the addressed Interface Object and Property and supports the requested ReadServiceID or WriteServiceID and the data (ServiceInfo) is properly formatted, but the MaS cannot execute the command.

The current Operation Mode does not allow the requested command.

octet 10	octet 11
Return Code	ServiceID
1 octet	1 octet
F3h	Read- or Write-ServicelD as in the request

Figure 30 - E_COMMAND_IMPOSSIBLE

FFh E_ERROR

(Common negative Return Code.)

This is the negative response without further Additional Information.

The code shall only be returned in case none of the above Return Codes applies.

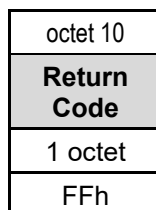
**Figure 31 - E_ERROR****4.8.1.2 Overview of the commands**

Table 34 below specifies the Operation Mode dependency of the commands. The commands shall be available exclusively in the Operation Modes as indicated.

EXAMPLE 18 Command 02h “Send local GO value on bus” and command 04h “Limit group service senders” shall only be available if the Operation Mode is “Diagnostic Mode”. These commands shall not be available if the Operation Mode is “Normal Mode”; if these commands are requested in that case, then the MaS shall respond with E_COMMAND_IMPOSSIBLE.

Table 34 – Overview of the commands and Operation Mode dependency

ServiceID	Command	Operation Mode
WriteServiceID 00h	Set local GO value	Normal Mode, Diagnostic Mode
WriteServiceID 01h	Send A_GroupValue_Write	Normal Mode, Diagnostic Mode
WriteServiceID 02h	Send local GO value on bus	Normal Mode, Diagnostic Mode
WriteServiceID 03h	Send A_GroupValue_Read	Normal Mode, Diagnostic Mode
WriteServiceID 04h	Limit group service senders	Diagnostic Mode
ReadServiceID 00h	Get GO Config	Normal Mode, Diagnostic Mode
ReadServiceID 01h	Get local GO value	Normal Mode, Diagnostic Mode

Please refer to the detailed specifications of the commands for the requirements on the MaS.

4.8.1.3 Write (A_FunctionPropertyCommand-PDU)**4.8.1.3.1 Basic format and overview****Overview WriteServiceIDs****Table 35 – Overview PID_GO_DIAGNOSTICS WriteServiceIDs**

WriteServiceID	Description
00h	Set local GO value
01h	Send A_GroupValue_Write
02h	Send local GO value on bus
03h	Read GO value from bus
04h	Limit GO service senders

4.8.1.3.2 WriteServiceID 00h - Set local GO value

The MaC shall use this command to set the value of a GO in the MaS. This shall allow the MaC to check whether an unexpected behaviour of the MaS may be due to an ignoring of the message by the MaS due to KNX Data Security restrictions (e.g. Roles and Permissions) or due to another reason, e.g. at application level or in the communication stack.

Operation Mode dependency

The MaS shall make the command “Set local GO value” available exclusively in the following Operation Modes.

- Normal Mode
- Diagnostic Mode

An overview of the Operation Mode dependency of the commands is given in Table 34.

Command execution

The Group Object Server in the MaS shall accept the command. This command shall not be subject to further checking of Permissions and Roles. This command shall be handled as an accepted A_GroupValue_Write-PDU for the GO. Its result shall however still be subject to the Group Object Config Flags for the GO. If the GO-value can be written, then the GO Server may set the update-flag of the GO. The execution of this command does not require that the MaS sends the value of this GO on the bus. However, it may be that the subsequent normal behaviour of the application causes a transmission of the GO-value, e.g. if there is a COV-condition for the transmission. In the same way, the command execution does not require that other GOs in the MaS that may be linked to the same GAs as this GO shall be influenced, but this may be a consequence of the application.

NOTE 35 This is thus to be interpreted as if an A_GroupValue_Write-PDU is received from the bus for this GO. It relates in the same way to the Config flags and the communication flags. It differentiates from receiving a Frame from the bus, as it for instance will not use GAs in the GA-Table or links in the Association Table.

NOTE 36 This command operates at the level of the Group Object and not at the level of the Group Address. Other Group Objects in the MaS that are linked to the same GA(s) as the GO that is accessed will not change value. To test for instance a “central off” functionality one would rather trigger the transmission at a sending GO using “WriteServiceID 01h - ” or “WriteServiceID 02h - Send local GO value on bus”.

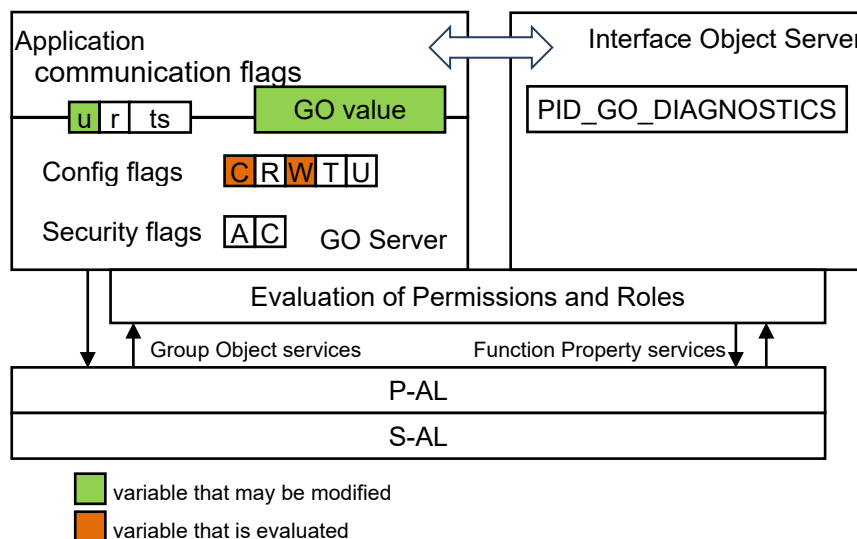


Figure 32 - PID_GO_DIAGNOSTICS working of WriteServiceID 00h “Set local GO value” (EXAMPLE)

a) Write

octet 10	octet 11	octet 12	octet 13	octet 14 to 14+n-1
Reserved	Write-ServiceID	ServiceInfo		
	Command	Group Object Number	Data: Object Value	
	1 octet	2 octets	n octets	
00h	00h			

Object Value: value that shall be given to the GO-value.

**Figure 33 - A_FunctionPropertyCommand-PDU
for WriteServiceID 00h - Set local GO value**

b) Positive Return Codes

If the command execution does not result in any error then the Return Code `E_GD_GO_STATUS_VALUE` (21h; see above) shall be returned. The Object Value shall be the current value of the GO.

c) Negative Return Codes

Exclusively the following Negative Return Codes shall be possible. Please refer to 4.8.1.1.7 for the specification of these Return Codes.

Table 36 – Negative Return Codes for WriteServiceID 00h - Set local GO value

Code	Name	Short description
A1h	<code>E_GD_GO_VOID</code>	The addressed data is a GO with a number that does not exist in this MaS.
A2h	<code>E_GD_CONFIG_FLAGS</code>	The GO Config flags do not allow handling this request.
A3h	<code>E_GD_GO_SIZE_MISMATCH</code>	The provided data is too short or too long for the value of the referred GO. This shall also be returned in case the field Data is missing, this is, if the size of the field <i>ServiceInfo</i> is only 2 octets.
F2h	<code>E_COMMAND_INVALID</code>	The server does not support the requested command.
FFh	<code>E_ERROR</code>	The code shall only be returned in case none of the above Return Codes applies.

4.8.1.3.3 WriteServiceID 01h - Send A_GroupValue_Write

The MaC shall use this command to make the MaS transmit an `A_GroupValue_Write`-PDU on the bus.

This shall allow the MaC to send a value on a GA, even if the IA of the MaC is not available in all devices receiving on that GA or if the MaC does not have a valid SeqNr for sending. The MaC uses the MaS to transmit the GO-value in its place.

Operation Mode dependency

The MaS shall make the command “Send `A_GroupValue_Write`” available exclusively in the following Operation Modes.

- Normal Mode
- Diagnostic Mode

An overview of the Operation Mode dependency of the commands is given in Table 34.

Command execution

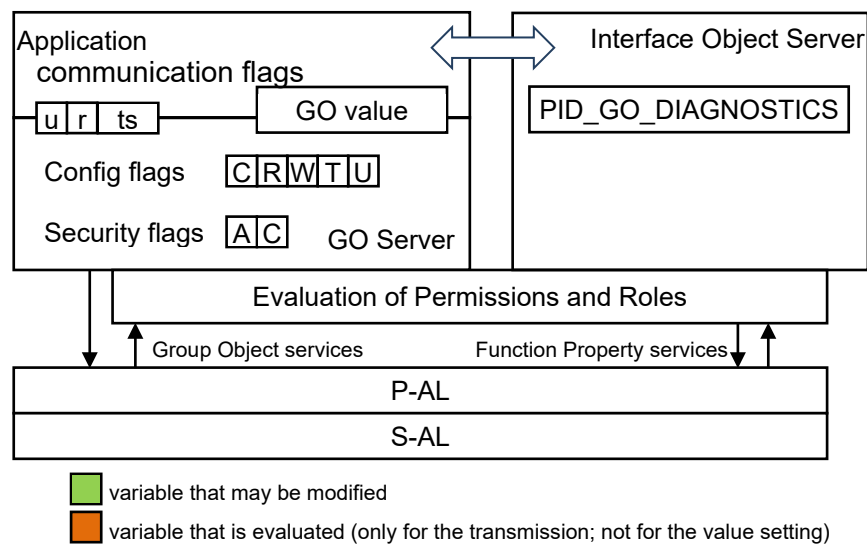


Figure 34 - PID_GO_DIAGNOSTICS working of WriteServiceID 01h “Send A_GroupValue_Write” (EXAMPLE)

NOTE 37 Please refer to “The relation between TSAPs and ASAPs” in [07]: the transmission of one GO-value via A_GroupValue_Write.req shall lead to an A_GroupValue_Write.ind for each other GO that is linked to the same GA. These indication service primitives will inherit the security service parameters par_auth and par_conf from the request service primitive. The further effects depend on the C- and W-flags of these GOs and on their GO Security Flags (PID_GO_SECURITY_FLAGS: see 6.3.15. However, all GOs linked to the same GA should normally have the same Security Flags (conf and auth), so, it should normally not happen that the transmission of a plain/secure GO leads to an update of a secure/plain other GO. The evaluation of the internal GO-config and GO Security flags is not indicated in the above Figure 34.

a) Write

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15... octet 15 + n - 1
Reserved	Write-ServiceID	ServiceInfo			
	Command	Flags	Group Address		Data
	1 octet	1 octet	2 octets		n octets
			MSB	LSB	MSB
00h	01h				

Figure 35 - A_FunctionPropertyCommand-PDU for WriteServiceID 01h - Send A_GroupValue_Write

- Flags:

Bit	7	6	5	4	3	2	1	0
Field	S	r	r	r	r	r	conf	auth
		0	0	0	0	0		

- Bit 7 (msb) “Selector” (S)

This flag shall indicate whether the *Data* shall be sent if possible in the 6 trailing bits after the 4 bit APCI in the field “data/APCI” in the A_GroupValue_Write-PDU.

This flag shall only be evaluated in case the field *Data* has a length of 1 octet and a value of less than 64.

0: The data shall be included in the 6 trailing bits “data/APCI” after the 4 bit APCI.

1: The data shall be included in the first next full octet after the 4 bit APCI.

In case the field *Data* is 1 octet long and has a value equal to or larger than 64 or in case the length is longer than 1 octet (even if the value is smaller than 64), the MaS shall ignore the value of this flag. This shall not be a condition for reporting with an error or for refusing the command.

- Bits 6 to 2 are reserved for further standard extensions and shall be 0.

- Bits 1 and 0 (lsb) shall have the same meaning as the GO Security flags as specified in [07].

- Group Address: This shall be the target GA of the Telegram to be sent.

- Data: This shall be the value that shall be sent on the bus.

The MaS shall do the following actions.

1. The MaS shall initiate the transmission of an A_GroupValue_Write-Telegram on the bus on the requested GA and according the requested Flags.

If the Flags require the use of Data Security, then the MaS shall accordingly secure the outgoing message using its own Sequence Number Sending and the key as found for the requested GA in its Group Key Table. Errors resulting from this shall be responded to the MaC with the appropriate Return Code.

The Return Code shall however be independent of the success or failure of the actual transmission. Failure of transmission, absence of a Layer 2 acknowledge for media where this is supported or Layer 2 repetitions shall for this not lead to a negative Return Code.

2. The MaS shall additionally pass an A_GroupValue_Write.ind to its own Application Layer with the used security service parameters (par_auth and par_conf) from the command; the link_index shall be “None”. All GOs linked to the requested GA shall be updated if this is accepted according these flags and according their GO config flags.

The MaS shall not inform the MaC on the outcome of this action, regardless of whether none, only a subset or all of the linked GOs is updated. (Please refer to the indications concerning reading out the local GO-values and the possible retries in [09] clause DMP_GA_DIAGNOSTICS_RCI).

The MaS shall handle this command 01h just as command 00h and initiate the update of the GO-value, ignoring possible Roles and Permissions. However, the GO Config flags shall be evaluated for setting the local GO-value.

NOTE 38 This means that the W-flag shall be evaluated for the setting of the GO-value through this command.

NOTE 39 If the size of the field *Data* does not match with the size of the associated internal GOs, then this will not lead to a negative Return Code to the requesting MaC. No Return Code is foreseen for this. The standard error handling in the AIL of the MaS applies concerning length verification of received group messages. Also, the message will be sent on the bus with the inappropriate length of the field *Data* in the *A_GroupValue_Write*-PDU!

b) Positive Return Codes

If the command execution does not result in any error then the Return Code *E_SUCCESS* (00h; see above) shall be returned.

c) Negative Return Codes

Exclusively the following Negative Return Codes shall be possible. Please refer to 4.8.1.1.7 for the specification of these Return Codes.

Table 37 – Negative Return Codes for WriteServiceID 01h - Send *A_GroupValue_Write*

eCode	Name	Short description
F8h	E_DATA_VOID	The <i>ServiceInfo</i> contains invalid data for any of the following fields. • <i>Flags</i> - Any of the reserved bits does not have the value 0. - An invalid combination of Flags is requested. - Security is requested for GA for which there is no GA Key. • <i>Group Address</i> - An unknown GA is requested. • <i>Data</i> - The field <i>Data</i> is missing. (The <i>ServiceInfo</i> is only 2 octets long.)
FFh	E_ERROR	The code shall only be returned in case none of the above Return Codes applies.

4.8.1.3.4 WriteServiceID 02h - Send local GO value on bus

The MaC shall use this command to request the MaS to send its current GO value on the bus. This shall allow the MaC to check whether an unexpected behaviour of the MaS may be due to an ignoring of the message by the MaS due to KNX Data Security restrictions (e.g. request of secure transmission of a DP for which there is no secure link) or due another ground, e.g. at application level.

Operation Mode dependency

The MaS shall make the command “Send local GO value on bus” available exclusively in the following Operation Modes.

- Normal Mode
- Diagnostic Mode

An overview of the Operation Mode dependency of the commands is given in Table 34.

Command execution

The MaS shall accept the command and initiate the normal transmission of the GO on the bus. This may influence the communication flags and shall be subject to evaluation of the Config flags and the Group Object Security flags (plain, authentication only or authentication and confidentiality).

NOTE 40 Please also refer to NOTE 37: the transmission of one GO-value may lead to the update of other GOs in the same device linked to the same GA.

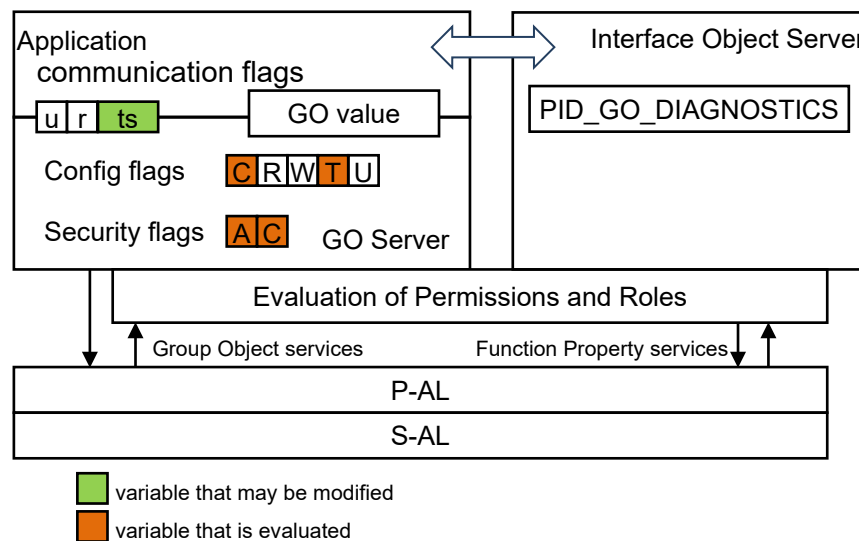


Figure 36 - PID_GO_DIAGNOSTICS working of WriteServiceID 02h “Send local GO value on bus” (EXAMPLE)

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12	octet 13
Reserved	Write-ServiceID	ServiceInfo	
	Command	Group Object Number	
	1 octet	2 octets	
		MSB	LSB
00h	02h		

Figure 37 - A_FunctionPropertyCommand-PDU for WriteServiceID 02h - Send local GO value on bus

b) Positive Return Codes

If the command execution does not result in any error then the Return Code *E_GD_GO_STATUS_VALUE* (21h; see above) shall be returned. The contained command shall be 02h. The Group Object Number shall be repeated from the request. The field *GO Status* shall at least be an immediate response by this Function Property about the initiation of the transmission request of the GO-value. This response *may* indicate the result (success or error) of the further handling of this request. This situation shall not be indicated in the response. If the MaC wants to know the *final result* of this command, it shall leave the MaS sufficient time (up to 6 s) to complete the command and then request the state through a subsequent “ReadServiceID 01h – Get local GO value” (see 4.8.1.4.3).

EXAMPLE 19 The transmission may fail because no GA is linked to the addressed GO. (This shall be marked in the flag L in the field GO Config in *E_GD_CONFIG*). The transmission may also fail because of misconfigured Data Security (missing Group Key), mismatches between Group Object Table, Group Object Association Table and Group Address Table or because of transmission failure on the medium.

Negative Return Codes

Exclusively the following Negative Return Codes shall be possible. Please refer to 4.8.1.1.7 for the specification of these Return Codes.

Table 38 – Negative Return Codes for WriteServiceID 02h - Send local GO value on bus

Code	Name	Short description
A1h	E_GD_GO_VOID	The addressed data is a GO with a number that does not exist in this MaS.
A2h	E_GD_CONFIG_FLAGS	The GO Config flags do not allow handling this request.
F2h	E_COMMAND_INVALID	The server does not support the requested command.
F3h	E_COMMAND_IMPOSSIBLE	The requested command is supported but requires the Operation Mode to be “Diagnostic Mode” and the current value is “Normal Mode”.
FFh	E_ERROR	The code shall only be returned in case none of the above Return Codes applies.

4.8.1.3.5 WriteServiceID 03h – Send A_GroupValue_Read

The MaC shall use this command to make the MaS transmit an A_GroupValue_Read-PDU on the bus.

Operation Mode dependency

The MaS shall make the command “Send A_GroupValue_Read” available exclusively in the following Operation Modes.

- Normal Mode
- Diagnostic Mode

An overview of the Operation Mode dependency of the commands is given in Table 34.

Command execution

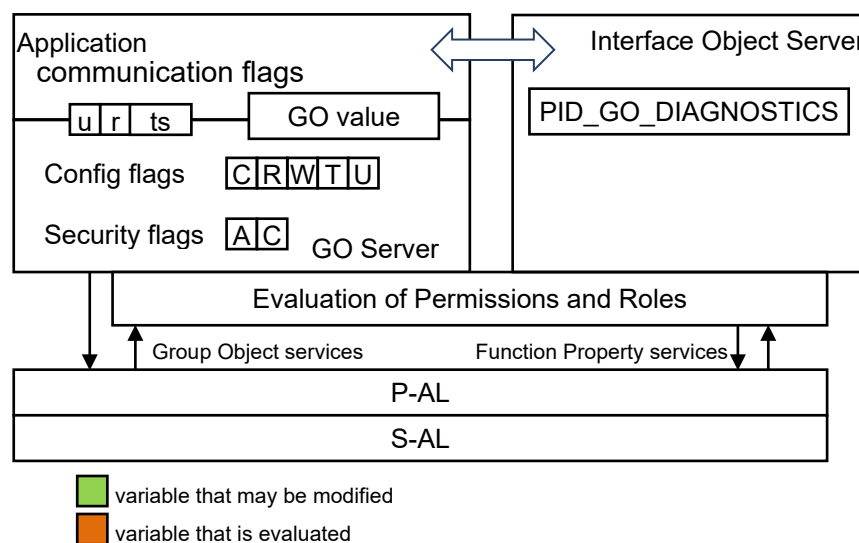


Figure 38 - PID_GO_DIAGNOSTICS working of WriteServiceID 03h “Send A_GroupValue_Read” (EXAMPLE)

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14
Reserved	Write-ServiceID	ServiceInfo		
	Command	Flags	Group Address	
	1 octet	1 octet	2 octets	
			MSB	LSB
00h	03h			

**Figure 39 - A_FunctionPropertyCommand-PDU
for WriteServiceID 03h – Send A_GroupValue_Read**

- Flags:

Bit	7	6	5	4	3	2	1	0
Field	r	r	r	r	r	r	conf	auth
	0	0	0	0	0	0		

- Bits 7 (msb) to 2 are reserved for further standard extensions and shall be 0.
- Bits 1 and 0 (lsb) shall have the same meaning as the GO Security flags as specified in [07].

- Group Address: This shall be the target GA of the Telegram to be sent.

- The MaS shall initiate the transmission of an A_GroupValue_Read-Telegram on the bus on the requested GA and according the requested Flags.

If the Flags require the use of Data Security, then the MaS shall accordingly secure the outgoing message using its own Sequence Number Sending and the key as found for the requested GA in its Group Key Table. Errors resulting from this shall be responded to the MaC with the appropriate Return Code.

The Return Code shall however be independent of the success or failure of the actual transmission. Failure of transmission, absence of a Layer 2 acknowledge for media where this is supported or Layer 2 repetitions shall for this not lead to a negative Return Code.

- The MaS shall additionally pass an A_GroupValue_Read.ind to its own Application Layer with the used security service parameters (par_auth and par_conf) from the command; the link_index shall be “None”. The read indication shall be passed to all GOs linked to the requested GA and these shall respond according the GO security flags and GO config flags.

The MaS shall not inform the MaC on the outcome of this action, regardless of whether none, only a subset or all of the linked GOs respond to the request.

NOTE 41 The definition of this command does not foresee that any resulting A_GroupValue_Response-PDU from the MaS itself or from any other device over the bus, is communicated back to the requesting MaC. The MaC can retrieve the result by monitoring the bus communication in parallel or by issuing an ReadServiceID 01h “Get local GO value” to a linked GO in the MaS afterwards.

b) Positive Return Codes

If the command execution does not result in any error then the Return Code E_SUCCESS (00h; see above) shall be returned.

NOTE 42 If the transmission of the A_GroupValue_Read-PDU fails, for instance because of security reasons or because there is no Layer-2 Acknowledge from the bus or because there is simply no response from any communication partner, then still this will result in the Positive Return Code.

c) Negative Return Codes

Exclusively the following Negative Return Codes shall be possible. Please refer to 4.8.1.1.7 for the specification of these Return Codes.

Table 39 – Negative Return Codes for WriteServiceID 03h – Send A_GroupValue_Read

Code	Name	Short description
F2h	E_COMMAND_INVALID	The server does not support the requested command.
F8h	E_DATA_VOID	The <i>ServiceInfo</i> contains invalid data for any of the following fields. • <i>Flags</i> - Any of the reserved bits does not have the value 0. - An invalid combination of Flags is requested. - Security is requested for GA for which there is no GA Key. • <i>Group Address</i> - An unknown GA is requested.
FFh	E_ERROR	The code shall only be returned in case none of the above Return Codes applies.

4.8.1.3.6 WriteServiceID 04h – Limit GO service senders

The MaC shall use this command to make that the MaS limits its incoming group communication to a set sender Individual Address only.

NOTE 43 This command is mainly motivated for use in use case 3 (see A.3): it shall prevent that after the “BinaryInput” sends its group Telegram to the “SwitchingActuator” (GroupValueWrite in Figure 84 and before the ETS subsequently read the GO value (GetLocalGOValue), any coincidental communication to the involved Group Object by any other device will overwrite the value set by the BinaryInput. Therefore, ETS should firstly filter the incoming group communication in the SwitchingActuator to the IA of the BinaryInput.

Operation Mode dependency

The MaS shall make the command “Limit GO service senders” available exclusively in the following Operation Modes.

- Diagnostic Mode

An overview of the Operation Mode dependency of the commands is given in Table 34.

Command execution

The MaS shall accept the command and immediately limit its received group communication to only the sender Individual Address as contained in the command.

- This shall only concern the services A_GroupValue_Read and A_GroupValue_Write. Other multicast services or other accesses to the Group Object (like further GO Diagnostics commands) shall not be limited by this command.
- The group messages shall still be acknowledged on Layer 2 if required by the medium.

The MaS shall automatically clear this limitation when the Operation Mode becomes no longer Diagnostic Mode (see 4.4.1.4).

a) Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12	octet 13
Reserved	Write-ServiceID	ServiceInfo	
	Command	Sender IA	
	1 octet	2 octets	
		MSB	LSB
00h	04h		

Figure 40 - A_FunctionPropertyCommand-PDU for WriteServiceID 04h – Limit GO service senders

b) Positive Return Codes

If the command execution does not result in any error then the Return Code E_SUCCESS shall be returned, without additional data.

c) Negative Return Codes

If the command execution encounters any error, then the Return Code E_ERROR shall be returned, without additional data.

Table 40 – Negative Return Codes for WriteServiceID 04h – Limit GO service senders

Code	Name	Short description
F3h	E_COMMAND_IMPOSSIBLE	The requested command is supported but requires the Operation Mode to be “Diagnostic Mode” and the current value is “Normal Mode”.

4.8.1.4 Read (A_FunctionPropertyState_Read-PDU)**4.8.1.4.1 Basic format and overview****Overview ReadServiceIDs**

Table 41 – Overview PID_GO_DIAGNOSTICS ReadServiceIDs

ReadServiceID	Description
00h	Get GO Config
01h	Get local GO value

4.8.1.4.2 ReadServiceID 00h – Get GO Config

The MaC shall use this command to get the configuration of a GO in the MaS. This shall allow the MaC to check whether an unexpected behaviour of the MaS may be due to an unexpected configuration of the GO or not.

NOTE 44 There is no command to set the GO Config; this is reserved for the normal configuration of the MaS according its Configuration Procedures.

Operation Mode dependency

The MaS shall make the command “Get GO Config” available exclusively in the following Operation Modes.

- Normal Mode
- Diagnostic Mode

An overview of the Operation Mode dependency of the commands is given in Table 34.

Command execution

The MaS shall respond with the current GO configuration of the indicated GO. The execution shall not be subject to or modify any runtime – or configuration flags.

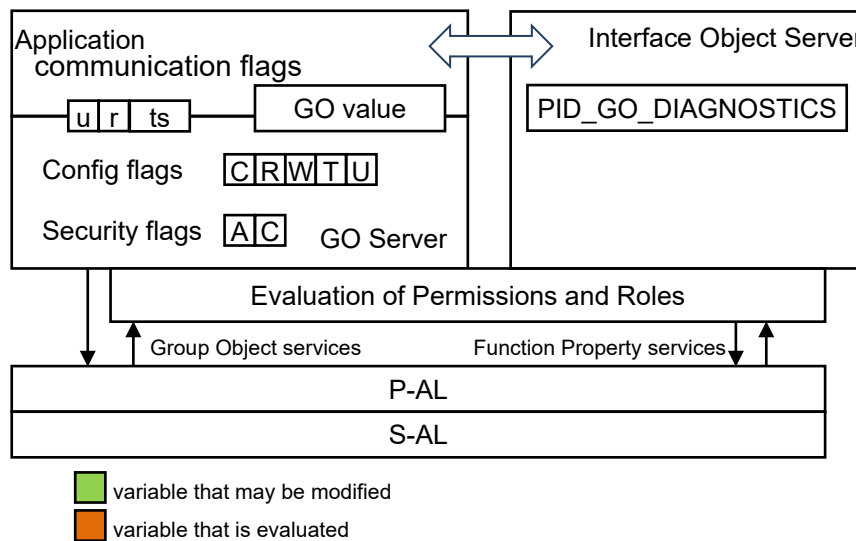


Figure 41 - PID_GO_DIAGNOSTICS working of ReadServiceID 00h “Get GO Config” (EXAMPLE)

a) Read (A_FunctionPropertyState_Read)

octet 10	octet 11	octet 12	octet 13
Reserved	Read-ServiceID	ServiceInfo	
	Command	Group Object Number	
	1 octet	2 octets	
		MSB	LSB
00h	00h		

Figure 42 - A_FunctionPropertyState_Read-PDU for ReadServiceID 00h – Get GO Config

b) Positive Return Codes

If the command execution does not result in any error then the Return Code E_GD_CONFIG (20h; see above) shall be returned.

c) Negative Return Codes

Exclusively the following Negative Return Codes shall be possible. Please refer to 4.8.1.1.7 for the specification of these Return Codes.

Table 42 – Negative Return Codes for ReadServiceID 00h – Get GO Config

Code	Name	Short description
A1h	E_GD_GO_VOID	The addressed data is a GO with a number that does not exist in this MaS.
F2h	E_COMMAND_INVALID	The server does not support the requested command.
FFh	E_ERROR	The code shall only be returned in case none of the above Return Codes applies.

4.8.1.4.3 ReadServiceID 01h – Get local GO value

The MaC shall use this command to read the current GO value from the MaS.

Operation Mode dependency

The MaS shall make the command “Get local GO value” available exclusively in the following Operation Modes.

- Normal Mode
- Diagnostic Mode

An overview of the Operation Mode dependency of the commands is given in Table 34.

Command execution

The MaS shall respond with the current value of the indicated GO. This may not modify any communication flag and it shall not be subject to any Config flag or Group Object Security flag. The MaS shall send a message on the bus to respond to the MaC, but it shall not send any group message on the bus as direct result of this command.

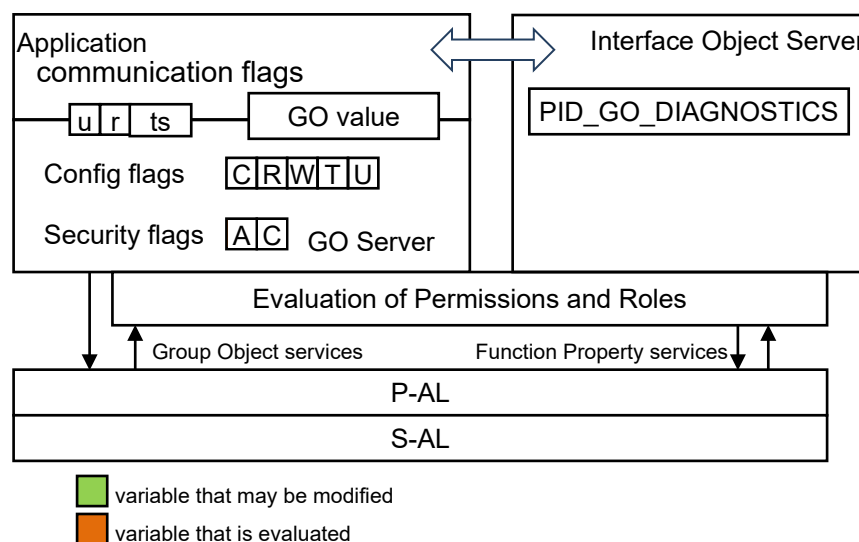


Figure 43 - PID_GO_DIAGNOSTICS working of ReadServiceID 01h “Get local GO value” (EXAMPLE)

a) Read (A_FunctionPropertyState_Read)

octet 10	octet 11	octet 12	octet 13
Reserved	Read-ServiceID	ServiceInfo	
	Command	Group Object Number	
	1 octet	2 octets	
		MSB	LSB
00h	01h		

Figure 44 - A_FunctionPropertyState_Read-PDU for ReadServiceID 01h – Get local GO value

b) Positive Return Codes

If the command execution does not result in any error then the Return Code E_GD_GO_STATUS_VALUE (21h; see above) shall be returned.

c) Negative Return Codes

Table 43 – Negative Return Codes for ReadServiceID 01h – Get local GO value

Code	Name	Short description
A1h	E_GD_GO_VOID	The addressed data is a GO with a number that does not exist in this MaS.
F2h	E_COMMAND_INVALID	The server does not support the requested command.
FFh	E_ERROR	The code shall only be returned in case none of the above Return Codes applies.

4.9 KNXnet/IP Parameter Object (Object Type 11)

For the functionality of the KNXnet/IP Parameter Object and the specification of its Properties, please refer to [16].

4.10 File Server Object (Object Type 13)

4.10.1 Overview

Table 44 – File Server Object

Property Name	Property Identifier	PDT/DPT	Description
Interface Object type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT	13 = File Server Object
MAX. APDU-Length	51 = PID_MAX_APDU_LENGTH_OUT	PDT_UNSIGNED_INT	Maximal APDU-length that shall be used during a file transfer.
File Command	52 = PID_FILE_COMMAND	PDT_FUNCTION	Command
File Path	53 = PID_FILE_PATH	PDT_UTF-8[] (alt. PDT_UNSIGNED_CHAR[])	Path to the directory or file
MAX. APDU-Length	56 = PID_MAX_APDU_LENGTH	PDT_UNSIGNED_INT	Maximal APDU-length supported by the File Server.
^a This Property is mandatory if long frames are supported.			

4.10.2 Property MAX. APDU-Length Out (PID = 51)

This Property shall be an optional writable Property and shall provide the maximum APDU-Length that shall be used by this File Server Object for the A_FileStream_InfoReport-service. This Property is mandatory if Long Frames are supported. With this Property a client can adjust the maximum used APDU length for an FTP procedure.

If this Property is not implemented then the used maximal APDU-length shall be 14.

4.10.3 Property File Command (PID = 52)

4.10.3.1 Definition

This Property shall be mandatory and shall be formatted as a PDT_FUNCTION i.e. the number of octets written shall be variable and the number of octets responded shall also be variable. Each command is described in detail below. The interpretation of the Return Code (Error Code) depends on the given Command. A read request to this Property shall always return the command result of the preceding command or “Command successful” if there has been no preceding command.

Prior to any File Command or access to any other Property of the File Server Object, a File Client shall at first request a File Handle from the File Server through the command “Get File Handle”. Only if a valid File Handle is returned, the File Client may use the current File Server Object.

The responded Return Code (Error Code) depends on the given command; these are specified for the different commands below.

4.10.3.2 Overview Return Codes

Table 45 – Summary of command independent Return Codes

Return Code	Code Name	Description
00h	Command successful	This is the general return code to indicate a successful command execution.
01h to 0Fh	Reserved	These Return Codes are reserved for future use and shall not be used.
10h	Invalid command	The preceding command is reserved or not supported by this File Server.
11h to 1Fh	Reserved	These Return Codes are reserved for future use and shall not be used.

Table 46 – Summary of command dependent Return Codes

Return Code	Code Name	Description
20h	Object busy	The File Server indicates that it is busy and cannot accept any file command in this File Server Object.
21h	Command not possible	The given command cannot be executed in the requested way.
22h	Not found	The current path name does not lead to any known file or directory.
23h	Write protected	The specified file or directory is write protected.
24h	Access denied	The specified file or directory is not accessible in the requested way.
25h	No more space available	The File Servers disk or storage memory is full. Incoming data cannot be stored.
26h	Directory not empty	The specified directory is not empty and can therefore not be removed.
27h to FFh	Reserved	These Return Codes are reserved for future use and shall not be used.

4.10.3.3 Command

4.10.3.3.1 Overview of the commands and general requirements

Most commands affect the treatment of a file and shall therefore always be executed with a file path or directory path as a parameter. This parameter shall either be written to the Property PID_FILE_PATH or shall be part of the command. This is specified for every individual command below.

If a file path or directory path is part of the command, the same coding rules shall be applied as defined for the Property File Path. See 4.10.4.

There are commands for an FTP based client, for an HTTP based clients and for both. This has the advantage that the same file can be read in different modes depending on its usage in the client.

EXAMPLE A heading part and a footing part can automatically be added to an HTML file, when it is read with an HTTP based client. This will not be done with an FTP based client.

Every command shall be a single, isolated event, this is, the File Server shall interpret any command without knowledge of any preceding commands. The exception to this rule shall be the command “Rename To”; this command shall only be executed if the preceding commands is a command “Rename From”.

Table 47 – File Commands

Command value	Command name	Parameter in other Properties	Mandatory / optional
00h	Reserved		O

Command value	Command name	Parameter in other Properties	Mandatory / optional
01h	Get File Handle		M
02h to 10h	Reserved		O
11h	Retrieve File (ftp)	file path	O
12h	Store File (ftp)	file path	O
13h	List Directory (ftp)	dir path	O
14h	Rename From (ftp)	file path / dir path	O
15h	Rename To (ftp)	file path / dir path	O
16h	Delete (ftp)	file path	O
17h	Remove Directory (ftp)	dir path	O
18h	Make Directory (ftp)	dir path	O
19h to 1Ch	Reserved		O
1Dh	Get File Size (ftp)	file path	O
1Eh	Get Empty Disk Space (ftp)	n/a	O
1Fh	Abort (http, ftp)	n/a	M ^a
20h	Reserved		O
21h	Get File (http)	file path	O
22h	Post File (http)	file path	O
23h to FFh	Reserved		O
^a This command is mandatory only if one of the following commands are implemented: Retrieve File, Get File, List Directory.			

4.10.3.3.2 Command “Get File Handle” (command value 01h)

This command shall cause the File Server to return a new valid File Handle.

Property File Command - Write (A_FunctionPropertyCommand-PDU)

octet 10
Command Value

Property File Command - Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11
Return Code	Return Value

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate a successful request of a File Handle. The Return Value shall denote the File Handle that shall be used by the File Server to identify the file in the A_FileStream_InfoReport service. File Handles shall be in the range of 0 to 15. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”. NOTE: All values 0 to 15 denote valid File Handles!
20h	Object busy	This Return Code shall be used by the File Server to indicate that it is busy and cannot accept any file command in this File Server Object. The Return Value shall denote the Object Index of the next free File Server Object. The decision by the File Client whether to try a second File Server Object can be based on this information. An Object Index value of 0 denotes that no other free File Server Object is available. An Object Index value of 1 to 255 is a valid index to a free File Server Object.

NOTE 45 If only one File Server Object is implemented, the Return Value may always be 0 (for the File Handle and for the next free File Server Object Index) and therefore may be a constant value.

4.10.3.3.2.1 Command “Retrieve File” (command value 11h)

This command shall cause the File Server to transfer a copy of the file of which the path is specified to the File Client. The status and contents of the file in the File Server shall be unaffected. For the transmission of the file the Management Procedure FTP_RetrieveFile shall be applied as specified in [09] clause “File Transfer Procedures”., this is, the service A_FileStream_InfoReport shall be called consecutively by the File Server.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command "Retrieve File". The File Server shall handle this request as specified in [09] clause "File Transfer Procedures".
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known file.
24h	Access denied	The File Server shall respond with this Return Code if the file cannot be read. NOTE This may be because the file is currently internally written, because another File Client is currently writing into this file, etc. This is not differentiated any further.

4.10.3.3.2.2 Command "Store File" (command value 12h)

This command shall cause the File Server to accept data from the File Client and store them as a file in its file system. If the file exists, then its contents shall be replaced. A new file shall be created if the file specified in the path name does not already exist. If the new file name is longer than the acceptable file name length, the file system may adjust the new file name automatically.

NOTE 46 This is equivalent to the store unique command in FTP.

For the transmission of the file the Management Procedure FTP_StoreFile shall be applied as specified in in [09] clause "File Transfer Procedures", this is, the service A_FileStream_InfoReport shall be called consecutively by the File Client.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command "Store File". The File Server shall handle this request as specified in in [09] clause "File Transfer Procedures".
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known directory.
23h	Write protected	The specified file is write protected.
24h	Access denied	The File Server shall respond with this Return Code if the file cannot be stored. NOTE 47 This may be because the file is currently internally written, because another File Client is currently writing to the file, etc. This is not differentiated any further.
25h	No more space available	The File Servers disk or storage memory is full. Incoming data cannot be stored.

4.10.3.3.3 Command "List Directory" (command value 13h)

This command shall cause the File Server to send a list of all entries of a directory of which the path is specified, to the client. A definition of the transfer format is given in [08] clause "Directory Listing Format". For the transmission of the directory listing the Management Procedure FTP_ListDirectory shall be applied as specified in [09] clause "File Transfer Procedures", this is, the service A_FileStream_InfoReport shall be called consecutively by the File Server.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	Directory Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “List Directory”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known directory.

4.10.3.3.4 Command “Rename From” (command value 14h)

This command shall cause the File Server to accept the path name as the name of the file to be renamed. It shall be followed by a command “Rename To”. To rename a file the Management Procedure FTP_Rename as specified in [09] clause “File Transfer Procedures” shall be used.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Rename From”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known file or directory.
23h	Write protected	The specified file is write protected.
24h	Access denied	The File Server shall respond with this Return Code if the file cannot be renamed. NOTE 48 This may be because the file is currently internally accessed in the device, because another File Client is currently accessing the file, etc. This is not differentiated any further.

4.10.3.3.4.1 Command “Rename To” (command value 15h)

This command shall specify the new path name of the file specified in the immediately preceding command “Rename From”. Together the two commands shall cause a file to be renamed. To rename a file the Management Procedure FTP_Rename as specified in [09] clause “File Transfer Procedures” shall be used.

If the new file name is longer than the acceptable file name length, the file system may adjust the new file name automatically.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Rename To”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server.
21h	Command not possible	The File Server shall respond with this Return Code if no command “Rename From” has preceded.
22h	Not found	The current path name does not lead to any known directory.

4.10.3.3.4.2 Command “Delete” (command value 16h)

This command shall cause the file specified in the path name to be deleted in the file system of the File Server.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Delete”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known file.
23h	Write protected	The specified file is write protected.
24h	Access denied	The File Server shall respond with this Return Code if the file cannot be deleted. NOTE 49 This may be because the file is currently internally accessed in the device, because another File Client is currently accessing the file, etc. This is not differentiated any further.

4.10.3.3.4.3 Command “Remove Directory” (command value 17h)

This command shall cause the directory specified in the path name to be deleted in the file system of the File Server. This command shall by the File Server only be executed if the directory is empty. If the directory contains files or other directories, then the directory shall not be deleted.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	Directory Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Remove Directory”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known directory.
23h	Write protected	The specified directory is write protected.
24h	Access denied	The File Server shall respond with this Return Code if the directory cannot be deleted. NOTE 50 This may be because the directory is currently internally accessed in the device, because another File Client is currently accessing the directory, etc. This is not differentiated any further.
26h	Directory not empty	The specified directory is not empty and can therefore not be removed.

4.10.3.3.4.4 Command “Make Directory” (command value 18h)

This command shall cause the directory specified in the path name to be created in the file system of the File Server. If the new directory name is longer than the acceptable directory name length, the File Server may adjust the new directory name automatically.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	Directory Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Remove Directory”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known parent directory.
23h	Write protected	The specified parent directory is write protected. A new directory cannot be created here.
24h	Access denied	The File Server shall respond with this Return Code if the directory cannot be created due to other reasons.
25h	No more space available	The File Servers disk or storage memory is full. A new directory cannot be created.

4.10.3.3.4.5 Command “Get File Size” (command value 1Dh)

This command shall cause the File Server to respond the size of a file specified by the path name. If the path name leads to a directory, the size shall be returned as 0.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14
	high	mhigh	mlow	low
Return Code	File Size (32 bit unsigned integer)			

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Get File Size”. The File Server shall respond the file size always as a 32 bit unsigned integer as defined above. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server. The file size is not transferred.
22h	Not found	The current path name does not lead to any known file or directory. The file size is not transferred.
24h	Access denied	The File Server shall respond with this Return Code if the file size cannot be evaluated. The file size is not transferred. NOTE 51 This may be because the file is currently internally written, because another File Client is currently writing into this file, etc. This is not differentiated any further.

4.10.3.3.4.6 Command “Get Empty Disk Space” (command value 1Eh)

This command shall cause the File Server to respond the remaining disk space of the File Server.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10
Command Value

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14
	high	mhigh	mlow	low
Return Code	Empty Disk Space (32 bit unsigned integer)			

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Get Empty Disk Space”. The File Server shall respond the empty disk space always as a 32 bit unsigned integer as defined above. If the empty disk space is larger than 4'294'967'295 (FFFFFFFFh) octets, then also this maximum number is returned. It has to be interpreted as 4'294'967'295 octets or more . The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server. The Empty Disk Space number is not transferred.

4.10.3.3.4.7 Command “Abort” (command value 1Fh)

This command shall cause the File Server to stop a running file transfer that has been initiated by any of the commands

- Retrieve File,
- List Directory or
- Get File.

NOTE 52 If a File Client wants to stop a post or store command, it signals the “end of transmission” by using a A_FileStream_InfoReport-PDU with a data length of 0.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10
Command Value

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command "Abort". The File Server shall handle this request as specified in [09] clause "File Transfer Procedures".
10h	Invalid command	The given command is reserved or not supported by this File Server.
21h	Command not possible	The File Server shall respond with this Return Code if there is no ongoing file transfer to this File Client.

4.10.3.3.5 Command "Get File" (command value 21h)

This command shall cause the File Server to transfer a copy of a file specified in the path name, to the client. During the transfer, the file may be treated by a pre- or post processor like PHP. A content type parameter is returned to the client, which defines the content of the file in more detail. With this information, the client can treat upcoming files in different ways (see also [08] "Short HTML File Format"). The status and contents of the file in the server shall be unaffected. For the transmission of the file the Management Procedure HTTP_GetFile shall be applied as specified in [09] clause "File Transfer Procedures", this is, the service A_FileStream_InfoReport shall be called consecutively by the File Server.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet n
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12
	Media-Type	Subtype
Return Code	Content-Type	

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Retrieve File”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”. The returned Content-Type is described below. It may easily be converted for http use.
10h	Invalid command	The given command is reserved or not supported by this File Server. The returned Content-Type is not transferred.
22h	Not found	The current path name does not lead to any known file. The returned Content-Type is not transferred.
24h	Access denied	The File Server shall respond with this Return Code if the file cannot be read. The returned Content-Type is not transferred. NOTE 53 This may be because the file is currently internally written, because another File Client is currently writing into this file, etc. This is not differentiated any further.

Possible Content-Types

Media-Type	Subtype	Content-Type Description
00h	00h	text/plain
	01h	text/html
	02h	text/short-html-200 (see also “Short HTML File Format in [08])
	03h	text/short-html-320 (see also “Short HTML File Format in [08])
	04h	text/short-html-480 ((see also “Short HTML File Format in [08])
	05h	text/short-html-640 (see also “Short HTML File Format in [08])
	06h	text/short-html-800 (see also “Short HTML File Format in [08])
	07h	text/short-html-1024 (see also “Short HTML File Format in [08])
	08h to FFh	reserved
01h	00h	image/jpeg
	01h	image/gif
	02h	image/png
	03h	image/tiff
	04h	image/g3fax
	05h to FFh	reserved
02h	00h	audio/basic
	01h to FFh	reserved
03h	00h	video/mpeg
	01h to FFh	reserved

Media-Type	Subtype	Content-Type Description
04h	00h	application/octet-stream
	01h	application/postscript
	02h	application/pdf
	03h	application/zip
	04h to FFh	reserved
05h to FFh	00h to FFh	reserved

NOTE 54 The Content Type definition is taken from the MIME-Type (Multipurpose Internet Mail Extension) definition described in RFC 2045, RFC 2046 and others.

4.10.3.3.5.1 Command “Post File” (command value 22h)

This command shall cause the File Server to accept data from the File Client and store them as a file in its file system. If the file exists, then its contents shall be replaced. A new file shall be created if the file specified in the path name does not already exist. If the new file name is longer than the acceptable file name length, the file system may adjust the new file name automatically. For the transmission of the file the Management Procedure HTTP_PostFile shall be applied as specified in [09] clause “File Transfer Procedures”, this is, the service A_FileStream_InfoReport shall be called consecutively by the File Client.

Property File Command Write (A_FunctionPropertyCommand-PDU)

octet 10	octet 11	octet 12 ... octet N
Command Value	File Path	

or

octet 10
Command Value

If the command is given without a File Path, the Property PID_FILE_PATH shall be used instead and therefore has to be set before the command is executed. If a File Path is included in the command, the Property PID_FILE_PATH shall be overridden.

Property File Command Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

Possible Command Results

Return Code	Code Name	Description
00h	Command successful	This Return Code shall be used by the File Server to indicate that it accepts the command “Store File”. The File Server shall handle this request as specified in [09] clause “File Transfer Procedures”.
10h	Invalid command	The given command is reserved or not supported by this File Server.
22h	Not found	The current path name does not lead to any known directory.
23h	Write protected	The specified file is write protected.

Return Code	Code Name	Description
24h	Access denied	The File Server shall respond with this Return Code if the file cannot be stored. NOTE 55 This may be because the file is currently internally written, because another File Client is currently writing to the file, etc. This is not differentiated any further.
25h	No more space available	The File Servers disk or storage memory is full. Incoming data cannot be stored.

4.10.3.4 Usage by the Management Server

The Management Server shall handle the Property File Command as specified for the separate commands above.

Invalid (Reserved) and not supported commands shall be responded as specified in Table 45.

4.10.3.5 Usage by the Management Client (FTP client)

If a “Retrieve File”, “List Directory” and “Get File” command has a response with no error to the client and no data are transferred, the client shall detect this “error” after a certain time-out.

NOTE 56 This time-out may depend on the implementation of the File Server, the size and complexity of the file system and on the command itself, and may last up to 30 s and more. No generally applicable time-out values can be given.

4.10.4 Property File Path (PID = 53)

4.10.4.1 Format

Property value index				
0 (start-index)	1	2	...	N
Length of FilePath	File Path			
	octet 1	octet 2	...	octet N

Figure 45 – Property File Path

This Property shall be mandatory and shall contain the path name in absolute notation i.e always starting from the root directory e.g. „/dirname/filename.txt“.

The maximum length of this Character Array depends on the implementation. This maximal length can be retrieved by the File Client by reading the Property Description with an A_PropertyDescription_Read and interpreting the field max_nr_of_elem.

File Servers that handle only one file may have this Property implemented as “read only”.

The used file system shall be compatible to UNIX / LINUX. The filepath shall be encoded according DPT_UTF-8 (DPT_ID = 28.001). The Property Datatype shall be PDT_UTF-8.

A path name shall at least be coded in standard 7 bit (MSB = 0) ASCII code using only uppercase letters, lowercase letters, numbers and the following four special characters: “/” (slash), “-” (hyphen), “_” (underline) and “.” (dot). 7 bit ASCII code is a subset of UTF-8 code.

Wildcards like “*.txt” shall not be used. Files shall be retrieved, renamed, deleted and so on, one after the other.

4.10.4.2 Usage by the Management Server (device)

There are no system variables like “current working directory”.

This is due to the fact that there is no open and close command to initiate and leave a command session (where system variables could be initialized). This requires that the path name is always in absolute notation.

The path name shall be deleted after the execution of any command. This rule only applies if the Property is “read / write”. A “read only” Property cannot be deleted.

4.10.4.3 Usage by the Management Client (FTP client)

The File Client shall set the Property PID_FILE_PATH to the value of the path name to which the subsequent command shall apply. If the path name is part of the command datagram, then this path name is used instead of the Property PID_FILE_PATH i.e. the Property PID_FILE_PATH shall be overridden.

4.10.5 Property MAX. APDU-Length (PID = 56)

This Property shall be an optional read-only Property and shall provide the maximum APDU-Length that is supported by this device for the A_FileStream_InfoReport-service. If this Property is not present the maximum APDU-Length of the device is valid for the A_FileStream_InfoReport-service (this is the value of the Property PID_MAX_APDU_LENGTH in the Device Object, or the value 14 if this Property is not available).

4.11 E-Mode Channel Object (Object Type 14)

4.11.1 General

4.11.1.1 Use and general requirements of the E-Mode Channel Object

This Interface Object shall hold information about the E-Mode Channel itself, about all in the E-Mode channel implemented Group Objects and Parameters.

Information in the various Properties on the same Property element index shall relate to the same Group Object or Parameter.

Please refer to [17] for the mandatory and optional Properties in the E-Mode Channel Object; Table 48 only gives the overview and is informative.

One instance of this Interface Object shall be implemented for each implemented E-Mode Channel.

There are no requirements towards the Object Indexes of the E-Mode Channel Objects: they do not need to be in consecutive order or be sorted in any way. There is no mandatory relation between the Object Indexes and the E-Mode Channel Number; this shall only be given through the Property *Channel Number*.

4.11.1.2 Group Objects

The GOs in an E-Mode Channel Object instance shall be numbered independently of the GOs of other E-Mode Channel Objects. The GOs of each separate E-Mode Channel shall be numbered starting from 1.

The GOs in the E-Mode Channel are numbered consecutively starting from 1 and have consecutive values without gaps. In the Properties that describe the GOs of one E-Mode Channel, data on the same Property Index shall refer to the same GO.

4.11.1.3 Property overview

Table 48 – E-Mode Channel Object (informative)

PID Value	PID Name	Property Name	PDT/DPT	Access
1	PID_OBJECT_TYPE	Interface Object Type	DPT_PropDataType (DPT_ID = 7.010)	3/1
2	PID_OBJECT_NAME	Interface Object Name	PDT_UNSIGNED_CHAR[] (DPT_UTF_8 (DPT_ID: 28.001)) PDT_REFERENCE	3/1
21	PID_DESCRIPTION	Description	PDT_UNSIGNED_CHAR[]	3/3
25	PID_VERSION	Version	PDT_VERSION (DPT_Version, DPT_ID = 217.001)	3/1
51	PID_CHAN_NUMBER	Channel Number	PDT_UNSIGNED_INT	3/1
52	PID_CHAN_CODE	Channel Code	PDT_UNSIGNED_INT	3/1
53	PID_CHAN_FLAGS	Channel Flags	PDT_GENERIC_02	3/1
54	PID_CHAN_FB_LIST	Functional Block List	PDT_UNSIGNED_INT[] (DPT_Value_2_Ucount, DPT_ID = 7.001)	3/1
55	PID_CHAN_ADJ_LISTS	Adjustable Channel Object List	PDT_UNSIGNED_CHAR[]	3/1
61	PID_GO_CCODES_LIST	Connection Codes	PDT_GENERIC_08[]	3/1
62	PID_GO_CFLAGS_LIST	Connection Flags	PDT_BITSET16[]	3/1
63	PID_OBJECTLINK	Object Link	PDT_FUNCTION	3/3
64	PID_GO_SUBUNIT	Subunit Number	PDT_UNSIGNED_CHAR[]	3/1
65	PID_GO_NAME_LIST	Group Object Names	PDT_GENERIC_10[] or PDT_REFERENCE[]	3/1
70	PID_PARAM_TYPES	Parameter Types	PDT_GENERIC_10[]	3/1
71	PID_PARAM_FLAGS	Parameter Flags	PDT_BITSET16[]	3/1
72	PID_PARAM_NAMES	Parameter Names	PDT_GENERIC_10[] or PDT_REFERENCE[]	3/1
73	PID_PARAM_UNITS	Parameter Units	PDT_GENERIC_10[] or PDT_REFERENCE[]	3/1
79	PID_PARAM_VALUES	Parameter Values	PDT_GENERIC_01[], or PDT_GENERIC_02[], or PDT_GENERIC_04[], or PDT_GENERIC_10[]	3/3

4.11.2 E-Mode Channel Object – PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the common specification of PID_OBJECT_TYPE.

The Object Type for the E-Mode Channel Object shall have the value 14.

4.11.3 E-Mode Channel Object – PID_OBJECT_NAME (PID = 2)

Please refer to 4.2.2 for the common specification of PID_OBJECT_NAME.

This Property shall contain the name of the E-Mode Channel. In this case, the PDT shall be PDT_UNSIGNED_CHAR[]. Alternatively, this Property may also only be a reference to the name of the E-Mode Channel. In that case the PDT shall be PDT_REFERENCE and the Property Value shall be the reference to the Property Name.

EXAMPLE 20 “Blinds Ch. 1”

The name shall in both cases be encoded as a string of variable length. Every textual character shall be encoded as DPT_UTF_8 (DPT_ID: 28.001).

The value of this Property is given by the product manufacturer and can typically not be changed by the planer or installer. If it is wanted that the product installer or – planner assigns an own textual description to this E-Mode Channel, then this can be done and stored in the E-Mode Management Client instead, and not in the E-Mode device or the below Property PID_DESCRIPTION can be used.

4.11.4 E-Mode Channel Object – PID_DESCRIPTION (PID = 21)

4.11.4.1 Resource definition - FEC

For the Interface Object Type independent specification of PID_DESCRIPTION, please refer to 4.2.21.

In a FEC device, this Property

- is mandatory in the E-Mode Channel Object, and
- shall have a minimal size of 10 characters; no maximal size is specified.

4.11.4.2 Usage by the Management Client

The Management Client may use PID_DESCRIPTION to store human readable textual information, entered by the installer, to help localising, identifying and describing the E-Mode Channel.

This shall help for reverse engineering E-Mode installations.

EXAMPLE 21 “Light Room 1”

4.11.5 E-Mode Channel Object – PID_VERSION (PID = 25)

- Property name: Channel Version
- Property Datatype: PDT_Version
- Datapoint Type: DPT_Version (DPT_ID = 217.001)
- Access: Data Property: 3/1

The Property Version (PID_VERSION) shall contain the manufacturer specific version of the E-Mode Channel.

4.11.5.1 Usage by the Management Server (device)

The manufacturer of the E-Mode Channel shall take care that the version indication in this Property is properly handled according the definition of DPT_Version in [14].

4.11.5.2 Usage by the Management Client

The Management Client may interpret this Property Version (PID_VERSION) in combination with the Properties Channel Code (PID_CHAN_CODE) and the manufacturer code of the device as retrieved through DD2. The E-Mode Management Client may store the manufacturer code, channel version and channel description of each E-Mode Channel of which it reads out the description. If the same values are returned later by another E-Mode Channel, the E-Mode Management Client may assume that these cached values are correct and up-to-date and may skip the further discovery of this E-Mode Channel. This speeds up the discovery procedure. This behaviour by the E-Mode Management Client is optional.

4.11.6 E-Mode Channel Object – PID_CHAN_NUMBER (PID = 51)

- Property name: Channel Number
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: DPT_Value_2_Ucount
- Access: Data Property: 3/1

4.11.6.1 Usage by the Management Server (device)

This Property shall contain the Channel Number of the E-Mode Channel.

Any Channel Number shall be within the range 1 to 255. The value 0 is reserved and shall not be used.

The Channel Number shall be a unique identifier for the E-Mode Channel Object within the FEC device. The implementer shall take care that no two E-Mode Channel Objects have the same value for the Property *Channel Number*.

This value shall amongst other be used in the Localisation Procedures with PID_LOCALISATION_REPORT (see 4.14.5) and PID_LOCALISATION_COMMAND (see 4.14.6).

4.11.6.2 Usage by the Management Client

The Management Client shall read out this Property during the discovery of the E-Mode Channel to link E-Mode Channel number with the Object Indexes of the E-Mode Channel Objects.

4.11.7 E-Mode Channel Object – PID_CHAN_CODE (PID = 52)

- Property name: Channel Code
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

This Property shall contain the Channel Code of the Basic E-Mode Channel that is described by this *E-Mode Channel* Object. If this E-Mode Channel is not an Adjustable E-Mode Channel, then this is the only Channel Code indication; otherwise, alternative E-Mode Channel Codes will be present in one or more *Channel Code* Properties in related instances of the *Adjustable E-Mode Channel* Object.

The E-Mode Channel Code shall be a 13 bit identifier. It shall be positioned on the least significant positions of the PID_CHAN_CODE-value; the three most significant bits of PID_CHAN_CODE shall be 0.

4.11.8 E-Mode Channel Object – PID_CHAN_FLAGS (PID = 53)

- Property name: Channel Flags
- Property Datatype: PDT_GENERIC_02
- Datapoint Type: None.
- Access: Data Property: 3/1

Table 49 – PID_CHAN_FLAGS

Bit Nr.	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
field symbol	r	r	r	r	r	r	r	r	r	r	LR	LC	r	O	I	A
	0	0	0	0	0	0	0	0	0	0						

- bits 0 (lsb):

name (abbreviation):	Adjustable (A)
description:	This specifies whether the E-Mode Channel is fully specified through this instance of the E-Mode Channel Object, or whether additional GO and Parameter information is available via one or more <i>Adjustable E-Mode Channel</i> Interface Objects.
encoding:	0b: This E-Mode Channel is not an adjustable E-Mode Channel. The GOs and Parameters of this Flexible E-Mode Channel are fully specified in this instance of the <i>E-Mode Channel</i> Object; there are no instances of the <i>Adjusted E-Mode Channel Object</i> Type for this Flexible E-Mode Channel. 1b: This E-Mode Channel is an adjustable E-Mode Channel. The GOs and Parameters of this Flexible E-Mode Channel may vary in number and function and are described in one or more instances of the <i>Adjusted E-Mode Channel Object</i> Type. If this bit is set, the Property PID_CHAN_ADJ_LISTS shall be available and shall contain at least one object index referring to an Object of type Adjusted E-Mode Channel.
 - bits 1:

name (abbreviation):	Input (I)
description:	This specifies whether the E-Mode Channel is an input E-Mode Channel or not.
encoding:	NOTE 57 This information does not affect the Configuration of this E-Mode Channel. It is merely considered to allow the E-Mode Controller to position the E-Mode Channel in a graphical representation of the link with another E-Mode Channel either on the left (input) or on the right (output). 0b: This E-Mode Channel is not an Input E-Mode Channel. 1b: This E-Mode Channel is an Input E-Mode Channel.
 - bit 2:

name (abbreviation):	Output (O)
description:	This specifies whether the E-Mode Channel is an output E-Mode Channel or not..
encoding:	0b: This E-Mode Channel is not an Output E-Mode Channel. 1b: This E-Mode Channel is an Output E-Mode Channel.
- An E-Mode Channel that is neither an Input Channel nor an output Channel (I = 0 and O = 0) shall be regarded as a Functional Module.
- It is possible that an E-Mode Channel has both an Input Channel functionality (I = 1) as well as output Channel functionality (O = 1).
- bit 3:

name (abbreviation):	reserved (r)
description:	This bit is reserved.
encoding:	This bit shall always be 0.
 - bit 4

name (abbreviation):	Localisation Command (LC)
description:	This field shall signal the support of a Localisation Command by this E-Mode Channel.
encoding:	0b: This E-Mode Channel <u>does not support</u> Localisation Commands. If a Localisation action is requested for this E-Mode Channel (see 4.14.6.3), this device shall not react. 1b: This E-Mode Channel supports Localisation Commands.

- bit 5

name (abbreviation):	Localisation Report (LR)
description:	This field shall signal the support of Localisation Reports by this E-Mode Channel.
encoding:	0b: This E-Mode Channel <u>does not support</u> Localisation Reports. While Localisation Mode is active in the E-Mode device, local operations on this E-Mode Channel's user interface will not cause the device to report such localisation action. 1b: This E-Mode Channel supports Localisation Reports.

The fields Localisation Command (LC) and Localisation Report (LR) are independent of each other. The field Localisation Action is typically set for an actuator E-Mode Channel but can also be set for a sensor E-Mode Channel and vice versa.

EXAMPLE 22A sensor E-Mode Channel may have a small LED (e.g. a status LED) that can be used for localisation.

EXAMPLE 23A binary output E-Mode Channel may have a hardware switch for local operation of the relay that can also be used for localisation.

Please refer to the requirements for PID_CHAN_FLAGS in [17] concerning the support of these fields.

- bits 15 to 6:
 These bits are reserved and shall be cleared in the device.
 The E-Mode Management Client shall ignore the value of these bits.

4.11.9 E-Mode Channel Object – PID_CHAN_FB_LIST (PID = 54)

- Property name: Functional Block List
- Property Datatype: PDT_UNSIGNED_INT[]
- Datapoint Type: DPT_Value_2_Ucount
- Access: Data Property: 3/1

The Property Functional Block List shall be an array containing **all** Interface Object Types defined for the Functional Blocks realised by this E-Mode Channel.

If a FB is used more than once in the E-Mode Channel, then it shall be reported as many times in this list.

This list may but does not have to be sorted.

4.11.10 E-Mode Channel Object – PID_CHAN_ADJ_LISTS (PID = 55)

- Property name: Adjustable Channel Object List
- Property Datatype: PDT_UNSIGNED_CHAR[]
- Datapoint Type: None.
- Access: Data Property: 3/1

The Property *Adjustable Channels Object List* shall be an array of the indexes of the *Adjusted E-Mode Channel Object* instances for this E-Mode Channel, if there are any.

The Property *Adjustable Channels Object List* shall have as many array elements as possible values that it supports for the parameter AJS. Every array element n ($n \neq 0$) shall specify the Object Index within the device that specifies the Adjustable Channel (Group Objects and Parameters) for the value n of AJS.

Adjustable Parameter Value	Property Value Array Index	Property Value
-	0	(current nr. of array elements)
1	1	Object Index of the Adjusted E-Mode Channel Object nr. 1
2	2	Object Index of the Adjusted E-Mode Channel Object nr. 2
3	3	Object Index of the Adjusted E-Mode Channel Object nr. 3
...	...	...

Table 50 – PID_CHAN_ADJ_LISTS format

If this Property is not implemented, then the bit *Adjustable* in PID_CHAN_FLAGS of the same E-Mode Channel Interface Object shall be cleared. If this Property is implemented, it shall contain at least one object index and the bit *Adjustable* in PID_CHAN_FLAGS shall be set.

If one or several value of the AJS is not supported in this E-Mode Channel, the array element corresponding to these values shall contain an Object Index equal to 00h. However, this E-Mode Channel that does not support every possible value of AJS should be differentiated from the similar E-Mode Channel that supports every possible value of AJS: this differentiation may be done by a different Channel Code or by a different version of the same Channel Code.

4.11.11 E-Mode Channel Object – PID_GO_CCOCODES_LIST (PID = 61)

- Property name: Connection Codes
- Property Datatype: PDT_GENERIC_08[]
- Datapoint Type: None.
- Access: Data Property: 3/1

This Property shall contain the Main Connection Codes and all Additional Connection Codes for all Group Objects of this E-Mode Channel.

4.11.11.1 Format

This Property Value shall have an array structure. The Property Value array index used in this Property shall refer to the same Group Object as referred to in the array element with the same index in the Properties PID_GO_CFLAGS_LIST, PID_GO_SUBUNIT and PID_GO_NAME_LIST.

EXAMPLE 24 The GO is listed on array index 2 in PID_GO_LIST shall be the same GO as the GO listed on array index 2 in this Property PID_GO_CCOCODES_LIST.

The array elements of the Property Value shall be values formatted according PDT_GENERIC_08. The most significant double octet shall be the Main Connection Code. The following double octets may be used for subsequent Additional Connection Codes, filling the Property Value from the left (most significant positions) to the right (least significant positions). Positions that are not used by Additional Connection Codes shall have the value 0000h.

Property Index	Property Value			
0	current_nr_of_elem = 3			
1	Main CC for GO 1	0000h	0000h	0000h
2	Main CC for GO 2	1 st Add CC for GO 2	2 nd Add CC for GO 2	0000h
3	Main CC for GO 3	1 st Add CC for GO 3	0000h	0000h

Figure 46 - PID_GO_CCOCODES_LIST (example)

4.11.11.2 Usage by the Management Client and – Server

The Connection Codes shall be used by the Management Client and the Management Server for Ctrl FEC and PB FEC as laid down in [10]. If this does not allow for the transmission of two octets Connection Codes AND if the higher octet of the Connection Code equals 00h then these Configuration Modes shall use single octet Connection Codes with only the least significant octet.

4.11.12 E-Mode Channel Object – PID_GO_CFLAGS_LIST (PID = 62)

- Property name: Connection Flags
- Property Datatype: PDT_BITSET16[]
- Datapoint Type: None.
- Access: Data Property: 3/1

4.11.12.1 Use

This Property shall contain all Connection Flags for all Group Objects of this E-Mode Channel.

4.11.12.2 Format

This Property shall have an array structure. The Property Value array index used in this Property shall refer to the same Group Object as referred to in the Property Value array element with the same index in the Properties PID_GO_CCOCODES_LIST, PID_GO_SUBUNIT and PID_GO_NAME_LIST.

EXAMPLE 25 If GO 8 is listed on array index 2 in PID_GO_LIST, then array index 2 in this Property PID_GO_CFLAGS_LIST shall be interpreted as the Connection Flags for GO 8.

The Property array elements shall be values formatted according to the format as specified in Table 51.

Bit Nr.	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
field symbol	r	r	r	r	r	r	r	r	r	r	r	r	r	I	O	X
	0	0	0	0	0	0	0	0	0	0	0	0	0			

Table 51 – PID_GO_CFLAGS_LIST (one Property Value array element)

For the definition and use of Visualisation DPs, please refer to [13] clause “Describe the Channel”.

- bit 0:
 - name (abbreviation): X (X)
 - description: The X-field shall indicate whether this GO may be connected more than once or not. This flag is only meaningful for an input DP.
 - EXAMPLE 26 In the definition of the channel CH_logic_AndOr (id 0200h), each of the four input Datapoints shall be connected only once, in order to guarantee the coherency of the logical result.
 - encoding:
 - 0b: This GO may be connected more than once.
 - 1b: This GO shall not be connected more than once.
- bit 1:
 - name (abbreviation): O (O)
 - description: The O-field shall indicate whether this GO is an Output DP or not.
 - encoding:
 - 0b: This GO is not an Output DP.
 - 1b: This GO is an Output DP.
- bit 2:
 - name (abbreviation): I (I)
 - description: The I-field shall indicate whether this GO is an Input DP or not.
 - encoding:
 - 0b: This GO is not an Input DP.

1b: This GO is an Input DP.

- bits 3 to 15

These bits are reserved and shall be cleared in the device.

The E-Mode Management Client shall ignore the value of these bits.

4.11.13 E-Mode Channel Object – PID_OBJECTLINK (PID = 63)

- Property name: Object Link
- Property Datatype: PDT_FUNCTION
- Datapoint Type: None.

For the common specification of PID_OBJECTLINK, please refer to the specification of the instance in the Device Object in clause 4.3.14.

4.11.14 E-Mode Channel Object – PID_GO_SUBUNIT (PID = 64)

- Property name: Subunit Number
- Property Datatype: PDT_UNSIGNED_CHAR[]
- Datapoint Type: DPT_Value_1_Ucount (5.010)

The Property Subunit Number shall be an array Property with number of elements equal to the number of Group Objects for this E-Mode Channel.

It shall specify for each Group Object n in this E-Mode Channel in Property Value Array element n the subunit number in the E-Mode Channel to which this Group Object belongs.

GO nr	Property Value Property Index	Property Value
-	0	(current nr. of array elements)
1	1	Subunit of GO 1
2	2	Subunit of GO 2
3	3	Subunit of GO 3
...	...	...

Table 52 – PID_GO_SUBUNIT format

PID_GO_SUBUNIT is always mandatory. If there are no Subunits, then the value of PID_GO_SUBUNIT shall be 0. Also the value of PID_GO_SUBUNIT of GOs that do not belong to a Subunit shall be 0. If there is only one Subunit, then the value for PID_GO_SUBUNIT for its DPs shall be 1.

Subunits shall be numbered consecutively, starting with 1.

4.11.15 E-Mode Channel Object – PID_GO_NAME_LIST (PID = 65)

- Property name: Group Object Names
- Property Datatype:
 - PDT_GENERIC_10[]
 - PDT_REFERENCE[]

- Datapoint Type: DPT_UTF_8 (DPT_ID: 28.001)

The Property *Group Object Names* shall be an array Property with number of elements equal to the number of *Group Objects* for this E-Mode Channel.

It shall specify for each Group Object the human readable description and identification of this Group Object.

The Property Value array index used in this Property shall refer to the same Group Object as referred to in the array element with the same index in the Properties PID_GO_CCOCODES_LIST, PID_GO_CFLAGS_LIST and PID_GO_SUBUNIT.

GO nr	Property Value Array Index.	PID_GO_NAME_LIST
		Property Value = GO names
	0	(current nr. of array elements)
1	1	"A Switch"
2	2	"A Dimming"
3	3	"A State"
	...	...

In this example, it can be read that "A State" is the name of the 4th Group Object in this device.

4.11.16 E-Mode Channel Object - PID_GO_DIAGNOSTICS (PID: 66)

- **Property name:** Group Object Diagnostics
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None
- **Access Policy:** See 4.8.1.
- **Access Level:** 3/3

4.11.16.1 Abstract Resource Definition

The Function Property *Group Object Diagnostics* (PID_GO_DIAGNOSTICS) in the *E-Mode Channel* Object shall have the same functionality as *Group Object Diagnostics* in the *Group Object Table* Object.

However, this Property shall relate exclusively to the GOs of the E-Mode Channel Object instance in which it is hosted. Every E-Mode Channel Object shall have at maximum one Property *Group Object Diagnostics*.

The field *Group Object Number* shall refer to the Group Object Index, which shall be the number of the Group Object relative within the E-Mode Channel Object.

NOTE 58 Every E-Mode Channel Object may thus have its own dedicated PID_GO_DIAGNOSTICS. It is an implementation choice for which E-Mode Channels PID_GO_DIAGNOSTICS is implemented and for which not.

NOTE 59 In an implementation, it may not be necessary to implement every instance of PID_GO_DIAGNOSTICS as a separate routine. This may be one single routine that is called with different GO reference depending on the Interface Object (E-Mode Channel Object) in which PID_GO_DIAGNOSTICS is called.

NOTE 60 This may possibly allow differentiating the access rights (security) to this Property per instance; this is, possibly different per E-Mode Channel Object.

NOTE 61 The same dependency exists in E-Mode as in S-Mode between the Group Object Diagnostics and the Operation Mode. The MaC shall also in E-Mode set the *Operation Mode* in the Applicationprogram Object to "Diagnostic Mode" as required or not per Command prior to accessing any PID_GO_DIAGNOSTIC. (According the Flexible E-Mode Channels (FEC) the Applicationprogram Object is also mandatory for FEC Profiles.)

4.11.16.2 Format and encoding

The format and the encoding shall be identical as PID_GO_DIAGNOSTICS as specified in 4.8.1. The field *Group Object Number* shall be interpreted as the *Group Object Index* within the E-Mode Channel Object in which this Property instance is hosted.

4.11.16.3 Usage by the MaS (device)

The MaS requirements are the same as in 4.8.1.

4.11.16.4 Usage by the MaC

The MaC requirements are the same as in 4.8.1.

4.11.17 E-Mode Channel Object – PID_PARAM_TYPES (PID = 70)

- Property name: Parameter Types
- Property Datatype: PDT_GENERIC_10[]
- Datapoint Type: None.

4.11.17.1 Format

The Property *Parameter Types* shall be an array Property with number of elements equal to the number of Parameters for this E-Mode Channel.

It shall specify for each Parameter n in this E-Mode Channel the Parameter Type Identifier, as defined in Table 53.

Table 53 – PID_PARAM_TYPES format

Parameter nr	Property Value Array Index	Property Value	
-	0	(current nr. of array elements)	2 octets
1	1	Parameter Type of Parameter 1	10 octets
2	2	Parameter Type of Parameter 2	10 octets
3	3	Parameter Type of Parameter 3	10 octets
...	...	...	

The encoding of the Parameter Type is flexible and shall depend on a main type indication in the most significant octet as specified in Table 54.

Table 54 – Parameter Type definitions

Format	Octet									
	0 (MSB) Type	1	2	3	4	5	6	7	8	9 (LSB)
Reserved	0	00h	00h	00h	00h	00h	00h	00h	00h	00h
Standard	1	Standard Parameter Type		00h	00h	00h	00h	00h	00h	00h
Bool	2	Type	00h	00h	00h	00h	00h	00h	00h	00h
Enum	3	Count	Reference Type	Object Type		Object Instance	Object Index	PID	0 (4 bit)	Start Index (12 bit)
INT8	4	Flags	Steps	00h	Minimum	00h	Maximum	00h	00h	00h
UINT8	5	Flags	Steps	00h	Minimum	00h	Maximum	00h	00h	00h
INT16	6	Flags	Steps	Minimum		Maximum		00h	00h	00h
UINT16	7	Flags	Steps	Minimum		Maximum		00h	00h	00h
FLOAT16	8	Flags	Steps	Minimum		Maximum		00h	00h	00h
Enum extended	9	Count	Reference Type	Object Type		Object Instance (12 bits)		PID (12 bits)		Start Index

NOTE 62 A possible future additional Parameter Type may be String.

◆ **Format 0: “Reserved”**

- Use: This Parameter Type format is reserved and shall not be used.

◆ **Format 1: “Standard”**

- Use: This Parameter Type format shall be used to refer to a Standard Parameter Type encoding.
If the Parameter is encoded according a standard Parameter Type then this format shall be used and none of the below formats.
- Encoding: The Parameter Type format shall be identified by the field *Standard Parameter Type* with value as given in the clause “Parameter Types” in Chapter 3/7/2 “Datapoint Types” ([14]).

◆ **Format 2: “Bool”**

- Use: This Parameter Type format shall be used to identify a Boolean Parameter.
- Encoding: **Type**
For the Boolean Parameter Type, standard labels are defined.
This avoids that the Management Server needs to define this PID_PARAM_UNITS as enum labels in a Text Catalogue Object, as is done for the type “Enum” below.
The value of Type shall equal the Subtype value of the 1 bit DPTs in [14].

Table 55 – Parameter Type

Type	Parameter Value	Label for Value
1	0	Off
	1	On
2	0	False
	1	True
3	0	Disable
	1	Enable
...	...	

EXAMPLE 27 If the Ctrl FEC Controller discovers a Parameter Format 2 “Bool” with Type 3, then it knows it can show on its display the texts “Disable” and “Enable”, which will stand for the values 0 and 1 respectively, without further reading a Text Catalogue Object.

NOTE 63 If none of the defined 1 bit DPTs can be meaningfully referred, a Boolean Parameter Format can also be described as an enumerated Parameter Format limited to two values only. See below.

◆ Format 3: “Enum”

- Use: This Parameter Type format shall be used to identify an enumeration Parameter. This matches a *real value*, which is written by the Management Client in the Management Server, with a *displayed value*, which is read by the Management Client from the Management Server and is displayed to the user.

- Encoding: **Count**

The real values of enumerated Parameter Type shall be encoded as unsigned 8 bit integers; the minimal real value shall always be 0 and the next real values shall be consecutive without gaps; the field *Count* shall contain the number of elements in the enumeration and shall thus equal the maximal value + 1.

Count shall at maximum have the value 191; the values 192 to 255 for *Count* are reserved and shall not be used.

NOTE 64 The real values set by the Management Client will be as requested by the installer within the consecutive range 0 to Count - 1. By implementation specific means, it is however possible to map these device internal to other values.

EXAMPLE 28 (Count = 3)

displayed value = User's choice	real value = Set by Management Client in Management Server	Management Server's internal mapping.
Frost protection	0	10 °C
Economy mode	1	18 °C
Comfort mode	2	21 °C

ObjIdx and PID

The displayed values for the enumerated Parameter Type format shall be stored in a Property, of any Interface Object, of type PDT_UTF_8 (A[n], DPT_UTF-8). This shall be a Property of an instance of the Text Catalogue Object (see 4.13), if the device allows of selection of the language of textual information. This shall be referred by the fields Reference Type, Object Type, Object Instance, Object Index, PID and Start Index, which shall all be encoded and used as specified for PDT_REFERENCE in [15]. The values of these fields shall in this way build a reference to the first displayed value of the enumeration; the next displayed values shall be read as is indicated in the field subfield Subtype of Reference Type (see [15]).

◆ Format 4: “INT8”

- Use: This Parameter Type format shall be used to identify a Parameter value encoded as an 8 bit signed integer value.

- Encoding: **Flags**

This field shall give common information for numerical Parameter Values.

Octet	1							
Bit	7	6	5	4	3	2	1	0
Name	r	r	r	r	r	r	Max	Min
Value	0	0	0	0	0	0		

- Min

- 0: The Parameter encoded according this Parameter Type does not have a minimal value. The smallest encodable minimal value shall be the minimal value for the Parameter Value.
- 1: The Parameter encoded according this Parameter Type does have a minimal value. This minimal value shall be read from the field “min” in this Parameter Type definition.

- Max

- 0: The Parameter encoded according this Parameter Type does not have a maximal value. The largest encodable maximal value shall be the maximal value for the Parameter Value.
- 1: The Parameter encoded according this Parameter Type does have a maximal value. This maximal value shall be read from the field “max” in this Parameter Type definition.

Steps

If not all possible values between the minimal value and the maximal value are supported or allowed, but if only a limited number of equally spread intermediate values is possible, the field *Steps* shall be used to indicate this possible number of intermediate values.

Steps shall be encoded as an 8 bit unsigned integer value.

Octet	2							
Bit	7	6	5	4	3	2	1	0
Name	Steps							
Value								

NOTE 65 This avoids the introduction of an Enumeration Parameter Type (see below).

EXAMPLE 29 If the minimal value is 10 and the maximal value is 20 and Steps equals 21 then the possible values shall be 10.0, 10.5, 11.0 etc. to 19.5 and 20.

The encoding of *Steps* shall be as follows.

- 0: infinite
- NOTE 66 This can be used for Parameter values that can be entered in an HMI as edit box input or keyboard input.
- 1 to 254: number of possible steps ("combo box selection")
- 255: reserved. Shall not be used.

Minimum

The field *Minimum* shall contain the minimal value for the Parameter Value encoded according this Parameter Type.

This field shall only be interpreted if the field *Min* in the field *Flags* is set.

This field shall be encoded as the value for this Parameter itself, in this case thus as an 8 bit signed integer value.

Maximum

The field *Maximum* shall contain the maximal value for the Parameter Value encoded according this Parameter Type.

This field shall only be interpreted if the field *Max* in the field *Flags* is set.

This field shall be encoded as the value for this Parameter itself, in this case thus as an 8 bit signed integer value.

◆ Format 5: "UINT8"

- Use: This Parameter Type format shall be used to identify a Parameter value encoded as an 8 bit unsigned integer value.
- Encoding: The encoding of the fields *Flags* and *Steps* shall be identical as for the Parameter Type format INT8 specified above.
The fields *Minimum* and *Maximum* shall be encoded as an 8 bit unsigned integer value.

◆ Format 6: "INT16"

- Use: This Parameter Type format shall be used to identify a Parameter value encoded as a 16 bit signed integer value.
- Encoding: The encoding of the fields *Flags* and *Steps* shall be identical as for the Parameter Type format INT8 specified above.
The fields *Minimum* and *Maximum* shall be encoded as 16 bit signed integer values.

◆ Format 7: "UINT16"

- Use: This Parameter Type format shall be used to identify a Parameter value encoded as a 16 bit unsigned integer value.
- Encoding: The encoding of the fields *Flags* and *Steps* shall be identical as for the Parameter Type format INT8 specified above.
The fields *Minimum* and *Maximum* shall be encoded as 16 bit unsigned integer values.

◆ Format 8: "FLOAT16"

- Use: This Parameter Type format shall be used to identify a Parameter value encoded as a "2-Octet Float Value" as specified in [14] (F₁₆; DPTs in the range 9.nnn).
- Encoding: The encoding of the fields *Flags* and *Steps* shall be identical as for the Parameter Type format INT8 specified [here](#) above.
The fields *Minimum* and *Maximum* shall be encoded as 16 bit float values (F₁₆; see also [14]).

◆ Format 9: "Enum extended"

- Use: Same as for Format 3.

- Encoding: **Count**

Same as for Format 3.

Object Type, Object Instance and PID

The displayed values for the enumerated Parameter Type format shall be stored in a Property, of any Interface Object, of type PDT_UTF_8 (A[n], DPT_UTF-8). This shall be a Property of an instance of the Text Catalogue Object (see 4.13), if the device allows of selection of the language of textual information. This shall be referred by the fields Reference Type, Object Type, Object Instance, PID and Start Index, which shall all be encoded and used as specified for PDT_REFERENCE in document [15]. The values of these fields shall in this way build a reference to the first displayed value of the enumeration; the next displayed values shall be read as is indicated in the field subfield Subtype of Reference Type (see [15]). The MaS shall only reply with this reference type only if the MaS effectively also supports Extended Property Services.

4.11.18 E-Mode Channel Object – PID_PARAM_FLAGS (PID = 71)

- Property name: Parameter Flags
- Property Datatype: PDT_BITSET16[]
- Datapoint Type: None.

The Property *Parameter Flags* shall be an array Property with number of elements equal to the number of Parameters for this E-Mode Channel.

It shall specify for each Parameter n in this E-Mode Channel in Property Value array element n its Parameter Flags.

Parameter nr	Property Value Array Index	Property Value
-	0	(current nr. of array elements)
1	1	Parameter Flags of Parameter 1
2	2	Parameter Flags of Parameter 2
3	3	Parameter Flags of Parameter 3
...	...	...

Table 56 – PID_PARAM_FLAGS format

The parameter flags of one Parameter shall be encoded as specified in Table 57.

Table 57 – Coding of Parameter Flags

Bit Nr.	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
field symbol	r	r	r	r	r	r	r	r	r	r	r	r	r	R	X	L
	0	0	0	0	0	0	0	0	0	0	0	0	0			

- bit 0 (lsb):
 name (abbreviation): Local (L)
 description: This flag shall indicate whether the Parameter is set locally or not.
 encoding:
 0: This Parameter is not set locally; this means that if it is attempted to set this Parameter Value from remote, that this will be accepted.
 1: This Parameter is set locally. An attempt to set this parameter remotely will be refused by the device; this Parameter is currently “read-only”.

- bit 1:
 - name (abbreviation): Void (X)
 - description: This flag shall indicate whether this Parameter is void or not. The value of a void Parameter shall have no effect on the behaviour of the application. This flag shall be independent of the value of the L-flag.
NOTE 67 A Parameter may for instance become void due to the value of another Parameter (such as AJS): the Parameter is still accessible but has no effect.
 - encoding:
 - 0: This Parameter is not void.
 - 1: This Parameter is void.
 - bit 2 (lsb):
 - name (abbreviation): Restart (R)
 - description: This flag shall indicate whether the changing of this Parameter will have immediate effect, or will only become effective after a restart of the device.
 - encoding:
 - 0: If this Parameter is changed, it will have immediate effect.
 - 1: If this Parameter is changed, it will have effect only after the next restart of the device.
- Ctrl FEC**
If the Controller modifies one or more Parameters for which the R flag is set, it shall at the end of the Configuration Procedure restart the device ¹⁶⁾.
- PB FEC**
Any device that hosts the Parameter shall keep track if during the PB-Mode linking one or more Parameters are modified of which the flag R is set. If this is the case, the device shall after the linking has completed autonomously execute a restart.
- bits 15 to 3:
 - These bits are reserved and shall be cleared in the device.
 - The E-Mode Management Client shall ignore the value of these bits.

¹⁶⁾ This allows that the Controller changes two more Parameters one after the other and that these only become active or evaluated together in a consistent way after a restart.

4.11.19 E-Mode Channel Object – PID_PARAM_NAMES (PID = 72)

- Property name: Parameter Names
- Property Datatype:
 - PDT_GENERIC_10[] (DPT_UTF_8 (DPT_ID: 28.001))
 - PDT_REFERENCE[]
- Datapoint Type: Every character to be encoded as DPT_UTF_8 (DPT_ID: 28.001)

The Property *Parameter Names* shall be an array Property with number of elements equal to the number of *Parameters* for this E-Mode Channel.

It shall specify for each Parameter *n* in this E-Mode Channel in Property Value array element *n* its parameter name, which shall be a human readable description and identification of the parameter.

Parameter nr	Property Value Array Index	Property Value
-	0	(current nr. of array elements)
1	1	Parameter Name of Parameter 1
2	2	Parameter Name of Parameter 2
3	3	Parameter Name of Parameter 3
...	...	...

Table 58 – PID_PARAM_NAMES format

Every parameter name shall be a 10 octet value, of which every octet shall encode one textual character according DPT_UTF_8 (DPT_ID: 28.001), starting with the most significant octet; unused octets shall be filled with 00h.

4.11.20 E-Mode Channel Object – PID_PARAM_UNITS (PID = 73)

- Property name: Parameter Units
- Property Datatype:
 - PDT_GENERIC_10[] (DPT_UTF_8 (DPT_ID: 28.001))
 - PDT_REFERENCE[]
- Datapoint Type: None.

The Property *Parameter Units* shall be an array Property with number of elements equal to the number of *Parameters* for this E-Mode Channel.

It shall specify for each Parameter *n* in this E-Mode Channel in Property Value array element *n* its parameter unit as shown in Table 59.

4.11.20.1 Format

PID_PARAM_UNITS shall be an array Property.

Every array element shall be a string of textual Characters each encoded according DPT_UTF_8 (DPT_ID: 28.001), as shown in Table 59.

Table 59 – PID_PARAM_UNITS format

Parameter nr	Property Value Array Index	Property Value
-	0	(current nr. of array elements)
1	1	Unit of Parameter 1
2	2	Unit of Parameter 2
3	3	Unit of Parameter 3
4	4	Unit of Parameter 4
...	...	...

The characters shall be left aligned on most significant positions (left) of the Property Value array element; unused Characters shall have the value 00h and shall be positioned on the least significant positions (right) of the Property Value array element.

Parameter nr	Property Value Array Index	Property Value								
-	0	4								
1	1	4Dh	69h	6Eh	75h	74h	65h	73h	00h	Minutes
2	2	B0h	43hh	00h	00h	00h	00h	00h	00h	°C
3	3	25h	00h	00h	00h	00h	00h	00h	00h	%
4	4	4Ch	75h	78h	00h	00h	00h	00h	00h	Lux
...	...									

Figure 47 – Example PID_PARAM_UNITS

Alternatively, the PDT may be PDT_REFERENCE, in case of which all array elements shall be references to strings in a Catalogue Object.

4.11.20.2 Usage by the Management Server (device)

PID_PARAM_UNITS shall hold in each array element n human readable textual information about the physical unit of the Parameter Value n or a reference to it.

4.11.20.3 Usage by the Management Client

The Management Client shall read out PID_PARAM_UNITS during the discovery of the device in order to be able to give human understandable representations of the Parameter Values in its user interface.

4.11.21 E-Mode Channel Object – PID_PARAM_VALUES (PID = 79)

- Property name: Parameter Values
- Property Datatype:
 - PDT_GENERIC_01[], or
 - PDT_GENERIC_02[], or
 - PDT_GENERIC_04[], or
 - PDT_GENERIC_10[]
- Datapoint Type: None.

The Property *Parameter Values* shall be an array Property with number of elements equal to the number of *Parameters* for this E-Mode Channel.

It shall specify for each Parameter *n* in this E-Mode Channel in Property Value array element *n* its parameter value as shown in Table 60.

Table 60 – PID_PARAM_VALUES format

Parameter nr	Property Value Array Index	Property Value
-	0	(current nr. of array elements)
1	1	Parameter Value of Parameter 1
2	2	Parameter Value of Parameter 2
3	3	Parameter Value of Parameter 3
4	4	Parameter Value of Parameter 4
...	...	...

The Property Datatype of this Property may vary according the size of the Parameter Values to be stored. The PDT shall be any of the type PDT_GENERIC_01, PDT_GENERIC_02, PDT_GENERIC_04 or PDT_GENERIC_10.

Each parameter value shall be left adjusted in the Property Value array element value: the msb of the parameter value shall be the msb of the Property Value array element value. If the Parameter Size is smaller than the PDT size, then the unused Property Value array element bits and octets shall be at the lower positions and shall be 0.

EXAMPLE 30 Table 61 shows an example of PID_PARAM_VALUES with PDT_GENERIC_02. (Cells for unused bits are left empty; in a device, these shall be filled with 0).

Table 61 – PID_PARAM_VALUES Example

Parameter Type	Parameter Size	Parameter Number	Property Value Array Index	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			0	4															
PART Boolean	1 bit	1	1	1															
PART UpDown_Switch_Action	2 bit	2	2	0	1														
PART COV Lux	1 octet	3	3	0	0	0	0	1	0	1	0								
PART Light_Value	2 octets	4	4	0	0	0	0	0	0	0	1	1	1	1	1	0	1	0	0

4.11.21.1 Usage by the Management Client

If the Management Client writes the Property Value then it shall clear the unused bits.

Every *Parameter Value* *n* shall be encoded as specified by the *Parameter Type* in the array with the same element number *n* in the Property *Parameter Types* (PID_PARAM_TYPES) as specified in 4.11.17.

If the FEC Channel is an Adjustable Channel then the Adjustable Parameter shall be the first parameter in the Property Parameter Values.

The Management Client should hold the run state machine of the Application Program prior to changing any Parameter Value. Otherwise, it is possible that the Management Server (device) gives a negative A_PropertyValue_Write.res if a Parameter value is changed that it does not allow changing during runtime.

4.11.21.2 Usage by the Management Server

The Management Server may negatively confirm the write access to Parameter values (negative A_PropertyValue_Write.res) if it is requested to write a Parameter value that cannot be changed without halting the Application Program.

The Management Server may use the unused fields in PID_PARAM_VALUES for internal purposes. If the Management Client however reads the value of PID_PARAM_VALUES, the Management Server shall guarantee that the response contains the value 0 for the unused fields.

4.12 Adjusted E-Mode Channel Object (Object Type 15)

4.12.1 General

Any instance of the Adjusted E-Mode Channel Object shall represent the adjustable data (description and configuration of Group Objects and Parameters) of one E-Mode Channel Object for one value of its Parameter AJS.

Please refer to [17] for the mandatory and optional Properties in the Adjusted E-Mode Channel Object.

NOTE 68 There are no requirements concerning the Object Indexes of the Adjusted E-Mode Channel Objects. These Object Indexes may or may not follow the Object Index of the parent E-Mode Channel Object and may or may not be consecutive.

Table 62 – Adjusted E-Mode Channel Object (informative)

PID Value	PID Name	Property Name	PDT/DPT	Access
1	PID_OBJECT_TYPE	Interface Object Type	DPT_PropDataType (DPT_ID = 7.010)	3/1
2	PID_OBJECT_NAME	Interface Object Name	- PDT_UNSIGNED_CHAR[] (DPT_UTF_8 (DPT_ID: 28.001)) - PDT_REFERENCE	3/1
61	PID_GO_CCODES_LIST	Connection Codes	PDT_GENERIC_08[]	3/1
62	PID_GO_CFLAGS_LIST	Connection Flags	PDT_BITSET16[]	3/1
63	PID_OBJECTLINK	Object Link	PDT_FUNCTION	3/3
65	PID_GO_NAME_LIST	Group Object Names	- PDT_GENERIC_10[] or - PDT_REFERENCE[]	3/1
70	PID_PARAM_TYPES	Parameter Types	PDT_GENERIC_10[]	3/1
71	PID_PARAM_FLAGS	Parameter Flags	PDT_BITSET16[]	3/1
72	PID_PARAM_NAMES	Parameter Names	- PDT_GENERIC_10[] or - PDT_REFERENCE[]	3/1
73	PID_PARAM_UNITS	Parameter Units	- PDT_GENERIC_10[] or - PDT_REFERENCE[]	3/1
79	PID_PARAM_VALUES	Parameter Values	- PDT_GENERIC_01[], or - PDT_GENERIC_02[], or - PDT_GENERIC_04[], or - PDT_GENERIC_10[]	3/3

4.12.2 Adjusted E-Mode Channel Object – PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the common specification of PID_OBJECT_TYPE.

The Object Type for the Adjusted E-Mode Channel Object shall have the value 15.

4.12.3 Adjusted E-Mode Channel Object – PID_OBJECT_NAME (PID = 2)

Please refer to 4.2.2 for the common specification of PID_OBJECT_NAME.

This Property *Object Name* shall contain or refer to the name of the Adjusted E-Mode Channel Object.

EXAMPLE 31 The first instance of the Object Adjusted E-Mode Channel, for AJS = 1, may have the name "PB Switch". The second instance of this Object, for AJS = 2, may have the name "PB Toggle".

4.12.4 Adjusted E-Mode Channel Object – PID_GO_CCODES_LIST (PID = 61)

Please refer to the specification of PID_GO_CCODES_LIST in 4.11.11.

In any IO *Adjusted E-Mode Channel*, this Property shall specify the Connection Codes that apply for the GOs used in this IO *Adjusted E-Mode Channel*. This list may differ in number and value compared to the related IO *E-Mode Channel Object* and its possible further IOs *Adjusted E-Mode Channel*.

4.12.5 Adjusted E-Mode Channel Object – PID_GO_CFLAGS_LIST (PID = 62)

Please refer to the specification of PID_GO_CFLAGS_LIST in 4.11.12.

4.12.6 Adjusted E-Mode Channel Object – PID_OBJECTLINK (PID = 63)

Please refer to the specification of PID_OBJECTLINK in 4.11.13.

When establishing links in an Adjustable E-Mode Channel, the PID_OBJECTLINK Function Property of the Adjusted E-Mode Channel Object corresponding to the current value of the AJS parameter shall be used, instead of the same Property of the E-Mode Channel Object.

4.12.7 Adjusted E-Mode Channel Object – PID_GO_DIAGNOSTICS (PID: 66)

- **Property name:** Group Object Diagnostics
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None
- **Access Policy:** See 4.8.1.
- **Access Level:** 3/3

4.12.7.1 Abstract Resource Definition

The Function Property *Group Object Diagnostics* (PID_GO_DIAGNOSTICS) in the Adjusted *E-Mode Channel* Object shall have the same functionality as *Group Object Diagnostics* in the *Applicationprogram* Object and in the E-Mode Channel Object.

However, this Property shall relate exclusively to the GOs of the Adjusted E-Mode Channel Object instance in which it is hosted. Every Adjusted E-Mode Channel Object shall have at maximum one Property *Group Object Diagnostics*.

The field *Group Object Number* shall refer to the Group Object Index, which shall be the number of the Group Object relative within the Adjusted E-Mode Channel Object.

The further requirements concerning format and encoding, usage by the MaS and usage by the MaS shall be identical as in 4.8.1 and 4.11.16.

4.12.8 Adjusted E-Mode Channel Object – PID_GO_NAME_LIST (PID = 65)

Please refer to the specification of PID_GO_NAME_LIST in 4.11.15.

4.12.9 Adjusted E-Mode Channel Object – PID_PARAM_TYPES (PID = 70)

Please refer to the specification of PID_PARAM_TYPES in 4.11.17.

4.12.10 Adjusted E-Mode Channel Object – PID_PARAM_FLAGS (PID = 71)

Please refer to the specification of PID_PARAM_FLAGS in 4.11.18.

4.12.11 Adjusted E-Mode Channel Object – PID_PARAM_NAMES (PID = 72)

Please refer to the specification of PID_PARAM_NAMES in 4.11.19.

4.12.12 Adjusted E-Mode Channel Object – PID_PARAM_UNITS (PID = 73)

Please refer to the specification of PID_PARAM_UNITS in 4.11.20.

4.12.13 Adjusted E-Mode Channel Object – PID_PARAM_VALUES (PID = 79)

Please refer to the specification of PID_PARAM_VALUES in 4.11.21.

4.13 Text Catalogue Object (Object Type 16)

Table 63 – Text Catalogue Object

	PID	Property Name	PDT and DPT	Description
1	PID_OBJECT_TYPE	Interface Object Type	DPT_PropDataType (DPT_ID = 7.010)	Type of the Interface Object
25	PID_VERSION	Version	PDT_VERSION (alt. PDT_GENERIC_02) DPT_Version (217.001)	Version of this Text Catalogue Object
51	PID_LOCALE_LIST	Locale List	PDT_GENERIC_04[] DPT_Locale_ASCII (231.001)	List of the supported languages and regions
52	PID_LOCALE_SELECTION	Locale Selection	PDT_GENERIC_04 DPT_Locale_ASCII (231.001)	To change the active language and region
53	PID_ACTIVE_LOCALE	Active Locale	PDT_GENERIC_04 DPT_Locale_ASCII (231.001)	To read the active language and region
60	PID_STRING_001	String 001	PDT_UTF_8[] DPT_UTF-8 (28.001)	Text string 001
...	...	...	...	...
nn	PID_STRING_mm	String mm	PDT_UTF_8[] DPT_UTF-8 (28.001)	Text string mm
...	...	...	...	...
200	PID_STRING_141	String 141	PDT_UTF_8[] DPT_UTF-8 (28.001)	Text string 141

4.13.1 General requirements

The Text Catalogue Object shall be used to hold in its Object Type dependent Properties amongst other Properties for storing and managing textual data.

There may be more than one instance of the Text Catalogue Object in a device. No standard functionality is foreseen for references or dependencies between Catalogue Object Instances.

EXAMPLE 32 If the language of one Catalogue Object is changed, the language of any other Catalogue Object is not affected. The Property Client thus has to observe for itself that when it accesses another Catalogue Object, that it should verify that IO's language.

4.13.2 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the common specification of PID_OBJECT_TYPE.

This Property shall contain Object Type of the Interface Object. The Text Catalogue Object shall have Object Type 16.

4.13.3 PID_VERSION (PID = 25)

Please refer to 4.2.25 for the general requirements for PID_VERSION.

This Property shall contain the version of the Text Catalogue Object.

4.13.4 PID_LOCALE_LIST (PID = 51)

- Property name: *Locale List*
- Property Datatype: PDT_GENERIC_04[]
- Datapoint Type: DPT_Locale_ASCII (see [14])

4.13.4.1 Abstract Resource definition

The Property *Locale List* shall be used to list the available language flavours and regional options of the textual data accessible in the device.

This Property shall allow discovery of the languages and regional dependencies for which textual information may be available in the device.

This list should be correct and complete. The following is the responsibility of the product manufacturer.

- This list should be complete. That is, it should contain combinations of language and region to which the textual data can be switched.
- This list should not contain languages or regions or combinations thereof that are not supported.

4.13.4.2 Encoding

This Property shall be an array of elements each encoded according DPT_Locale_ASCII.

There is no requirement concerning the sorting of this list.

Property Value Array Index	Value	Contents	Comment
0	4	nr. of elements	
1	6465h 4445h	"de DE"	German Germany
2	656Eh 4742h	"en GB"	English Great-Britain
3	6E6Ch 4245h	"nl BE"	Dutch Belgium
4	656Eh 5553h	"en US"	English United States

Figure 48 – PID_LOCALE_LIST

4.13.4.3 Usage by the Management Server (device)

The Management Server shall list in the *Locale List* all locale options (languages and regions) that it provides for the accessible textual information in it.

4.13.4.4 Usage by the Management Client

The Management Client shall read this Property to discover all possible regional options for which textual information can be accessed in the device.

If this Property is not available, the Management Client may assume that the language- and regional options of the textual data in the device cannot be selected.

4.13.5 PID_LOCALE_SELECTION (PID = 52)

- Property name: *Locale Selection*
- Property Datatype: PDT_GENERIC_04
- Datapoint Type: DPT_Locale_ASCII (see [14])

4.13.5.1 Abstract Resource definition

The Property Locale Selection shall allow the Management Client to set one language and region for the Text Catalogue Object.

4.13.5.2 Encoding

This Property shall be encoded according DPT_Locale_ASCII.

Property Value Array Index	Value	Contents	Comment
0	1	nr. of elements	
1	656Eh 4742h	"en GB"	English Great-Britain is requested

4.13.5.3 Usage by the Management Server (device)

The Management Server shall use this Property Value to learn the language and regional options selected by the Management Client.

This Property shall always be writeable.

If the language and regional options of the textual Properties can be switched, then the Property Server shall switch the contents of the textual Properties in this Text Catalogue Object according to the value set by the Management Client. This means that the Management Server shall on a change of PID_LOCALE_SELECTION to a supported language and region, accordingly change the Values of the Properties PID_STRING_001 to PID_STRING_141.

The Management Server may activate the selected language and region either as soon as possible or only after a restart.

When the selected language and region become active, the Management Server shall set the value of PID_ACTIVE_LOCALE accordingly.

Error handling

If an A_PropertyValue_Write-PDU is received with a value for PID_LOCALE_SELECTION for a language and region that are not supported by the Management Server, then the Management Server shall not switch the language or regional options and shall respond with a negative A_PropertyValue_Response-PDU.

4.13.5.4 Usage by the Management Client

The Management Client shall use PID_LOCALE_SELECTION to select the active language and regional options in the hosting Text Catalogue Object. After changing PID_LOCALE_SELECTION, the Management Client should read PID_ACTIVE_LOCALE. If the value set for PID_LOCALE_SELECTION and the read value from PID_ACTIVE_LOCALE do not equal, and if the Management Client did not get a negative response when setting PID_LOCALE_SELECTION, then the Management Client may assume that the Management Server requires a restart before the selected locale becomes effective.

4.13.6 PID_ACTIVE_LOCALE (PID = 53)

- Property name: *Active Locale*
- Property Datatype: PDT_GENERIC_04
- Datapoint Type: DPT_Locale_ASCII (see [14])

4.13.6.1 Abstract Resource definition

In this Property, the Management Server shall at any time hold the currently active language and regional options.

4.13.6.2 Encoding

This Property shall be encoded according DPT_Locale_ASCII.

Property Value Array Index	Value	Contents	Comment
0	1	nr. of elements	
1	656Eh 4742h	"en GB"	English Great-Britain is active

4.13.6.3 Usage by the Management Server (device)

The Management Server shall in this Property at any time hold the currently active language and regional options.

This Property Value shall be any of the elements contained in PID_LOCALE_LIST (see 4.13.4), if available.

4.13.6.4 Usage by the Management Client

The Management Client shall read PID_ACTIVE_LOCALE to learn about the active language and regional options in the *Text Catalogue Object*.

In particular, when the Management Client has requested a change of the locale options through PID_LOCALE_SELECTION, it shall read PID_ACTIVE_LOCALE to check whether the selected language and regional options become effective immediately or only after a restart.

4.13.7 PID_STRING_001 (PID = 60) to PID_STRING_141 (PID = 200)

- Property name: String 001 to String 141
- Property Datatype: PDT_UTF_8 (A[n], DPT_UTF-8)
- Datapoint Type: See above.

4.13.7.1 Abstract Resource definition

These Properties shall be used to hold the textual data of the *Text Catalogue Object*.

The Property Datatypes shall be PDT_UTF_8 (A[n]).

These Properties are intended to be referred to by textual Properties in other Interface Objects in the Management Server, that shall then be of type PDT_REFERENCE (see [15]). The semantic meaning of the Properties PID_STRING_001 to PID_STRING_141 is thus only given by the referring Properties. It should be noted that PDT_REFERENCE allows referring to a start_index within an array Property. This allows that one Property holds more than one string.

The Text Catalogue Object shall at least contain the Property PID_STRING_001; additional Properties in the range PID_STRING_002 to PID_STRING_141 shall have subsequent Property Identifiers.

4.13.7.2 Usage by the Management Server (device)

The Management Server shall make available in these Properties textual data that can be read out in one or more languages selectable by the Management Client.

These Properties may be referred by textual Properties in other Interface Objects by using PDT_REFERENCE as alternative Property Datatype.

If the Management Server accepts a change of language of the *Text Catalogue Object* through PID_LOCALE_SELECTION by the Management Client, then the Management Server shall make accessible through these Properties the textual data according the selected language and regional options. This may require a restart of the Management Server.

The correctness and the completeness of these language values are the responsibility and choice of the device manufacturer. If for a certain language, not all textual information is translated, then it may be that the textual information is provided when read out in another language than the activated language.

EXAMPLE 33 If "Dutch" is selected, but for "Ausgang A" the manufacturer has not given the Dutch translation, then still "Ausgang A" may be provided when the textual Property Value be read out.

4.13.7.3 Usage by the Management Client

The Management Client shall read the Values of the Properties PID_STRING_001 to PID_STRING_141 – as far as implemented in the Management Server – to read textual information.

4.14 E-Mode Device Object (Object Type 18)

4.14.1 Overview

Table 64 – E-Mode Device Object

PID	Property Name	PDT and DPT	Description
1 PID_OBJECT_TYPE	Interface Object Type	DPT_PropDataType (DPT_ID = 7.010)	Type of the Interface Object
60 PID_LOCALISATION_MODE	Localisation Mode	DPT_State (DPT_ID = 1.011) PDT_BINARY_INFORMATION (alt.: PDT_UNSIGNED_CHAR)	To activate and inactivate the Localisation Mode.
61 PID_LOCALISATION_REPORT	Localisation Report	PDT_UNSIGNED_INT U ₁₆	To report on human operations performed for localisation.
62 PID_LOCALISATION_COMMAND	Localisation Command	PDT_GENERIC_03 U ₈ U ₁₆	To receive localisation commands.

4.14.2 General requirements

The E-Mode Device Object shall be used to hold in its Object Type dependent Properties amongst other Properties for the Property based Management of E-Mode FEC devices for:

- localisation,
- E-Mode Channel independent Parameters
- etc.

4.14.3 E-Mode Device Object – PID_OBJECT_TYPE (PID = 1)

Please refer to clause 4.2.1 for the common specification of PID_OBJECT_TYPE.

The Object Type for the E-Mode Device Object shall have the value 18.

4.14.4 E-Mode Device Object – PID_LOCALISATION_MODE (PID = 60)

- Property name: Localisation Mode
- Property Datatype: PDT_BINARY_INFORMATION (alt.: PDT_UNSIGNED_CHAR)
- Datapoint Type: DPT_State (DPT_ID = 1.011)

4.14.4.1 Usage by the Management Server (device)

This Property shall control whether the *Localisation Mode* is inactive or active.

NOTE 69 Please refer to [10] for the definition of *Localisation Mode*.

If the Management Client sets PID_LOCALISATION_MODE = Active, then the Management Server shall immediately activate *Localisation Mode*. It shall also temporarily store the Source Address of the Management Client as long as the *Localisation Mode* is active. If the Management Client sets PID_LOCALISATION_MODE = Inactive, then the Management Server shall immediately inactivate *Localisation Mode* and may clear the Individual Address information of the Management Client.

The default value shall be 0 = Inactive.

Once *Localisation Mode* has become Active, the Management Server shall start a time-out timer of 3:30 minutes. This time-out timer shall be restarted with each activation or repeated activation of the *Localisation Mode* through PID_LOCALISATION_MODE by the Management Client. If this time-out timer expires, the Management Server shall autonomously inactivate *Localisation Mode*.

NOTE 70 It is not foreseen that the Management Server informs the Management Client in any way if it autonomously inactivates its *Localisation Mode*.

4.14.4.2 Usage by the Management Client

6	Octet 7							Octet 8							Octet 9							Octet 10							Octet 11													
°)	APCI							object_type														property_id							value													
1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0									
APC	APC	APC	APC	APC	APC	APC	APC	E-Mode Device Object = 18														PID_LOCALISATION _MODE = 60																				
1	1	1	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	1	1	1	1	0	0							

°) last two bits of octet 6

v : value of the *Localisation Mode*

Figure 49 – A_NetworkParameter_Write-PDU with Localisation Mode (example)

The Management Client shall activate the *Localisation Mode* in the Management Server by setting PID_LOCALISATION_MODE to 1 via broadcast communication mode in all devices supporting this feature in the installation or via point-to-point connectionless communication mode in one single device. As the *Localisation Mode* automatically times out in the Management Server after 3:30 minutes, the Management Client shall resend the localisation request every 1 minute as long as it is required by the installer via the Controller user interface.

```

/* Activate the Localisation Mode in all FEC devices supporting this feature in the installation */
NM_NetworkParameter_Write_R(ASAP = void; comm_mode = broadcast; hop_count_type_request = Network Layer
Parameter; object_type = E-Mode Device Object, PID = PID_LOCALISATION_MODE, priority = system,
value = 1)
OR
/* Activate the Localisation Mode in one single device */
NM_NetworkParameter_Write_R(ASAP = Device.IA; comm_mode = point-to-point connectionless;
hop_count_type_request = "Network Layer Parameter"; object_type = E-Mode Device Object,
PID = PID_LOCALISATION_MODE, priority = system, value = 1)

/* The FEC devices shall store the Source Address of this Procedure for later use. */

```

If the Controller wants to quit the Localisation Mode, it shall explicitly do this (this is; it shall not wait until the *Localisation Mode* automatically becomes inactive in the devices).

```
/* Inactivate the Localisation Mode in all FEC devices in the installation.*/
NM_NetworkParameter_Write_R(ASAP = void; comm_mode = broadcast; hop_count_type_request = "Network Layer
Parameter"; object_type = E-Mode Device Object, PID = PID_LOCALISATION_MODE, priority = system,
value = 0)
```

If the Controller wants to read out the Localisation Mode of a device, it shall use the point-to-point connectionless communication mode.

```
/* Read the Localisation Mode in one single device.*/
NM_NetworkParameter_Read_R(ASAP = Device.IA; comm_mode = point-to-point connectionless;
hop_count_type_request = "Network Layer Parameter"; object_type = E-Mode Device Object,
PID = PID_LOCALISATION_MODE, priority = system)
```

4.14.5 E-Mode Device Object – PID_LOCALISATION_REPORT (PID = 61)

- Property name: Localisation Report
- Property Datatype: PDT_UNSIGNED_INT
- Datapoint Type: None.

4.14.5.1 Format

The Property *Localisation Report* shall have the format as specified in Figure 50.

octet	octet 0								octet 1							
bit nr	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
field	Channel Number															
value																

Figure 50 – Localisation Report format

- **octet 0 and 1: Channel Number**

definition: This shall contain the number of the device Channel on which a user localisation action is performed. This shall be the value of the Property PID_CHAN_NUMBER of the E-Mode Channel for which the localisation action is reported.

encoding: 2 octets: U₁₆

E-Mode Channel numbers shall be counted starting from 1.

The Channel Number 0000h shall be reserved to denote “the entire device” or all E-Mode Channels.

4.14.5.2 Usage by the Management Server (device)

If *Localisation Mode* is active in a FEC device and if a user localisation action is performed, then the FEC device shall send an A_NetworkParameter_InfoReport-PDU in point-to-point communication mode to the IA of the Management Client that has activated the *Localisation Mode* as follows.

6		Octet 7							Octet 8							Octet 9							Octet 10							Octet 11							Octet 12							Octet 13																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
°)		APCI							object_type														property_id							test_info							test_result																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
1		0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
APCI		APCI		APCI		APCI		APCI		APCI		APCI		APCI		APCI		APCI		E-Mode Device Object = 18														PID_LOCALISATION _REPORT = 61							reserved 00h							Channel Number																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																								
1		1	1	1	1	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

°) last two bits of octet 6

Figure 51 – A_NetworkParameter_InfoReport-PDU with Localisation Report (example)

```
/* Report on a localisation action on E-Mode Channel n. */
DMP_InterfaceObjectInfoReport_RCI(mpp_ASAP = Controller.IA, mpp_Obj.Type = E-Mode Device Object,
  mpp_Prop.ID = PID_LOCALISATION_REPORT, mpp_Test_Info = 00h, mpp_Test_Result = Channel Number)
```

NOTE 71 The field test_info shall be 00h. This field can be used for later extensions.

The *Localisation Report* is typical for but not limited to sensors.

4.14.5.3 Usage by the Management Client

The Management Client shall accept the A_NetworkParameter_InfoReport-PDU from the Management Server only under the following conditions:

- 1 it has before activated the *Localisation Mode* in that Management Server, and
- 2 this *Localisation Mode* is not yet inactive, this is, the Management Client has itself not yet inactivated the *Localisation Mode* in that Management Server.

In all other cases, the Management Client shall ignore the information report on PID_LOCALISATION_-REPORT.

If the Management Client accepts the Localisation report then it shall interpret it appropriately in its internal data and in its user interface.

4.14.6 E-Mode Device Object – PID_LOCALISATION_COMMAND (PID = 62)

- Property name: Localisation Command
- Property Datatype: PDT_GENERIC_03
- Datapoint Type: None.

NOTE 72 This clause uses the term “device localisation action”. For the definition please refer to [01].

4.14.6.1 Format

octet	octet 0								octet 1								octet 2							
bit nr	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
field	r	r	r	r	r	r	P	S	Channel Number															
value	0	0	0	0	0	0																		

Figure 52 – Localisation Command format

- octet 0 - bit 7 to 2:
name (abbreviation): reserved (r)
description: These bits are reserved.
encoding: These bits shall be cleared by the E-Mode Management Client.
The Management Server (device) shall ignore the value of these bits and shall not react.

- octet 0 - bit 1:

name (abbreviation):	Physical (P)
description:	It is possible that an E-Mode Channel exhibits two different localisation actions. This bit <i>Physical</i> shall indicate whether the device localisation action shall be physical action or not.
	EXAMPLE 34 A shutter channel may for localisation perform a short movement. This is the Physical localisation action. It is interesting if the human is present at the location where the E-Mode Channel takes effect. It may additionally or alternatively exhibit a status LED on the housing. An alternative localisation action could blink this LED. This is the “non-physical localisation action”. This is interesting if the human is present in front of the electrical distribution cabinet.
encoding:	0: Physical action 1: Not a physical action It is optional to support “Not a physical action” for a Management Server. If this is not supported and a Localisation command is received with this value then the Management Server shall not react.
- octet 0 - bit 0:

name (abbreviation):	Start (S)
description:	This bit shall indicate whether the device localisation action shall be started or stopped.
encoding:	0: Stop 1: Start
- octet 1 and 2:

name (abbreviation):	Channel Number
description:	This shall be the Channel Number for which it is requested that the device localisation action be started or stopped. This shall be the value of the Property PID_CHAN_NUMBER of the E-Mode Channel for which the localisation action is requested.
encoding:	The value shall be binary encoded. 0000h: The device localisation action shall be started or stopped for all E-Mode Channels in this device or for the entire device. 0001h to FFFFh: The device localisation action shall be started or stopped for the E-Mode Channel given by this value.

4.14.6.2 Usage by the Management Server (device)

The *Localisation Command* is typical for but not limited to actuators.

The Management Server shall interpret write accesses to PID_LOCALISATION_COMMAND exclusively if *Localisation Mode* is active.

If *Localisation Command* is written, then the Management Server shall perform its device localisation action for the contained E-Mode Channel as requested.

The device localisation action shall be started or stopped as indicated by the field *Start*. The device localisation action shall at the latest stop if *Localisation Mode* becomes inactive in the Management Server.

4.15 RF Medium Object (Object Type 19)

4.15.1 Overview

The RF Medium Object shall deal with the features of the KNX RF Communication Medium like:

- physical acknowledge, and
- Slow/Fast frequency transmission/reception, and
- Call RF channel.

This Interface Object shall be used to get the physical requirements of the RF device and shall thus enable the MaC to verify the compatibility between KNX RF devices in terms of RF capabilities (e.g. RF scanning and RF transmission). According to the physical requirements of each device, the MaC may enable or not the link between devices. The MaC shall be able to give additional information to the device along with data of the link (e.g. Fast or Slow transmission, acknowledge slot number...).

4.15.2 RF Medium Object for cEMI

Some of the cEMI Additional Information for KNX RF Multi may also be available in the local management between cEMI Client and cEMI Server.

The local management gives the possibility to a manufacturer to define dedicated behaviour of the cEMI Server device, independently of any L_Data messages.

EXAMPLE 35 A script that contains KNX RF Multi data Frames can be run in different modes. The same list of RF Frames is re-used, several times, just preceded by a different setting of the Management Server.

Some of the relevant local management Properties are placed within the RF Medium Object. Please refer to [17] for the conditions on the mandatory - or optional support of the RF Medium Object and the Properties.

These Properties are used mainly to configure the cEMI Server's Physical Layer. They shall not be used to cause the transmission of an RF message.

These are some examples for local device management functions.

- Setting the reception mode of the cEMI Server (e.g. Fast, Slow, Fast & Slow)
- Transmitting a carrier on a specific frequency.
- Transmitting a modulated signal on a specific frequency.
-

For cEMI Server local management, the RF Medium Object (Object Type 10) may hold the Properties as specified in Table 65.

Table 65 – Properties in the RF Medium Object (informative)

Property Identifier	PID	Description, remark
PID_RF_MULTI_TYPE	51	Set the KNX RF Type to ready or to Multi.
PID_RF_MULTI_PHYSICAL_FEATURES	52	Expose the supported features of KNX RF Multi.
PID_RF_MULTI_CALL_CHANNEL	53	Set the Call Channel for KNX RF Multi Fast - and Slow frequencies.
PID_RF_MULTI_OBJECT_LINK	54	GA management of the KNXRF Multi specific features.
PID_RF_MULTI_EXT_GA_REPEATED	55	Management of repeated KNX RF Multi GAs.
PID_TRANSMISSION_MODE	70	To manage the KNX RF Multi transmission mode of the RF Physical Layer
PID_RECEPTION_MODE	71	To manage the KNX RF Multi reception mode of the RF Physical Layer
PID_TEST_SIGNAL	72	To manage the transmission of a test signal on the specific frequency
PID_FAST_ACK	73	To manage the Fast Ack in reception
PID_FAST_ACK_ACTIVATE	74	To activate or deactivate the Fast Ack management in reception

4.15.3 PID_RF_MULTI_TYPE (PID = 51)

4.15.3.1 Format

- Property name: RF Multi Type
- Property Datatype: PDT_GENERIC_01
- Datapoint Type: none

4.15.3.1.1 Abstract Resource definition

The Property *RF Multi Type* shall indicate and control whether the device is or operates as a KNX RF Ready device or as a KNX RF Multi device.

4.15.3.1.2 Format

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved	reserved	reserved	RF-Multi Type
Values	0	0	0	0	0	0	0	0, 1

Bit	Name	Description	Encoding
0	RF-Multi Type	<i>RF-Multi Type</i> shall indicate whether the KNX RF device is configured as “KNX RF Ready” type or as “KNX RF Multi” type	0: KNX RF Multi Device set in KNX RF Ready type 1: KNX RF Multi Device set in KNX RF Multi type
1 to 7	reserved	These bits are reserved and shall always be 0.	not applicable

4.15.3.2 Usage by the Management Client

The MaC shall use the Property *RF Multi Type* to set the value of the *KNX RF Type* in the device.

- If the parameter PID_RF_MULTI_TYPE is set to KNX RF Ready type value then it is recommended to use the Property PID_OBJECT_LINK in the Device Object for the link management.
- If the parameter PID_RF_MULTI_TYPE is set to KNX RF Multi type value then it is recommended to use the Property PID_RF_MULTI_OBJECT_LINK in the RF Medium Object for the link management.

The MaC shall only modify this parameter in a MaS that is in ex-factory state. This shall avoid any inconsistency in the device configuration.

EXAMPLE 36 If this Property is set in an already configured MaS, then this may lead to a mixture of links added in RF Multi type mode with Fast Ack management and links added in RF Ready type mode without Fast Ack management.

4.15.3.3 Usage by the Management Server

Please refer to the Profile definition in [17] for the mandatory and optional support of the Property *RF Multi Type*.

This Property shall only be implemented in the KNX RF Multi stack. A native KNX RF 1.1 or KNX RF Ready Device shall not implement this Property and react according the Property error handling if this Property is addressed. This shall indicate that the device is a KNX RF 1.1 or native KNX RF Ready device, this is, with RF-Multi Type = 0

- If this parameter is set to the KNX RF Ready type, then the KNX RF Multi device shall act as a KNX RF Ready device. The specific RF Multi functionalities like multi frequencies, fast physical acknowledge shall not be supported by the device. This value shall enable the KNX RF Multi device (set in KNX RF Ready type) to be compatible and configurable with KNX RF 1.1 devices.
- If this parameter is set to the KNX RF Multi type, then the KNX RF Multi device shall act as a KNX RF Multi device. It shall then be possible to manage all the RF Multi fonctionnalities of the device, like multi frequencies, fast physical acknowledge (according to the physical requirements of the device). If this parameter is set in the KNX RF Multi type, the device shall neither be compatible nor configurable with a KNX RF 1.1 device.

If this Property is read or written, then the response shall contain the KNX RF type value that is actually written in the device.

The device that answers with $b_0 = 0b$ shall be a KNX RF Multi device that is set previously in the “KNX RF Ready” type.

4.15.4 PID_RF_MULTI_PHYSICAL_FEATURES (PID = 52)

4.15.4.1 Format

- Property name: RF Multi Physical Features
- Property Datatype: PDT_BITSET8
- Datapoint Type: none

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	PhysAck	TxSx	TxFx	Scan Sx	Scan Fx
Values	0	0	0	0, 1	0, 1	0, 1	0, 1	0, 1

Bit	Name	Description	Encoding
0	Scan Fx	scan of the fast Fx RF channels	0: The device does not scan the fast Fx RF channels in runtime and thus cannot receive Frames transmitted on a fast Fx RF channel. 1: The device scans the fast Fx RF channels in runtime and thus can receive Frames transmitted on a fast Fx RF channel.
1	Scan Sx	scan of the slow Sx RF channels	0: The device does not scan the slow Sx RF channels in runtime and thus cannot receive Frames transmitted on a slow Sx RF channel. 1: The device scans the slow Sx RF channels in runtime and thus can receive Frames transmitted on a slow Sx RF channel.

Bit	Name	Description	Encoding
2	TxFx	transmission on the fast Fx RF channels	0: The device cannot transmit Frames on the fast Fx RF channels. 1: The device is able to transmit Frames on the fast Fx RF channels.
3	TxSx	transmission on the slow Sx channels	0: The device cannot transmit Frames on the slow Sx RF channels. 1: The device is able to transmit Frames on the slow Sx RF channels.
4	PhysAck	management (Tx and Rx) of the fast physical acknowledge	0: The device is not able to manage a fast physical acknowledge in runtime. 1: The device is able to manage a fast physical acknowledge.
5 to 7	reserved	These bits are reserved and shall always be 0.	not applicable

4.15.4.2 Usage by the Management Server (device)

This Property shall be read-only.

If this Property is read, then the Management Server shall respond with the RF Multi physical features value.

4.15.4.3 Usage by the Management Client

The MaC shall read the Property RF Multi Physical Features to learn the RF multi physical capabilities of the device.

4.15.5 PID_RF_MULTI_CALL_CHANNEL (PID = 53)

4.15.5.1 Format

- Property name: RF Multi Call Channel
- Property Datatype: PDT_GENERIC_01
- Datapoint Type: none

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	Fast Frequency Call Channel		Slow Frequency Call Channel	
Values	0	0	0	0				

Bit	Name	Description	Encoding
0 to 1	Slow Frequency Call channel	This field shall indicate the slow frequency call channel.	00b: Call channel is the S1 slow frequency. 01b: Call channel is the S2 slow frequency. 10b: reserved for future use. 11b: reserved for future use.
2 to 3	Fast Frequency Call channel	This field shall indicate the fast frequency call channel.	00b: Call channel is the F1 fast frequency. 01b: Call channel is the F2 fast frequency. 10b: Call channel is the F3 fast frequency. 11b: reserved for future use.
4 to 7	reserved	These bits are reserved and shall always be 0.	not applicable.

4.15.5.2 Usage by the Management Server

The KNX RF Multi devices shall use the Slow – and Fast RF call channels as given by this Property value.

If this Property is read or written, then the response shall contain the KNX RF call channel value that is actually written in the device.

4.15.5.3 Usage by the Management Client

The MaC shall use the Property *RF Multi Call Channel* to set the value of the RF Call channel for the slow and the fast frequencies of the device.

4.15.6 PID_RF_MULTI_OBJECT_LINK (PID = 54)

4.15.6.1 Format

- Property name: RF Multi Object Link
- Property Datatype: PDT_Function
- Datapoint Type: none

4.15.6.1.1 Write (A_FunctionPropertyCommand-PDU)

4.15.6.1.1.1 Format

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16	octet 17	octet 18	octet 19	octet 20	octet 21
Flags	00	SN high	SN	SN	SN	SN	SN low	GA high	GA low	Handle high	Handle low

octet 22	octet 23
Physical	AckSlot Nb

Octet 12 to 17: Serial number

In case of RF Multi link added/deleted in sending mode and physical acknowledge requested in runtime, the field SN is the KNX Serial Number of the receiver.

In case of RF Multi link added/deleted in non-sending mode, the field SN is the KNX Serial Number part of the Extended Group Address.

Octet 10: Flags

The octet 10 shall indicate whether a link to the referred Group Object shall be established or shall be broken. In case the link shall be added, it shall also indicate whether the link shall be a sending Group Address or not.

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved	reserved	Add/Delete	Sending
Values	0	0	0	0	0	0		

Bit	Name	Description and Encoding
-----	------	--------------------------

- | | | |
|---|---------|---|
| 0 | Sending | <p>In case the contained Extended GA is added, then this field shall indicate whether it shall be the sending GA for the referred GO or not.</p> <p>This field shall only be interpreted in case the flag “<i>Add/Delete</i>” equals 0; in case the field “<i>Add/Delete</i>” equals 1, the value of the flag <i>Sending</i> shall be don't care.</p> <p>0: not sending: The contained GA shall not be the sending GA for the referred GO.</p> <p>1: sending: The contained GA shall be the sending GA for the referred GO.</p> |
|---|---------|---|

Error and exception handling

- If a link is added to a GO to which the contained GA is already linked, but different value of the field *Sending*, then this new value of the flag *Sending* shall be used.
- A GA set as "sending" shall take precedence on other previous sending GA for that GO.

- | | | |
|---|------------|--|
| 1 | Add/Delete | <p>It shall indicate whether the contained extended GA shall be added to or removed from the list of GA assigned to the referred GO.</p> <p>0: add: The contained GA shall be added to the list of GAs assigned to the referred GO.</p> <p>1: delete: The contained GA shall be removed from the list of GAs assigned to the referred GO</p> |
|---|------------|--|

2 to 7	reserved	These bits are reserved and shall always be 0.
--------	----------	--

Octet 20 and 21: Group Object Handle

The octet 20 and 21 shall contain the high and low octets of the Group Object Handle and shall identify the Group Object on which the extended Group Address must be set or deleted.

Octet 22: Physical Features (Fx and Sx frequencies)

The octet 22 and 23 contain data to manage the KNX RF Multi links.

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved			
Values	0	0	0	0	0			

Bit	Name	Description	Encoding
0		KNX RF Multi Fast Fx frequency bit:	- For an Output Datapoint 0: A Frame shall not be transmitted on a fast Fx frequency RF channel in runtime. 1: A Frame shall be transmitted on a fast Fx frequency RF channel in runtime. - For an Input Datapoint 0: No Frame is received on a fast Fx frequency RF channel in runtime. 1: A Frame may be received on a fast Fx frequency RF channel in runtime.
1		KNX RF Multi Slow Sx frequency bit:	- For an Output Datapoint 0: A Frame shall not be transmitted on a slow Sx frequency RF channel in runtime. 1: A Frame shall be transmitted on a slow Sx frequency RF channel in runtime. - For an Input Datapoint 0: No Frame is received on a slow Sx frequency RF channel in runtime. 1: A Frame may be received on a slow Sx frequency RF channel in runtime.
2		KNX RF Ready Fast F1 frequency bit	- For an Output Datapoint: 0: A Frame (KNX RF Ready Frame) shall not be transmitted on the fast F1 frequency RF channel in runtime. 1: A Frame (KNX RF Ready Frame) shall be transmitted on the fast F1 frequency RF channel in runtime - For an Input Datapoint: 0: No Frame (KNX RF Ready Frame) is received on a fast F1 frequency RF channel in runtime. 1: A Frame (KNX RF Ready Frame) may be received on a fast F1 frequency RF channel in runtime.
3 to 7	reserved	These bits are reserved and shall always be 0.	not applicable

Octet 23: Acknowledge Slot Number

The octets 22 and 23 shall contain data to manage the KNX RF Multi links.

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
N a m e	reserved	Physical Acknowledge	Physical Acknowledge Slot Number					
Values	0							

Bit	Name	Description	Encoding
0	Physical Acknowledge	Physical Acknowledge Slot Number	- For an output Datapoint 00h to 3Fh: If a Frame is transmitted on this Datapoint, a physical acknowledge shall be expected to be received on the specified ack slot number in runtime for this link. This means that if the output Datapoint is linked to N receivers, then N links have to be added on this output Datapoint. Each link containing the KNX Serial Number and the ack slot number used by the receiver for the sending of the physical acknowledge. - For an input Datapoint 00h to 3Fh: If a Frame is received on this Datapoint, a physical acknowledge shall be sent in the specified ack slot number in runtime for this link.
6	Physical Acknowledge		- For an output Datapoint 0: The Frame shall be transmitted with no physical acknowledge request in runtime. 1: The Frame shall be transmitted with a physical acknowledge request in runtime. - For an input Datapoint 0: If a Frame is received on this link, then the device shall never answer with a fast physical acknowledge because the sender will not request a physical acknowledge in runtime. The value in the physical slot number field shall not be used and shall be set to 0. 1: If a Frame is received on this link with the physical acknowledge bit set in the Frame, then the device shall answer with a fast physical acknowledge. The value of the acknowledge slot time shall be contained in the physical slot number field.
7	reserved	This bit is reserved and shall always be 0.	not applicable

4.15.6.1.1.2 Response (A_FunctionPropertyState_Response-PDU)

octet 10
return Code

The response shall contain the return code of the operation.

Return Code:

00h:	SUCCESS (assignment /deletion done or already existed / not present)
FDh:	ERR_RF_PHYSICAL (any element or combination of elements from the physical features is not supported or incorrect)
FEh:	ERR_OBJECT (object does not exist)
FFh:	ERR_TABLEFULL (no more space in link table)

4.15.6.1.1.2.1 Usage by the Management Client

The MaC shall use the function “RF Multi Object Link” to assign a new receiving GA to a GO or to delete a GA from a GO. Input parameters shall be the GO-Handle to identify the GO, the Extended Group Address to be assigned/deleted, physical features of the link (frequency and acknowledge slot number).

NOTE 74 The Extended Group Address may include the own KNX Serial Number of the device, in this case an internal assignment shall be established / deleted.

4.15.6.1.2 Read (A_FunctionPropertyState_Read-PDU)

4.15.6.1.2.1 Format

octet 10	octet 11
Code	Iterator

Octet 10: Code

The octet 10 shall indicate whether the read command is used to go through the list of links or to know which part of the link table is available.

- Code: 00h: Read the RF multi links through an iterator.
- 01h: Read unused and total size of the link table. In this case, the response shall contain the return code together with two octets for the number of free RF Multi links and two other octets of the total size of the RF Multi link table.

4.15.6.1.2.2 Response in case of Code = 00h (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16	octet 17	octet 18	octet 19	octet 20	octet 21
Return code	Flags	SN high	SN	SN	SN	SN	SN low	GA high	GA low	Handle high	Handle low

octet 22	octet 23
Physical	AckSlotNb

Octet 11: Flags

The octet 11 shall indicate whether the contained GA is the sending GA for the referred GO or not.

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved	reserved	reserved	Sending
Values	0	0	0	0	0	0	0	

Bit	Name	Description	Encoding
0	Sending	This field <i>Sending</i> shall indicate whether the contained extended GA is the sending GA for the referred GO or not.	0: not sending: The contained GA shall not be the sending GA for the referred GO. 1: sending: The contained GA shall be the sending GA for the referred GO.
1 to 7	reserved	These bits are reserved and shall always be 0.	not applicable

Octet 12 to 17: Serial number

In case of RF Multi link added/deleted in sending mode and physical acknowledge requested in runtime, the field SN is the KNX Serial Number of the receiver.

In case of RF Multi link added/deleted in non-sending mode, the field SN is the KNX Serial Number part of the Extended Group Address.

Octet 20 and 21: Group Object Handle

The octet 20 and 21 shall contain the high and low octets of the GO-Handle, to identify the GO on which the Extended Group Address shall be set or deleted.

Octet 22: Physical Features (Fx and Sx frequencies)

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Name	reserved	reserved	reserved	reserved	reserved			
Values	0	0	0	0	0			

Bit	Name	Description	Encoding
0		KNX RF Multi FastFx frequency bit	- For an output Datapoint 0: A Frame shall not be transmitted on a fast Fx frequency RF channel in runtime. 1: A Frame shall be transmitted on a fast Fx frequency RF channel in runtime. - For an input Datapoint 0: No Frame shall be received on a fast Fx frequency RF channel in runtime. 1: A Frame may be received on a fast Fx frequency RF channel in runtime
1		KNX RF Multi Slow Sx frequency bit	- For an output Datapoint 0: A Frame shall not be transmitted on a slow Sx frequency RF channel in runtime. 1: A Frame shall be transmitted on a slow Sx frequency RF channel in runtime. - For an input Datapoint 0: No Frame shall be received on a slow Sx frequency RF channel in runtime. 1: A Frame may be received on a slow Sx frequency RF channel in runtime.
2		KNX RF Ready Fast F1 frequency	- For an output Datapoint 0: A Frame (KNX RF Ready Frame) shall not be transmitted on the fast F1 frequency RF channel in runtime. 1: A Frame (KNX RF Ready Frame) shall be transmitted on the fast F1 frequency RF channel in runtime. - For an input Datapoint 0: No Frame (KNX RF Ready Frame) shall be received on a fast F1 frequency RF channel in runtime. 1: A Frame (KNX RF Ready Frame) may be received on a fast F1 frequency RF channel in runtime.

Bit	Name	Description	Encoding
3 to 7	reserved	These bits are reserved and shall always be 0.	not applicable

Octet 23: Acknowledge Slot Number

Field	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
N a m e	reserved	Physical Ack now ledg e	Physical Acknowledge Slot Number					
V a l u e s	0							

Bit	Name	Description	Encoding
5 t o 0	Physical Acknow ledge Slot Number	Physical Acknowledge Slot Number	- For an output Datapoint: 00h to 3Fh: If a Frame is transmitted on this Datapoint then a physical acknowledge shall be expected to be received on the specified ack slot number in runtime for this link. - For an input Datapoint: 00h to 3Fh: If a Frame is received on this Datapoint then a physical acknowledge shall be sent in the specified ack slot number in runtime for this link.
6	Physical Acknow ledge	Physical Acknowledge	- For an output Datapoint: 0: The Frame shall be transmitted with no physical acknowledge request in runtime. 1: The Frame shall be transmitted with a physical acknowledge request in runtime. - For an input Datapoint: 0: If a Frame is received on this link, then the device shall never answer with a fast physical acknowledge. The value in the physical slot number field shall not be used and shall be set to 0. 1: If a Frame is received on this link with the physical acknowledge bit set in the Frame then the device shall answer with a fast physical acknowledge. The value of the acknowledge slot time shall be contained in the physical slot number field
7	reserved	This bit is reserved and shall always be 0.	not applicable

4.15.6.1.2.3 Usage by the Management Client

The MaC shall use the function Read “RF Multi Object Link” with Code = 00h to read all Extended Group Addresses assigned to the GOs in a device. Therefore this Property shall serve as an iterator over the Group Object Association Table. The input parameter shall be an iterator that shall count the links in the device.

If the Return Code equals to 00h, then the response shall contain the Extended Group Address, the associated Group Object handle, the value of the Flags octet and the physical features in case an additional Extended Group Address is available.

If no more links is present, the return code shall be FFh.

4.15.6.1.2.4 Response in case of Code = 01h (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14
Return code	NFL high	NFL low	TNL high	TNL low

Octet 11 to 12: NFL (Number of Free Links)

The octets 11 and 12 contain the high and low octets of the number of free links, in the link table.

Octet 13 to 14: TNL (Total Number of Links)

The octets 13 and 14 shall contain the high and low octets of the total number of links, in the link table.

4.15.6.1.2.5 Usage by the Management Client

The MaC shall use the function Read “RF Multi Object Link” with Code = 01h to read the extension capabilities of the device. By this, the MaC shall be able to know if it can add new links to the device, before adding the links.

If the Return Code equals to 00h, then the response shall contain the number of free (or unused) links and the total number of links.

If no more links is present, the return code shall be FFh.

4.15.7 PID_RF_MULTI_EXT_GA_REPEATED (PID = 55)

4.15.7.1 Format

- Property name: RF Multi Extended Group Address Repeated
- Property Datatype: PDT_FUNCTION
- Datapoint Type: none

4.15.7.1.1 Write (A_FunctionPropertyCommand-PDU)

4.15.7.1.2 Format

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16	octet 17	octet 18	octet 19	octet 20	octet 21
code	00	SN high	SN	SN	SN	SN	SN low	GA high	GA low	reserved	reserved

Octet 10: Code

- Code: 00h: set Repeated Extended Group Address
- 01h: delete Repeated Extended Group Address
- 02h: delete all entries of the array (octet from 12 to 21 are in this case not used)

Octet 12 to 19

This field shall contain the Extended Group Address to add/delete in the array.

This shall be an array of octets containing the Extended Group Addresses (KNX Serial Number on 6 octets and GA on 2 octets) and 2 reserved octets.

This array shall contain the Extended Group Addresses of the Frames that shall be repeated. All Frames that do not comply with any Extended Group Address in this array shall be discarded.

If the GA equals 0000h in the Extended Group Address then all Frames containing the KNX Serial Number of this Extended Group Address shall be repeated (whatever the GA is in the Frame).

In case of deletion of an entry from the table, the exact value of SN and GA (so the Extended Group Address) shall be removed from the table. If only the SN is given in the delete command, no further deletion is done on other extended Group Address entries that would contain the SN.

Octet 20 to 21

Reserved octets: these octets shall be set to 0.

4.15.7.1.3 Response (A_FunctionPropertyState_Response-PDU)

octet 10
Return Code

The response shall contain the return code of the operation.

Return Code

- 00h: SUCCESS (assignment /deletion done or already existed / not present)
 FFh: ERR_TABLEFULL (no more space in repeated table)

4.15.7.2 Read (A_FunctionPropertyState_Read-PDU)

4.15.7.2.1 Format

octet 10	octet 11
Code	Iterator

Octet 10: Code

The octet 10 shall indicate whether the read command is used to go through the list of repeated Group Addresses or to know which part of the repeated Group Address Table is available.

- Code: 00h: Read the RF Extended Group Zddress repeated through an iterator.
 01h: Read unused and total size of the link table. In this, the response shall contain a return code together with two octets for the number of unused Extended Group Address repeated and two other octets of the total size of the Extended Group Address repetition table

4.15.7.2.2 Response in case of Code = 00h (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14	octet 15	octet 16	octet 17	octet 18	octet 19	octet 20	octet 21
Return code	Iterator	SN high	SN	SN	SN	SN	SN low	GA high	GA low	00	00

4.15.7.2.3 Usage by the Management Client

The MaC shall use the function Read “RF Multi Extended Group Address Repeated” with Code = 00h to read all the Extended Group Addresses that are repeated in a device. Therefore this Property shall serve as an iterator over the repeater table. The input parameter shall be an iterator that shall count the repeated Extended Group Addresses in the device.

If the ReturnCode equals 00h then the response shall repeat the iterator value from the Read and shall contain the return code 00h and the repeated Extended Group Address shall be available. If no more repeated Extended Group Address is present, the return code shall be FFh.

4.15.7.2.4 Response in case of Code = 01h (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11	octet 12	octet 13	octet 14
Return code	NFT high	NFT low	TNT high	TNT low

Octet 11 to 12: NFT (Number of Free entries in the repetition Table)

The octets 11 and 12 contain the high and low octets of the number of free entries, in the repetition table.

Octet 13 to 14: TNT (Total Number of entries in the repetition Table)

The octets 13 and 14 shall contain the high - and low octets of the total number of entries, in the repetition table.

4.15.7.2.5 Usage by the Management Client

The MaC shall use the function Read “RF Multi extended group address repeated” with Code = 01h to read the extension capabilities of the device. By this, the MaC shall be able to know if it can add new entries to the repetition table in the device, before adding them.

If the Return Code equals to 00h, then the response shall contain the number of free (or unused) entries and the total number of entries in the repetition table.

If no free entry is present in the repetition table, the return code shall be FFh.

4.15.8 PID_RF_DOMAIN_ADDRESS (PID = 56)

In mask 2311h, PID_RF_DOMAIN_ADDRESS (PID = 56) is mandatory in the RF Medium Object.

It may additionally be implemented as PID_RF_DOMAIN_ADDRESS (PID = 82) in the Device Object. If that is the case then the MaS shall take care that that value is at all-time synchronised with the value of PID_RF_DOMAIN_ADDRESS in the RF Medium Object.

4.15.9 PID_RF_RETRANSMITTER (PID: 57)

- **Property name:** RF Retransmitter Flag
- **Property Datatype:** PDT_BINARY_INFORMATION
- **Datapoint Type:** DPT_Bool (DPT_ID: 1.003)

4.15.9.1 Abstract Resource definition

This Property shall control whether the RF Retransmitter function of the device is inactive or active.

4.15.9.2 Format

This shall be a 1 bit Property that shall be encoded as follows.

- 0: Disable: The KNX RF Retransmitter function in the MaS shall be disabled.
- 1: Enable: The KNX RF Retransmitter function in the MaS shall be enabled.

The default value shall be 0: Disable.

4.15.9.3 Usage by the Management Server (device)

The MaS shall enable and disable its KNX RF Retransmitter functionality according to the setting of this *RF Retransmitter Flag*.

If the MaC sets *RF Retransmitter Flag* to “Enable” respectively “Disable”, then the MaS shall immediately activate respectively inactivate the retransmission RF Telegrams.

4.15.9.4 Usage by the Management Client

The MaC shall set this value according to the selection by the installer.

4.15.10 PID_RF_BIDIR_TIMEOUT (PID: 60)

- **Property name:** Bidirectional Mode Time-out
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None

This Property *Bidirectional Mode Time-out* is designed for semi-directional devices. These devices shall enable bidirectional mode according to clause “Semi-directional devices and bidirectional mode” in [03]. This Property shall allow the MaC to retrigger the time-out for the bidirectional mode and thus keep the bidirectional mode active in the MaS as long as necessary.

This Property shall not change the default time-out for bidirectional mode as specified in [03]; it shall only retrigger or stop the bidirectional mode.

4) Write (A_FunctionPropertyCommand-PDU)

Octet 10
RF_Bidir_Time-Out (second)

RF_Bidir_TimeOut shall be the duration in seconds, from 1 s to 254 s, during which the RF bidirectional mode shall be active after reception of this Telegram.

NOTE 75 The value 255 s is not foreseen, as it is used to signal a permanent bidirectional device is the below responses.

- If the MaC writes *RF_Bidir_TimeOut* to a value different from 0 then the MaS shall activate RF bidirectional mode and start the RF Bidirectional Mode Time-out timer with the value as given in the command.

NOTE 76 This command can actually only be received if bidirectional mode is already active in the MaS, by any of the triggers as specified in [03].

- If the MaC writes the value 0 for the field *RF_Bidir_TimeOut* then the MaS shall immediately stop the RF Bidirectional Mode Time-out timer and deactivate RF bidirectional communication immediately after sending out the below response.
- If the MaC writes a value for the field *RF_Bidir_TimeOut* that is larger than the maximal RF Bidir timeout that is supported by the MaS, including the value 255, then the MaS shall do the following.
 - It shall retrigger the Bidirectional Mode time-out timer with the maximal value that it supports.
 - It shall respond with both field “Max RF_Bidir_TimeOut” and “Time Left” containing this value.

Response (A_FunctionPropertyState_Response-PDU)

Octet 10	Octet 11	Octet 12
Return Code	Max RF_Bidir_TimeOut	Time Left

Return Code 00h: SUCCESS

 FFh: ERROR

Max RF_Bidir_TimeOut The MaS shall fill this field with the maximum time that it can be in in RF bidirectional mode, from 0 s to 254 s.

For permanent bidirectional devices, this value shall be 255 s (FFh) and shall denote an infinite bidirectional mode time.

Time Left For semi-directional devices, this shall be the current value of the Bidirectional Mode Time-out timer, this is, it shall be the time, in seconds, until the device will deactivate the RF bidirectional mode. This shall be a value in the range from 0 s to 254 s.

For permanent bidirectional devices, this value shall be 255 s (FFh) and shall denote an infinite bidirectional mode time.

5) Read (A_FunctionPropertyState-Read-PDU)

No Data

The MaC shall use the function read PID_RF_BIDIR_TIMEOUT to read the value and state of the RF bidirectional mode and the time-out timer. The MaS shall indicate in the response the maximum time-out value and the current time left value.

The format, contents and interpretation of the response (A_FunctionPropertyState_Response-PDU) shall be identical as to the above response to the A_FunctionProperty_Command.

6) A_SystemNetworkParameter_Write

The encoding and the requirements for the MaS shall be identical as for accessing this Property through using the A_FunctionPropertyCommand-PDU as specified above. (The MaS does however not send any response.)

This shall allow the MaC to maintain the bidirectional mode enabled in multiple devices in case longer lasting (Network) Configuration Procedures are performed and it wants to avoid that any MaS disables its bidirectional mode.

4.15.11 PID_RF_DIAG_SA_FILTER_TABLE[] (PID: 61)

- **Property name:** RF Diagnose Source Address Filter Table
- **Property Datatype:** PDT_GENERIC_03[]
- **Datapoint Type:** none

4.15.11.1 Abstract Resource definition

The Property *RF Source Address Filter Table* shall contain the Individual Addresses of the RF devices for which the MaS (device) shall measure the link quality.

This table shall be readable and writeable.

Please refer to the Management Procedure NMP_LinkQuality_Measure in [09] for how the Properties PID_RF_DIAG_SA_FILTER_TABLE, PID_RF_DIAG_QUALITY_TABLE and PID_RF_DIAG_PROBE work together.

4.15.11.2 Format

This shall be an array Property; the minimal size shall be 1.

Array Index	Filter Info	Value
1	1 st Filter Info	1 st Individual Address (2 octets)
2	2 nd Filter Info	2 nd Individual Address (2 octets)
n		

Figure 54 - Structure of RF Source Address Filter Table

NOTE 77 This table is not sorted.

- Filter Info

Bit	7	6	5	4	3	2	1	0
Description	Reserved =0	Reserved =0	Reserved =0	Reserved =0	0: don't check repeat count 1: check repeat count	Repeat count value [0-7]		

4.15.11.3 Usage by the MaS (device)

If the MaS (device) receives any RF Telegram with a SA and repeat count value equal to one of the elements this table, then the MaS shall use the received signal strength of this Telegram to update the corresponding array element in the *RF Diagnose Quality Table* (see 4.15.12).

A Source Address can be present in more than one element.

The value 0000h shall be an invalid value for the field *Individual Address*. The MaC may use this value to denote an unused entry in the array Property. The MaS shall not evaluate entries with this value for the Individual Address, regardless of the repeat count value.

NOTE 78 This allows that the diagnosis can differentiate between communication directly from the device sending with that Source Address and communication from that same sender that is repeated by one or more RF Retransmitters.

If the MaC writes any element of this table, the MaS shall reset the associated element in the *RF Diagnose Quality Table* (see 4.15.12), even if the element is overwritten with the same data.

Implementation restrictions

Please refer to the Profile specifications for the mandatory and optional support of PID_RF_DIAG_SA_FILTER_TABLE[], PID_RF_DIAG_QUALITY_TABLE[] and PID_RF_DIAG_PROBE.

If the implementation is mandatory, then it is allowed to limit the number of entries in PID_RF_DIAG_SA_FILTER_TABLE[], PID_RF_DIAG_QUALITY_TABLE[] to only one single entry.

4.15.12 PID_RF_DIAG_QUALITY_TABLE[] (PID: 62)

- **Property name:** RF Diagnose Quality Table
- **Property Datatype:** PDT_GENERIC_04[]
- **Datapoint Type:** none

4.15.12.1 Abstract Resource definition

The *RF Diagnose Quality Table* shall for each Source Address contained in the RF Source Address Filter Table contain the calculated link quality information based on the received and filtered Telegrams from that Source Address.

Please refer to the Management Procedure NMP_LinkQuality_Measure in [09] for how the Properties PID_RF_DIAG_SA_FILTER_TABLE, PID_RF_DIAG_QUALITY_TABLE and PID_RF_DIAG_PROBE work together.

4.15.12.1.1 Format

The *RF Diagnose Quality Table* shall be an array Property as shown in Figure 55.

Array Index	Nr of Telegrams (1 octet)	Link Quality		
		Min	Max	Average
		(1 octet) dB	(1 octet) dB	(1 octet) dB
1		0 to 255	0 to 255	0 to 255
2				
...				
n				

Figure 55 - Structure of RF Diagnose Quality Table

The Link Quality is defined as follows.

Link Quality [dB] = Measured RSSI of received frame [dBm] – RSSI at sensitivity level of the receiver [dBm]

This shall be a read-only Property. The default value shall be 0 for all fields. The MaC can only clear the contents of any entry in this Property by writing the corresponding element in the Property RF Diagnose Source Address Filter Table (see 4.15.11).

4.15.12.2 Usage by the MaS (device)

Every element of the *RF Diagnose Quality Table* with array index *m* shall relate 1-to-1 to the element of the *RF Diagnose Source Address Filter Table* with the same array index *m*.

- **Nr of Telegrams**
The MaS (device) shall count in this field all received Telegrams with Source Address and repeat count value as in the corresponding entry in the *RF Diagnose Source Address Filter Table*. If more than 255 matching Telegrams are received then the *Nr of Telegrams* shall stay at 255, but the Link Quality (min, max and average) shall still be updated.
- **Link Quality (in dB)**
The Link Quality shall be the difference between the RSSI of the received Telegram and the sensitivity of the MaS. This value will normally be greater than 0. A big value will indicate a good communication between two products.

- Min
This shall be the minimal value of the Link Quality calculated for all received Telegrams for this entry.
- Max
This shall be the maximal value of the Link Quality calculated for all received Telegrams for this entry.
- Average
This shall be the floating average value of the Link Quality calculated for all received Telegrams for this entry. This shall be updated with each received matching Telegram nr n as follows.

$$average_n = \frac{7 \times average_{n-1} + new\ value}{8}$$

For the first received Telegram, there is no $average_{n-1}$; the average shall therefore be initiated with the Link Quality of this first received Telegram.

Reset of an element

The MaS shall manage the *RF Diagnose Quality Table* and shall make it contain exactly the same number of elements as the *RF Diagnose Source Address Filter Table*. The MaS shall clear (value 0) all fields in an entry in the *RF Diagnose Quality Table* if the MaC writes the corresponding element in the *RF Diagnose Source Address Filter Table*.

- If the MaC writes a new Property array element in the *RF Diagnose Source Address Filter Table* beyond the current nr of elements, then the MaS (device) shall also grow the *RF Diagnose Quality Table* to have the same length and fill the newly created entry or entries with 0.
- The clearing shall happen on any write event to *RF Diagnose Source Address Filter Table*, even with identical data.

Implementation restrictions

Please refer to the Profile specifications for the mandatory and optional support of PID_RF_DIAG_SA_FILTER_TABLE[], PID_RF_DIAG_QUALITY_TABLE[] and PID_RF_DIAG_PROBE.

If the implementation is mandatory, then it is allowed to limit the number of entries in PID_RF_DIAG_SA_FILTER_TABLE[], PID_RF_DIAG_QUALITY_TABLE[] to only one single entry.

4.15.12.3 Usage by the MaC

The MaC shall read out any entry in this array Property to learn the Link Quality of the RF communication between this MaS and the KNX RF device with the given IA.

In order to clear any data in this Property, the MaC shall access the corresponding entries in the Property *RF Diagnose Source Address Filter Table*.

4.15.13 PID_RF_DIAG_PROBE (PID: 63)

- **Property name:** RF Diagnose Probe
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None

4.15.13.1 Abstract Resource definition

The Property *RF Diagnose Probe* (PID_RF_DIAG_PROBE) shall support the measuring of the Link Quality between the MaS and another KNX RF device. It shall allow the MaC to trigger the MaS to transmit this Property State to the network, of which the other KNX RF device can measure the signal strength and calculate the Link Quality.

Please refer to the Management Procedure NMP_LinkQuality_Measure in [09] for how the Properties PID_RF_DIAG_SA_FILTER_TABLE, PID_RF_DIAG_QUALITY_TABLE and PID_RF_DIAG_PROBE work together.

4.15.13.2 Format

PID RF DIAG PROBE shall be a Function Property that shall be formatted as follows.

4.15.13.3 Usage by the MaS (device)

1) Write (A_FunctionPropertyCommand-PDU)

Octet 10
Trigger: 00h

The field *Trigger* shall have the fixed value 00h. Other values are reserved for later extensions and shall not be used. If the MaS (device) receives a value for this field *Trigger* that it does not support, then it shall give a negative Function response.

Response (A FunctionPropertyState Response-PDU)

Octet 10	Octet 11
test_info	test_result
00h	00h

The MaS shall respond to the MaC with this A_FunctionPropertyState_Response-PDU in point-to-point connectionless communication mode.

NOTE 79 The trigger is specified as a constant 00h. Future alternative values may control the information to be sent, like the RF Channel to use, to what receivers to send it and the communication mode (e.g. point-to-point connectionless).

Additionally, the MaS (device) shall send the state of this function using the service

A NetworkParameter InfoReport in broadcast communication mode. The field test result shall be 00h.

Octet 6										Octet 7								Octet 8								Octet 9								Octet 10								Octet 11								Octet 12										
										APCI										parameter_type																test_info								test_result																
																				object_type																																property_id								
																					RF Medium Object																63								00								00h							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0					
							APCI																																																					
							1	1	1	1	0	1	1	0	1	1																																												

Figure 56 – A NetworkParameter InfoReport - RF Diagnose Probe message (example)

2) Read (A FunctionPropertyState-Read-PDU)

If the MaC reads that status of the Function Property `PID_RF_DIAG_PROBE`, then the MaS shall respond with the same value as on the write confirmation.

Octet 10	Octet 11
test_info	test_result
00h	00h

NOTE 80 The octet 10, test_info, shall in this be seen as an identifier of the type of information that is transmitted. Future values may indicate additional octets in the field test_info or different information in the field test_result.

The MaS shall not transmit any second message like A_NetworkParameter_InfoReport on the network in this case.

Implementation restrictions

Please refer to the Profile specifications for the mandatory and optional support of PID_RF_DIAG_SA_FILTER_TABLE[], PID_RF_DIAG_QUALITY_TABLE[] and PID_RF_DIAG_PROBE.

If the implementation is mandatory, then it is allowed to limit the number of entries in PID_RF_DIAG_SA_FILTER_TABLE[], PID_RF_DIAG_QUALITY_TABLE[] to only one single entry.

4.15.14 PID_TRANSMISSION_MODE ((PID = 70))

4.15.14.1 General requirements

- Property name: Transmission Mode
- Property Datatype: PDT_ENUM8
- Datapoint Type: None

The data Property *Transmission Mode* shall control the transmission mode of the Physical Layer implemented in the cEMI Server.

The cEMI Client shall write the Property *Transmission Mode* to change the transmission mode of the Physical Layer implemented in the cEMI Server.

The cEMI Client shall read the Property *Transmission Mode* to get the current transmission mode of the cEMI Server's Physical Layer.

The value of the Property PID_TRANSMISSION_MODE shall be formatted as an 8 bit enumeration data type and shall be interpreted as specified in Table 66.

The interpretation of PID_TRANSMISSION_MODE depends on the device in which the cEMI Server is hosted and is specified in the following clauses.

Table 66 - Interpretation of the data in PID_TRANSMISSION_MODE

Value	Description
00h	Reserved for future use of the KNX system
01h	Transmission on the single F1 frequency (RF1.R)
02h	Transmission on the single F1 frequency (RF1.M)
03h	Transmission on the single F2 frequency (RF1.M)
04h	Transmission on the single F3 frequency (RF1.M)
05h	Transmission on the single S1 frequency (RF1.M)
06h	Transmission on the single S2 frequency (RF1.M)
07h	Transmission on the fast Fx frequencies (RF1.M)
08h	Transmission on the slow Sx frequencies (RF1.M)
09h	Reserved for future use of the KNX system
0Ah	Transmission on the single F1 frequency (RF2.R)
0Bh	Transmission on the single F1 frequency (RF2.M)
0Ch	Transmission on the single F2 frequency (RF2.M)
0Dh	Transmission on the single F3 frequency (RF2.M)
0Eh	Transmission on the single S1 frequency (RF2.M)

Value	Description
0Fh	Transmission on the single S2 frequency (RF2.M)
10h	Transmission on the fast Fx frequencies (RF2.M)
11h	Transmission on the slow Sx frequencies (RF2.M)
12h	Reserved for future use of the KNX system
13h	Transmission on the single F1 frequency (RF5.M)
14h	Transmission on the single F2 frequency (RF5.M)
15h	Transmission on the single F3 frequency (RF5.M)
16h	Transmission on the single S1 frequency (RF5.M)
17h	Transmission on the single S2 frequency (RF5.M)
18h	Transmission on the fast Fx frequencies (RF5.M)
19h	Transmission on the slow Sx frequencies (RF5.M)
1Ah	Reserved for future use of the KNX system
1Bh to FFh	Reserved for future use of the KNX system

The cEMI Client may read the Property *Transmission Mode* to get the current transmission mode.

If a message is transmitted between the last setting of the transmission frequency to Sx or Fx (using writing the Property or AddInfo) and reading the Property, then the returned value shall not return value neither Fx nor Sx.

4.15.14.2 Intended use and mandatory behaviour

This Property is mainly intended for test - and debugging purposes.

The Property value should be stored in volatile memory. The Property value shall be cleared after a reset of the MaS or after disconnection from the Management Client (cEMI Client). After a reconnection of the MaC (cEMI Client), or after a reset, the default values should be used by the Management Server.

4.15.15 PID_RECEPTION_MODE (PID = 71)

4.15.15.1 General requirements

- Property name: Reception Mode
- Property Datatype: PDT_ENUM8
- Datapoint Type: None

The data Property *Reception Mode* shall control the reception mode of the Physical Layer implemented in the cEMI Server.

The cEMI Client shall write the Property *Reception Mode* to change the reception mode of the Physical Layer implemented in the cEMI Server.

The cEMI Client shall also read the Property *Reception Mode* to get the current reception mode of the cEMI Server's Physical Layer.

The value of the Property *Reception Mode* shall be formatted as an 8 bit enumeration data type and shall be interpreted as specified Table 67.

The interpretation of Property *Reception Mode* shall depend on the device in which the cEMI Server is hosted and is specified in the following clauses.

Table 67 - Interpretation of the data in PID_RECEPTION_MODE

Value	Description
00h	Reserved for future use of the KNX system
01h	Reception on the single F1 frequency (RF1.R)
02h	Reception on the single F1 frequency (RF1.M) ¹
03h	Reception on the single F2 frequency (RF1.M)
04h	Reception on the single F3 frequency (RF1.M)
05h	Reception on the single S1 frequency (RF1.M)
06h	Reception on the single S2 frequency (RF1.M)
07h	Reception on the fast Fx frequencies (RF1.M)
08h	Reception on the slow Sx frequencies (RF1.M)
09h	Reception on the fast Fx and the slow Sx frequencies (RF1.M)
0Ah	Reception on the single F1 frequency (RF2.R)
0Bh	Transmission on the single F1 frequency (RF2.M)
0Ch	Transmission on the single F2 frequency (RF2.M)
0Dh	Transmission on the single F3 frequency (RF2.M)
0Eh	Transmission on the single S1 frequency (RF2.M)
0Fh	Transmission on the single S2 frequency (RF2.M)
10h	Transmission on the fast Fx frequencies (RF2.M)
11h	Transmission on the slow Sx frequencies (RF2.M)
12h	Reception on the fast Fx and the slow Sx frequencies (RF2.M)
13h	Transmission on the single F1 frequency (RF5.M)
14h	Transmission on the single F2 frequency (RF5.M)
15h	Transmission on the single F3 frequency (RF5.M)
16h	Transmission on the single S1 frequency (RF5.M)
17h	Transmission on the single S2 frequency (RF5.M)
18h	Transmission on the fast Fx frequencies (RF5.M)
19h	Transmission on the slow Sx frequencies (RF5.M)
1Ah	Reception on the fast Fx and the slow Sx frequencies (RF5.M)
0Bh to FFh	Reserved for future use of the KNX system

NOTE 81 As the RF1.M and RF1.R physically use the same frequencies, both types of Frame can be received in this case.

4.15.15.2 Intended use and mandatory behaviour

This Property is mainly intended for test - and debugging purposes.

The Property value should be stored in volatile memory. The Property value shall be cleared after a reset of the MaS or after disconnection from the Management Client (cEMI Client). After a reconnection of the MaC (cEMI Client), or after a reset, the default values should be used by the Management Server.

4.15.16 PID_TEST_SIGNAL (PID = 72)

4.15.16.1 General requirements

- Property name: Test Signal
- Property Datatype: PDT_GENERIC_02
- Datapoint Type: None

The Property *Test Signal* shall control the frequency and the form of the signal to be transmitted by the cEMI Server's Physical Layer.

The cEMI Client shall write the Property *Test Signal* to request the transmission of a signal for debugging purposes.

The value of the Property *Test Signal* shall be formatted as two consecutive 8 bit enumeration and shall be interpreted as specified in Table 68.

The interpretation of the Property *Test Signal* shall depend on the device in which the cEMI Server is hosted and is specified in the following clauses.

Table 68 - Interpretation of the data in PID_TEST_SIGNAL

Field	Octet 1	Octet 2	Octet 3
Name	Signal frequency	Signal form	Signal duration

Octet	Name	Description	Encoding
1	Signal frequency	Indicate the frequency of the signal or stop the signal.	00h: stop the current carrier transmission and resumes reception 01h: F1 Ready frequency (RF1.R) 02h: F1 frequency (RF1.M) 03h: F2 frequency (RF1.M) 04h: F3 frequency (RF1.M) 05h: S1 frequency (RF1.M) 06h: S2 frequency (RF1.M) 07h: Reserved for future use of the KNX system 08h: Reserved for future use of the KNX system 09h: Reserved for future use of the KNX system 0Ah: F1 Ready frequency (RF2.M) 0Bh: F1 frequency (RF2.M) 0Ch: F2 frequency (RF2.M) 0Dh: F3 frequency (RF2.M) 0Eh: S1 frequency (RF2.M) 0Fh: S2 frequency (RF2.M) 10h: Reserved for future use of the KNX system 11h: Reserved for future use of the KNX system 12h: Reserved for future use of the KNX system 13h: F1 frequency (RF5.M) 14h: F2 frequency (RF5.M) 15h: F3 frequency (RF5.M) 16h: S1 frequency (RF5.M) 17h: S2 frequency (RF5.M) 18h: Reserved for future use of the KNX

Octet	Name	Description	Encoding
			system 19h: Reserved for future use of the KNX system 1Ah: Reserved for future use of the KNX system 1Bh to FFh: Reserved for future use of the KNX system
2	Signal form	Indicate the form of the signal	00h: carrier on $f_c - f_{dev}$ 01h: carrier on $f_c + f_{dev}$ 02h: 0/1 chips modulated signal 03h to FFh: Reserved for future use of the KNX system
3	Signal duration	Indicate the duration of the signal	00h: no time duration (permanent transmission) 01h to FFh: duration in seconds

f_c : centre frequency.

f_{dev} : FSK deviation.

The cEMI Client shall read the Property *Test Signal* to get the current status of the carrier command. If the cEMI Server is transmitting a carrier, it shall return the used signal frequency and the signal form. If there is no current transmission of a signal, the octet 1 shall contain the value 00h and the Signal form octet shall have no meaning.

The new start of a test signal shall stop the possible current test signal and restart the transmission of the new test signal.

4.15.16.2 Intended use and mandatory behaviour

This Property is mainly intended for test - and debugging purposes.

The Property value should be stored in volatile memory. The Property value shall be cleared after a reset of the MaS or after disconnection from the Management Client (cEMI Client). After a reconnection of the MaC (cEMI Client), or after a reset, the default values should be used by the Management Server.

4.15.17 PID_FAST_ACK (PID = 73)

4.15.17.1 General requirements

- Property name: Fast Ack
- Property Datatype: PDT_GENERIC_02[]
- Datapoint Type: none

The data Property *Fast Ack* shall control the Fast Ack Slots that the cEMI's Physical Layer has to manage when receiving a RF message.

The cEMI Client shall write the Property *Fast Ack* for each Fast Ack slot that the cEMI Server device's Physical Layer shall fill along with the Info Octet to write in the Fast Ack.

The value of the data Property *Fast Ack* shall be formatted as 2 octets as specified in Table 69.

The interpretation of the Property *Fast Ack* shall depend on the device in which the cEMI Server is hosted and is specified in the following clauses.

The Fast Ack shall be organised as an array of 64 elements. Each element shall be a structured data made of two octets as described in Table 69.

The array shall be composed of 64 elements. The first element (first Fast Ack at Ack Slot 0) shall be located at index 1 of the array. The last element (last Fast Ack at Ack Slot 63) shall be located at index 64.

Table 69 - Interpretation of the data in PID_FAST_ACK

Field	Octet 1	Octet 2
Name	Enable/Disable	Fast Ack Info Octet
Values		

Table 70 – PID_FAST_ACK property description

Octet	Name	Description	Encoding
1	Enable/Disable	Indicate if the management of the Ack Slot should be enabled or disabled	00h: Disable 01h: Enable 02h to FFh: Reserved for future use of the KNX system
2	Fast Ack Info byte	Indicate the Fast Ack Info byte value to write in the related Fast Ack.	0 to 255: Fast Ack Info byte value

The cEMI Client shall read the Property *Fast Ack* to get one or several Fast Ack management status. This shall be done by using the NumberOfElement and StartIndex parameters. One element (Fast Ack) shall be composed of 2 octets and the 2 octets shall be the same as the 2 octets that are written for this Fast Ack in the write request.

Multiple Fast Ack elements can be read in the read request by setting the right NumberOfElement and StartIndex parameters.

4.15.17.2 Intended use and mandatory behaviour

This Property is mainly intended for test - and debugging purposes.

The Property value should be stored in volatile memory. The Property value shall be cleared after a reset of the MaS or after disconnection from the Management Client (cEMI Client). After a reconnection of the MaC (cEMI Client), or after a reset, the default values should be used by the Management Server.

4.15.18 PID_FAST_ACK_ACTIVATE (PID = 74)

4.15.18.1 General requirements

- Property name: Fast Ack Activate
- Property Datatype: PDT_BINARY_INFORMATION
- Datapoint Type: None

The Property *Fast Ack Activate* shall control the activation and deactivation of the Fast Ack management by the cEMI Server's Physical Layer when receiving RF messages with Fast Ack management required.

The cEMI Client shall write the Property *Fast Ack Activate* with value 1 to activate the management of the Fast Ack by the cEMI Server.

The value of the Property *Fast Ack Activate* shall be formatted as an 8 bit field data type and shall be interpreted as specified in Table 71.

The interpretation of PID_FAST_ACK_ACTIVATE shall depend on the device in which the cEMI Server is hosted and is specified in the following clauses.

Table 71 - Interpretation of the data in PID_FAST_ACK_ACTIVATE

Field	b7	b6	b5	b4	b3	b2	b1	b0
Name	reserved	reserved	reserved	reserved	reserved	reserved	reserved	Activation deactivation
Values	0	0	0	0	0	0	0	

Table 72 – PID_FAST_ACK_ACTIVATE property description

Octet	Name	Description	Encoding
0	Activation/ deactivation	This field shall indicate if the Fast Ack management has to be activated or deactivated in the cEMI Server	0b: deactivate the Fast Ack management 1b: activate the Fast Ack management
1 to 7	Reserved for future use of the KNX system	These bits are reserved and shall always be 0	Not applicable

The cEMI Client shall read the Property *Fast Ack Activate* to get the current state, activated or deactivated, of the Fast Ack management in the cEMI Server device's Physical Layer.

Intended use and mandatory behaviour

This Property is mainly intended for test - and debugging purposes.

The Property value should be stored in volatile memory. The Property value shall be cleared after a reset of the MaS or after disconnection from the Management Client (cEMI Client). After a reconnection of the MaC (cEMI Client), or after a reset, the default values should be used by the Management Server.

4.15.19 PID_RF_TYPES_SUPPORTED (PID = 75)

- **Property name:** RF Types Supported
- **Property Datatype:** PDT_BITSET8 (alt.: PDT_GENERIC_01)
- **Datapoint Type:** None.
- **Access:**
 - cEMI Services for local management (M_Prop...) ¹⁷⁾

Service:	M/O
Read:	M
Write:	X

- Data Property services (A_PropertyValue...) ¹⁸⁾

Service:	M/O	Role:	Security:	Access level:
Read:	O	any	plain	3
Write:	X	none	n/a	n/a

List: This Property shall neither be on the White List, the Black List or the Intermediate List.

4.15.19.1 Abstract Resource definition

The Property RF Types Supported shall indicate the RF medium types that are supported by the MaS. This shall mainly serve for discovery by the MaC.

4.15.19.2 Format

PID_RF_TYPES_SUPPORTED shall be composed of 8 bits (B₈) as defined in Figure 57.

Bit	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
Meaning	reserved	reserved	reserved	reserved	reserved	reserved	KNX RF Multi	KNX RF Ready
Value	0	0	0	0	0	0		1

Figure 57 – PID_RF_TYPES_SUPPORTED

A bit shall be set if the corresponding flavour of the RF Communication Medium is supported: otherwise, it shall be cleared.

- For all KNX RF devices, the minimal requirement is to support KN RF Ready. Therefore, b₀ shall always be set.
- If the MaS is a KNX RF Multi device, b₁ shall be set. Otherwise, it shall be cleared.
- The reserved bits b₇ to b₂ shall not be used and shall be 0.

4.15.19.3 Usage by the Management Server (device)

This Property shall be read-only. It shall be accessible as indicated above.

¹⁷⁾ The cEMI services for local device management do not exhibit any security feature. This thus has to be supported by the cEMI host protocol, like KNXnet/IP Tunnelling or USB.

¹⁸⁾ This concerns access via the bus, over the L_Data-service, as well as access through the cEMI Transport Layer using the cEMI T_Data_Individual or T_Data_Connected to transfer an A_PropertyValue_nnn,-PDU.

4.15.19.4 Usage by the Management Client

The MaC shall read this Property to discover the RF Medium Types that are implemented in the MaS.

EXAMPLE 37 This may be necessary before concluding on the possible values that can be set in `PID_RF_MULTI_TYPE`.

EXAMPLE 38 This may be necessary for the MaC to discover the cEMI AddInfo –types that can be expected and that can be used in case of a cEMI Server.

4.16 Group Address Table (GrAT)

4.16.1 Abstract Resource definition

The GrAT shall be a shared Resource of both the Data Link Layer and the group oriented Transport Layer.

The Data Link Layer shall use the Group Address Table as a look up reference to check whether it should pass a received frame to the upper layers or not. The group oriented Transport Layer shall consult the Group Address Table to map an incoming LSAP (Group Address) to a TSAP ¹⁹⁾ in receiving direction and vice-versa in sending direction.

Default state (optional)

The default state is not standardised but is implementation specific. This may be any of the following. Other default states are possible as well.

- The Group Address Table contains no Group Addresses.
- The Group Address Table contains an implementation specific number of default Group Addresses; the size has an according value.

If the Group Address Realisation is controlled through a Load State Machine, then this may be Loaded or Unloaded.

4.16.2 Access Control to the Group Address Table

If the MaS supports KNX Secure Communication, then it shall support Access Control to the Group Address Table. The write access to the contents and any other parameter related to the Group Address Table shall be limited to the Role "Tool". Other Roles shall not have write access but may have read access.

EXAMPLE 39 Parameters related to the Group Address Table can be the write access to the Load State Machine (memory mapped or Property based), Group Address Table length, pointers to the memory mapped location etc.

NOTE 82 Read access is no problem, as the Group Addresses are at runtime anyhow visible unencrypted on the bus.

¹⁹⁾ TSAP may also be referenced by "cr_id" ("communication reference identifier" or "communication relationship identifier"). Previous publications may use the term "connection number".

4.16.3 Group Address Table - Realisation Type 1

Used by: Mask 0010h, 0011h, 0012h
Mask 0020h in compatibility mode , 0021h in compatibility mode

4.16.3.1 Format

Structure:	Address	Group Address Table	TSAP (Connection Nr.)
	Low memory		
1 octet		Length	
2 octets		Individual Address	0
2 octets		Group Address Nr. 1	1
2 octets		Group Address Nr. 2	2
2 octets		Group Address Nr. 3	3
...	...	...	...
	High memory		

The Group Addresses shall be sorted in ascending order with increasing memory locations. Connection Nr. 1 refers to the first Group Address.

Max. Length: The maximum length is implementation dependent. It shall be checked whether this maximum value is not exceeded.

GAs: 16 bit

4.16.3.2 Location

The location shall be fixed at address 0116h.

4.16.3.3 Usage by the Management Server (device)

4.16.3.3.1 Data Link Layer

The following specification is only valid for the L_Data Frame format. For the other frame formats (L_Busmon, L_Sys_Data), the Group Address Table shall not be evaluated.

a) Frame transmission

The Group Address Table is not evaluated.

b) Frame reception

If Data Link Layer finds the frame it has composed from the Physical Layer to be correct, it shall evaluate the contained Domain Address (if relevant), destination_address and destination_address_flag (DAF) for deciding on passing to the frame to the Data Link Layer User or not.

The Data Link Layer shall only evaluate the GrAT under the following conditions:

1. the DoA equals the DoA of the device (if relevant), and
2. the DAF indicates the Destination Address is a Group Address, and
3. the frame contains a group message ²⁰.

If these conditions are not fulfilled, the behaviour shall be as described in the Data Link Layer specifications for the various media.

The GrAT shall be evaluated as follows:

²⁰ Thus no broadcast message.

- ◆ The behaviour of the Data Link Layer shall depend on the length of the Group Address Table (address 116h)

The Length (GroupAddressTableLength) shall contain the number of Group Addresses in the Group Address Table including the Individual Address.

EXAMPLE If there are 5 Group Addresses in the Group Address Table, the GroupAddressTableLength = 6.

Length Behaviour

- 0 ◆ Frame acceptance

The Data Link Layer shall pass all group frames to the Data Link Layer User irrespective of the contained Group Address.

- ◆ Frame acknowledgement

The condition for Group Frame acknowledgment based on the evaluation of the Group Address Table shall be considered as fulfilled ²¹⁾.

- 1 ◆ Frame acceptance

No Group Frame shall be passed to the Data Link Layer User.

- ◆ Frame acknowledgement

No Group Frame shall be acknowledged.

- >1 The Data Link Layer shall evaluate the Group Address Table as follows.

The Data Link Layer shall compare the destination_address to the subsequent Group Addresses in the Group Address Table, starting from connection nr. 1. This procedure shall stop on either one of the 3 following stop conditions:

1. the destination_address is found in the Group Address Table, or
2. the value of the destination_address is lower than the last compared Group Address of the Group Address Table ²²⁾.
3. the last compared Group Address entry in the Group Address Table has the connection_number equal to the Address Table length, this is when the end of the Group Address Table is reached.

- ◆ Frame acceptance

Only if the stop condition 1 is fulfilled, this is, if the destination_address is found in the Group Address Table, the Data Link Layer shall pass the received Group Frame to the Data Link Layer User by means of an L_Data-service.

For the other stop conditions, the received Group Frame shall not be passed to the Data Link Layer User and the Group Frame shall be removed.

- ◆ Frame acknowledgement

If the device does not support the non-selective Data Link Layer acknowledge, then only if the stop condition 1 is fulfilled, the condition for Group Frame acknowledgment based on the evaluation of the Group Address Table shall be regarded as fulfilled.

If the device supports non-selective Data Link Layer acknowledge the condition for Group Frame acknowledgment based on the evaluation of the Group Address Table may be always fulfilled.

²¹⁾ This means that the acknowledge condition resulting from the Group Address Table evaluation is positive. Other conditions, typically medium specific, additionally apply for confirming the message.

²²⁾ For this reason, the Group Address entries in the Group Address Table have to be sorted. This allows for a fast evaluation algorithm.

4.16.3.3.2 Transport Layer

The Transport Layer specifications (see [06]) specify that the TSAP used in the T_Data_Group service shall be a reference to a Group Address.

If using this standardized Group Address Table format, the TSAP shall be equal to the Connection Number.

a) Frame transmission

The Transport Layer shall use the value of the TSAP parameter in the T_Data_Group.req service primitive as connection number in the Group Address Table. The value of the Group Address found for this connection number shall be used as destination_address parameter in the N_Data_Group.req service primitive that shall be passed to the Network Layer.

b) Frame reception

The value of the parameter destination_address of the N_Data_Group.ind (N_Data_Group.con) shall be searched in the Group Address Table.

If the destination_address is found, the connection number at which it is found shall be taken as value for the parameter TSAP in the T_Data_Group.ind (T_Data_Group.con).

If the destination_address is not found, the value for the parameter TSAP in the T_Data_Group.ind shall be set to FFh to indicate an unknown connection number.

4.16.3.4 Usage by the Management Client

4.16.3.4.1 Remote access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to download the Group Address Table Format 1.

It shall be checked on the forehand by the Management Server if the download of the Group Address Table does not conflict with other Resources ²³⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Set Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, 00h)
;      Set Group Address Table Length to 1 24)
DMP_MemWrite_RCo(0116h, 0116h, 01h)
;      Write the Group Address entries of the Group Address Table.
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemWrite_RCo(0119h, 0119h+2*AddressTableLength-3, GrAT)
;      Set the Group Address Table length back to the value correct for the application.
;      AddressTableLength is the length of the group address table as defined in
;      ;      clause 4.16.3.1 "Format" above.
DMP_MemWrite_RCo(0116h, 0116h, LLh)
;      Check the correct Group Address Table Length
DMP_MemRead_RCo(0116h, 0116h)
;      If the received length is not equal to the programmed length, abort the Configuration Procedure.
```

²³⁾ Examples

1. Group Address that have been inserted compared to the Group Address Table currently running in the Management Server, which would make the Group Objects Association Table invalid.
2. Increased number of Group Address which require a larger address space than currently available in the Management Client, forcing other recourses, such as the Group Objects Association Table, to be shifted.

²⁴⁾ This makes the Link Layer does no longer pass L_Data-frames containing group communication to the upper layers

```
;      Clear Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, FFh)
;      Restart the device
DMP_Restart
```

4.16.3.4.2 Local access

```
;      Connect to the device
DMP_Connect_LEmi1
;      Set Runtime Error Flags
DMP_MemWrite_LEmi1(010Dh, 010Dh, 00h)
;      Set Group Address Table Length to 1
DMP_MemWrite_LEmi1(0117h, 0118h, PPPPh)
;      Write the Group Address entries of the Group Address Table.
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemWrite_LEmi1 (0119h, 0119h+2*AddressTableLength-3, GrAT)
;      Set the Group Address Table length back to the value correct for the application.
;      AddressTableLength is the length of the group address table as defined in
;      ;      clause 4.16.3.1 "Format" above.
DMP_MemWrite_LEmi1 (0116h, 0116h, LLh)
;      Clear Runtime Error Flags
DMP_MemWrite_LEmi1 (010Dh, 010Dh, FFh)
;      Restart the device
DMP_Restart_LEmi1
```

4.16.3.4.3 Internal Access

There are no special conditions for an internal Management Client (application) to handle the Group Address Table – Realisation Type 1. The resources can be accessed directly either by the functionality provided by the host processor or by dedicated access routines (API) to handle the Resources.

In case this Resource is located in memory protected by a checksum, the internal Management Client shall take care of this, by either reducing the range of the checksum so that this Resource is not included, or by making this checksum is recalculated (for instance via a dedicated API).

4.16.4 Group Address Table - Realisation Type 2

Used by: Mask 0020h, 0021h

4.16.4.1 Format

The format is as specified in clause 4.16.3.1 “Format” above.

4.16.4.2 Location

The pointer to the GrAT shall be written to the Address Table Load State Machine in the Absolute Code/Data Allocation Record. For this location, the value of this pointer shall however not be evaluated. Instead, the Group Address Table shall be located at the fixed address 0116h.

4.16.4.3 Usage by the Management Server (device)

The usage by the Management Server for the Group Address Table Format 2 is as specified in clause 4.16.3.3 “Usage by the Management Server (device)” above.

NOTE 83 System 2 does not allow putting a non-sending association in the first *n* associations if it is used for a sending association with a higher number. This may lead to the fact that the MaS allows for fewer associations than what may be assumed. Therefore, this model should not be used to its limits (near the maximal number of associations).

4.16.5 Group Address Table - Realisation Type 4

Used by: Mask 1012h, 1013h

4.16.5.1 Format

Structure:	Address	Group Address Table		TSAP (Connection Nr.)
	Low memory			
1 octet	0116h	Length		
2 octets	0117h	Individual Address		0
2 octets	0119h	R1	Group Address Nr. 1	1
2 octets	011Bh	R2	Group Address Nr. 2	2
2 octets	011Dh	R3	Group Address Nr. 3	3
...	...	...	...	...
	High memory			

The Group Addresses shall be sorted in ascending order with increasing memory locations. Connection Nr. 1 shall refer to the first Group Address.

Max. Length: The maximum length is implementation dependent. It shall be checked whether this maximum value is not exceeded.

GAs: 15 bit

The msb of every Group Address entry is the Group Responder Flag for that device for that Group Address (Rx = 1: the device is group responder for that Group Address ²⁵).

It shall be checked whether this maximum value is not exceeded.

4.16.5.2 Location

The location shall be fixed at address 0116h.

4.16.5.3 Usage by the Management Server (device)

4.16.5.3.1 Link Layer

The following specification is only valid for the L_Data Frame format. For the other frame formats (L_Busmon, L_Sys_Data), the Group Address Table shall not be evaluated.

The evaluation of the Group Address Table Realisation Type 4 shall be identical as specified in clause 4.16.3.3 “Usage by the Management Server (device)” above, with the additional evaluation of the bit Rx for every connection nr. X.

a) Frame transmission

The Group Address Table shall not be evaluated.

b) Frame reception

If Data Link Layer finds the frame it has composed from the Physical Layer to be correct, it shall evaluate the contained Domain Address (if relevant), destination_address and destination_address_flag (DAF) for deciding on passing the frame to the Data Link Layer User or not.

²⁵) Usage: See Powerline Communication Medium specification.

The Data Link Layer shall only evaluate the GrAT under the following conditions:

1. The DoA equals the DoA of the device (if relevant), and
2. t the DAF indicates the Destination Address is a Group Address, and
3. the frame contains a group message.

If these conditions are not fulfilled, the behaviour shall be as described in the Data Link Layer specifications for the various media.

The GrAT shall be evaluated as follows:

- ◆ The behaviour of the Data Link Layer shall depend on the length of the Group Address (0116h).

Length	Behaviour
--------	-----------

- | | |
|---|--------------------|
| 0 | ◆ Frame acceptance |
|---|--------------------|

The Data Link Layer shall pass all group frames to the Link Layer User irrespective of the contained Group Address.

- ◆ Frame acknowledgement

The condition for Group Frame acknowledgment based on the evaluation of the Group Address Table shall be considered as fulfilled under the extra condition that the Main Responder Flag is set ²⁶⁾.

- | | |
|---|--------------------|
| 1 | ◆ Frame acceptance |
|---|--------------------|

No Group Frame shall be passed to the Data Link Layer User.

- ◆ Frame acknowledgement

No Group Frame shall be acknowledged.

²⁶⁾ This means that the acknowledge condition resulting from the Group Address Table evaluation is positive. Other conditions, typically medium specific, additionally apply for confirming the message.

Length Behaviour

- >1 The Data Link Layer shall evaluate the Group Address Table as follows.
 The Data Link Layer shall compare the destination_address to the subsequent Group Addresses in the Group Address Table, starting from connection nr. 1. This procedure shall stop on either one of the 3 following stop conditions:
1. the destination_address is found in the Group Address Table, or
 2. the value of the destination_address is higher than the last compared Group Address of the Group Address Table ²⁷⁾.
 3. the last compared Group Address entry in the Group Address Table has the connection_number equal to the Address Table length, this is when the end of the address table is reached.
- ◆ Frame acceptance
 Only if the stop condition 1 is fulfilled, this is, if the destination_address is found in the Group Address Table, the Data Link Layer shall pass the received Group Frame to the Data Link Layer User by means of an L_Data-service.
- For the other stop conditions, the received Group Frame shall not be passed to the Data Link Layer User and the Group Frame shall be removed.
- ◆ Frame acknowledgement
 The received frame shall be acknowledged only if both of the following conditions are fulfilled:
1. The Group Address Table evaluation procedure is stopped by the stop condition 1, this is, the Group Address is found in the Group Address Table, and
 2. In the Connection X at which the Group Address is found, the bit Rx is set.

4.16.5.3.2 Transport Layer

In the Transport Layer specifications (see [06]) specify that the TSAP used in the T_Data_Group service shall be a reference to a Group Address.

If using this standardized Group Address Table format, the TSAP shall be equal to the Connection Number.

a) Frame transmission

The Transport Layer shall use the value of the TSAP parameter in the T_Data_Group.req service primitive as connection number in the Group Address Table. The value of the Group Address found for this connection number is used as destination_address parameter in the N_Data_Group.req service primitive which shall be passed to the Network Layer.

b) Frame reception

The value of the parameter destination_address of the N_Data_Group.ind (N_Data_Group.con) shall be searched in the Group Address Table.

If the destination_address is found, the connection number at which it is found shall be taken as value for the parameter TSAP in the T_Data_Group.ind (T_Data_Group.con).

If the destination_address is not found, the value for the parameter TSAP in the T_Data_Group.ind shall be set to FFh to indicate an unknown connection number.

²⁷⁾ For this reason, the Group Address entries in the Group Address Table have to be sorted. This allows for a fast evaluation algorithm.

4.16.5.4 Usage by the Management Client

4.16.5.4.1 Remote access

The access by a remote Management Client is as specified in clause 4.16.3.4.1 "Remote access" above.

4.16.5.4.2 Local access

The access by a local Management Client is as specified in clause 4.16.3.4.2 "Local access" above.

4.16.5.4.3 Internal access

The access by an internal Management Client is as specified in clause 4.16.3.4.3 "Internal Access" above.

4.16.6 Group Address Table - Realisation Type 5

Used by: Mask 3012h, 4012h configured in S-Mode

The specification of this Resource is only valid if it is and will be managed in S-Mode configuration. If the device hosting this Resource is managed by another Configuration Mode, the locations for this Resource can still be accessed; however, these accessed values shall not be interpreted according the specifications below.

4.16.6.1 Format

Structure:	Address	Group Address Table	Connection Nr. (TSAP)
	Low memory		
1 octet	0116h	Length	
2 octets	0117h	Individual Address	0
2 octets	0119h	Group Address Nr. 1	1
2 octets	011Bh	Group Address Nr. 2	2
2 octets	011Dh	Group Address Nr. 3	3
	...	...	...
	High memory		

The Group Addresses are not sorted. Connection Nr. 1 shall refer to the first Group Address.

Max. Length: The maximum length is implementation dependent. It shall be checked whether this maximum value is not exceeded.

GAs: 16 bit

4.16.6.2 Location

The location shall be fixed at address 0116h.

4.16.6.3 Usage by the Management Server (device)

4.16.6.3.1 Data Link Layer

The following specification is only valid for the L_Data Frame format. For the other frame formats (L_Busmon, L_Sys_Data), the Group Address Table shall not be evaluated.

a) Frame transmission

The Group Address Table shall not be evaluated.

b) Frame reception

If Data Link Layer finds the frame it has composed from the Physical Layer to be correct, it shall evaluate the contained Domain Address (if relevant), destination_address and destination_address_flag (DAF) and for deciding on passing to the frame to the Data Link Layer User or not.

The Data Link Layer shall only evaluate the GrAT under the following conditions:

1. The DoA equals the DoA of the device (if relevant), and
2. the DAF indicates the destination address is a group address, and
3. the frame contains a group message.

If these conditions are not fulfilled, the behaviour shall be as described in the Data Link Layer specifications for the various media.

The GrAT shall be evaluated as follows.

- ◆ The behaviour of the Data Link Layer shall depend on the length of the Group Address Table (address 0116h)

Length	Behaviour
--------	-----------

0	
---	--

- | | |
|--|--------------------|
| | ◆ Frame acceptance |
|--|--------------------|

	The Data Link Layer shall pass all group frames to the Data Link Layer User irrespective of the contained Group Address.
--	--

- | | |
|--|---------------------------------------|
| | ◆ Frame acknowledgement when relevant |
|--|---------------------------------------|

	The condition for Group Frame acknowledgment based on the evaluation of the Group Address Table shall be considered as fulfilled ²⁸⁾ .
--	---

1	
---	--

- | | |
|--|--------------------|
| | ◆ Frame acceptance |
|--|--------------------|

	No Group Frame shall be passed to the Data Link Layer User.
--	---

- | | |
|--|-------------------------|
| | ◆ Frame acknowledgement |
|--|-------------------------|

	No Group Frame shall be acknowledged.
--	---------------------------------------

²⁸⁾ This means that the acknowledge condition resulting from the Group Address Table evaluation is positive. Other conditions, typically medium specific, additionally apply for confirming the message.

Length Behaviour

- >1 The Data Link Layer shall evaluate the Group Address Table as follows.
- The Data Link Layer shall compare the destination_address to the subsequent Group Addresses in the Group Address Table, starting from connection nr. 1. This procedure shall stop on either one of the 3 following stop conditions:
1. the destination_address is found in the Group Address Table, or
 2. the last compared Group Address entry in the Group Address Table has the connection_number equal to the Address Table length, this is when the end of the address table is reached.
- ◆ Frame acceptance
- Only if the stop condition 1 is fulfilled, this is, if the destination_address is found in the Group Address Table, the Data Link Layer shall pass the received Group Frame to the Data Link Layer User by means of an L_Data-service.
- For the other stop condition, the received Group Frame shall not be passed to the Data Link Layer User and the Group Frame shall be removed.
- ◆ Frame acknowledgement
- Only if the stop condition 1 is fulfilled, the condition for Group Frame acknowledgment based on the evaluation of the Group Address Table shall be considered as fulfilled.

4.16.6.3.2 Transport Layer

The Transport Layer specifications (see [06]) require that the TSAP used in the T_Data_Group service shall be a reference to a Group Address.

If using this standardized Group Address Table format, the TSAP shall be equal to the Connection Number.

a) Frame transmission

The Transport Layer shall use the value of the TSAP parameter in the T_Data_Group.req service primitive as connection number in the Group Address Table. The value of the Group Address found for this connection number shall be used as destination_address parameter in the N_Data_Group.req service primitive that shall be passed to the Network Layer.

b) Frame reception

The value of the parameter destination_address of the N_Data_Group.ind (N_Data_Group.con) shall be searched in the Group Address Table.

If the destination_address is found, the connection number at which it is found shall be taken as value for the parameter TSAP in the T_Data_Group.ind (T_Data_Group.con).

If the destination_address is not found, the value for the parameter TSAP in the T_Data_Group.ind shall be set to FFh to indicate an unknown connection number.

4.16.6.4 Usage by the Management Client

4.16.6.4.1 Remote access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to download the Group Address Table - Realisation Type 5.

It shall be checked on the forehand by the Management Server if the download of the Group Address Table does not conflict with other resources ²⁹⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Set Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, 00h)
;      Set Group Address Table Length to 1 30)
DMP_MemWrite_RCo(0116h, 0116h, 01h)
;      Write the Group Address entries of the Group Address Table.
;      Note that this management procedure foresees the optional splitting up of the data
;          ;          in blocks of 12 octets.
DMP_MemWrite_RCo(0119h, 0119h+2*AddressTableLength-3, GrAT)
;      Set the Group Address Table length back to the value correct for the application.
DMP_MemWrite_RCo(0116h, 0116h, LLh)
;      Check the correct Group Address Table Length
DMP_MemRead_RCo(0116h,0116h)
;      If the received length is not equal to the programmed length, abort the Configuration Procedure.
;      Clear Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, FFh)
;      Restart the device
DMP_Restart
```

²⁹⁾ Examples

1. Group Address that have been inserted compared to the Group Address Table currently running in the Management Server, which would make the Group Objects Association Table invalid.
2. Increased number of Group Address which require a larger address space than currently available in the Management Client, forcing other recourses, such as the Group Objects Association Table, to be shifted.

³⁰⁾ This makes the Data Link Layer does no longer pass L_Data-frames containing group communication to the upper layers.

4.16.7 Group Address Table – Realisation Type 6

Used by: System 300

4.16.7.1 Location

The Group Address Table - Realisation Type 6 shall be implemented as Interface Object of the type Addressable Object (Object Type = 0001h).

4.16.7.2 Format

The Address Table Interface Object shall only include the Group Addresses of the device. It shall provide the management operations for uploading and downloading of the Group Address Table.

The Group Address Table Realisation Type 6 shall support both the L_Data_Standard frames format as well as the L_Data_Extended frame format. For each Frame Format there shall be a corresponding Group Address Table. One AddressTable Object shall contain exactly one of the Properties Address Table Format 0 (PID_TABLE = 23) or Address Table Format 1 (PID_ADDRTAB1 = 52) with one given Address Table format as specified below. A device may support multiple Frame Formats and therefore more than one instance of the Addressable Object is possible. The Property “Address Table Format 1” shall only be valid for the frame format given in the Property “Extended Frame Format” (PID_EXT_FRAMEFORMAT = 51). If the Property “Extended Frame Format” does not exist the Frame Format shall be 0 (L_Data_Standard frame format).

The Group Address Table – Realisation Type 6 shall provide the Properties as specified in Table 77 and defined in detail in the further clauses. For the requirements about which Properties are mandatory and optional and for the access levels, please refer to Annex A in [17].

Table 73 – Group Address Table Interface Object – Realisation Type 6

Property Name	Property Identifier	PDT/DPT	Use
Interface Object Type	1 = PID_OBJECT_TYPE	See 4.2.1.	BCU/LTE
Interface Object Name	2 = PID_OBJECT_NAME	See 4.2.2.	BCU/LTE
Load Control	5 = PID_LOAD_STATE_CONTROL	See 4.2.5.	BCU/LTE
Address Table Format 0 ^a	23 = PID_TABLE	PDT_GENERIC_02[]	BCU/LTE
Extended Frame Format	51 = PID_EXT_FRAMEFORMAT	PDT_UNSIGNED_CHAR	LTE
Address Table Format 1 ^a	52 = PID_ADDRTAB1	PDT_GENERIC_04[]	LTE
^a Each instance of the Group Address Table Object shall hold exactly one of the Properties “Address Table Format 0” or “Address Table Format 1”.			

4.16.7.2.1 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This Property shall contain Object Type of the Interface Object. The Group Address Table shall have Object Type 0001h.

4.16.7.2.2 PID_OBJECT_NAME (PID = 2)

Please refer to 4.2.2 for the general requirements for PID_OBJECT_NAME.

This Property shall contain the name of the Address Table.

4.16.7.2.3 PID_LOAD_STATE_CONTROL (PID = 5)

Please refer to 4.2.5 for the general requirements for PID_LOAD_STATE_CONTROL.

4.16.7.2.4 PID_TABLE (PID = 23)

This Property “Address Table Format 0” shall include an array with the Group Address Table. Only Group Addresses used by the device shall be in the table. If the current length is 0 all Group Addresses shall be accepted. No Group Address shall be accepted if the load control state differs from loaded (e.g.: unloaded or loading). The Group Addresses shall be filled in ascending order. The current length of the array shall be the number of entries in the Address Table. The maximum length shall be calculated via the Load Control “Relative Allocation” (for further information please refer to [09] clause “Load Control Record: Relative Allocation”) or shall be fixed in the device.

Table 74 – Property Address Table Format 0 - value

Property Value array index (current nr of elem)	Description	Size	TSAP
0	Current length	2 octets	
1	Group Address 1	2 octets	1
2	Group Address 2	2 octets	2
...	...	...	
n	Group address n	2 octets	n

The maximum length of the Address Table Property shall be read with an A_PropertyDescription_Read service.

4.16.7.2.5 PID_EXT_FRAMEFORMAT (PID = 51)

This Property is reserved for LTE-Mode devices. The Extended Frame Format Property shall include the value of Extended Frame Format (0001b to 1111b) for which the Address Table Property shall be valid³¹⁾. If the Extended Frame Format Property does not exist the Address Table shall be valid for frame type 0 (standard frame).

4.16.7.2.6 PID_ADDRTAB1 (PID = 52)

This special LTE-Mode Property Address Table Format 1 shall control the support of Extended Frames. One address entry shall consist of four octets. The first two octets shall include the base address and the third and fourth a mask with the valid bits of the base address. The rule for address matching for reception shall be:

If (Group Address AND mask) is equal to (base address AND mask) then match

This Property shall only include the zone addresses for one Extended Frame Format. If more than one Extended Frame Format is supported in a device multiple instances of the Address Table Object with PID_ADDRTAB1 shall exist. The number of Address Table Objects is theoretically not limited in a device. The Address Table Format 1 Property shall only include values for reception, transmission addresses are specified in the application.

The maximum length shall be calculated via the Load Control “Relative Allocation” or shall be fixed in the device.

³¹⁾ The Extended Frame Format field is specified in [03].

Table 75 - Address Table Format 1 - Example 1

Base Address	Mask	Matching addresses
1234h	FFFFh	1234h
1230h	FFFFh	1230h
1200h	FFF0h	1200h – 120Fh (sniffer)
1204h	FF0Fh	12x4h, x=0h - Fh (sniffer)
1534h	FFFFh	1534h
...		

Table 76 - Example for a device with more than one Address table

Address Table Instance	Interface Object Type	Local Index (device Object = 0)	Frame Format (PID = 51)	Address Table Property	Address Table Values	Description
1	1	1	00h	23	1001h, 1002h, 1003h,...	S-Mode Group Address Table
2	1	2	04h	52	1234.FFFFh, 1230.FFFFh, 1200.FFF0h 1204.FF0Fh 1534.FFFFh, 1530.FFFFh, 1500.FFF0h	Geographical Tag 1 Address Table
3	1	3	05h	52	2534.FFFFh, 2530.FFFFh, 2500.FFF0h	Geographical Tag 2 Address Table
4	1	4	06h	52	00A1.FFFFh	Application Specific Tag Address Table
5	1	5	07h	52	0654.FFFFh	Unassigned Tags Address Table

4.16.7.4 Usage by the Management Client

4.16.7.4.1 General

For the Property Based Management no memory allocation is needed. The data shall be written directly to the Property Address Table. The maximum size of the table can be read via a description read in the maximum number of elements in the Address Table Property. To prevent the connectionless access to Address Table it is recommended to not use the lowest access level for this Interface Object.

The complete download of the Group Address Table – Realisation Type 6 shall be done in the following way:

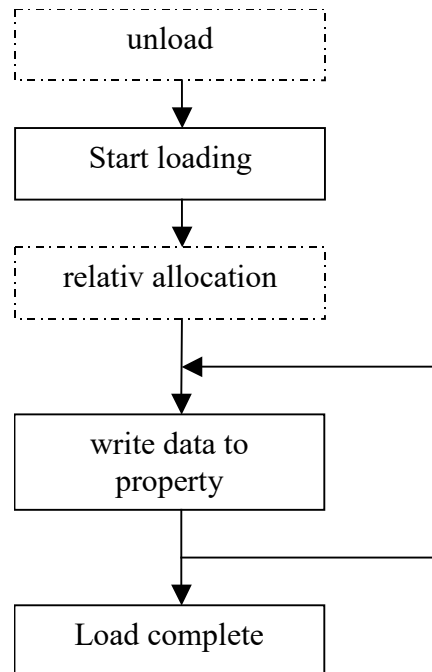


Figure 58 - Load Procedure

4.16.7.4.2 Remote Access

The following Load Controls shall be supported for the management of the Group Address Table – Realisation Type 6.

Load Control	Subtype	Description	Supported
00h		No operation	M
01h		Start Loading	M
02h		Load Completed	M
03h		Additional Load Controls	
	0Ah	Relative Allocation	M
04h		Unload	M

4.16.7.4.3 Local Access

Local Access by a Management Client to the Group Address Table – Realisation Type 6 is not specified.

4.16.8 Group Address Table – Realisation Type 7

Used by: System B

4.16.8.1 Location

The Group Address Table - Realisation Type 7 shall be implemented as Interface Object of the type Addressable Object (Object Type = 0001h). This Interface Object shall provide the Property `PID_TABLE_REFERENCE` for memory mapped access to the table data.

4.16.8.2 Format

The Group Address Table Object shall contain the Group Address Table length, and the Group Addresses (LSAPs) of the device.

It shall provide the management operations for the up- and download of Group Address Tables.

The Group Address Table – Realisation Type 7 provides the Properties as specified in Table 77 and defined in detail in the further clauses. For the requirements about which Properties are mandatory and optional and for the access levels please refer to Annex A in [17].

Table 77 – Group Address Table Interface Object – Realisation Type 7

Property Name	Property Identifier	PDT/DPT
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL
Table Reference	7 = PID_TABLE_REFERENCE	PDT_UNSIGNED_LONG
Table	23= PID_TABLE	PDT_UNSIGNED_INT[]
Memory Control Table	27= PID_MCB_TABLE	PDT_GENERIC_08[]
Error code	28= PID_ERROR_CODE	PDT_ENUM8
PL110 Group Responder Table ^a	53= PID_GROUP_RESPONDER- _TABLE	PDT_BITSET8[]
^a This Property is mandatory for PL110 devices. For all other media this Property shall not be implemented.		

4.16.8.2.1 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This Property shall be the Object Type of the Interface Object. The Group Address Table shall have Object Type 0001h.

4.16.8.2.2 PID_LOAD_STATE_CONTROL (PID = 5)

Please refer to 4.2.5 for the general requirements for PID_LOAD_STATE_CONTROL.

4.16.8.2.3 PID_TABLE_REFERENCE (PID = 07)

Please refer to 4.2.7 for the general requirements for PID_TABLE_REFERENCE.

4.16.8.2.4 PID_TABLE (PID = 23) for Group Address Table - Realisation Type 7

This Property shall contain the Group Address Table – Realisation Type 7.

The contents of this Property shall only be valid in the load state ‘loaded’. Writing shall only have effect in the load state ‘Loading’.

For the Realisation Type 7 of the Group Address Table, the Property Table shall contain an array with the Group Address Table. Only Group Addresses (LSAPs) used by the device shall be in the table. If the current length is 0 all Group Addresses shall be accepted. To accept no Group Address at all the Load State shall be set to a state different from “Loaded” (e.g.: “Unloaded” or “Loading”). The Group Addresses shall be filled in ascending order. The current length shall be the number of entries in the Address Table. The maximum length of the Address Table Property shall be read via A_Property-Description_Read. The Maximum Length shall be calculated via the load control “*relative allocation*” (for further information please refer to [09] clause “Load Event Data Relative Allocation” and [10] in the Configuration Procedures that use this Load Control) or shall be fixed in the device.

Table 78 - Group Address Table – Realisation Type 7

Property Value array index (current nr of elem)	Description	Size	TSAP
0	Current length	2 octets	
1	Group Address 1	2 octets	1
2	Group Address 2	2 octets	2
...	...	...	
n	Group address n	2 octets	n

Every Group Address entry shall comply with the format and full range as specified in clause 3.4 and [04].

4.16.8.2.5 PID_GROUP_RESPONSER_TABLE (PID = 53)

This Property is mandatory for PL110 devices. For all other media this Property shall not be implemented.

Each bit in this bit field shall define the Data Link Layer acknowledge generation of one associated LSAP.

Table 79 - Group Responder Flag (GRF) Table

Octet#	Bit#							
	7	6	5	4	3	2	1	0
Octet 0	Max. Length (LSAPs DIV 8) + 1							
Octet 1	GRF 8	GRF 7	GRF 6	GRF 5	GRF 4	GRF 3	GRF 2	GRF 1
Octet 2	GRF 16	GRF 15	GRF 14	GRF 13	GRF 12	GRF 11	GRF 10	GRF 9
...								
Octet n	...	...	...	...	...	...	...	...

4.16.8.2.6 PID_MCB_TABLE (PID = 27)

Please refer to the Interface Object Type independent specification of PID_MCB_TABLE in 4.2.27.

For the Group Address Table Object only the checksum feature is intended to be used but using the same mechanism as for the Application Program objects.

For this case the maximal table length (no. of elements) shall be set to '1' and the read and write access shall be set to the lowest access level.

4.16.8.2.7 PID_ERROR_CODE (PID = 28)

Please refer to 4.2.28 for the general requirements for PID_ERROR_CODE.

4.16.8.3 Usage by the Management Server

The usage by the Management Server (device) shall be identical as for Group Address Table – Realisation Type 4 (see 4.16.5), except for the frame acknowledgement.

◆ Frame acknowledgement

The received frame shall be acknowledged only if both of the following conditions are fulfilled:

1. The Group Address Table evaluation procedure is stopped by the stop condition 1, this is, the Group Address is found in the Group Address Table, and

2. In the Property `PID_GROUP_RESPONSER_TABLE` the bit is set at Property array element $(\text{Group Address DIV } 8) + 1$ at bit position $(\text{Group Address} - 1) \text{ MOD } 8$.

4.16.9 Group Address Table – Realisation Type 8

Used by mask 5705h.

4.16.9.1.1 Format

Structure:	Address	Group Address Table	TSAP (Connection Nr.)
	Low memory		
1 octet		Length	
2 octets		Individual Address	0
2 octets		Group Address Nr. 1	1
2 octets		Group Address Nr. 2	2
2 octets		Group Address Nr. 3	3
		...	...
	High memory		

The Group Addresses shall be sorted in ascending order with increasing memory locations. Connection Nr. 1 shall refer to the first Group Address.

Max. Length: The maximum length is implementation dependent. It shall be checked whether this maximum value is not exceeded.

GAs: 16 bit

4.16.9.2 Location

The Location shall be fixed at address 4000h.

4.16.10 Group Address Table - E-Mode Realisation Type 1 (GrAT – Easy 1)

Used by: Ctrl-Mode fixed DMA
and mask 0012h in E-Mode

4.16.10.1 Format

See clause 4.16.3.1.

4.16.10.2 Location

The location is fixed at address 0116h.

4.16.10.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and mask 0012h the following applies.

Before memory mapped access to a device, the E-Mode Controller shall set the error flags to 00h to stop the application. A device shall reject Link Services sending a negative response (`start_index = 0` and no `group_address_list`) if error flags are set to 00h, not depending on communication mode (error flags definition is given in [22]).

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

4.16.10.4 Usage by the Management Server (device)

See clause 4.16.3.3.

4.16.10.5 Usage by the Management Client

4.16.10.5.1 Remote access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to read and to write the GrAT – Easy 1 and to get the maximum length of the GrAT – Easy 1.

4.16.10.5.1.1 Read GrAT – Easy 1

```
;      Connect to the device
DMP_Connect_RCo
;      Get GrAT – Easy 1 Group Address Table Length
DMP_MemRead_RCo(0116h, 0116h, AddressTableLength)
;      Read the Group Address entries of the GrAT – Easy 1.
;      Note that this Management Procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemRead_RCo(0119h, 0119h+2*AddressTableLength-3, GrAT)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.16.10.5.1.2 Get maximum length GrAT – Easy 1

The calculation of the maximum length of the GrAT – Easy 1 is based on the Association Table following directly behind the GrAT – Easy 1 without space between the tables.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table Pointer
DMP_MemRead_RCo(0111h, 0111h, GrAssocTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
;      Calculate the maximum length of GrAT – Easy 1:
;      ⇒ AddressTableMaxLength = (0100h + GrAssocTabPtr - 0116h - 3)/2 32)
```

4.16.10.5.1.3 Write GrAT – Easy 1

It shall be checked on beforehand by the Management Server if the download of the GrAT – Easy 1 does not conflict with other Resources ³³⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Set Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, 00h)
;      Set Group Address Table Length to 1 34)
```

³²⁾ The calculation is based on the memory block reserved for GrAT – Easy 1, including 3 octets for GrAT – Easy 1 Table length and the Individual Address. The Association Table directly follows in memory the GrAT – Easy 1 (please refer to [09] “Appendix 5 : Controller Mode download Procedures”).

³³⁾ EXAMPLE Group Address that have been inserted compared to the GrAT – Easy 1 currently running in the Management Server, which would make the Group Objects Association Table invalid.

```

DMP_MemWrite_RCo(0116h, 0116h, 01h)
;   Write the Group Address entries of the Group Address Table.
;   Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemWrite_RCo(0119h, 0119h+2*AddressTableLength-3, GrAT)
;   Set the Group Address Table length back to the value correct for the application.
DMP_MemWrite_RCo(0116h, 0116h, LLh)
;   Check the correct Group Address Table Length
DMP_MemRead_RCo(0116h, 0116h)
;   If the received length is not equal to the programmed length,
;   abort the Configuration Procedure.
;   Clear Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, FFh)
;   Disconnect from the device
DMP_Disconnect_RCo

```

4.16.10.5.2 Local access

Not applicable.

4.16.10.5.3 Internal Access

Not applicable.

4.16.11 Group Address Table - E-Mode Realisation Type 2 (GrAT – Easy 2)

Used by: Ctrl-Mode reloc. DMA
and mask 0020h, 0021h and 0701h in E-Mode

4.16.11.1 Format

See clause 4.16.4.1.

4.16.11.2 Location

The location of the GrAT – Easy 2 is indicated by the pointer PID_TABLE_REFERENCE (PID = 7) of the Group Address Table Object. The object_index of the Group Address Table Object shall be 1 (please refer [10] “Appendix 5 : Controller Mode download Procedures”).

The Property PID_TABLE_REFERENCE shall be a read only Property.

4.16.11.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and Masks 0020h, 0021h and 0701h the following applies.

A device receiving Link Services when Group Address Table, Group Object Association Table, Group Object Table and Application load states are different from *loaded* shall reject these services sending a negative response (start_index = 0 and no group_address_list), not depending on the communication mode.

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

4.16.11.4 Usage by the Management Server (device)

Please refer to clause 4.16.4.3 “Usage by the Management Server (device)”.

³⁴⁾ This makes the Data Link Layer does no longer pass L_Data-frames containing group communication to the upper layers.

4.16.11.5 Usage by the Management Client

4.16.11.5.1 Remote Access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to read and to write the GrAT – Easy 2 and to get the maximum length of the GrAT – Easy 2.

4.16.11.5.1.1 Read GrAT – Easy 2

```
;      Connect to the device
DMP_Connect_RCo
;      Get GrAT – Easy 2 Table pointer
DMP_InterfaceObjectRead_R(object_index = 01h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAddrTabPtr)
;      Get Group Address Table Length
DMP_MemRead_RCo(GrAddrTabPtr, GrAddrTabPtr, AddressTableLength)
;      Read the Group Address entries of the Group Address Table.
;      Note that this management procedure foresees the optional splitting up
;      of the data in blocks of 12 octets.
DMP_MemRead_RCo(GrAddrTabPtr + 3, GrAddrTabPtr + 2 * AddressTableLength, GrAT)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.16.11.5.1.2 Get maximum length GrAT – Easy 2

The calculation of the maximum length of the GrAT – Easy 2 is based on the Group Association Table following directly behind the GrAT – Easy 2 without space between the tables.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Address Table pointer
DMP_InterfaceObjectRead_R(object_index = 01h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAddrTabPtr)
;      Get Group Association Table pointer
DMP_InterfaceObjectRead_R(object_index = 02h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAssocTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
;      Calculate the maximum length Group Address Table Format 2:
;      ⇒      AddressTableMaxLength = (GrAssocTabPtr - GrAddrTabPtr - 4)/2 35)
```

³⁵⁾ The calculation is based on the memory block reserved for GrAT – Easy 2, including 4 octets for Table length, Individual Address and checksum. The Association Table is close to the Group Address Table (refer [09] “Appendix 5 : Controller Mode download Procedures”).

4.16.11.5.1.3 Write GrAT – Easy 2

It shall be checked on beforehand by the Management Server if the download of the GrAT – Easy 2 does not conflict with other Resources ³⁶⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Address Table pointer 37)
DMP_InterfaceObjectRead_R(object_index = 01h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAddrTabPtr)
;      Set Group Address Table "Loading" 38)
DMP_LoadStateMachineWrite_RCo_IO(object_index = 01h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
    nr_of_elem = 1, Load)
;      Write the Group Address entries of the Group Address Table.
;      Note that this management procedure foresees the optional splitting up
;      of the data in blocks of 12 octets.
DMP_MemWrite_RCo(GrAddrTabPtr +3, GrAddrTabPtr +2*AddressTableLength, GrAT)
;      Set the Group Address Table length back to the value correct for the application.
DMP_MemWrite_RCo(GrAddrTabPtr, GrAddrTabPtr, LLh)
;      Check the correct Group Address Table Length
DMP_MemRead_RCo(GrAddrTabPtr, GrAddrTabPtr)
;      If the received length is not equal to the programmed length, abort the Configuration Procedure.
;      Set Group Address Table "Loaded"
DMP_LoadStateMachineWrite_RCo_IO(object_index = 01h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
    nr_of_elem = 1, LoadCompleted)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.16.11.5.2 Local Access

Not applicable.

4.16.11.5.3 Internal Access

Not applicable.

³⁶⁾ Example: Group Address that have been inserted compared to the Group Address Table currently running in the Management Server, which would make the Group Objects Association Table invalid.

³⁷⁾ Can be skipped if the pointer value is known from previous Management Procedures as for instance as specified in clause 4.10.10.5.1.2 “Get maximum length GrAT – Easy 2”.

³⁸⁾ It is recommended to check the load via DMP_LoadStateMachineRead_RCo state before setting a new state. If the previous state is "Loading" it's the decision of the client to handle this state. All other states except "Loaded" shall stop the procedure.

4.17 Group Object Association Table (GrOAT)

4.17.1 Abstract Resource definition

The Group Objects Association Table shall be a parameter of the Application Layer. It shall store the relationship between Transport Layer Service Access Points (TSAPs) and Application Layer Service Access Points (ASAP). The information on this relationship is needed when mapping the Multicast Communication Mode messages A_GroupValue_Read, A_GroupValue_Response and A_GroupValue_Write to T_Data_Group-messages and vice versa. The TSAP shall be an index in the Group Address Table. The ASAP shall be the Group Object number. The lowest ASAP shall be 0.

The ASAP shall in this Resource be a unique identifier for a Group Object to the Application Layer. Please refer as well to [07]. The ASAP for this Resource shall thus be a Group Object Number.

Default state (optional)

The default state is not standardised but is implementation specific. This may be any of the following. Other default states are possible as well.

- The Group Object Association Table contains no associations; the Current Size is 0.
- The Group Object Association Table contains an implementation specific number of default associations; the size has an according value.

If the Group Object Association Table Realisation is controlled through a Load State Machine, then this may be Loaded or Unloaded.

4.17.2 Access Control to the Group Object Association Table

If the MaS supports KNX Secure Communication, then it shall support Access Control to the Group Object Association Table. The write access to the contents and any other parameter related to the Group Object Association Table shall be limited to the Role “Tool”. Other Roles shall not have write access but may have read access.

EXAMPLE 40 Parameters related to the Group Object Association Table can be the write access to the Load State Machine (memory mapped or Property based), Table length, pointers to the memory mapped location etc.

4.17.3 Group Object Association Table - Realisation Type 1

Used by: Mask 0010h, 0011h, 0012h
 Mask 0020h in compatibility mode , 0021h in compatibility mode
 Mask 1012h, 1013h
 Mask 3012h, 4012h configured in S-Mode

4.17.3.1 Format

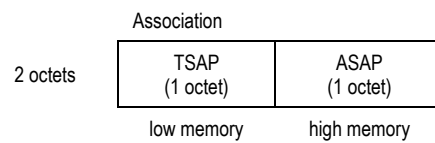
The Group Object Association Table shall be mapped in memory and shall have the following format.

		Group Objects Association Table		Association Nr.
Low memory				
1 octet	TableAddress	Current Size		
2 octets	TableAddress +1	TSAP	ASAP	0
2 octets	TableAddress +3	TSAP	ASAP	1
2 octets	TableAddress +5	TSAP	ASAP	2
...	...	...	...	...
High Memory				

GrAssocTabPtr = pointer to the Group Objects Association Table

The Current Size shall indicate the number Group Associations in the GrOAT.

An association shall always have the following format:



4.17.3.2 Location

The location of the Group Objects Association Table shall be given by the value contained in the Resource “Group Objects Association Table Pointer” (GrAssocTabPtr) incremented with an offset of 100h.

$$\text{TableAddress} = 100\text{h} + \text{GrAssocTabPtr}$$

GrAssocTabPtr

- **Format**
Single octet value
- **Location**
Address 111h
- **Value**
Valid values are within the range 00h and FFh.
Due to the location of other Resources (Group Address Table, EEPROM Checksum) this value can be only between 19h and FEh.
- **Usage by Management Server**
The Management Server shall take the value given at this location incremented with 100h to find the location of the Group Objects Association Table.
- **Usage by Management Client**
The Management Client shall set this value according to the location of the Group Objects Association Table.

4.17.3.3 Usage by Management Server (Device)

The algorithm for evaluation of the Group Objects Association Table shall depend on the communication way.

4.17.3.3.1 Frame transmission

When mapping a group message received from the Application Layer User to a T_Data_Group-message, the Application Layer shall map the contained ASAP to exactly one TSAP. To this, a “sending association” shall be defined.

The sending association shall be the Association with association number equal to the value of the ASAP. It is not checked whether the contained ASAP is the one for which the Application Layer User requested the transmission.

EXAMPLE ASAP nr. 4 is always transmitted using the association number 4.

4.17.3.3.2 Frame reception

The parameter TSAP, contained in the received T_Data_Group.ind and T_Data_Group.con services, shall be searched in the associations in the Group Objects Association Table. For every ³⁹⁾ instance of the TSAP that is found, the Application Layer

- shall take the ASAP that is associated to it by the association in which the TSAP is found, and
- shall generate the appropriate Application Layer service primitive ⁴⁰⁾, as specified in [07], using that ASAP as parameter, and
- shall stop the search for instances of the TSAP in the GrOAT when the end of the GrOAT is reached.

4.17.3.4 Usage by Management Client

4.17.3.4.1 Build rules

The evaluation mechanism by the Management Server leads to the following build rules for the Management Client.

- The size of the GrOAT shall at least be equal to the number of ASAPs.
- For each ASAP, the single TSAP on which its services shall be sent, shall be contained in the association with association number equal to the ASAP number.
- Non-sending associations can be put:
 - ▲ in the range of the sending associations (association numbers $\leq$ number ASAPs), if the TSAP is not used for a sending association with a higher association number.
 - ▲ in the range of the non-sending associations (association numbers $>$ number of ASAPs)
- Unused associations shall be indicated by a TSAP = FEh.

4.17.3.4.2 Remote access

```
;      Connect to the device
DMP_Connect_RCo
;      Set Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, 00h)
;      Set Group Address Table Length to 1. 41)
DMP_MemWrite_RCo(0116h, 0116h, 01h)
;      Write the Group Object Association Table
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemWrite_RCo(*GrAssocTabPtr, *GrAssocTabPtr+CurrentSize*2)
;      Set the GrAssocTabPtr to the correct value
DMP_MemWrite_RCo(0111h, 0111h, GrAssocTabPtr)
;      Set the Group Address Table length back to the value correct for the application.
DMP_MemWrite_RCo(0116h, 0116h, LLh)
;      Check the correct Group Address Table Length
DMP_MemRead_RCo(0116h,0116h)
;      If the received length is not equal to the programmed length, abort the Configuration Procedure.
;      Clear Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, FFh)
;      Restart the device
DMP_Restart
```

³⁹⁾ This means that the reception of one T_Data_Group-service leads to as many A_Group_Value_Xxx-services as the TSAP is contained in the Group Object Association Table.

⁴⁰⁾ A_GroupValue_Read.ind, A_GroupValue_Response.ind and A_GroupValue_Write.ind.

⁴¹⁾ This makes the Data Link Layer does no longer pass L_Data-frames containing group communication to the upper layers.

4.17.4 Group Object Association Table - Realisation Type 2

Used by: Mask 0020h, 0021h (both not in “compatibility mode”)

4.17.4.1 Format

The format is as specified in clause 4.17.3.1 “Format” above.

4.17.4.2 Location

The location of the Group Object Association Table – Realisation Type 2 shall be given by the value of the Property `PID_TABLE_REFERENCE` (`PID = 7`) of the System Interface Object “Group Object Association Table” (`object_type = 02h`). The Management Server (device) shall take care to set the `PID_TABLE_REFERENCE` to the value given by the Management Client when allocating the memory for this Resource.

4.17.4.3 Usage by the Management Server (device)

The algorithm for evaluation of the Group Objects Association Table shall depend on the communication way.

4.17.4.3.1 Frame transmission

When mapping a group message received from the Application Layer User to a `T_Data_Group`-message, the Application Layer shall map the contained ASAP to exactly one TSAP. To this, a “sending association” shall be defined.

The sending association shall be the Association with association number equal to the value of the ASAP. The Application Layer shall check whether the contained ASAP is the one for which the Application Layer User requested the transmission. If positive, the association shall be used for processing the transmission request. If negative, the transmission request shall be confirmed negatively by the Application Layer.

EXAMPLE ASAP nr. 4 is transmitted using the association number 4 if the ASAP entry in that association is equal to 4.

4.17.4.3.2 Frame reception

The usage by the Management Server for frame reception is as specified in clause 4.17.3.3.2 “Frame reception” above.

4.17.5 Group Object Association Table - Realisation Type 6

Used by: System 300
System B

4.17.5.1 Location

The Group Object Association Table – Realisation Type 6 shall be implemented as Interface Object of the type `Associationtable Object` (`Object Type = 0002h`).

For System B, this Interface Object shall provide the Property `PID_TABLE_REFERENCE` for memory mapped access to the table data. Please refer to [17] for the mandatory and optional Properties of this Interface Object in function of the Profile.

4.17.5.2 Format

The Group Object Association Table – Realisation Type 6 shall consist of one Interface Object of the type Associationtable Object (Object Type = 0002h). It shall provide the management operations for downloading. For the requirements about which Properties are mandatory and optional and for the access levels, please refer to Annex A in [17].

Table 80 - Association Table Interface Object

Property Name	Property Identifier	PDT (DPT)
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT
Interface Object Name	2 = PID_OBJECT_NAME	PDT_UNSIGNED_CHAR[]
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL
Table Reference	07 = PID_TABLE_REFERENCE	PDT_UNSIGNED_LONG
Table	23 = PID_TABLE	PDT_GENERIC_02[] ^a
		PDT_GENERIC_04[] ^a
Memory Control Table	27 = PID_MCB_TABLE	PDT_GENERIC_08[]
Error code	28 = PID_ERROR_CODE	PDT_ENUM8
^a It is mandatory to implement either one flavour of this Property.		

4.17.5.2.1 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This Property shall contain the Object Type of the Interface Object. The Associationtable Object shall have Object Type 0002h.

4.17.5.2.2 PID_OBJECT_NAME (PID = 2)

Please refer to 4.2.2 for the general requirements for PID_OBJECT_NAME.

This Property shall contain the name of the Association Table.

4.17.5.2.3 PID_LOAD_STATE_CONTROL (PID = 5)

Please refer to 4.2.5 for the general requirements for PID_LOAD_STATE_CONTROL.

4.17.5.2.4 PID_TABLE_REFERENCE (PID = 07)

Please refer to 4.2.7 for the general requirements for PID_TABLE_REFERENCE.

4.17.5.2.5 PID_TABLE (PID = 23)

This Property “Association Table” shall contain the actual Group Object Association Table which shall associate the Group Objects, through their Group Object Number (Application Layer Service Access Point = ASAP) with the Transport Layer Service Access Point (TSAP). The TSAP shall be the index in the Group Address Table format 0 (S-Mode Group Address Table with Frame Format = 0) starting with index 1.

The Association Table can be of format 0 or format 1. The Management Client can determine which format is implemented via an A_PropertyDescription_Read. In the A_PropertyDescription_Response-PDU the type field specifies either format 0 or format 1. This requires the implementation of full Interface Objects.

The format of the Group Object Association Table and the Property Datatype of PID_TABLE shall also be identical to the indication given by the field “Group Object Association Table Management” in the Property PID_MGT_DESCRIPTOR (see 4.3.23.3) if this Property is implemented.

The maximum number of elements in the Property shall show the maximum size of the table and current length shall show the current usage. In each association the high octet shall be the TSAP and the low octet shall be the ASAP number, both starting with value 1.

In case of transmission the first matching entry in the Association Table shall be used.

If the current length is set to 0 a “standard Association Table” shall be used (this is: ASAP = TSAP number). The Maximum Length shall be calculated via the load control relative allocation or shall be fixed in the device. The maximum length of the Property *Table* shall be read with an A_PropertyDescription_Read-service.

The content of the Property PID_TABLE shall only be valid in the load state ‘loaded’. Writing shall only have effect in the load state ‘Loading’.

PID_TABLE for the Group Object Association Table – format 0

In each association the high octet shall be the TSAP and the low octet shall be the ASAP, both starting with value 1.

Table 81 – PID_TABLE for the Group Object Association Table – format 0

Connection number (TSAP)	Group Object number (ASAP)	Property Array Index
Length (2 octet)		
TSAP #1 (1 octet)	ASAP #1 (1 octet)	1
TSAP #2 (1 octet)	ASAP #2 (1 octet)	2
TSAP #3 (1 octet)	ASAP #3 (1 octet)	3
...	...	...

PID_TABLE for the Group Object Association Table – format 1

In each association the most significant two octets shall be the TSAP and the least significant two octets shall be the ASAP number; both shall start with value 1.

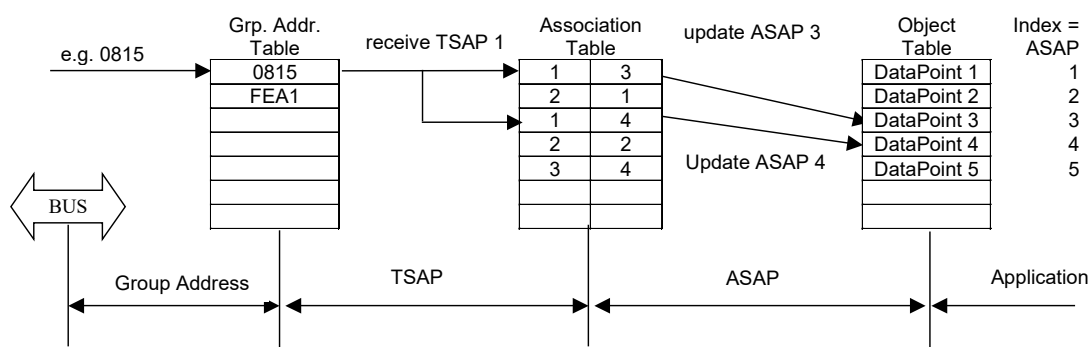
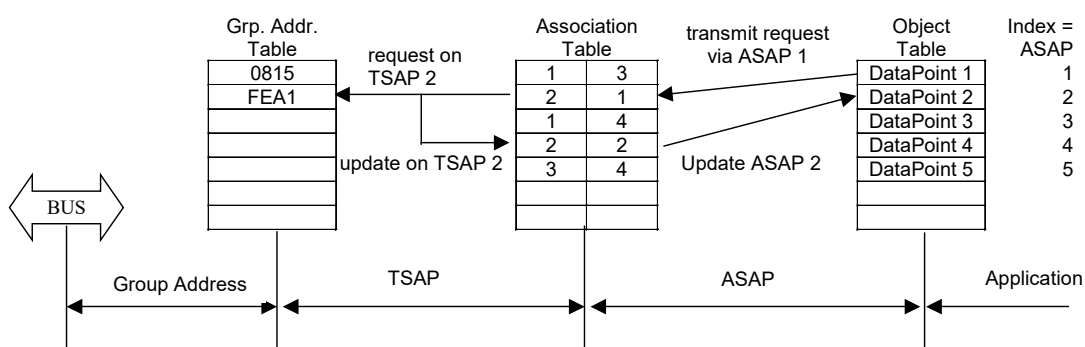
Table 82 – PID_TABLE for the Group Object Association Table – format 1

Connection number (TSAP)	Group Object number (ASAP)	Property Array Index
Length (2 octets)		
TSAP #1 (2 octets)	ASAP #1 (2 octets)	1
TSAP #2 (2 octets)	ASAP #2 (2 octets)	2
TSAP #3 (2 octets)	ASAP #3 (2 octets)	3
...	...	...

Table 83 - Example for an Association Table

Property Array Index		TSAP (starting with 1)	ASAP (starting with 1)
0	Association table length (2 octets)	--	---
1	1. Group Association	1	1
2	2. Group Association	2	2
3	3. Group Association	1	5
....			

The sending ASAP shall be the first matching ASAP (like in System 7). No other sorting shall be required. If no association is found no request (in transmission) or update (in reception) shall be generated. For more details please refer to [07] clause 3.1 “The relation between TSAPs and ASAPs”.

**Figure 59 – Reception of a frame with Group Address 0815h (example)****Figure 60 – Transmission of Group Object with ASAP 1**

4.17.5.2.6 PID_MCB_TABLE (PID = 27)

Please refer to the Interface Object Type independent specification of PID_MCB_TABLE in 4.2.27.

For the Group Object Association Table object only the checksum feature is intended to be used but using the same mechanism as for the application program objects.

For this case the maximal table length (no. of elements) shall be set to ‘1’ and the read and write access shall be set to the lowest access level.

4.17.5.2.7 PID_ERROR_CODE (PID = 28)

Please refer to 4.2.28 for the general requirements for PID_ERROR_CODE.

4.17.5.3 Usage by the Management Client

4.17.5.3.1 General

The sequence is identical to the sequence as specified in 4.16.7.4 “Usage by the Management Client”.

For the Property Based Management no memory allocation is needed. The data shall be written directly to the Property Association Table. The maximum size of the table can be read via a description read in the maximum number of elements in the Association Table Property. To prevent the connectionless access to the Association Table it is recommended to not use the lowest access level for this Interface Object.

4.17.5.3.2 Remote Access

The following Load Controls shall be supported for the management of the Group Object Association Table - Realisation Type 6.

Load Control	Sub-type	Description	Supported
00h		No operation	M
01h		Start Loading	M
02h		Load Completed	M
03h		Additional Load Controls	
	0Ah	Relative Allocation	M
04h		Unload	M

4.17.6 Group Object Association Table - Realisation Type 8

Used by: Mask 5705h

4.17.6.1 Format

		Group Objects Association Table		Association Nr.
Low memory				
1 octet	TableAddress	Current Size		
2 octets	TableAddress +1	TSAP	ASAP	0
2 octets	TableAddress +3	TSAP	ASAP	1
2 octets	TableAddress +5	TSAP	ASAP	2
...	...	...	...	...
High Memory				

The Current Size shall indicate the number Group Associations in the GrOAT.

4.17.6.2 Location

The location of the Group Object Association Table – Realisation Type 8 - shall be given by the value of the Property PID_TABLE_REFERENCE (PID = 7) of the System Interface Object “Group Object Association Table” (object_type = 02h).

4.17.7 Group Object Association Table - E-Mode Realisation Type 1 (GrOAT – Easy 1)

Used by: Ctrl-Mode fixed DMA
and mask 0012h in E-Mode

4.17.7.1 Format

Please refer to clause 4.17.3.1 “Format”.

4.17.7.2 Location

The location of the GrOAT – Easy 1 shall be given by the value contained in the Resource “Group Objects Association Table Pointer” (GrAssocTabPtr) incremented with an offset of 100h.

$$\text{TableAddress} = 100\text{h} + \text{GrAssocTabPtr}$$

GrAssocTabPtr

- Format
Single octet value
- Location
Address 111h
- Value
Valid values shall be within the range 00h and FFh.
Due to the location of other Resources (Group Address Table, EEPROM Checksum) this value can be only between 19h and FEh.
- Usage by Management Server
The Management Server shall take the value given at this location incremented with 100h to find the location of the GrOAT – Easy 1.
- Usage by Management Client
The Management Client shall **read only** the value **to find out** the location of the GrOAT – Easy 1.

4.17.7.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and mask 0012h the following applies.

Before memory mapped access to a device, the E-Mode Controller shall set the error flags to 00h to stop the application. A device shall reject Link Services sending a negative response (start_index = 0 and no group_address_list) if error flags are set to 00h, not depending on communication mode (error flags definition is given in [22]).

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

4.17.7.4 Usage by Management Server (Device)

Please refer to 4.17.3.3 “Usage by Management Server (Device)”.

4.17.7.5 Usage by Management Client

The evaluation mechanism by the Management Server leads to the following build rules for the Management Client.

- The size of the GrOAT – Easy 1 shall at least be equal to the number of ASAPs.
- For each ASAP, the single TSAP on which its services shall be sent, shall be contained in the association with association number equal to the ASAP number.
- Non-sending associations can be put:
 - in the range of the sending associations (association numbers $\leq$ number ASAPs), if the TSAP is not used for a sending association with a higher association number.
 - in the range of the non-sending associations (association numbers $>$ number of ASAPs)

Unused sending associations shall be indicated by a TSAP = FEh.

Unused receiving associations shall be indicated by a TSAP = FDh.

4.17.7.5.1 Remote access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to read and to write the GrOAT – Easy 1 and to get the maximum length of the GrOAT – Easy 1.

4.17.7.5.1.1 Read GrOAT - Easy Format 1

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table Pointer
DMP_MemRead_RCo(0111h, 0111h, GrAssocTabPtr)
;      Get Group Object Association Table Length
DMP_MemRead_RCo(0100h +GrAssocTabPtr, 0100h +GrAssocTabPtr, GrOAT_Length)
;      Read the Group Object Association Table
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemRead_RCo(0100h +GrAssocTabPtr+1, 0100h +GrAssocTabPtr +2*GrOAT_Length, GrOAT)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.17.7.5.1.2 Get maximum length GrOAT – Easy 1

The calculation of the maximum length of the GrOAT – Easy 1 shall be based on the Group Object Table following directly behind the Association Table without space between the tables.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table Pointer
DMP_MemRead_RCo(0111h, 0111h, GrAssocTabPtr)
;      Get Group Object Table Pointer
DMP_MemRead_RCo(0112h, 0112h, GrTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
;      Calculate the maximum length of Group Association Table Format 1:
;      ⇒   AssocTableMaxLength = (GrTabPtr - GrAssocTabPtr -1)/2 42)
```

⁴²⁾ The calculation is based on the memory block reserved for Association Table, including 1 octet for the “Length”. The Group Object Table is close to the Association Table (refer to the KNX Handbook Supplement 1 “Easy Configuration” Appendix 5)

4.17.7.5.1.3 Write GrOAT – Easy 1

It shall be checked on beforehand by the Management Server if the download of the GrOAT – Easy 1 does not conflict with other Resources ⁴³⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table Pointer 44)
DMP_MemRead_RCo(0111h, 0111h, GrAssocTabPtr)
;      Set Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, 00h)
;      Set Group Address Table Length to 1 45)
DMP_MemWrite_RCo(0116h, 0116h, 01h)
;      Write the Group Object Association Table
;      Note that this management procedure foresees the optional splitting up
;      of the data in blocks of 12 octets.
DMP_MemWrite_RCo(0100h +GrAssocTabPtr, 0100h +GrAssocTabPtr +2*GrOAT_Length, GrOAT) 46)
;      Set the Group Address Table length back to the value correct for the application.
DMP_MemWrite_RCo(0116h, 0116h, LLh)
;      Check the correct Group Address Table Length
DMP_MemRead_RCo(0116h, 0116h)
;      If the received length is not equal to the programmed length, abort the Configuration Procedure.
;      Clear Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, FFh)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.17.7.5.2 Local Access

Not applicable.

4.17.7.5.3 Internal Access

Not applicable.

4.17.8 Group Object Association Table – E-Mode Realisation Type 2 (GrOAT – Easy 2)

Used by: Ctrl-Mode reloc. DMA
and mask 0020h, 0021h in E-Mode

4.17.8.1 Format

Please refer to clause 4.17.4.1 “Format”.

4.17.8.2 Location

The location of the GrOAT – Easy 2 shall be indicated by the pointer PID_TABLE_REFERENCE (PID = 7) of the Association table Object. The object_index of the Association table Object shall be 2 ([10] clause “Appendix 5: Controller Mode download Procedures”).

⁴³⁾ Example:Associations that have been inserted compared to the Group Association Table currently running in the Management Server, which would make the Group Address Table invalid.

⁴⁴⁾ Can be skipped if the pointer value is known from previous actions e.g. as in clause 4.11.6.5.1.2 “Get maximum length GrOAT – Easy 1”.

⁴⁵⁾ This makes the Data Link Layer does no longer pass L_Data-frames containing group communication to the upper layers.

⁴⁶⁾ This includes the new GrOAT_Length value.

The Property PID_TABLE_REFERENCE shall be a read only Property.

4.17.8.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and Masks 0020h, 0021h the following applies.

A device receiving Link Services when Group Address Table, Group Object Association Table, Group Object Table and Application load states are different from *loaded* shall reject these services sending a negative response (start_index = 0 and no group_address_list), not depending on the communication mode.

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

4.17.8.4 Usage by the management server (device)

Please refer to 4.17.4.3.

4.17.8.5 Usage by the Management Client

The evaluation mechanism by the Management Server leads to the following build rules for the Management Client.

- The size of the GrOAT – Easy 2 shall at least be equal to the number of ASAPs.
- For each ASAP, the single TSAP on which its services shall be sent, shall be contained in the association with association number equal to the ASAP number.
- Non-sending associations can be put:
 - in the range of the sending associations (association numbers $\leq$ number ASAPs), if the TSAP is not used for a sending association with a higher association number.
 - in the range of the non-sending associations (association numbers $>$ number of ASAPs)

Unused sending associations shall be indicated by a TSAP = FEh.

Unused receiving associations shall be indicated by a TSAP = FDh.

4.17.8.5.1 Remote Access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to read and to write the GrOAT – Easy 2 and to get the maximum length of the GrOAT – Easy 2.

4.17.8.5.1.1 Read GrOAT – Easy 2

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table pointer
DMP_InterfaceObjectRead_R(object_index = 02h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAssocTabPtr)
;      Get Group Object Association Table Length
DMP_MemRead_RCo(GrAssocTabPtr, GrAssocTabPtr, GrOAT_Length)
;      Read the Group Object Association Table.
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemRead_RCo(GrAssocTabPtr + 1, GrAssocTabPtr + 2*GrOAT_Length, GrOAT)
;      Disconnect from the device
DMP_Disconnect_RCo
```


4.17.8.5.1.2 Get maximum length GrOAT – Easy 2

The calculation of the maximum length of the GrOAT – Easy 2 shall be based on the Group Object Table following directly behind the Association Table without space between the tables. The Group Object Table shall be the first part of the application program, the Group Object Table pointer shall match with the Property PID_TABLE_REFERENCE.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table Pointer
DMP_InterfaceObjectRead_R(object_index = 02h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAssocTabPtr)
;      Get Group Object Table Pointer
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
;      Calculate the maximum length Group Association Table Format 2:
;      ⇒      AssocTableMaxLength = (GrTabPtr - GrAssocTabPtr - 2)/2 47)
```

4.17.8.5.1.3 Write GrOAT – Easy 2

It shall be checked on beforehand by the management server if the download of the GrOAT – Easy 2 does not conflict with other Resources ⁴⁸⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table pointer 49)
DMP_InterfaceObjectRead_R(object_index = 02h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAssocTabPtr)
;      Set Group Object Association Table "Loading" 50)
DMP_LoadStateMachineWrite_RCo_IO(object_index = 02h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
    nr_of_elem = 1, Load)
;      Write the Group Object Association Table.
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemWrite_RCo(GrAssocTabPtr, GrAddrTabPtr + 2* GrOAT_Length, GrOAT) 51)
;      Set Group Object Association Table "Loaded"
DMP_LoadStateMachineWrite_RCo_IO(object_index = 02h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
    nr_of_elem = 1, LoadCompleted)
;      Disconnect from the device
DMP_Disconnect_RCo
```

⁴⁷⁾ The calculation is based on the memory block reserved for GrOAT – Easy 2, including 2 octets for the “Length” and checksum. The Group Object Table is close to the GrOAT – Easy 2 (refer to [09] Appendix 5 “Controller Mode download Procedures”).

⁴⁸⁾ Example: Associations that have been inserted compared to the GrOAT – Easy 2 currently running in the Management Server, which would make the GrOAT – Easy 2 invalid.

⁴⁹⁾ Can be skipped if the pointer value is known from previous actions as for instance specified in 4.11.7.5.1.2 “Get maximum length GrOAT – Easy 2”.

⁵⁰⁾ It is recommended to check the load state via DMP_LoadStateMachineRead_RCo before setting a new state. If the previous state is "Loading" it's the decision of the client to handle this state. All other states except "Loaded" shall stop the procedure

⁵¹⁾ This includes the new GrOAT_Length value

4.17.8.5.2 Local Access

Not applicable.

4.17.8.5.3 Internal Access

Not applicable.

4.17.9 Group Object Association Table – E-Mode Realisation Type 3 (GrOAT – Easy 3)

Used by: Easy Ctrl reloc. DMA
and mask 0701h in E-Mode

4.17.9.1 Format

The format shall be identical to the format of the “Group Object Association Table – Realisation Type 1” as specified in 4.17.3.1 “Format”.

4.17.9.2 Location

The location of the GrOAT – Easy 3 shall be indicated by the pointer `PID_TABLE_REFERENCE` (`PID = 7`) of the Association Table Object. The `object_index` of the Association Table Object shall be 2 (refer to [10] Appendix 5 “Ctrl-Mode download procedures”).

The property `PID_TABLE_REFERENCE` shall be a read only Property.

4.17.9.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and Mask 0701h the following applies.

A device receiving Link Services when Group Address Table, Group Object Association Table, Group Object Table and Application load states are different from *loaded* shall reject these services sending a negative response (`start_index = 0` and no `group_address_list`), not depending on the communication mode.

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

4.17.9.4 Usage by the management server (device)

The algorithm for evaluation of the Group Objects Association Table shall depends on the communication way.

4.17.9.4.1 Frame transmission

When mapping a group message received from the Application Layer User to a `T_Data_Group`-message, the Application Layer shall map the contained ASAP to exactly one TSAP. To this, a "sending association" shall be defined.

The sending association shall be the first matching entry in the Group Object Association Table. The Application Layer shall check whether the contained ASAP is the one for which the Application Layer User requested the transmission. If positive, the association shall be used for processing the transmission request. If negative, the transmission request shall be confirmed negatively by the Application Layer.

4.17.9.4.2 Frame reception

The parameter TSAP, contained in the received T_Data_Group.ind and T_Data_Group.con services, shall be searched in the associations in the Group Objects Association Table. For every instance of the TSAP that is found, the Application Layer shall

- take the ASAP that is associated to it by the association in which the TSAP is found, and
- generate the appropriate Application Layer service primitive, as specified in [07], using that ASAP as parameter.

The search for instances of the TSAP in the GrOAT shall stop when the end of the GrOAT is reached.

4.17.9.5 Usage by the Management Client

The evaluation mechanism by the Management Server leads to the following build rules for the Management Client.

- The sorting of the table is different from mask 0020h, 0021h, 0012h.
- All entries in the Group Object Association Table may be used for sending, not only the first ones.
- If there are several connection numbers connected to a certain SAP the entry with the lowest association number shall be used. This means that the sending connection number shall be before all other connection numbers for a certain SAP.
- If there are several SAPs connected to one connection number the sending SAP shall send a message to the associated connection number and initiate an internal update to all other SAPs. In this case the sorting is of no importance.

4.17.9.5.1 Remote Access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to read and to write the GrOAT – Easy 3 and to get the maximum length of the GrOAT – Easy 3.

4.17.9.5.1.1 Read GrOAT – Easy 3

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table pointer
DMP_InterfaceObjectRead_R(object_index = 02h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAssocTabPtr)
;      Get Group Object Association Table Length
DMP_MemRead_RCo(GrAssocTabPtr, GrAssocTabPtr, GrOAT_CurrentSize)
;      Read the Group Object Association Table.
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemRead_RCo(GrAssocTabPtr + 1, GrAssocTabPtr + 2*GrOAT_CurrentSize, GrOAT)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.17.9.5.1.2 Get maximum length GrOAT – Easy 3

The calculation of the maximum length of the GrOAT – Easy 3 shall be based on the Group Object Table following directly behind the Group Object Association Table without space between the tables. The Group Object Table shall be the first part of the Application Program, the Group Object Table pointer shall match with the property PID_TABLE_REFERENCE.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table Pointer
DMP_InterfaceObjectRead_R(object_index = 02h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAssocTabPtr)
;      Get Group Object Table Pointer
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
;      Calculate the maximum length Group Association Table Format 2:
;      ⇒      AssocTableMaxLength = (GrTabPtr - GrAssocTabPtr - 2)/2 52)
```

4.17.9.5.1.3 Write GrOAT – Easy 3

It shall be checked on beforehand by the Management Server if the download of the GrOAT – Easy 3 does not conflict with other Resources ⁵³⁾. If this cannot be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Association Table pointer 54)
DMP_InterfaceObjectRead_R(object_index = 02h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrAssocTabPtr)
;      Set Group Object Association Table "Loading" 55)
DMP_LoadStateMachineWrite_RCo_IO(object_index = 02h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
    nr_of_elem = 1, Load)
;      Write the Group Object Association Table.
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemWrite_RCo(GrAssocTabPtr, GrAddrTabPtr + 2* GrOAT_CurrentSize, GrOAT) 56)
;      Set Group Object Association Table "Loaded".
DMP_LoadStateMachineWrite_RCo_IO(object_index = 02h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
    nr_of_elem = 1, LoadCompleted)
;      Disconnect from the device
DMP_Disconnect_RCo
```

⁵²⁾ The calculation is based on the memory block reserved for GrOAT – Easy 3, including 2 octets for the current size and checksum. The Group Object Table is close to the GrOAT – Easy 3 (see [09] Appendix 5 “Ctrl-Mode download procedures”).

⁵³⁾ Example: Associations that have been inserted compared to the GrOAT – Easy 3 currently running in the Management Server, which would make the GrOAT – Easy 3 invalid.

⁵⁴⁾ Can be skipped if the pointer value is known from previous actions as for instance specified in 4.11.8.5.1.2

⁵⁵⁾ It is recommended to check the load state via DMP_LoadStateMachineRead_RCo before setting a new state. If the previous state is "Loading" it's the decision of the client to handle this state. All other states except "Loaded" shall stop the procedure

⁵⁶⁾ This includes the new GrOAT_CurrentSize value.

4.17.9.5.2 Local Access

Not applicable.

4.17.9.5.3 Internal Access

Not applicable.

4.18 Group Object Table

4.18.1 Abstract Resource Definition

The Group Object-Table shall contain general information about the internal Group Objects. It shall provide management operations for the internal Group Object Table. Elements of the Group Object Table shall be referred by the Association Table.

4.18.2 Access Control to the Group Object Table

If the MaS supports KNX Secure Communication, then it shall support Access Control to the Group Object Table. The write access to the contents and any other parameter related to the Group Object Table shall be limited to the Role “Tool”. Other Roles shall not have write access but may have read access.

EXAMPLE 41 Parameters related to the Group Object Table can be the write access to the Load State Machine (memory mapped or Property based), Table length, pointers to the memory mapped location etc.

4.18.3 Group Object Table - Realisation Type 1

Used by: Mask 0010h, 0011h, 0012h
Mask 0020h in compatibility mode , 0021h in compatibility mode
Mask 1012h, 1013h
Mask 3012h, 4012h configured in S-Mode

4.18.3.1 Format

The Group Object Table is mapped in memory and has the following format:

Low memory		Group Objects Table	Descriptor Nr.
1 octet	TableAddress	Current Size	
1 octet	TableAddress+1	RAM-Flags-Table Pointer	
3 octets	TableAddress+2	Group Object Descriptor	0
3 octets	TableAddress+5	Group Object Descriptor	1
3 octets	TableAddress+8	Group Object Descriptor	2
...	...		...
...	High Memory		

The Current Size shall indicate the number of Group Objects in the GrOT.

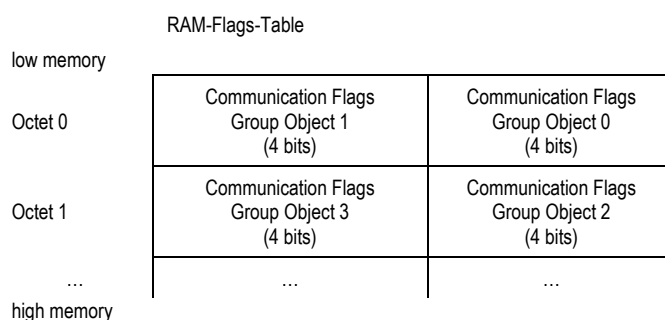
4.18.3.1.1 RAM-Flags-Table Pointer

This single octet pointer contains the start address of the RAM-Flags-Table. This pointer shall point to User-RAM.

4.18.3.1.1.1 RAM-Flags-Table

The RAM-Flags Table forms the data interface between the user application and the Group Object Server.

- Format



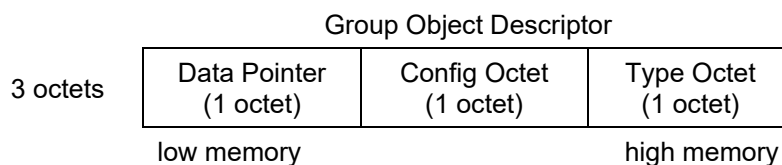
The communication flags are encoded as follows:

Bit	Name	Coding
3	update-flag	0 = not updated 1 = the value of the group object has been from the bus by an A_GroupValue_Write.ind or an A_GroupValue_Read.res.
2	data request flag	0 = idle/response 1 = data-request
1	transmission status	00 = idle/OK
0		01 = idle/error 10 = transmitting 11 = transmit request

4.18.3.1.2 Group Object Descriptor

4.18.3.1.2.1 Format

A group object descriptor always has the following format:



Data Pointer

The Data Pointer points to the MSB of the Group Object Value, that shall always start at octet boundary. Unused leading bits shall be set to 0. The address segment into which this pointer points is indicated by the “Segment Selector” field in the Config Octet, as specified below.

Config Octet

- Format: 1 octet

Bit	Field name
7	Shall be 1
6	Transmit Enable
5	Segment Selector Type
4	Write Enable
3	Read Enable
2	Communication Enable
1	Transmission Priority
0	

- Value

transmission priority: specifies the transmission-priority that is used for transmitting frames from this group object.

00	system priority
10	urgent priority
01	normal priority
11	low priority

communication enable: enables or disables communication via this group object

0	disabled	messages for this group object are not handled
1	enabled	messages for this group object are handled, in function of the other flags

read enable: specifies the behaviour of the Group Object server for this group object when an A_GroupValue_Read.ind is received for this group object.

0	disabled	the A_GroupValue_Read.ind service is ignored. no response is transmitted.
1	enabled	the group object server answers to an A_GroupValue_Read.ind to this group object with an A_GroupValue_Read.res containing this group objects value, under the condition that also the communication enable flag is set.

It is possible to read the value from another internal object.

write enable: specifies the behaviour of the Group Object server for this group object when an A_GroupValue_Write.ind is received for this group object.

0	disabled	the A_GroupValue_Write.ind service is ignored. no update of the group object value.
1	enabled	the group object server updates the group object value with the value contained in the A_GroupValue_Write.ind service, under the condition that also the communication enable flag is set.

segment selector ⁵⁷⁾: specifies the start address of the segment into which the data pointer points. The value of the data pointer has to be incremented with the value given below, in function of the segment selector, to obtain the address of the group object value.
This segment selector allows locating the group object value in RAM or in EEPROM.

0 segment = 00h

1 segment = 100h

transmission enable: specifies the behaviour of the Group Object server when the transmit request flag is set (by the user application).

0 disabled the transmit-request flag is ignored. No A_GroupValue_Write.req or A_GroupValue_Read.req is generated.

1 enabled if the transmit request flag is set, it is evaluated and the appropriate Application Layer service primitive is generated, under the condition that the communication flag is set.

NOTE 84 In mask version 0010h, this field is not supported. The value of this bit shall be always 1.

Type Octet

This field specifies the size of the group object value ⁵⁸⁾.

- Format

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Reserved. Shall be 0.	Reserved. Shall be 0.	Value-type					

- Coding

Type (dec.)	Value Size
0	1 bit
1	2 bits
2	3 bits
3	4 bits
4	5 bits
5	6 bits
6	7 bits
7	1 octet
8	2 octets
9	3 octets
10	4 octets
11	6 octets
12	8 octets

⁵⁷⁾ This was previously named “memory type”.

⁵⁸⁾ The formerly specified “VARDATA” size is no longer allowed.

Type (dec.)	Value Size
13	10 octets
14	14 octet

4.18.3.2 Location

The location of the Group Object Table – Realisation Type 1 is given by the value contained in the resource “Group Objects Table Pointer” (GrObjTabPtr) incremented with an offset of 100h.

TableAddress = 100h + GrObjTabPtr

GrObjTabPtr

- Format
Single octet value
- Location
Address 112h
- Value
Valid values are within the range 00h and FFh.
Due to the location of other resources (Group Address Table, EEPROM Checksum) this value can be only between 19h and FEh.
- Usage by Management Server
The Management Server takes the value given at this location incremented with 100h to locate the Group Objects Association Table.
- Usage by Management Client
The Management Client shall set this value according to the location of the Group Objects Association Table.

4.18.4 Group Object Table - Realisation Type 2

Used by: Mask 0020h, 0021h

4.18.4.1 Format

The format is identical as the “Group Object Table - Realisation Type 1” as specified in clause 4.18.3 above, except for the Group Object Config Octet.

Config Octet

- Format: 1 octet

This Config Octet Type contains an “Update Enable” field. The other fields are identical as above.

Bit	Name
7	Update Enable
6	Transmit Enable
5	Segment Selector Type
4	Write Enable
3	Read Enable
2	Communication Enable
1	Transmission Priority
0	

update enable: specifies if the Group Object Value is updated if an A_GroupValue_Read.res service primitive is received for that Group Object.

- 0 disabled the value of the Group Object is not updated. The update-flag in the RAM-Flags-Table for that Group Object is not set.
- 1 enabled the value of the Group Object is updated by the value contained in the A_GroupValue_Read.res service primitive. The update-flag in the RAM-Flags-Table for that Group Object is set.

4.18.5 Group Object Table - Realisation Type 6

Used by: System 300

4.18.5.1 Location

The Group Object Table – Realisation Type 6 shall be implemented as Interface Object of the object type Group Object Table Object (Object Type = 0009h).

4.18.5.2 Format

The Group Object Table – Realisation Type 6 shall consist of one Interface Object of the object type Group Object Table (Object Type = 0009h) and shall have the Properties as specified in Table 89. It shall provide the management operations for downloading. For the requirements about which Properties are mandatory and optional and for the access levels, please refer to Annex A in [17].

**Table 84 - Group Object Table Interface Object
for Group Object Table – Realisation Type 6**

Property Name	Property Identifier	PDT/DPT	
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT	Group Object Table
Interface Object Name	2 = PID_OBJECT_NAME	PDT_UNSIGNED_CHAR[]	Name of the Table
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL	See description below (4.18.5.2.3)
Object Table	51 = PID_GRPOBJTABLE	PDT_GENERIC_06[]	Group Object Table
Extended Group Object Reference	52 = PID_EXT_GRPOBJREFERENCE	PDT_GENERIC_08[]	Group Object Reference

4.18.5.2.1 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This Property shall contain the Object Type of the Interface Object. The Group Object Table shall have Object Type 0009h.

4.18.5.2.2 PID_OBJECT_NAME (PID = 2)

Please refer to 4.2.2 for the general requirements for PID_OBJECT_NAME.

4.18.5.2.3 PID_LOAD_STATE_CONTROL (PID = 5)

For further information please refer to clause 4.23 “Load State Machine”. This Property is mandatory if the Application Program is loadable. If this Property does not exist the Group Object Table shall not be loadable.

4.18.5.2.4 PID_GRPOBJTABLE (PID = 51)

The Group Object Table shall be divided into two Properties.

The Property Object Table (PID_GRPOBJTABLE) shall include the relevant parts for the Management Client that shall be writeable.

The Property Extended Group Object Reference (PID_EXT_GRPOBJREFERENCE) shall include the internal reference to the data that may be readable and optionally writeable or completely hidden.

Array elements within the Properties PID_GRPOBJTABLE and PID_EXT_GRPOBJREFERENCE with the same index shall refer to the same Group Object.

The Property Object Table shall include the tool relevant part of an Extended Group Object. In this table every Group Object shall refer to a Property of an Interface Object of the device. This Property shall replace or be additional to the standard Group Object Table Property (PID_TABLE).

Format

Configuration	Datatype (High part of DPT)	Datatype (low part of DPT)
2 octets	2 octet	2 octet

Configuration

Bitnr	Name	Values	
0, 1	transmission priority	00b = system 10b = urgent 01b = normal 11b = low	Sets the transmission priority for a sending frame
2	communication enable	0 = disabled 1 = enabled	Enables the communication to the bus
3	read enable	0 = disabled 1 = enabled	Enables a read from bus
4	Write enable	0 = disabled 1 = enabled	Enables a write from bus
5	Memory segment	0 / 1	device internal memory segment selector
6	transmit enable	0 = disabled 1 = enabled	Enables a read/write to bus
7	response update enable	0 = disabled 1 = enabled	Update is used as a write if enable. (Responses from other devices leading to updates of Datapoints in own device)
8 to 15	Reserved	0	

Datatype

The field Datatype shall contain the 4 octet ($U_{16}U_{16}$) Datapoint Type identifier (DPT_ID) according which the Datapoint value is encoded. The data length to be transmitted is implicit in the high part of the Datapoint Type.

The Data point Type is as described [14]. E.g.:

```
1.001 DPT_Switch {Off=0, On=1}
...
5.001 DPT_Scaling {0.. 100%}
...
9.001 DPT_Value_Temp {273...+670760}...
```

4.18.5.2.5 PID_EXT_GRP OBJ REFERENCE (PID = 52)

The Property Extended Group Object Reference shall include an Extended Group Object Table. In this table every Group Object shall refer to a Property of an Interface Object of the device. This Property shall be additional to the Group Object Table Property (PID_GRP OBJ TABLE). The Property array element from this Property shall refer to the Property array element from PID_GRP OBJ TABLE with the same start index. This Property shall describe the relation between Interface Object properties and Group Objects.

Format

2 octet	1 octet	1 octet	2 octet		1 octet	1 octet
Object Type	Object Instance	Property Identifier	Reserved (4 bit)	Start Index (12 bit)	Bit Offset	Conversion

Object Type

The Object Type shall include the type of the Interface Object that shall be equal to the value of the Object Type Property.

Object Instance

The Object Instance shall include the local Interface Object Instance. The Object Instance shall for each Object Type start with 1 and shall ascend with the local Interface Object index in the device.

Property Identifier

The Property Identifier shall be the identifier of the Property in the Interface Object referred by the Object Index.

Reserved

These 4 bits are reserved (always 0).

Start Index

This shall be the start index of the Property array (in case the Property data is no array, the start index shall be 1). This field shall be the start index in a Property and shall be 12 bit long. Only one array element of the Property can be accessed. Therefore the number of elements is fixed = 1 and not contained in the table.

Bit Offset

The Bit Offset shall be used to address single bits or only a part of a Property in a device. Bit offset shall start from “left” / MSB.

EXAMPLE From a structured Property with the type $U_{16}B_8$ the attribute Bit 0 of the B_8 field is mapped to a Boolean Group Object $\Rightarrow$ Bit Offset = 23

Conversion

This value shall identify a conversion function. Value 0 shall specify that there is no conversion. Values from 1 to 200 shall be reserved for standard conversions. Values from 201 to 254 are reserved for device specific conversions. Value 255 shall be reserved for escape.

A list of standard conversions will be defined for new application specific DPT in a separate document.

4.18.5.4 Usage by the Management Client

4.18.5.4.1 General

The sequence is identical to the sequence as specified in 4.16.7.4 “Usage by the Management Client”.

4.18.5.4.2 Remote Access

The following Load Controls shall be supported for the management of the Group Object Table - Realisation Type 6.

Load Control	Subtype	Description	Supported
00h		No operation	M
01h		Start Loading	M
02h		Load Completed	M
03h		Additional Load Controls	
	0Ah	Relative Allocation	M
04h		Unload	M

4.18.5.4.3 Local Access

Local Access by a Management Client to the Group Address Table – Realisation Type 6 is not specified.

4.18.6 Group Object Table - Realisation Type 7

Used by: System B

4.18.6.1 Location

The Group Object Table – Realisation Type 7 shall be implemented as Interface Object of the object type Group Object Table Object (Object Type = 0009h).

4.18.6.2 Format

The Group Object Table – Realisation Type 7 shall contain general information about the internal Group Objects. It shall provide management operations for the internal Group Object Table. Elements of the Group Object Table shall be referred by the Association Table. For the requirements about which Properties are mandatory and optional and for the access levels, please refer to Annex A in [17].

**Table 85 - Group Object Interface Object
for Group Object Table – Realisation Type 7**

Property Name	Property Identifier	PDT (DPT)	Description
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT	Group Object Table Object
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL	for further Information see Load / State- machines
Table Reference	07 = PID_TABLE_REFERENCE	PDT_UNSIGNED_LONG	This shall be a reference to the memory mapped access to the data of this Resource.
Table	23 = PID_TABLE	PDT_GENERIC_02[]	Group Object table Format 1
Memory Control Table	27 = PID_MCB_TABLE	PDT_GENERIC_08[]	Subsegmentation of memory space and check sum
Error code	28 = PID_ERROR_CODE	PDT_ENUM8	Specifies the reason for load state "ERROR"

4.18.6.2.1 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This Property shall contain the Object Type of the Interface Object. The Group Object Table shall have Object Type 0009h).

4.18.6.2.2 PID_LOAD_STATE_CONTROL (PID = 5)

For further information please refer to clause 4.23 "Load State Machine".

4.18.6.2.3 PID_TABLE_REFERENCE (PID = 07)

Please refer to 4.2.7 for the general requirements for PID_TABLE_REFERENCE.

A Management Client shall use the value of this Property as reference for memory mapped access to the Group Object Table – Realisation Type 7.

4.18.6.2.4 PID_TABLE (PID = 23) – for Group Object Table – Realisation Type 7

4.18.6.2.4.1.1 Format

The Group Object Table Property format 1 shall include an array of Group Object descriptions.

The current length shall be the number of entries in the Group Object Table. The Maximum Length shall be calculated via the Load Control *Data Relative Allocation* or shall be fixed in the device.

The content of this Property shall only be valid in the load state *Loaded*. Writing shall only have effect in the load state *Loading*.

Table 86 - Group Object Table format

Property array index, i.e. the relative memory address		ASAP
0	Length (2 octets)	
1	Group Object descriptor (2 octets)	1
2	Group Object descriptor (2 octets)	2
n	...	n

Table 87 - Group Object Descriptor

Bit #	15	14	13	12	11	10	9	8
Meaning	Response Update enable 0 = disable 1 = enable	Transmit enable 0 = disable 1 = enable	Value Read on initialisation 0 = disable 1 = enable	Write enable 0 = disable 1 = enable	Read enable 0 = disable 1 = enable	Communi- cation enable 0 = disable 1 = enable	Transmission priority 00b = system 10b = urgent 01b = normal 11b = low	
Bit #	7	6	5	4	3	2	1	0
Meaning	Value Field Types							

4.18.6.2.4.1.2 Transmission Priority, Communication Enable, Read Enable, Write Enable, Transmit Enable and Response Update Enable

These fields shall have identical functionality and coding as the fields with the same name as specified in clauses 4.18.3.1.2.1 and 4.18.4.1.

4.18.6.2.4.1.3 Value read on initialisation

This shall specify whether or not the RAM-flags of the Group Object shall be set after reset of the application and consequently an A_GroupValue_Read shall be issued to update this Group Object value

- 0 disabled The Group Object value shall not be updated after reset.
- 1 enabled The Group Object value shall be updated after reset.

NOTE 85 'Value Read on Initialisation' is a new feature introduced with System B.

As this feature has effect on busload especially after a reset of a complete installation this feature should be used very carefully (e.g. it should not be set by default in an average application).

4.18.6.2.4.1.4 Value Field Types

Table 88 - Value Field Types

Code	Symbol	Value size
0	Unsigned Integer (1)	1 bit
1	Unsigned Integer (1)	2 bit
2	Unsigned Integer (1)	3 bit
3	Unsigned Integer (1)	4 bit
4	Unsigned Integer (1)	5 bit
5	Unsigned Integer (1)	6 bit
6	Unsigned Integer (1)	7 bit
7	Octet 1	1 octet
8	Octet 2	2 octets
9	Octet 3	3 octets
10	Octet 4	4 octets
11	Octet 6	6 octets
12	Octet 8	8 octets
13	Octet 10	10 octets
14	Octet 14	14 octets
15	Octet 5	5 octets
16	Octet 7	7 octets
17	Octet 9	9 octets
18	Octet 11	11 octets
19	Octet 12	12 octets
20	Octet 13	13 octets
21	Octet 15	15 octets
22	Octet 16	16 octets
23	Octet 17	17 octets
...	...	...
254	Octet 248	248 octets
255	Octet 252	252 octets ⁵⁹⁾

4.18.6.2.5 PID_MCB_TABLE (PID = 27)

Please refer to clause 4.2.27 for the Interface Object Type independent specification of PID_MCB_TABLE.

This optional Property shall divide the segments ‘application program 1’ and ‘application program 2’ into multiple subsegments with access rights definable for each subsegment and carrying the checksum for the subsegments.

For the Group Object Interface Object only the checksum feature shall be used but using the same mechanism as for the Application Program objects.

⁵⁹⁾ Due to the fact that the frame length of an extended frame is encoded with a maximum value of 254 (255 = ESC Code) and that these maximum 254 octets cover the data plus two octets for the APCI and Transport Control the maximum data length is 252 octets.

For this case the table length (nr. of elements) shall be set to ‘1’ and the read and write access shall be set to the lowest access level.

The CRC checksum calculation shall provide a higher consistency while using the differential download.

The Management Server (device) shall properly calculate the CRC on the transition from the Load State *Loading* to the Load State *Loaded* according to the segment’s memory content right from beginning. Its value shall be valid in the load state *Loaded* only.

The 16 bit CRC shall be calculated according to the CRC16-CCITT specification.

For the specification of the Memory Control Block Table, please refer to 4.2.27 (Table 12).

For the specification of the CRC Control Byte please refer to clause 4.2.27.1.1.

4.18.6.2.6 PID_ERROR_CODE (PID = 28)

Please refer to 4.2.28 for the general requirements for PID_ERROR_CODE.

4.18.7 Group Object Table – E-Mode Realisation Type 1
(GrOT – Easy 1)

Used by: Ctrl-Mode fixed DMA
 and mask 0012h in E-Mode

4.18.7.1 Format

The Group Object Table is mapped in memory and has the following format ⁶⁰⁾:

		Group Object Table	Descriptor Nr.
		Low memory	
1 octet	TableAddress	Length	
1 octet	TableAddress+1	RAM-Flags-Table Pointer	
3 octets	TableAddress+2	Group Object Descriptor	0
3 octets	TableAddress+5	Group Object Descriptor	1
3 octets	TableAddress+8	Group Object Descriptor	2
...	...		...
...	High Memory		

The length shall indicate the number of Group Objects in the GrOT.

4.18.7.1.1 RAM-Flags-Table Pointer

Not applicable.

4.18.7.1.2 Group Object Descriptor

Not applicable.

4.18.7.2 Location

The location of the GrOT – Easy 1 is given by the value contained in the Resource “Group Object Table Pointer” (GrTabPtr) incremented with an offset of 100h.

TableAddress = 100h + GrTabPtr

⁶⁰⁾ The contents of the GrOT itself except the Length is not used by the Ctrl-Mode, but the general structure is needed for calculation of the absolute length of the GrOT.

GrObjTabPtr

- Format
Single octet value
- Location
Address 112h
- Value
Valid values are within the range 00h and FFh.
Due to the location of other Resources (Group Address Table, EEPROM Checksum) this value can be only between 1Ah and FEh.
- Usage by Management Server
The Management Server takes the value given at this location incremented with 100h to find the location of the GrOT – Easy 1.
- Usage by Management Client
The Management Client shall **read only** the value **to find out** the location of the GrOT – Easy 1.

4.18.7.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and mask 0012h the following applies.

Before memory mapped access to a device, the E-Mode Controller shall set the error flags to 00h to stop the application. A device shall reject Link Services sending a negative response (start_index = 0 and no group_address_list) if error flags are set to 00h, not depending on communication mode (error flags definition is given in [22]).

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

4.18.7.4 Usage by the Management Server (Device)

Please refer to 4.18.3.3 “Usage by the Management Server (Device)”.

4.18.7.5 Usage by the Management Client

4.18.7.5.1 Remote Access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to get the pointer of the GrOT – Easy 1 ⁶¹⁾.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Table Pointer
DMP_MemRead_RCo(0112h, 0112h, GrTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
;      Calculate the absolute pointer to Group Object Table Format 1:
;      ⇒      AbsGrTabPtr = 0100h +GrTabPtr
```

4.18.7.5.2 Local Access

Not applicable.

4.18.7.5.3 Internal Access

Not applicable.

4.18.8 Group Object Table – E-Mode Realisation Type 2 (GrOT – Easy 2)

Used by: Ctrl-Mode reloc. DMA
 and mask 0020h, 0021h in E-Mode

4.18.8.1 Format

The format is identical to the “Group Object Table – E-Mode Realisation Type 1” as specified in clause 4.18.7.1 “Format” above.

4.18.8.2 Location

The location of the GrOT – Easy 2 shall be indicated by the pointer PID_TABLE_REFERENCE (PID = 7) of the Application Program Object. The object_index of the Application Program Object shall be 3. (Please refer to [10] Appendix 5 “Controller Mode download Procedures”).

The property PID_TABLE_REFERENCE shall be a read only Property.

4.18.8.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and masks 0020h and 0021h the following applies.

A device receiving Link Services when Group Address Table, Group Object Association Table, Group Object Table and Application load states are different from *loaded* shall reject these services sending a negative response (start_index = 0 and no group_address_list), not depending on the communication mode.

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

⁶¹⁾ The use of this procedure may be integrated into the calculation of the memory reference for easy parameter handling, as specified in clause 4.13.2.4.1 “Remote accessRemote access”.

4.18.8.4 Usage by the Management Server (Device)

Please refer to clause 4.18.4.3.

4.18.8.5 Usage by the Management Client

4.18.8.5.1 Remote Access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any Management Client to get the pointer of the GrOT – Easy 2 ⁶²⁾.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Table Pointer
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.18.8.5.2 Local Access

Not applicable.

4.18.8.5.3 Internal Access

Not applicable.

4.18.9 Group Object Table – E-Mode Realisation Type 3 (GrOT – Easy 3)

Used by: Easy Ctrl reloc. DMA
and mask 0701h in E-Mode

4.18.9.1 Format

The Group Object Table shall have the following format

	Address	Group Objects Table	Descriptor nr
1 octet	TableAddress	Current Size	
2 octets	TableAddress + 1	RAM-Flags-Table Pointer	
4 octets	TableAddress + 3	Group Object Descriptor	0
4 octets	TableAddress + 7	Group Object Descriptor	1
4 octets	TableAddress + 11	Group Object Descriptor	2
...	...		...

The current size shall indicate the number of Group Objects in the GrOT.

4.18.9.2 Location

The location of the GrOT – Easy 3 shall be indicated by the pointer PID_TABLE_REFERENCE (PID = 7) of the Application Program Object. The object_index of the Application Program Object shall be 3. (please refer to [10] Appendix 5 “Ctrl-Mode download procedures”).

⁶²⁾ The use of this procedure may be integrated into the calculation of the memory reference for easy parameter handling, see 4.13.3.4.1.

This Property shall point to the base address of the Group Object Table. This mandatory Property shall be set by the device. The Property `PID_TABLE_REFERENCE` shall be a read only Property. The address range shall be limited to 64k.

4.18.9.3 Resource data integrity in case of dual access by Link Services and memory mapped access

Please refer to the general requirements in 2.3.4.

In particular for the E-Mode Controller and masks 0701h the following applies.

A device receiving Link Services when Group Address Table, Group Object Association Table, Group Object Table and Application load states are different from *loaded* shall reject these services sending a negative response (`start_index = 0` and no `group_address_list`), not depending on the communication mode.

An E-Mode Controller receiving rejected Link Services shall indicate in a proper way to the installer that the download procedure has failed.

4.18.9.4 Usage by the Management Server (Device)

This shall be identical as the usage by the Management Server for the Group Object Table - Realisation Type 2 as specified in 4.18.4.3.

4.18.9.5 Usage by the Management Client

4.18.9.5.1 Remote Access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any management client to get the pointer of the GrOT – Easy 3⁶³⁾.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Table Pointer
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrTabPtr)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.18.9.5.2 Local Access

Not applicable.

4.18.9.5.3 Internal Access

Not applicable.

⁶³⁾ The use of this procedure may be integrated into the calculation of the memory reference for easy parameter handling, see 4.13.3.4.1.

4.19 Easy Parameter Block Table (PaBT)

4.19.1 Abstract Resource definition

Please refer to the [10] clause Appendix 6 “Structure for Parameters”/

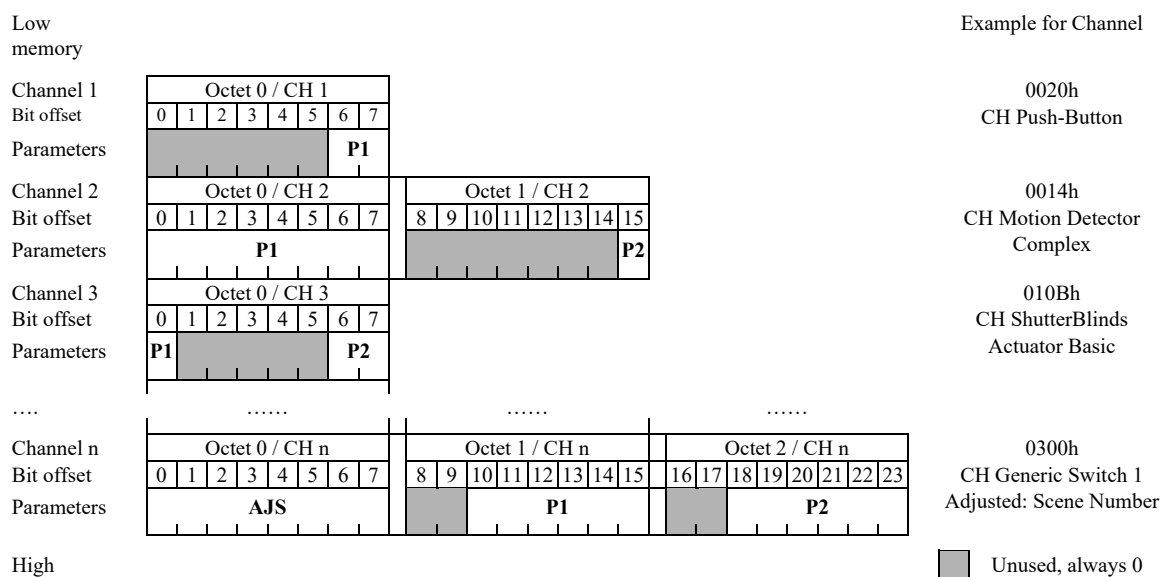
4.19.2 Parameter Block Table – E-Mode Realisation Type 1 (PaBT – Easy 1)

Used by: Ctrl-Mode fixed DMA
and mask 0012h in E-Mode

4.19.2.1 Format

Please refer to [10] Appendix 6 “Structures for Parameters”.

Format – Example (Informative)



4.19.2.2 Location

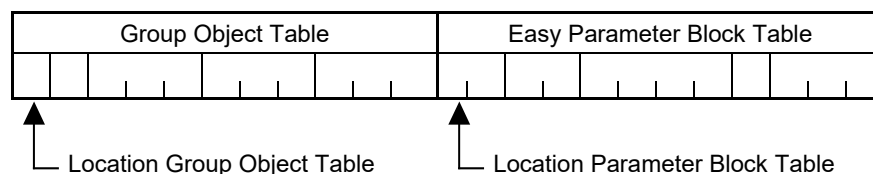
The location of the PaBT – Easy 1 is given by the value contained in the Resource “Group Object Table Pointer” (GrTabPtr) incremented by an offset of 100h and the Group Object Table “Length”. The location is calculated based on these two Resources.

4.19.2.2.1 GrObjTabPtr

The pointer is as specified in clause 4.18.7.2 “Location above.

4.19.2.2.2 Parameter Block location

The PaBT – Easy 1 is located directly behind the Group Object Table, as specified in [10] Appendix 5 “Controller Mode download procedures”.



4.19.2.2.3 Parameter Block calculation

$\text{ParaTabPtr} = 100\text{h} + \text{GrObjTabPtr} + 3 * \text{GrOT_Length} + 2$

4.19.2.3 Usage by the Management Server (device)

The Easy Parameter Block Table is used by the application running in the device in accordance to the supported E-Mode channels and the E-Mode Channel Code specification (please refer to [19] in the various Chapters with E-Mode Channel definitions). The table entries are normally read by the Application but in some cases depending on the supported E-Mode Channel the table entries are also written by the application (e.g. scene controller).

4.19.2.4 Usage by the Management Client

4.19.2.4.1 Remote access

The following Configuration Procedure specifies the standard sequence of Management Procedures that shall be followed by any Management Client to read and to write the PaBT – Easy 1.

4.19.2.4.1.1 Read PaBT – Easy 1

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Table Pointer
DMP_MemRead_RCo(0112h, 0112h, GrTabPtr)
;      Calculate the absolute pointer to Group Object Table Format 1
;      ⇒ AbsGrTabPtr = 0100h +GrTabPtr
;      Get Group Object Table Length
DMP_MemRead_RCo(AbsGrTabPtr, AbsGrTabPtr, GrOT_Length)
;      Calculate the absolute pointer to Parameter Block Table Easy Format 1
;      ⇒ AbsParaTabPtr = AbsGrTabPtr + 3*GrOT_Length +2
;      Read the Parameter Block Table Easy Format 1 64)
;      Note that this management procedure foresees the optional splitting up of the data in blocks of 12 octets.
DMP_MemRead_RCo(AbsParaTabPtr, AbsParaTabPtr + AbsParaTabLength 65) -1, PaBT)
;      Disconnect from the device
DMP_Disconnect_RCo
```

⁶⁴⁾ The Easy Parameter Block Table can be read entirely or a single parameter value can be read from the device. In both cases, interpretation of a parameter in the complete read sequence or fetching a single parameter the position of the single parameter inside a parameter block shall be calculated as bit offset according the easy Channel code definition ([16] in the various Chapters with E-Mode Channel definitions) and [09] Appendix 5 “Controller Mode download procedures”.

⁶⁵⁾ The parameter AbsParaTabLength is the sum of the parameter blocks of all channels in the device, counted in octets.

4.19.2.4.1.2 Write a Parameter ⁶⁶⁾ to PaBT – Easy 1

It shall be checked on beforehand by the Management Server if the writing to the Parameter Table does not conflict with other Resources ⁶⁷⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Table Pointer 68)
DMP_MemRead_RCo(0112h, 0112h, GrTabPtr)
;      Calculate the absolute pointer to Group Object Table Format 1
;      ⇒ AbsGrTabPtr = 0100h + GrTabPtr
;      Get Group Object Table Length 68)
DMP_MemRead_RCo(AbsGrTabPtr, AbsGrTabPtr, GrOT_Length)
;      Calculate the absolute pointer to PaBT – Easy 1 68)
;      ⇒ AbsParaTabPtr = AbsGrTabPtr + 3*GrOT_Length + 2
;      Calculate the absolute parameter block location of the parameter block, where the parameter is in 69)
;      ⇒ AbsParaBlkAdr = AbsParaTabPtr + sum (previous channels in the device)
;      Calculate the absolute parameter location using the bitoffset according
;      the Easy Channel code definition 70)
;      ⇒ AbsParaAdr = AbsParaBlkPtr + bitoffset/8
;      Read the parameter value from the PaBT – Easy 1 71)
;      Note that this management procedure foresees the optional splitting up
;      of the data in blocks of 12 octets.
DMP_MemRead_RCo(AbsParaAdr, AbsParaAdr + ParaLength - 1, ParaValue)
;      Mask the new parameter value to ParaValue, not affecting unused bits 71).
;      Set Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, 00h)
;      Write the parameter value to the PaBT – Easy 1
;      Note that this management procedure foresees the optional splitting up
;      of the data in blocks of 12 octets.
DMP_MemWrite_RCo(AbsParaAdr, AbsParaAdr + ParaLength 72) - 1, ParaValue)
;      Clear Runtime Error Flags
DMP_MemWrite_RCo(010Dh, 010Dh, FFh)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.19.2.4.2 Local Access

Not applicable.

4.19.2.4.3 Internal Access

Not applicable.

⁶⁶⁾ Parameters are typically written as a single action, depending on the installation procedures. This shall not prevent from writing the whole PaBT – Easy 1 or the parameters of a complete E-Mode channel in one step.

⁶⁷⁾ Example: setting of an adjustable parameter in most cases make the Group Address Table and the Group Association Table invalid.

⁶⁸⁾ Can be skipped if the pointer value is known from previous actions e.g. as specified in clause 4.13.2.4.1.1 “Read PaBT – Easy 1”.

⁶⁹⁾ Minimal Information for E-Mode Channels is the block length, see [16] in the various Chapters with E-Mode Channel definitions.

⁷⁰⁾ See [16] in the various Chapters with E-Mode Channel definitions.

⁷¹⁾ Can be skipped, if the parameter value starts and ends on an octet border (one or more complete octets).

⁷²⁾ The length of the single parameter depends on the parameter type, see [16] in the various Chapters with E-Mode Channel definitions.

4.19.3 Parameter Block Table – E-Mode Realisation Type 2 (PaBT – Easy 2)

Used by: Easy Ctrl reloc. DMA
and Mask 0020h, 0021h in E-Mode

4.19.3.1 Format

The format is as specified in clause 4.19.2.1 “Format” above.

4.19.3.2 Location

The location of the PaBT – Easy 2 is given by the value contained in the Resource “Group Object Table Pointer” (PID_TABLE_REFERENCE in Object 3) and the Group Object Table "Length". The location is calculated based on these two Resources

4.19.3.2.1 GrObjTabPtr

The pointer is as specified in clause 4.18.7.2 “Location” above.

4.19.3.2.2 Parameter Block location

The Parameter Block location is as specified in clause 4.19.2.2.2 “Parameter Block location” above.

4.19.3.2.3 Parameter Block calculation

$\text{ParaTabPtr} = \text{GrObjTabPtr} + 3 * \text{GrOT_Length} + 2$

4.19.3.3 Usage by the Management Server (Device)

The usage is as specified in clause 4.19.2.3 “Usage by the Management Server (device)” above.

4.19.3.4 Usage by the Management Client

4.19.3.4.1 Remote Access

The following Configuration Procedure specifies the standard sequence of Management Procedures that shall be followed by any Management Client to read and to write the PaBT – Easy 2.

4.19.3.4.1.1 Read PaBT – Easy 2

```

;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Table pointer
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrTabPtr)
;      Get Group Object Table Length
DMP_MemRead_RCo(GrTabPtr, GrTabPtr, GrOT_Length)
;      Calculate the absolute pointer to Parameter Block Table Easy Format 2:
;      ⇒ AbsParaTabPtr = GrTabPtr + 3*GrOT_Length +2
;      Read the Parameter Block Table Easy Format 1 73)
;      Note that this management procedure foresees the optional splitting up
;      of the data in blocks of 12 octets.
DMP_MemRead_RCo(AbsParaTabPtr, AbsParaTabPtr + AbsParaTabLength 74) -1, PaBT)
;      Disconnect from the device
DMP_Disconnect_RCo

```

4.19.3.4.1.2 Write a Parameter ⁷⁵⁾ to PaBT – Easy 2

It shall be checked on beforehand by the Management Server if the download of the Group Address Table does not conflict with other Resources ⁷⁶⁾. If this cannot be excluded, the download of the other Resources shall be integrated in the following procedure.

```

;      Connect to the device
DMP_Connect_RCo
;      Get Group Object Table pointer 77)
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_TABLE_REFERENCE, start_index = 1,
    nr_of_elem = 1, GrTabPtr)
;      Get Group Object Table Length 77)
DMP_MemRead_RCo(GrTabPtr, GrTabPtr, GrOT_Length)
;      Calculate the absolute pointer to Parameter Block Table Easy Format 2 77)
;      ⇒ AbsParaTabPtr = GrTabPtr + 3*GrOT_Length +2
;      Calculate the absolute parameter block location of the parameter block,
;      in which the parameter resides 78)
;      ⇒ AbsParaBlkAdr = AbsParaTabPtr + sum (previous channels in the device)
;      Calculate the absolute parameter location using the bitoffset
;      according the Channel Code definition 79)
;      ⇒ AbsParaAdr = AbsParaBlkPtr + bitoffset/8
;      Read the parameter value from the PaBT – Easy 2 80)

```

⁷³⁾ The Easy Parameter Block Table can be read entirely or a single parameter value can be read from the device. In both cases, interpretation of a parameter in the complete read sequence or fetching a single parameter the position of the single parameter inside a parameter block shall be calculated as bit offset according [09] Appendix 5 “Ctrl-Mode download procedures”.

⁷⁴⁾ The parameter AbsParaTabLength is the sum of the parameter blocks of all channels in the device, counted in octets.

⁷⁵⁾ Parameters are typically written as single action, depending on the installation procedures. This shall not prevent to write the whole PaBT or the parameters of a complete E-Mode Channel in one step.

⁷⁶⁾ Example: setting of an adjustable parameter in most cases make the Group Address Table and the Group Association Table invalid.

⁷⁷⁾ Can be skipped if the pointer value is known from previous actions e.g. as specified in clause 4.13.3.4.1.1 “Read PaBT – Easy 2”.

⁷⁸⁾ Minimal Information for E-Mode channels is the block length; please refer to [16] in the various Chapters with E-Mode Channel definitions.

⁷⁹⁾ Please refer to [16] in the various Chapters with E-Mode Channel definitions.

```
; Note that this management procedure foresees the optional splitting up
; of the data in blocks of 12 octets.
DMP_MemRead_RCo(AbsParaAdr, AbsParaAdr + ParaLength 81) -1, ParaValue)
; Mask the new parameter value to ParaValue, not affecting unused bits 80)
; Set Application Program load state machine to "Loading" 82).
DMP_LoadStateMachineWrite_RCo_IO(object_index = 03h, PID = =PID_LOAD_STATE_CONTROL, start_index = 1,
nr_of_elem = 1, Load)
; Write the parameter value to the PaBT – Easy 2
; Note that this management procedure foresees the optional splitting up
; of the data in blocks of 12 octets.
DMP_MemWrite_RCo(AbsParaAdr, AbsParaAdr + ParaLength -1, ParaValue)
; Set Application Program load state machine to "Loaded".
DMP_LoadStateMachineWrite_RCo_IO(object_index = 03h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
nr_of_elem = 1, LoadCompleted)
; Disconnect from the device
DMP_Disconnect_Rco
```

4.19.3.4.2 Local Access

Not applicable.

4.19.3.4.3 Internal Access

Not applicable.

⁸⁰⁾ Can be skipped, if the parameter value starts and ends on an octet border (one or more complete octets).

⁸¹⁾ The length of the single parameter depends on the parameter type; please refer to the KNX Handbook Supplement 12 “Channel Codes”

⁸²⁾ It is recommended to check the load state via DMP_LoadStateMachineRead_RCo before setting a new state. If the previous state is "Loading" it's the decision of the client to handle this state. All other states except "Loaded" shall stop the procedure

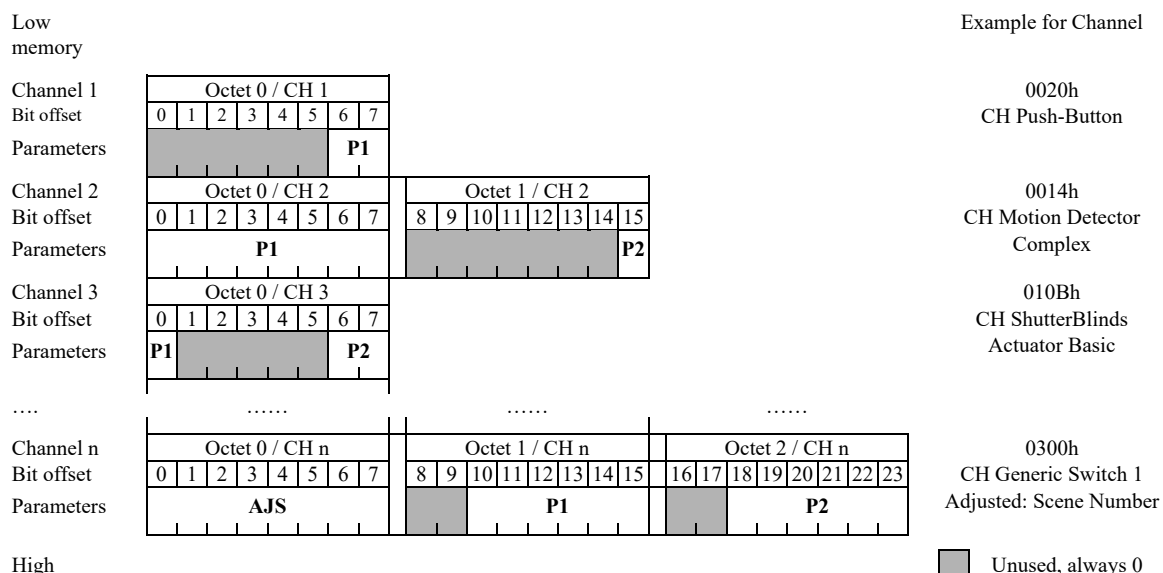
4.19.4 Parameter Block Table – E-Mode Realisation Type 3 (PaBT – Easy 3)

Used by: Easy Ctrl reloc. DMA
and mask 0701h in E-Mode

4.19.4.1 Format

Please refer to [10] “Appendix 6: Structures for Parameters”.

Format – Example (Informative)



4.19.4.2 Location

The location of the PaBT – Easy 3 shall be indicated by the pointer `PID_PARAM_REFERENCE` (`PID = 51`) of the Application Object. This Property shall point to base address of the Parameter Block. The `object_index` of the Application Object shall be 3.

The Property `PID_PARAM_REFERENCE` shall be a read only Property. This mandatory property shall be set by the device; it shall be of the type `PDT_UNSIGNED_INT` (16 bit) and shall be read only. The address range shall be limited to 64k.

4.19.4.3 Usage by the Management Server (Device)

The Easy Parameter Block Table shall be used by the application running in the device in accordance to the supported E-Mode Channels and the E-Mode Channel Code specification (please refer to [19] in the various Chapters specifying the E-Mode Channels). The table entries are normally read by the application but in some cases depending on the supported E-Mode Channel the table entries are also written by the application (e.g. scene controller).

4.19.4.4 Usage by the Management Client

4.19.4.4.1 Remote Access

The following Configuration Procedure specifies the standardized sequence of Management Procedures that shall be followed by any management client to read and to write the PaBT – Easy 3.

4.19.4.4.1.1 Read PaBT – Easy 3

```
;      Connect to the device
DMP_Connect_RCo
;      Get Parameter Block Pointer
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_PARAM_REFERENCE, start_index = 1,
    nr_of_elem = 1, PaBTPtr)
;      Read the Parameter Block Table E-Mode Format 1 83)
;      Note that this Management Procedure foresees the optional splitting up
;      of the data in blocks of 12 octets.
DMP_MemRead_RCo(PaBTPtr, PaBTPtr + AbsParaTabLength 84) -1, PaBT)
;      Disconnect from the device
DMP_Disconnect_RCo
```

4.19.4.4.1.2 Write a Parameter ⁸⁵⁾ to PaBT – Easy 3

It shall be checked on beforehand by the Management Server if the download of the Group Address Table does not conflict with other Resources ⁸⁶⁾. If this can not be excluded, the download of the other Resources shall be integrated in the following procedure.

```
;      Connect to the device
DMP_Connect_RCo
;      Get Parameter Block Pointer 87)
DMP_InterfaceObjectRead_R(object_index = 03h, PID = PID_PARAM_REFERENCE, start_index = 1,
    nr_of_elem = 1, PaBTPtr)
;      Calculate the absolute parameter block location of the parameter block,
;      in which the parameter resides 88)
;      ⇒ AbsParaBlkAdr = PaBTPtr + sum (size of previous channel parameters in the device)
;      Calculate the absolute parameter location using the bit offset
;      according the E-Mode Channel Code definition 89)
;      ⇒ AbsParaAdr = AbsParaBlkPtr + bitoffset/8
```

⁸³⁾ The E-Mode Parameter Block Table can be read entirely or a single parameter value can be read from the device. In both cases, interpretation of a parameter in the complete read sequence or fetching a single parameter the position of the single parameter inside a parameter block shall be calculated as bit offset according [09] clause “Appendix 5: Ctrl-Mode download procedures”.

⁸⁴⁾ The parameter AbsParaTabLength shall be the sum of the parameter blocks of all E-Mode Channels in the device, counted in octets.

⁸⁵⁾ Parameters are typically written as single action, depending on the installation procedures. This shall not prevent to write the whole PaBT or the parameters of a complete channel in one step.

⁸⁶⁾ Example: setting of an adjustable parameter in most cases make the Group Address Table and the Group Association Table invalid.

⁸⁷⁾ Can be skipped if the pointer value is known from previous actions e.g. as specified in clause 4.11.8.5.1.2.

⁸⁸⁾ Minimal Information for E-Mode Channels is the block length; please refer [16] in the various Chapters specifying the E-Mode Channels.

⁸⁹⁾ Please refer [16] in the various Chapters specifying the E-Mode Channels.

```

;   Read the parameter value from the PaBT – Easy 3 90)
;   Note that this Management Procedure foresees the optional splitting up
;   of the data in blocks of 12 octets.
DMP_MemRead_RCo(AbsParaAdr, AbsParaAdr + ParaLength 91) -1, ParaValue)
;   Mask the new parameter value to ParaValue, not affecting unused bits.
;   Set Application Program load state machine to "Loading" 92).
DMP_LoadStateMachineWrite_RCo_IO(object_index = 03h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
nr_of_elem = 1, Load)
;   Write the parameter value to the PaBT – Easy 3
;   Note that this Management Procedure foresees the optional splitting up
;   of the data in blocks of 12 octets.
DMP_MemWrite_RCo(AbsParaAdr, AbsParaAdr + ParaLength -1, ParaValue)
;   Set Application Program load state machine to "Loaded".
DMP_LoadStateMachineWrite_RCo_IO(object_index = 03h, PID = PID_LOAD_STATE_CONTROL, start_index = 1,
nr_of_elem = 1, LoadCompleted)
;   Disconnect from the device
DMP_Disconnect_Rco

```

4.19.4.4.2 Local Access

Not applicable.

4.19.4.4.3 Internal Access

Not applicable.

4.20 Application Program

4.20.1 Abstract Resource Definition

The Application Program Object shall contain global information about the internal user Application Program. It shall provide management operations for the internal user Application Program.

4.20.2 Application Program – Realisation Type 6

Used by: System 300

4.20.2.1 Location

The Application Program – Realisation Type 1 shall be implemented as Interface Object of the object type Application Program Object (Object Type = 0003h).

4.20.2.2 Format

The Application Program – Realisation Type 1 shall consist of one Interface Object of the object type Application Program Object (Object Type = 0003h) and shall have the Properties as specified in [17]. It shall provide the management operations for downloading.

All application specific parameters of the Application Program shall be set in application specific Interface Objects with the service A_PropertyValue_Write (See Application Interworking Standards in [19]).

⁹⁰⁾ Can be skipped, if the parameter value starts and ends on an octet border (one or more complete octets).

⁹¹⁾ The length of the single parameter depends on the parameter type; please refer [16] in the various Chapters specifying the E-Mode Channels.

⁹²⁾ It is recommended to check the load state via DMP_LoadStateMachineRead_RCo before setting a new state. If the previous state is "Loading" it's the decision of the client to handle this state. All other states except "Loaded" shall stop the procedure.

Table 89 - Application program Interface Object

Property Name	Property Identifier	Type	Description
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT	Application Object 3
Interface Object Name	2 = PID_OBJECT_NAME	PDT_UNSIGNED_CHAR[]	Name of the Application program
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL	See description below
Run Control	6 = PID_RUN_STATE_CONTROL	PDT_CONTROL	for further Information see Run / State machines
Application Version	13= PID_PROGRAM_VERSION	PDT_GENERIC_05	Version of Application Program
PEI Type	16= PID_PEI_TYPE	PDT_UNSIGNED_CHAR	Required PEI-Type for User

4.20.2.2.1 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This Property shall contain the Object Type of the Interface Object. The Applicationprogram Object shall have Object Type 0003h.

4.20.2.2.2 PID_OBJECT_NAME (PID = 2)

Please refer to 4.2.2 for the general requirements for PID_OBJECT_NAME.

This Property shall contain the name of the Application Program.

4.20.2.2.3 PID_LOAD_STATE_CONTROL (PID = 5)

Please refer to 4.2.5 for the general requirements for PID_LOAD_STATE_CONTROL.

4.20.2.2.3.1 General

The sequence is identical to the sequence as specified in 4.16.7.4 “Usage by the Management Client”.

4.20.2.2.3.2 Remote Access

The following Load Controls shall be supported for the management of the Application Program – Realisation Type 6.

Load Control	Subtype	Description	Supported
00h		No operation	M
01h		Start Loading	M
02h		Load Completed	M
04h		Unload	M

4.20.3 Application Program – Realisation Type 7

Used by: System B

4.20.3.1 Use

The Application Program – Realisation Type 7 is named “Application Program 1” in System B. It shall contain global information about the internal user Application Program 1. It shall provide management operations for the internal user Application Program 1.

4.20.3.2 Location

The Application Program 1 Object shall be implemented as Interface Object of the object type *Applicationprogram Object* (Object Type = 0003h).

4.20.3.3 Format

4.20.3.3.1 Overview

Table 90 - Application Program 1 Interface Object

Property Name	Property Identifier	PDT/DPT	Description
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT	Applicationprogram Object
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL	See description below
Run Control	6 = PID_RUN_STATE_CONTROL	PDT_CONTROL	See 4.24.2 “Run State Machine – Realisation Type 1 (Property based)”
Application Version	13 = PID_PROGRAM_VERSION	PDT_GENERIC_05	Version of Application Program 1
PEI-Type	16 = PID_PEI_TYPE	PDT_UNSIGNED_CHAR	Required PEI-Type for User
Table Reference	07 = PID_TABLE_REFERENCE	PDT_UNSIGNED_LONG	Pointer to Application Program 1 Base Address
Memory Control Table	27 = PID_MCB_TABLE	PDT_GENERIC_08[]	Subsegmentation of memory space and checksum
Error code	28 = PID_ERROR_CODE	PDT_ENUM8	Specifies the reason for Load State “ERROR”

4.20.3.3.2 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This shall be the Object Type of the Interface Object (Application Program 1 Object = 0003h). This property is always mandatory and shall always be readable.

4.20.3.3.3 PID_LOAD_STATE_CONTROL (PID = 5)

Please refer to clause 4.2.5 for the specification of PID_LOAD_STATE_CONTROL and to clause 4.23 for the specification of the Load State Machine.

4.20.3.3.4 PID_RUN_STATE_CONTROL (PID = 6)

Please refer to clause 4.2.6 for the specification of PID_RUN_STATE_CONTROL and to clause 4.24.2 for the specification of the Run State Machine.

The Property Run Control shall control the Application Program 1 of the device.

The minimum implemented possible states and the run state machine define state changes of an executable part.

4.20.3.3.5 PID_PEI_TYPE (PID = 16)

Please refer to clause 4.2.16 for the specification of PID_PEI_TYPE.

4.20.3.3.6 PID_TABLE_REFERENCE (PID = 07)

Please refer to clause 4.2.7 for the specification of PID_TABLE_REFERENCE.

When the Application Program 1 object is *Unloaded* then the Management Server shall set the value to zero. When the Management Client has successfully allocated memory then the value shall be set to the absolute memory location from where the table will be downloaded. When memory allocation is not successful then the value shall be set to zero.

4.20.3.3.7 PID_MCB_TABLE (PID = 27)

Please refer to clause 4.2.27 for the Interface Object Type independent specification of PID_MCB_TABLE.

This optional Property shall divide the segment into multiple subsegments with access rights definable for each subsegment and carrying the checksum for the subsegments.

The use case for this subsegmentation is to separate code and parameter sections. A code section is normally protected with the highest access level; a parameter section with a lower access level. The maximum length of this table (nr. of elements) shall be fixed within the device and shall be read with an A_PropertyDescription_Read. The minimum length is '1'.

For the specification of the Memory Control Block Table, please refer to 4.2.27 (Table 12).

For the specification of the CRC Control Byte please refer to clause 4.2.27.1.1.

4.20.3.3.8 PID_ERROR_CODE (PID = 28)

Please refer to clause 4.2.28.

4.21 Application Program 2

4.21.2 Application Program 2 – Realisation Type 7

Used by: System B

4.21.2.1 Use

The *Application Program 2* shall contain global information about the internal Application Program 2. It shall provide management operations for the internal Application Program 2.

4.21.2.2 Location

The Application Program 2 Object shall be implemented as Interface Object of the object type *Interfaceprogram Object* (Object Type = 0004).

4.21.2.3 Format

4.21.2.3.1 Overview

Table 91 - Application Program 2 Interface Object

Property Name	Property Identifier	PDT/DPT	Description
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT	Interfaceprogram Object
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL	See description below
Run Control	6 = PID_RUN_STATE_CONTROL	PDT_CONTROL	for further Information see Run / State machines
Application Version	13 = PID_PROGRAM_VERSION	PDT_GENERIC_05	Version of Application Program
PEI-Type Required	16 = PID_PEI_TYPE	PDT_UNSIGNED_CHAR	Required PEI-Type for User
Pointer to base address Format 1	07 = PID_TABLE_REFERENCE	PDT_UNSIGNED_LONG	Pointer to address of Application Program 2 base
Memory Control Table	27 = PID_MCB_TABLE	PDT_GENERIC_08[]	Sub-Segmentation of memory space and check sum
Error code	28 = PID_ERROR_CODE	PDT_ENUM8	Specifies the reason for load state "ERROR"

4.21.2.4 PID_OBJECT_TYPE (PID = 1)

Please refer to 4.2.1 for the general requirements for PID_OBJECT_TYPE.

This shall be the Object Type of the Interface Object (Application Program 2 Object = 0004h). This Property is always mandatory and shall always be readable.

4.21.2.5 PID_LOAD_STATE_CONTROL (PID = 5)

Please refer to clause 4.2.5 for the specification of PID_LOAD_STATE_CONTROL and to clause 4.23 for the specification of the Load State Machine.

4.21.2.6 PID_RUN_STATE_CONTROL (PID = 6)

Please refer to clause 4.2.6 for the specification of PID_RUN_STATE_CONTROL and to clause 4.24.2 for the specification of the Run State Machine.

The transitions between the 'Ready' and the 'Running' states shall always be made automatically by the system depending on the run conditions. The states in the device that have to be reached to fulfil the run conditions depend upon the application and are, therefore, not standardized. For example it could be possible that the same or different run conditions apply for application program 1 and application program 2. It is manufacturer's responsibility to take care that the device cannot reach a critical state when the application program is started.

For diagnostic purposes, it shall be possible to start and stop Application Program 2 (if the run conditions are fulfilled).

4.21.2.7 PID_TABLE_REFERENCE (PID = 07)

Please refer to clause 4.2.7 for the specification of PID_TABLE_REFERENCE.

When the Application Program 2 object is Unloaded then the Management Server shall set the value to zero. When the Management Client has successfully allocated memory then the value shall be set to the absolute memory location from where the table will be downloaded. When memory allocation is not successful then the value shall be set to zero.

4.21.2.8 PID_MCB_TABLE (PID = 27)

Please refer to clause 4.2.27 for the Interface Object Type independent specification of PID_MCB_TABLE.

This optional Property shall divide the segment into multiple subsegments with access rights definable for each subsegment and carrying the checksum for the subsegments.

The use case for this subsegmentation is to separate code and parameter sections. A code section is normally protected with the highest access-level a parameter section with a lower access-level. The maximal length of this table (nr. of elements) shall be fixed within the device and shall be read with an A_PropertyDescription_Read. The minimum length is '1'.

For the specification of the Memory Control Block Table, please refer to 4.2.27 (Table 12).

For the specification of the CRC Control Byte please refer to clause 4.2.27.1.1.

4.21.2.9 PID_ERROR_CODE (PID = 28)

Please refer to clause 4.2.28.

4.22 KNX Serial Number

4.22.1 Abstract Resource Definition

4.22.1.1 Abstract Resource definition

The KNX Serial Number shall be a unique identifier for the device.

Whether or not a device shall support a KNX Serial Number can be derived from [17].

The owner of the micro controller shall ensure the global uniqueness of the leading four octets within the specific manufacturer's code space.

4.22.1.2 Format

Regardless to the Realisation Type, the KNX Serial Number shall be encoded according the specification of DPT_SerNum (DPT_ID = 221.001; see [14]). The Property Datapoint Type shall be PDT_GENERIC_06.

This is illustrated by the frame examples underneath ⁹³⁾.

6	Octet 7								Octet 8								Octet 9								Octet 10								Octet 11								Octet 12								Octet 13															
°)									Manufacturer code								Manufacturer code								Number incremented with each BAU																																							
1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0															
APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																																								
1	1	1	1	1	0	1	1	1	0	0	m	m	m	m	m	m	m	m	m	m	m	m	m	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n	n															

⁹³⁾ last two bits of octet 6

Figure 61 - A_IndividualAddress_SerialNumber_Read-PDU (example)

⁹³⁾ The indicated octet numbers are valid for TP1.

Octet 6						Octet 7						Octet 8						Octet 9						Octet 10						Octet 11																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
						APCI						Object_index						Property_id						nr_of_ele m		start_index																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
						1	1	1	1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 62 - A_PropertyDescription_Response-PDU (example)

4.22.1.3 Usage by the Management Server (device) – general

The support of the KNX Serial Number may encompass the support of the following features:

- KNX Serial Number Realisation Type 1 (memory mapped; see 4.22.2), and/or
- KNX Serial Number Realisation Type 2 (Property; see 4.22.3), and/or
- the AL-services A_IndividualAddress-SerialNumber_Read and A_IndividualAddressSerialNumber_Write

However, these features cannot be combined in any combination. Instead, the following rules apply.

1. If Interface Objects (reduced or full) are supported and the AL services A_IndividualAddress-SerialNumber_Read and A_IndividualAddressSerialNumber_Write are supported too, then the Property PID_SERIAL_NUMBER shall be present in the device.
2. If Interface Objects (reduced or full) are supported and the Property PID_SERIAL_NUMBER is present then the AL services A_IndividualAddress-SerialNumber_Read and A_IndividualAddressSerialNumber_Write shall be supported too.
3. Devices with memory mapped KNX Serial Number may support the AL services A_Individual-AddressSerialNumber_Read and A_IndividualAddressSerialNumber_Write without the support of the Property PID_SERIAL_NUMBER. These devices have no Interface Object server.

The allowed combinations are specified for each relevant Profile in [17]. If no explicit definition is available then these default rules shall apply.

4.22.2 KNX Serial Number - Realisation Type 1

4.22.2.1 Format

Please refer to 4.22.1.2.

4.22.2.2 Location

In this KNX Serial Number – Realisation Type 1, the KNX Serial Number is exclusively accessible via accessible via the Application Layer services A_IndividualAddressSerialNumber_Read and A_IndividualAddressSerialNumber_Write. It is not accessible via memory mapped access, via a Property in an Interface Object or any other.

4.22.3 KNX Serial Number - Realisation Type 2

4.22.3.1 Format

Please refer to 4.22.1.2.

4.22.3.2 Location

For the Realisation Type 2, the KNX Serial Number is realised as PID_SERIAL_NUMBER; please refer to 4.2.11.

4.23 Load State Machine

4.23.1 Abstract Resource definition

The Load State Machine shall be able to manage all kinds of downloadable configuration data including executable code. More than one Load State Machine is possible in one device.

A Load State Machine for executable code is possibly combined with a Run State Machine. The state of the Load State Machine shall be stored in non-volatile memory (e.g. EEPROM, flash).

4.23.2 Load State Machine – Realisation Type 1 (Property based)

4.23.2.1 Format

Realisation Type 1 for the Load State Machine shall be a Property of Property Datatype PDT_CONTROL. The format shall differ between the read access, as specified in Table 92 and write access, as specified in Table 93.

The transitions between the states shall be less than 30 seconds. This time shall be the minimum delay during which a MaC shall wait after sending a load event until interpretation of the load event with the expected state transition to be failed.

4.23.2.2 Location

The load procedure for device configuration data specifies a general mechanism for changing the different programmable parts of a device. Each programmable part shall have exactly one dedicated Load State Machine that controls the load process. This Load State Machine shall be accessed by an associated Interface Object with the Property Load Control (PID_LOAD_STATE_CONTROL, see also 4.2.5).

The Load Procedure for device configuration data specifies a general mechanism for changing the different loadable parts of a device. Each loadable part shall be represented by exactly one dedicated Interface Object. The Load State Machine of the loadable part shall be accessed exclusively through the dedicated Property PID_LOAD_STATE_CONTROL within that Interface Object.

The value of the Property Load Control shall be stored in non-volatile memory, because it shall be preserved also on power fail.

4.23.2.3 Usage by the Management Server (device)

4.23.2.3.1 States of the Load State Machine

Each Property Load Control shall provide the state of the Load State Machine by providing read access to the Property value. The load states (data read from the Property Load Control) shall be encoded as follows.

Table 92 - Load States

Load State	Value	M/O ^a	Remark
Unloaded	0	M	No data is loaded. The data that is associated with the Property Load Control is not valid (e.g.: Address Table is not valid).
Loaded	1	M	Valid data is loaded. Only in this state the associated data shall be considered as valid; in all other states the data shall be considered as invalid
Loading	2	M	Load process is active.
Error	3	M	Error in data detected or error during load process.
Unloading	4	O	Unload process is active
LoadCompleting	5	O	Intermediate state between Loading and Loaded, e.g. checksum calculation process is active
^a M: mandatory O: optional			

4.23.2.3.2 Events of the Load State Machine

There are two types of events that can cause a transition in the Load State Machine:

1. external events are write accesses to the Property Load Control by a Management Client, and
2. internal events in the Management Server.

The reaction of the Management Server is already partly specified under 4.23.2.1 “Format” above.

Illegal additional events (e.g. invalid values written to the Property) shall be ignored and shall not lead to a change of state.

The Load State Machine shall enter the state Error in case of an error. Unknown events shall be ignored.

The event *UnloadCompleted* is mostly only an internal event. Therefore, a Management Client may never observe the state Unloading.

The event *LoadCompletingCompleted* is only an internal event. The event LoadCompleted may cause the Load State Machine to change firstly to the intermediate state LoadCompleting and then, by the internal event LoadCompletingCompleted to state Loaded. For a Load State Machine supporting state LoadCompleting, a MaC may observe the state LoadCompleting.

If the Management Server (MaS) goes offline during the state LoadCompleting, the Load State Machine shall send state LoadCompleting at least once before going offline, i.e. with the A_PropertyValue_Response PDU sent as reaction to the received A_PropertyValue_Write PDU containing the load event LoadCompleted.

Write accesses to the Property Load Control shall be events to the Load State Machine. The encoding of the different Load Events (data written to the Property Load Control) shall be as specified in Table 93.

Table 93 - Load Events

Load Event	Value	Reaction of the Load State Machine
No Operation	00h	Nothing
Start Loading	01h	This shall be interpreted as the request to start the loading of the loadable part.
Load Completed	02h	This shall be interpreted as the request to complete the Loading of the loadable part if this event occurs during the state Loading. Possible checksum shall be calculated, all data shall be declared as valid and the Load State shall change to the state Loaded, if no internal errors occurs and no error in the data is detected. If the transition from Loading to Loaded takes more than 2 seconds, e.g. by complex checksum calculation, the Load State shall first change to intermediate state LoadCompleting and change to Loaded after calculation is finished.
Additional Load Controls	03h	This Load Event shall be used to transfer additional load information like memory allocation (absolute or relative), application entry points or special run conditions for an application. See clause 3.27 "DM_LoadStateMachineWrite" in [09].
Unload	04h	This Load Event shall be interpreted as the request to unload the loadable part. The loadable data shall be declared as invalid. The data is undefined.

4.23.2.3.3 Load State Machine

The state transition diagram as specified in Figure 63 shall be valid for all Load State Machines.

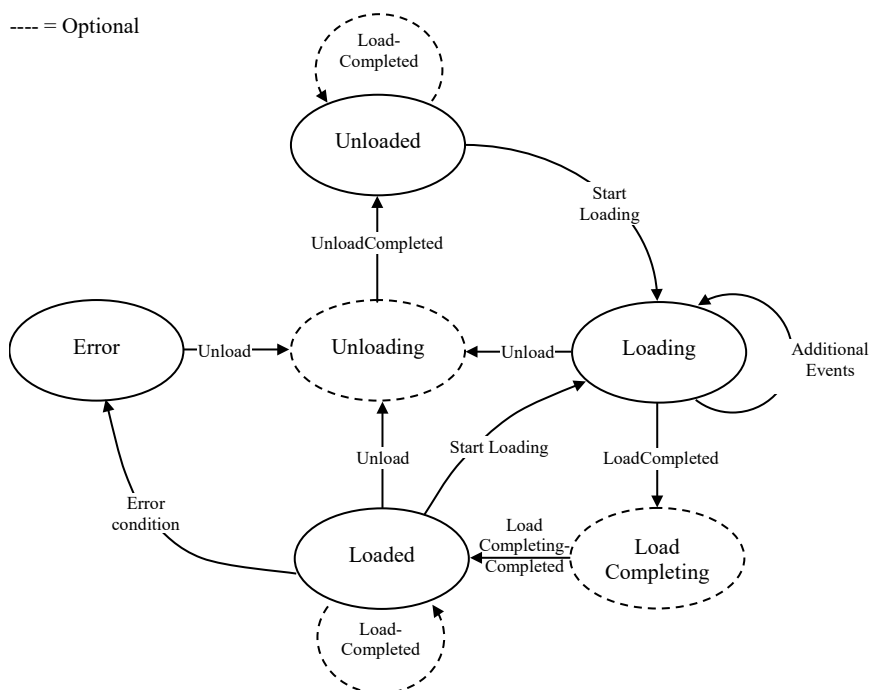


Figure 63 – Example Load State Machine

Table 94 - Load State Machine transition table

Load Event	Load State					
	Unloaded (00h)	Loaded (01h)	Loading (02h)	Error (03h)	Unloading (04h)	LoadCompleting (05h) ^{*)}
No Operation (00h)	Unloaded	Loaded	Loading	Error	Unloading	LoadCompleting
Start Loading (01h)	Loading	Loading	Loading	Error	Error	R: Error O: I: LoadCompleting →M: Loaded
Load Completed (02h)	R: Unloaded O: Error	R: Loaded O: Error	I: LoadCompleting →M: Loaded	Error	Error	R: Error O: I: LoadCompleting →M: Loaded
Additional /Segment Load Controls (03h)	R: Unloaded O: Error	Error	Loading	Error	Error	R: Error O: I: LoadCompleting →M: Loaded
Unload (04h)	Unloaded	I: Unloading →M: Unloaded	I: Unloading →M: Unloaded	I: Unloading →M: Unloaded	I: Unloading →M: Unloaded	R: I: Unloading →M: Unloaded O: I: LoadCompleting →M: Loaded
Device Restart	Unloaded	Loaded (Error in case of error detection at start-up)	R: Loading O: Error	Error	Unloaded	R: Unloaded O: I: LoadCompleting →M: Loaded
LEGEND: I: intermediate state M: mandatory O: optional R: recommended →: mandatory transition from a possible intermediate state to a final state *) Note: a device may be offline during state LoadCompleting and therefore not react to load events from a MaC. Transition from state LoadCompleting to state Loaded is an internal event. Therefore it may not be possible to provoke any state transition or a device restart from a MaC. Ignore load event: if the device may communicate, it shall return unchanged state LoadCompleting						

For events that are client errors, more than one transition is allowed; the recommended transitions should be implemented but the optional transitions may be implemented alternatively. For devices with a non-loadable application the Property Load Control shall be read-only and shall have the fix value Loaded. For transitions that need more than two seconds the intermediate state is mandatory.

4.23.2.4 Usage by the Management Client

4.23.2.4.1 Sequence of downloading data via a Load State Machine

The event UnloadCompleted is mostly only an internal event. Therefore, a Management Client may never observe the state Unloading.

The complete download of loadable data controlled by a Load State Machine (e.g. Address- or Association table) shall be done as specified in Figure 64.

If a device contains more than one Load State Machine, dependencies between these Load State Machines have to be defined in the Profiles (see [17]) of these devices.

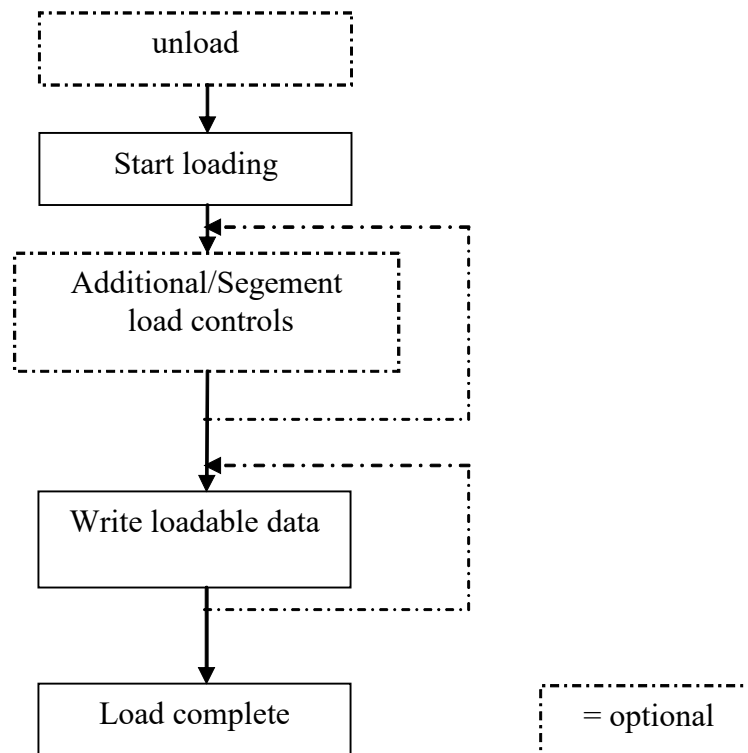


Figure 64 - Load Procedure

The event LoadCompletingCompleted is only an internal event.

If the MaC observes load state LoadCompleting, it shall

- control the load state by repeated reading of PID_LOAD_STATE_CONTROL and continue with further access only after load state has changed to LoadCompleted
- try to keep alive an established TL-connection. This is accomplished by the repeated reading of PID_LOAD_STATE_CONTROL, with a time period that is shorter than the TL-timeout. The period for reading shall not exceed half the TL-timeout, i.e. 3 seconds.

If the MaS responds during state LoadCompleting, an established TL-connection is kept alive; otherwise, the TL-connection will close after TL-timeout.

NOTE 86 A device may be offline during state LoadCompleting. A running TL-connection may be lost if the duration of state LoadCompleting exceeds the TL-timeout.

If a before established TL-connection breaks down, the MaC shall try to re-establish the connection periodically during the maximum transition time and once more when the maximum transition time has passed.

4.23.3 Load State Machine – Realisation Type 2 (Memory mapped)

This Realisation Type is not specified in this version of this document.

4.24 Run State Machine

4.24.1 Abstract Resource definition

The Run State Machine shall control the execution of an executable part of the device.

The executable part may be loaded via a Load State Machine or may be fixed in the device. If the executable part is controlled by a Management Client the Run State Machine is mandatory.

4.24.2 Run State Machine – Realisation Type 1 (Property based)

4.24.2.1 Format

Realisation Type 1 for the Run State Machine shall be a Property of Property Datatype PDT_CONTROL⁹⁴). The format shall differ between the read access, as specified in Table 95 and write access, as specified in Table 96.

For the minimum implementation the Run State Machine may be read-only. In this case the Run State shall show only the current state of the executable part. No external events can be transferred to the Run State Machine.

The transitions between the states shall be less than 30 seconds.

4.24.2.2 Location

This Realisation Type of the Run State Machine shall be accessed by an associated Interface Object with the Property Run Control (PID_RUN_STATE_CONTROL), see also 4.2.6.

EXAMPLE The Property Run Control (PID_RUN_STATE_CONTROL) shall be available for the Interface Object Application Program.

The implemented possible states of an executable part that is loadable via a Load State Machine are defined by the Run State Machine. The Run State shall be stored in volatile memory (e.g. RAM).

4.24.2.3 Usage by the Management Server (device)

4.24.2.3.1 States of the Run State Machine

The states of the minimum Run State Machine (data reading from the Property Run Control) shall be encoded as specified in Table 95.

⁹⁴) Existing implementations of mask 0020h and mask 0701h use PDT_UNSIGNED_CHAR. Any new implementation of these masks and any other Profile shall use PDT_CONTROL.

Table 95 - Run States

State	Value	M/O ^a	Remarks
Halted	0	M	The executable part shall be halted or not loaded. (There shall be an automatic transition to ready if the Program is loaded.)
Running	1	M	The executable part shall be running.
Ready	2	M	The executable part shall be ready for being executed, but not yet running. If the run-conditions are fulfilled the executable part shall start automatically.
Terminated	3	O	The executable part shall be terminated and shall no longer start automatically. It shall only start again through an event "Restart" to the Run State Machine or through a reset of the device.
Starting	4	M	The executable part shall be starting and not yet ready. This state is only mandatory if an executable part needs more than 2 seconds for start-up.
Shutting down	5	M	The executable part shall be shutting down. This state is only mandatory if the executable part needs more than 2 seconds for shutting down.
^a M: mandatory O: optional			

The state Terminated is optional.

4.24.2.3.2 Events of the Run State Machine

It shall be possible to start (if the run conditions are fulfilled) and stop the executable part for diagnostic purposes. The data for the Property Run Control shall be encoded as specified in Table 96.

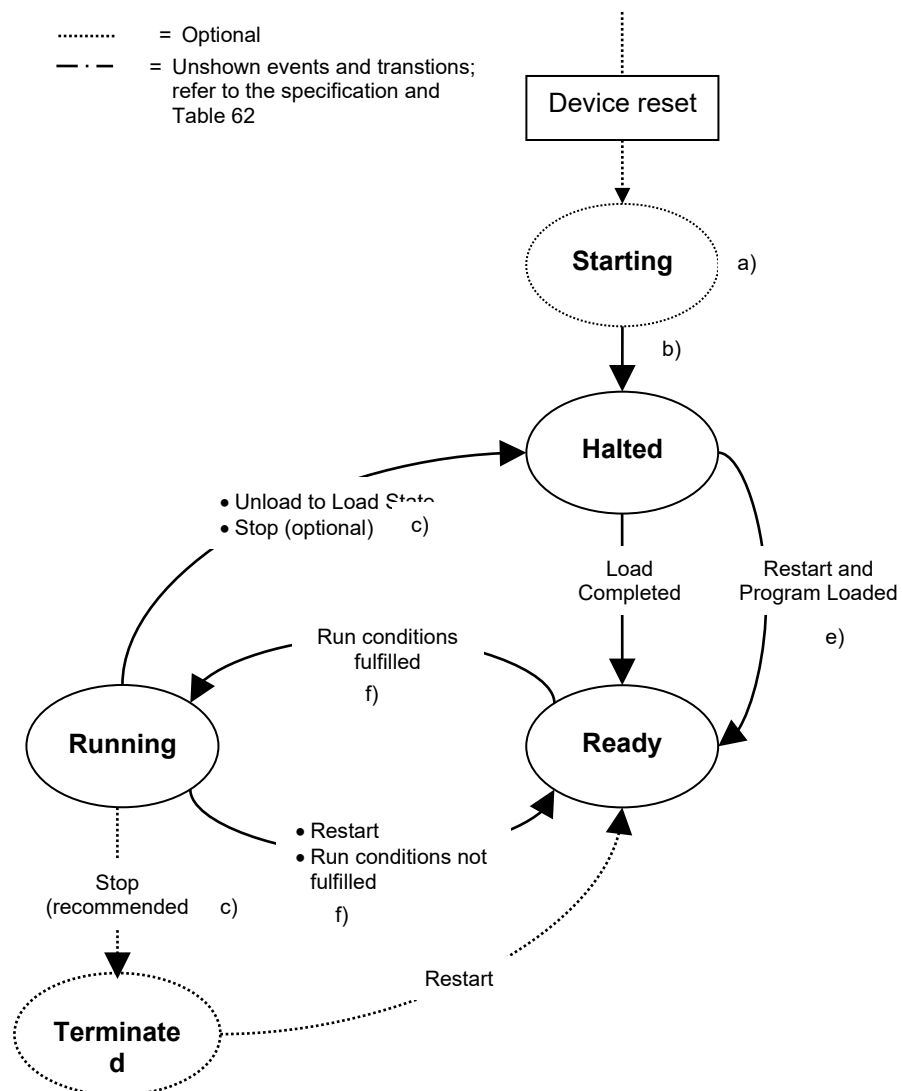
Table 96 - Run State Events (Writing to Run Control)

Run State Event	Value	Remarks
NOP	0	No operation.
Restart	1	Request to restart the executable part.
Stop	2	Request to stop the executable part.

All events for the Run State Machine are optional.

4.24.2.3.3 Run state machine

Below the state transition diagram for the Run State Machine of the executable part is drawn.



**Figure 65 - Minimum Run State Machine
(not all events and transitions are shown)**

- a) The states *Starting* and *Shutting Down* (not shown in the diagram) shall be read-only states. This means that during these states no state transition shall be done on an external event.
- b) The transition from *Starting* to *Halted* shall be done internally.
The transition to *Shutting Down* shall be device - and application specific.
- c) Even if not shown explicitly, the event *Stop* shall always lead to the state *Terminated*; this shall only be possible if the corresponding Load State Machine is in the state *Loaded*.
- d) After a reset the Run State Machine shall always start in the state *Halted*.
- e) Then the system shall automatically make the transition to the state *Ready* if the corresponding Load State Machine is in the state *Loaded*.
The automatic transition shall only be made at start-up. If the Run State Machine is manually stopped it shall not restart automatically.

The run state *Halted* shall only entered after start-up or if the corresponding Load State is not *Loaded*.

Starting and Shutting Down are intermediate states that may never appear.

- f) The transitions between the run states *Ready* and the *Running* shall always be made automatically by the system depending on the run conditions. The run conditions are hardware specific and/or application specific.

Examples of Run conditions:

- Correct PEI-type
- Correct checksum
- Hardware check OK
- Ordernumber OK
- The Load State Machine of the corresponding application segment has the Load State equal to “Loaded”.

No error state is defined for the Run State Machine. Unknown events shall be ignored.

Table 97 - Run State Machine transition table

Event	State					
	Halted (00h)	Running (01h)	Ready (02h)	Terminated(03h)	Starting (04h)	Shutting down (05h)
No Operation (00h)	Halted	Running	Ready	Terminated	I: Starting → M: Halted	I: Shutting down M: Halted
Restart and executable part loaded (01h)	I: Halted → I: Ready → M: Running	I: Halted → I: Ready → M: Running	I: Halted → I: Ready → M: Running	I: Halted → I: Ready → M: Running	I: Starting → M: Halted	I: Shutting down M: Halted
Restart and executable part unloaded (01h)	Halted	Halted	Halted	Halted	I: Starting → M: Halted	I: Shutting down M: Halted
Stop (02h)	Terminated	R: Terminated O: Halted ^a	Terminated	Terminated	I: Starting → M: Halted	I: Shutting down M: Halted
Unload to corresponding Load State	Halted	I: Shutting down M: Halted	Halted	Halted	I: Starting → M: Halted	I: Shutting down M: Halted
Device Restart and executable part loaded (PowerUp)	I: Halted → I: Ready → M: Running	I: Halted → I: Ready → M: Running	I: Halted → I: Ready → M: Running	I: Halted → I: Ready → M: Running	I: Starting → M: Halted	I: Shutting down M: Halted
Device Restart and executable part unloaded (PowerUp)	Halted	not applicable	not applicable	Halted	I: Starting → M: Halted	I: Shutting down M: Halted
LEGEND: I: intermediate state M: mandatory O: optional R: recommended →: mandatory transition from a possible intermediate state to a final state						
^a BCU 2 goes to Terminated and masks 0300h and 2300h to Halted.						

The transition from Ready to Running shall be done automatically if the run conditions are fulfilled. The state Shutting down shall either lead an automatic restart of the device or to a power off and shall therefore always be an intermediate state.

For transitions that need more than two seconds the intermediate states Starting and Shutting down are mandatory.

Optional additional error information can be made available according the specifications of Functional Block “Technical Alarm”, see Chapter 7/1/4 “Techical Alarm”.

4.24.2.3.4 Dependencies between Load - and Run State Machine

One of the run conditions of the Run State Machine is the state of the Load State Machine. The executable part can only start if the Load State is Loaded. For an executable part that cannot be loaded the Run State may be read-only and may always be in the state Running.

4.24.2.4 Usage by the Management Client

These Run State Machine – Realisation Type 1 shall be accessed by standard Property services (A_PropertyValue_Write, A_PropertyValue_Read).

The standard error handling shall apply for the services A_PropertyValue_Read and A_PropertyValue_Write.

Starting and Shutting Down are intermediate states that may never appear.

4.24.3 Run State Machine – Realisation Type 2 (Memory mapped)

This Realisation Type is not specified in this version of this document.

4.25 OptionReg

4.25.1 Abstract Resource definition

The OptionReg shall contain the EEPROM Option Register.

4.25.2 OptionReg – Realisation Type 1

Used by: masks 0010h, 0011h, 0012h

4.25.2.1 Format

Bits 0 and 1 of this octet are determined by the processor hardware (see datasheet).

Octet	Bit		Encoding (d = default value)
1	0	SEC (MC68HC05 specific)	
	1	EE1P (MC68HC05 specific)	0: EEPROM address range 0120h to 01FFh is protected 1: EEPROM address range 0120h to 01FFh is writeable
	2	Reserved for system software. Shall be 1	
	3	Reserved for system software. Shall be 1	
	4	Reserved for system software. Shall be 1	
	5	Reserved for system software. Shall be 1	
	6	Reserved for system software. Shall be 1	
	7	Reserved for system software. Shall be 1	

4.25.2.2 Location

Address 0100h

4.26 Programming Mode (prog_mode)

4.26.1 Abstract Resource definition

Programming Mode shall be the Resource in the MaS that shall control the MaS to accept well-defined services and expose well-defined behaviour. These are not listed here; dependency of the *Programming Mode* is part of the specification of these services and Resources.

EXAMPLE 42 In function of the current value of Programming Mode, the service A_IndividualAddress_Read and A_IndividualAddress_Write will be accepted or not.

Please refer to the use of *Programmierung Mode* in [07] and [09].

Autonomous inactivation of Programming Mode

This is an optional feature for the MaS. Please refer to [17] for the conditions for the support of this feature.

If this feature is supported, then, if the *Programming Mode* becomes enabled, by any means, the MaS (device) shall start a time-out timer of 4 minutes.

This timer shall count down and it shall not be possible to restart this timer.

If the timer expires, the MaS shall autonomously and automatically disable its *Programming Mode*.

A semi-directional KNX RF S-Mode device shall additionally inactivate its *Programming Mode* if its bidirectional mode becomes disabled.

4.26.2 Programming Mode – Realisation Type 1

This is the Property PID_PROGMode as specified in 4.3.5.

4.26.3 Programming Mode – Realisation Type 2

Used by:

- Ctrl-Mode fixed DMA
- Ctrl-Mode reloc DMA
- masks 0012h 0020h, 0021h, 0701h in E-Mode

4.26.3.1 Format and encoding

This Resource may share its storage location with other data on the same memory location inside the device. These other data may be different depending of the mask version of the device. The Resource for activation of the Programming Mode is always the same.

The Resource for activating the Programming Mode shall consist of two bits. The location inside the memory is specified in Figure 66.

curr_prog_mode							
octet 1							
7	6	5	4	3	2	1	0
p_parity	X	X	X	X	X	X	prog_mode

Figure 66 – Shared variable curr_prog_mode

The bit **prog_mode** (bit 0) shall serve both for setting and clearing the Programming Mode as well as for reflecting the state of the Programming Mode. If read with value “1” then the Programming Mode is active, if read with value “0” then the Programming Mode is not active. If the Programming Mode is active, then the condition for reacting on the services **A_IndividualAddress_Read** and **A_IndividualAddress_Write** (see [07]) are fulfilled.

If the Programming Mode is activated or deactivated by user interaction at the device (e.g. via programming button) this shall also be reflected in this bit value. The programming mode may also be indicated at the device (e.g. programming LED) that shall then display the state of the bit **prog_mode**.

Bit 1 to bit 6 are the shared bits. They shall be don't care in context of Programming Mode. Even if the whole octet is read and written back they shall not be changed when activating and deactivating the Programming Mode.

The variable **p_parity** shall be the parity bit of the complete octet. The don't care bits 1 to 6 may have some essential importance for the system stack and so the complete range bit 0 to bit 6 may be parity controlled. With this also the variable **prog_mode** shall be covered by the parity check of this systems.

Even if the value of the don't care bits 1 to 6 is not evaluated, the state ⁹⁵⁾ of the variable **p_parity** shall always be changed if the variable **prog_mode** is changed. This means that if the value of **prog_mode** is changed from “0” to “1” or from “1” to “0” then the variable **p_parity** shall be inverted.

4.26.3.2 Location

The location of **curr_prog_mode** shall be the memory address 0060h.

4.26.3.3 Usage by Management Server (device)

The Management Server shall reflect the internal state of the Programming Mode to the bit **prog_mode**. If the Programming Mode is active, then the value of **prog_mode** shall be set to “1”. If the Programming Mode is not active, then the value of **prog_mode** shall be set to “0”. This is independent of the method by which the Programming Mode is activated or deactivated.

If **prog_mode** is set from “0” to “1” by the client then the Programming Mode shall be activated. If the **prog_mode** is set from “1” to “0” by the client then the Programming Mode shall be deactivated.

The variable **p_parity** shall always have a valid value. If no parity checking is used for the octet then the value of **p_parity** shall be ignored.

The reaction of an invalid parity value is manufacturer specific ⁹⁶⁾.

4.26.3.4 Usage by Management Client

4.26.3.4.1 General requirements

The Management Client shall set the Programming Mode via read – modify – write of the variable **curr_prog_mode**. The variable **prog_mode** shall be set to the value “1” if Programming Mode shall be activated. The variable **prog_mode** shall be set to the value “0” if Programming Mode shall be deactivated. The variable **p_parity** shall be inverted, if the value of **prog_mode** is changed.

The following Configuration Procedures specify the activation and deactivation of the Programming Mode.

⁹⁵⁾ It is assumed, that a proper running system has always a valid setting of the variable **p_parity**.

⁹⁶⁾ Typically the system is restarted if **p_parity** is invalid.

4.26.3.4.2 Activation Programming Mode

Procedure: Activate prog_mode

```
    ; Connect to the device.
DMP_Connect_RCo(IA = Device_IA)
    ; Get the current programming mode of the device (curr_prog_mode).
DMP_MemRead_RCo(Addr = 0060h, length = 1)
if prog_mode = 0
    then new_prog_mode = 1
        ; Write the new programming mode.
        DMP_ProgModeSwitch_RCo(mode = new_prog_mode)
end if
    ; Disconnect from the device.
DMP_Disconnect_RCo()
```

4.26.3.4.3 Deactivation Programming Mode

Procedure: Deactivate prog_mode

```
    ; Connect to the device.
DMP_Connect_RCo(IA = Device_IA)
    ; Get the current programming mode of the device curr_prog_mode.
DMP_MemRead_RCo(Addr = 0060h, length = 1)
if prog_mode = 1
    then new_prog_mode = 0
        ; Write the new programming mode.
        DMP_ProgModeSwitch_RCo(mode = new_prog_mode)
end if
    ; Disconnect from the device.
DMP_Disconnect_RCo()
```

4.26.3.4.4 Local access

The behaviour for the local access to **prog_mode** for Management Clients is the same as specified. Instead of the connection oriented procedures (RCo) the according local procedures for Emi1 or Emi2 has to be used.

4.27 Group telegram rate limitation

4.27.1 Abstract Resource definition

“Group Telegram Rate Limitation” shall be a parameter of the Application Layer. Its value shall represent the number of group telegrams that shall at maximum be sent by a device within a time period or an interframe time between two telegrams. Telegrams that are no group telegrams shall not be limited by group telegram rate limitation functionality.

4.27.3 Telegram Rate Limitation – Realization Type 2

Used by Mask 07B0h, 17B0h

4.27.3.1 Format and location

The Telegram Rate Limitation parameter shall consist of two Properties located in the Device Object.

- PID_GROUP_TELEGR_RATE_LIMIT_TIME_BASE : see 4.3.34.
- PID_GROUP_TELEGR_RATE_LIMIT_NO_OF_TELEGR: see 4.3.34.

4.27.3.2 Discovery

Whether or not a Management Server supports Telegram Rate Limitation – Realisation Type 2 can be discovered by the Management Client by scanning these Properties in the Device Object

DMP_InterfaceObjectScan_R.

5 Resources for Couplers

5.1 Resources for mask 0910h and mask 0911h

- A Couplers shall route Frames that have a Group Address as well Frames that have an Individual Address as destination address using the approach given in [05] clause 2.4.2.4.2 “Routing in case of an Individual Destination Address: Line Coupler” and clause 2.4.2.4.3 “Routing in case of an Individual Destination Address: Backbone Coupler”.
- It shall be possible to parameterize the Coupler in such a way, that independent of the communication mode and the hop counter the Coupler always generates a Layer 2 acknowledge after reception of a correct Frame.
- It shall be possible to at least route Frames with Group Addresses from Main Group 0 to 13 individually and from Main Group 14 or higher either generally or individually.
- Depending on the Individual Address, it shall be possible to use the Coupler as Line - or Backbone Coupler (optionally as KNX TP1 Repeater).

– If used as Backbone Coupler:

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Individual Address	x	x	x	x	0	0	0	0	0	0	0	0	0	0	0	0

Bit 0 to 11 = 0.

The Backbone Coupler separates the Backbone Line from the Main Line.

– If used as Line Coupler:

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Individual address	x	x	x	x	Q	Q	Q	Q	0	0	0	0	0	0	0	0

Bit 8 to 11 $\neq$ 0 and bit 0 to 7 = 0.

The Line Coupler separates the Main Line from the Line.

– If used as a KNX TP1 Repeater:

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Individual Address	x	x	x	x	Q	Q	Q	Q	D	D	D	D	D	D	D	D

Bit 8 to 11 $\neq$ 0 and bit 0 to 7 $\neq$ 0.

The KNX TP1 Repeater separates physical segments in a Line.

- For bus load tests for Couplers, please refer to [04]. For corresponding tests, see [20].
- It shall be possible to reduce the number of repetitions from Primary Side to secondary Side and vice versa in case of use as a KNX TP1 Repeater for Telegrams in point-to-point as well as in point-to-multipoint communication mode.
- In case of power down of the Primary Side, it is recommended that according to parameterization the Coupler generates an error message.
- The following Profiles are defined for Couplers: Mask 0910h and mask 0911h.
 - These Profiles shall not be used for new developments of Couplers: their implementation shall be exclusively used for re-assessment of existing mask 0910h and 0912h implementations.

- The Coupler Profiles 0910h and 0911h connect a Primary Side and a Secondary Side and contain two modified BCU 1 implementations, one per connection, and 4 kb static RAM to which both BCU 1 implementations have access. The Filter Table is stored in the upper 3,5 kb of the RAM, corresponding to 28 672 Group Addresses. The lower 0,5 kb of RAM is split into 20 Telegram buffers, 10 in either direction, with a length of 23 octets each. These buffers are used for passing Telegrams between the processors and hence to the connections (Primary Side and Secondary Side) thereby providing the routing function. The processor connected to the Primary Side is defined as the master.
- ▲ The Filter Table for the Masks 0910h and 0911h is organized as bit fields: If there is a Group Address represented in the Filter Table, then the corresponding bit is set to '1'. The method of numbering within an octet is from the lowest value to the highest value bit (from D0 to D7). If e.g. on memory address 021Dh the value 01h is written, the Group Address included in the Filter Table can then be calculated as follows:
 1. The contents corresponds to 00000001b in binary form (signifying one set Group Address).
 2. Up till address 021Dh, it is possible to store 1DHh x 8 = E8h Group Addresses
 3. Group Address, bit position is 0 or E8h + 0 = E8h = 0000 0000 1110 1000 B = 0/232
- ▲ The memory model of mask 0910h and 0911h (which shall be implemented in both processors) deviates from the BCU 1 Profile in the following way :

Address	mask 0910h	mask 0911h
109h to 10Ch	unused	unused
10Fh	identical to BCU1	unused
112h	unused	MPhyRstCnt
113h	unused	MGrRstCnt
114h	unused	SPhyRstCnt
115h	unused	SGrRstCnt
116h	shall be 1	shall be 1
119h	LKstat	LKstat
11Ah	LKerror	LKerror
11Bh	LKdef1	LKdef2
11Ch	LKgrpconf	LKgrpconf
11Dh to 1FEh	unused	unused

Possible settings of above parameters.

a) Lkdef

Setting	Meaning	mask 0910h	mask 0911h
xxxxxx11	ignore errors in group table	X	X
xxxxxx10	in case of group table errors, route all telegrams of the error group	X	X
xxxxxx01	in case of group table errors, block all telegrams of the error group	X	X
xxx1xxxx	group addresses > 6FFFh are transmitted	X	X
xxx0xxxx	group addresses > 6FFFh are blocked	X	X
xx1xxxxx	group table is cyclically checked	X	X
xx0xxxxx	group table is not cyclically checked	X	X
1xxxxxxx	Main Line; routing of physical addresses activated		X
0xxxxxxx	Main Line; routing of physical addresses deactivated		X
x1xxxxxx	sub line; routing of physical addresses activated		X
x0xxxxxx	sub line; routing of physical addresses deactivated		X
xxxx1xxx	Main Line; IACK in case of illegal addressing deactivated		X
xxxx0xxx	Main Line; IACK in case of illegal addressing activated		X
xxxxx1xx	sub line; IACK in case of illegal addressing deactivated		X
xxxxx0xx	sub line; IACK in case of illegal addressing activated		X

b) Lkgrpconf

Setting	Meaning	mask 0910h	mask 0911h
xxxxxx11	group telegrams pass routed main to sub line	X	X
xxxxxx10	group telegrams blocked main to sub line	X	X
xxxxxx01	group telegrams pass not routed main to sub line	X	X
xx11xxxx	group telegrams pass routed sub line to main	X	X
xx10xxxx	group telegrams blocked sub line to main	X	X
xx01xxxx	group telegrams pass not routed sub line to main	X	X

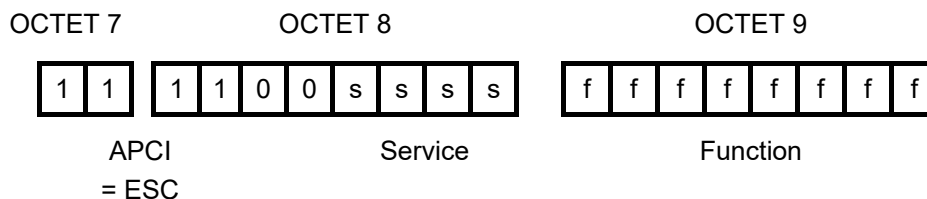
c) Lkerror

Setting	Meaning	mask 0910h	mask 0911h
1111111 1	no errors	X	X
xxxxxxx0	telegram length error	X	X
xxxxxxx0x	group table error	X	X
xxxxx0xx	slave error after reset	X	X
xxx0xxxx	illegal telegram in TX buffer	X	X
xx0xxxxx	connection error between two processors	X	X

d) Lkstat

Setting	Meaning	mask 0910h	mask 0911h
xxxx1110	test mode	X	X
xxxx1101	repeater	X	X
xxxx1011	line coupler	X	X
xxxx0111	bus coupling unit	X	X

– Group Functions



The Service shall be encoded as follows.

Dh: A_RouterStatus_Read
 Eh: A_RouterStatus_Response
 Fh: A_RouterStatus_Write

The Function shall be encoded as follows.

33h Both Lines Normal
 22h Both Lines Block
 11h Both Lines Free
 03h Main Line to sub line Normal
 02h Main Line to sub line Block
 01h Main Line to sub line Free
 30h sub line to Main Line Normal
 20h sub line to Main Line Block
 10h sub line to Main Line Free

5.2 Resources for mask 0912h

5.2.1 Filter Table Realisation Type 2

Used by: Mask 0912h

5.2.1.1 General

Because of the memory mapped access for the Filter Table, the structure of the Filter Table shall be as implicitly fixed as specified in for masks 0910h and 0911h above. In mask 0911h Couplers the Filter Table is limited to 28 672 Group Addresses. In mask 0912h the Filter Table supports filtering of full 16 bit Group Addressing. For compatibility reasons the Filter Table shall be accessed using the already specified Coupler specific AL-services. The address 0000h shall be reserved for the broadcast address and shall not be evaluated in the Filter Table.

5.2.1.2 Location

The Filter Table Realisation Type 2 shall start at memory address 0200h and shall end on memory address 21FFh.

5.2.1.3 Format

The bit position and memory address of the octet in the Filter Table shall be related by the following.

The Filter Table of the Coupler mask 0912h shall be organized as bit fields. If there is a Group Address present in the Filter Table, the corresponding bit shall be set to '1'. The method of numbering within an octet shall be from the lowest value to the highest value bit (from D0 to D7).

EXAMPLE 43 If on memory address 021Dh the value 01h is written, then the Group Address included in the Filter Table can then be calculated as follows.

1. The content corresponds to 00000001b in binary form (signifying one set Group Address).
2. Up till address 021Dh, it is possible to store 1Dh x 8 = E8h Group Addresses.
3. Group Address, bit position is 0 or E8h + 0 = E8h = 0000 0000 1110 1000b = 0/232.

5.2.1.4 Usage by the Management Client and the Management Server

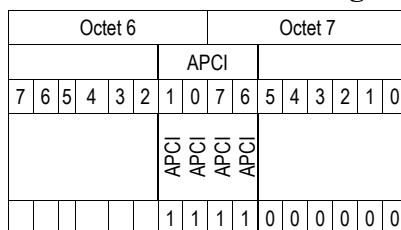


Figure 67 – A_FilterTable_Open-PDU (example)

Before any access to the Filter Table it has to be enabled using this service. On reception of the A_FilterTable_Open-PDU, the Management Server shall set the LSM of the Filter Table implemented in the Router Object to LOADED. There shall be no transition to the state ERROR due to this service or missing this service. There shall also be no dependency to the LTE Filter Table.

5.2.2 Coupler Parameters

5.2.2.1 Normal conditions

5.2.2.1.1 Introduction and overview

A Coupler according mask 0912h has two Management Servers implemented. One memory mapped based MaS for the compatibility with Coupler mask 0911h and one MaS for a Property based tool access. The Configuration Procedures for mask 0912h use a mixture of both MaS.

The memory mapped access is used for the Filter Table. Using the Function Property access can result in a faster loading of the Filter Table, but not in all cases. So it makes no sense to implement new MaC procedures for that.

For parameters the Property based access is used. The set of parameters in mask 0912h is expanded compared to the set of parameters in Couplers mask 0911h.

Mapping of Property based Parameters to memory mapped Parameters

As the memory mapped parameters are only a subset of the parameters located in Properties the mapping of Property based parameters into memory mapped parameters is not completely possible and therefore not mandatory. There is no mapping of Property based Parameters into memory mapped parameters.

Mapping of memory mapped parameters to Property based Parameters

The parameters and diagnostic data of the implementations of mask 0911h shall be memory mapped Resources. In mask 0912h these Resources shall be implemented in Properties and for compatibility reasons also as memory mapped parameters. The synchronisation of both Resources shall ensure that memory mapped parameter settings done by ETS are available in the Property based parameters for an ETS configuration using the new Configuration Procedure. The synchronisation shall be performed after Restart or after a timeout. The timeout is triggered by write access on the memory mapped Resources. For the synchronisation a maximal timeout of 6 s after ETS configuration is allowed.

An overview to the memory mapped Resources is shown in Table 98.

Table 98 – Memory mapped Resources in masks 0911h and 0912h

Address	Mask 0911h	Mask 0912h
010Eh	RouteCnt	PID_ROUTING_COUNT in Device Object
0112h	MPhyRstCnt	PID_MAIN_LCCONFIG
0113h	MGrRstCnt	PID_MAIN_LCGRPCONFIG
0114h	SPhyRstCnt	PID_SUB_LCCONFIG
0115h	SGrRstCnt	PID_SUB_LCGRPCONFIG

Address	Mask 0911h	Mask 0912h
011Bh	LKdef2	PID_MAIN_LCGRPCONFIG PID_SUB_LCGRPCONFIG PID_MAIN_LCCONFIG PID_SUB_LCCONFIG
011Ch	LKgrpconf	PID_MAIN_LCGRPCONFIG PID_SUB_LCGRPCONFIG
011Dh to 1FEh	unused	unused

5.2.2.1.2 MPhyRstCnt

MPhyRstCnt shall contain the number of Data Link Layer repetitions that the Coupler shall perform on its Primary Side for Frames in point-to-point connectionless - or connection-oriented communication mode after the initial transmission is acknowledged with “NACK”, “no L2-ack” or BUSY. The bit configuration is shown in Figure 68.

7	6	5	4	3	2	1	0
number of repetition in case of BUSY			reserved		number of repetitions in case of NACK		

Figure 68 – Format of MPhyRstCnt

This Parameter shall be mapped to bit 2 of PID_MAIN_LCCONFIG.

Bit 2 in PID_MAIN_LCCONFIG shall be set to 0 in case “number of repetition in case of BUSY” and “number of repetitions in case of NACK” are set 0 or 1.

Optional behaviour

- Bit 2 in PID_MAIN_LCCONFIG shall be set to 0 in case of “number of repetitions in case of NACK” is set 0 or 1.
- Bit 2 in PID_MAIN_LCCONFIG shall be set to 1 in case of “number of repetition in case of BUSY” or “number of repetitions in case of NACK” is set greater than or equal to 2.

Optional behaviour

- Bit 2 in PID_MAIN_LCCONFIG shall be set to 1 in case of “number of repetitions in case of NACK” is set greater than or equal to 2.

5.2.2.1.3 MGrRstCnt

MGrRstCnt shall contain the number of Data Link Layer repetitions that the Coupler shall perform on its Primary Side for Frames sent in multicast – and broadcast communication mode after the initial transmission is acknowledged with “NACK”, “no L2-ack” or BUSY. The bit configuration is identical as in Figure 68.

This Parameter shall be mapped to bit 4 of PID_MAIN_LCCONFIG for broadcast communication mode and to bit 4 of PID_MAIN_LCGRPCONFIG for point-to-multipoint, connectionless (multicast) mode.

The mapping of this Parameter shall also reset bit 4 (BROADCAST_LOCK) in PID_MAIN_LCGRPCONFIG.

NOTE 87 This is not implemented in current implementations of mask 0912h.

Bit 4 in PID_MAIN_LCCONFIG and PID_MAIN_LCGRPCONFIG shall be set to 0 in case “number of repetition in case of BUSY” and “number of repetitions in case of NACK” are set 0 or 1.

Optional behaviour

- Bit 4 in PID_MAIN_LCCONFIG and PID_MAIN_LCGRPCONFIG shall be set to 0 in case “number of repetitions in case of NACK” is set 0 or 1.
- Bit 4 in PID_MAIN_LCCONFIG and PID_MAIN_LCGRPCONFIG shall be set to 1 in case of “number of repetition in case of BUSY” or “number of repetitions in case of NACK” is set greater than or equal to 2.

Optional behaviour

- Bit 4 in PID_MAIN_LCCONFIG and PID_MAIN_LCGRPCONFIG shall be set to 1 in case of “number of repetitions in case of NACK” is set greater than or equal to 2.

5.2.2.1.4 SPhyRstCnt

SPhyRstCnt shall contain the number of Data Link Layer repetitions that the Coupler shall perform on its Secondary Side for Frames sent in point-to-point connectionless - or connection-oriented communication mode after the initial transmission is acknowledged with “NACK”, “no L2-ack” or “BUSY”. The bit configuration shall be identical as in Figure 68.

This Parameter shall be mapped to bit 2 of PID_SUB_LCCONFIG.

Bit 2 in PID_SUB_LCCONFIG shall be set to 0 in case of “number of repetition in case of BUSY” and “number of repetitions in case of NACK” are set 0 or 1.

Optional behaviour

- Bit 2 in PID_SUB_LCCONFIG shall be set to 0 in case of “number of repetitions in case of NACK” is set 0 or 1.
- Bit 2 in PID_SUB_LCCONFIG shall be set to 1 in case of “number of repetition in case of BUSY” or “number of repetitions in case of NACK” is set greater than or equal to 2.

Optional behaviour

- Bit 2 in PID_SUB_LCCONFIG shall be set to 1 in case of “number of repetitions in case of NACK” is set greater than or equal to 2.

5.2.2.1.5 SGrRstCnt

SGrRstCnt shall contain the number of Data Link Layer repetitions that the Coupler shall perform on its Secondary Side for Frames sent in multicast – and broadcast communication mode after the initial transmission is acknowledged with “NACK”, “no L2-ack” or BUSY. The bit configuration is shown in Figure 68.

This Parameter shall be mapped to bit 4 of PID_SUB_LCCONFIG for broadcast communication mode and PID_SUB_LCGRPCONFIG for group communication mode.

The mapping of this Parameter shall also reset bit 4 (BROADCAST_LOCK) in PID_SUB_LCGRPCONFIG.

NOTE 88 This is not implemented in current implementations of mask 0912h.

Bit 4 in PID_SUB_LCCONFIG and PID_SUB_LCGRPCONFIG shall be set to 0 in case of “number of repetition in case of BUSY” and “number of repetitions in case of NACK” is set 0 or 1.

Optional behaviour

- Bit 4 in PID_SUB_LCCONFIG and PID_SUB_LCGRPCONFIG shall be set to 0 in case of “number of repetitions in case of NACK” is set 0 or 1.
- Bit 4 in PID_SUB_LCCONFIG and PID_SUB_LCGRPCONFIG are set to 1 in case of “number of repetition in case of BUSY” or “number of repetitions in case of NACK” is set greater than or equal to 2.

Optional behaviour

- Bit 4 in PID_SUB_LCCONFIG and PID_SUB_LCGRPCONFIG shall be set to 1 in case of “number of repetitions in case of NACK” is set greater than or equal to 2.

5.2.2.1.6 LKdef2

Settings of Parameters in LKdef2 shall be mapped according Table 2.

Table 2 – Mapping of Parameter LKdef2

Setting								Meaning	Profile	
b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀		0911h	0912h
x	x	x	x	x	x	1	1	Ignore errors in Filter Table.	M	O
x	x	x	x	x	x	1	0	In case of Filter Table errors, route all telegrams of the error group.	M	O
x	x	x	x	x	x	0	1	In case of Filter Table errors, block all telegrams of the error group	M	O
x	x	x	1	x	x	x	x	Group Addresses > 6FFFh shall be forwarded.	M	PID_SUB_LCGRPCONFIG = XXXX01XX PID_MAIN_LCGRPCONFIG = XXXX01XX
x	x	x	0	x	x	x	x	Group Addresses >6FFFh shall be blocked	M	PID_SUB_LCGRPCONFIG = XXXX10XX PID_MAIN_LCGRPCONFIG = XXXX10XX
x	x	1	x	x	x	x	x	The Filter Table shall be cyclically checked.	M	O
x	x	0	x	x	x	x	x	The Filter Table shall not be checked cyclically.	M	O
1	x	x	x	x	x	x	x	The routing of telegrams in point-to-point connectionless - and connection-oriented communication mode from the Primary Side to the Secondary Side shall be enabled.	M	PID_MAIN_LCCONFIG = XXXXXX11
0	x	x	x	x	x	x	x	All frames in point-to-point connectionless - and connection-oriented communication mode from the Primary Side to the Secondary Side shall be routed independent of the Coupler Mode	M	PID_MAIN_LCCONFIG = XXXXXX01
x	1	x	x	x	x	x	x	The routing of telegrams in point-to-point connectionless - and connection-oriented communication mode from the Secondary Side to the Primary Side shall be enabled.	M	PID_SUB_LCCONFIG = XXXXXX11
x	0	x	x	x	x	x	x	All frames in point-to-point connectionless - and connection-oriented communication mode from the Secondary Side to the Primary Side shall be routed independent of the Coupler Mode	M	PID_SUB_LCCONFIG = XXXXXX01
x	x	x	x	1	x	x	x	Primary Side: send Layer 2 acknowledge only in case of routing	M	PID_MAIN_LCCONFIG = 011XXXXX
x	x	x	x	0	x	x	x	Primary Side: always send Layer 2 acknowledge independent of routing	M	PID_MAIN_LCCONFIG = 100XXXXX
x	x	x	x	x	1	x	x	Secondary Side: send Layer 2 acknowledge only in case of routing	M	PID_SUB_LCCONFIG = 011XXXXX
x	x	x	x	x	0	x	x	Secondary Side: always send Layer 2 acknowledge independent of routing	M	PID_SUB_LCCONFIG = 100XXXXX

5.2.2.1.7 LKgrpconf

Settings of Parameters in LKgrpconf shall be mapped according Table 3.

Table 3 – Mapping of Parameter LKgrpconf

Setting								Meaning	Profile	
b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀		0911h	0912h
x	x	x	x	x	x	1	1	Telegrams in multicast communication mode from Primary Side to Secondary Side shall be routed in function of the evaluation of the Filter Table.	M	PID_MAIN_LCGRPCONFIG = XXXXXX11
x	x	x	x	x	x	1	0	Telegrams in multicast communication mode from Primary Side to Secondary Side shall be blocked.	M	PID_MAIN_LCGRPCONFIG = XXXXXX10
x	x	x	x	x	x	0	1	Telegrams in multicast communication mode from Primary Side to Secondary Side shall all be routed without evaluation of the Filter Table.	M	PID_MAIN_LCGRPCONFIG = XXXXXX01
x	x	1	1	x	x	x	x	Telegrams in multicast communication mode from Secondary Side to Primary Side shall be routed in function of the evaluation of the Filter Table.	M	PID_SUB_LCGRPCONFIG = XXXXXX11
x	x	1	0	x	x	x	x	Telegrams in multicast communication mode from Secondary Side to Primary Side shall be blocked.	M	PID_SUB_LCGRPCONFIG = XXXXXX10
x	x	0	1	x	x	x	x	Telegrams in multicast communication mode from Secondary Side to Primary Side shall all be routed without evaluation of the Filter Table.	M	PID_SUB_LCGRPCONFIG = XXXXXX01

5.2.2.2 Error and exception handling

The parameters of Couplers mask 0911h shall be a subset of the mask 0912h parameters. The Configuration Procedure ensures the possibility for mixing in the configuration of a mask 0912h Coupler the Configuration Procedures of mask 0911h (memory mapped) with the Configuration Procedures of mask 0912h (Property Based). For further details about the Configuration Procedures, please refer to [10].

5.2.3 Load State Machine – Realisation Type 1 (Property based)

The Load State Machine transition table allows an optional transition from state “Loaded” to “Error” in case of an event “Load Completed”. This is not allowed for mask version 0912h Couplers. Mask 0912h shall stay in state “Loaded” in case of an error.

5.3 Resources for Coupler Model 2.0

5.3.1 Filter Table Realization Type 3

- **Access levels:** 3/2
- **Access Policy:** 3FF/00C

5.3.1.1 Format and encoding

5.3.1.1.1 Format

The Filter Table Realisation Type 3 shall be organised as a memory mapped bit-field of 65 536 bits and thus 8 192 octets. Each bit shall uniquely correspond to one Group Address. The full 16 bit KNX GA encoding range shall be supported.

The bit corresponding to a GA can be calculated as follows. “GA_value” shall be the numerical value of the GA.

- The address of the octet in which the bit resides shall be calculated as follows.

$$\text{octet_address} = \text{GA_value} \div 8$$
- The position of the bit within that octet shall be calculated as follows. The bits within the octet shall be numbered from 0, least significant position, to 7, most significant position.

$$\text{bit_position} = \text{GA_value} \bmod 8$$

EXAMPLE 44 For the GA 7/1879 in the Filter Table, the following is calculated.

The octet address will be $3F57h \div 8 = 07EAh$

The bit position will be $3F57h \bmod 8 = 7$.

5.3.1.1.2 Encoding

Each bit in the Filter Table corresponding to a GA shall comply with the following encoding.

- 0: The Filter Table evaluation for this Group Address shall be negative: the message with the GA corresponding to this bit in the Filter Table shall not be passed.
- 1: The Filter Table evaluation for this Group Address shall be positive: the message with the GA corresponding to this bit in the Filter Table shall be passed.

NOTE 89 This solely concerns the check of the passing of a message in function of the Filter Table. Other conditions, like other Coupler Parameters and the hop_count of the Telegram, may exist that influence the passing of the Telegram.

Exceptions

- The Group Address 0000h is no valid KNX Group Address. The bit 0 in the Filter Table, the least significant bit of octet 0, shall be “don’t care”. It shall not be evaluated.
- For the routing of Frames in (system) broadcast address communication mode, the Filter Table shall not be evaluated.

5.3.1.2 Default value and Master Reset

The default contents of the Filter Table – Realisation Type 3 shall be implementation specific. The Coupler Model 2.0 shall assume this value on the following conditions. The MaC shall make no assumptions on the contents of the Filter Table.

Erase Code	Effect
01h Confirmed Restart	not influenced
02h Reset to default state	implementation default
03h Reset IA	not influenced

Erase Code	Effect
04h Reset AP	not influenced
05h ResetParam	implementation default
06h ResetLinks	not influenced
07h Reset to default without IA	implementation default
08h Erase persistently stored application data	not influenced
n.a. Local Reset to default state	implementation default
n.a. Power Cycle	not influenced

5.3.1.3 Location

The Filter Table realisation Type 3 shall be a memory mapped Resource. It shall start at the memory address indicated by the Property `PID_TABLE_REFERENCE` of the Router Object. This Property, and hence the memory location of the Filter Table, shall comply with the common specification of `PID_TABLE_REFERENCE` in 4.2.7.

There are no requirements concerning the value of the memory location and hence the value of `PID_TABLE_REFERENCE`.

5.3.1.4 Usage by the Management Server (device)

The below specifies the standard, minimal MaS side interpretation of the Filter Table. Additionally, the MaS may have further, implementation specific parameters that influence the evaluation of the Filter Table.

The Filter Table Realisation type 3 shall be controlled and managed by a Load State Machine – Realisation Type 1 (Property based) as given in clause 4.23.2.

This shall be the Load State Machine for the Filter Table that is used for the Medium Interface that is represented by this Router Object..

If the Load State of the Filter Table is *Loaded*

If the MaS (Coupler) evaluates the Filter Table for concluding to pass a Frame with a given GA or not, then this evaluation shall be positive if the bit corresponding to the GA in the Filter Table is set. If the bit is cleared, then this evaluation shall be negative.

If the Load State of the Filter Table is any other than *Loaded*

The evaluation of the Filter Table shall always be negative.

NOTE 90 This solely concerns the evaluation of the Filter Table in concluding on passing a Frame. Other conditions exist as well: the Network Layer State Machine (see [05]) and the standard – and implementation specific Parameters.

5.3.1.5 Usage by the Management Client

The MaC shall respect the Load State Machine if modifying the Filter Table.

The MaC shall allocate the Filter Table memory using the Additional Load Control for memory allocation and then retrieve the pointer to the memory located from `PID_TABLE_REFERENCE`.

When writing the Filter Table, the MaC shall access the Filter Table using `DMP_MemWrite_RCoV` or `DMP_UserMemWrite_RCoV` in function of the memory addresses that are accessed: it shall use `DMP_MemWrite_RCoV` to access the memory below and equal to `FFFFh` and `DMP_UserMemWrite_RCoV` to access the memory above `FFFFh`.

5.3.2 Device Object

5.3.2.1 PID_LOAD_STATE_CONTROL (PID = 5)

- **Property name:** Load Control
- **Property Datatype:** PDT_CONTROL
- **Datapoint Type:** None.
- **Access Policy:** 3FF/0CC
- **Access Policy:** 3/2

This use of PID_LOAD_STATE_CONTROL shall comply with the common specification given in clause 4.23.2 “Load State Machine – Realisation Type 1 (Property based)”.

As concerning the Load Events to be supported and the Load Events that may be returned by the Coupler Model 2.0, please refer to Volume 6 “Profiles”.

The below specifies deviations or additional requirements for the Coupler Model 2.0.

5.3.2.1.1 Usage by the MaS

The MaS (Coupler) shall additionally comply with the following.

- If the Load State is “Loaded” then the following shall apply.
 - The Application Program, if implemented, shall run.
 - The standard – and the implementation specific parameters of the Network Layer may be interpreted.

EXCEPTION: The interpretation of PID_PL110_SBC_CONTROL and PID_RF_SBC_CONTROL is not implementation specific and shall not depend on the Load State. These Properties shall be evaluated as specified in 4.5.14.1 and 4.5.15.1 respectively.
- If the Load State is any other than “Loaded” then the following shall apply.
 - The Application Program, if implemented, shall not run.
 - This Load State gives a clear indication that the MaS (Coupler) is under configuration by the MaC. It is implementation specific whether or not the standard – and the implementation specific parameters are interpreted.

EXCEPTION: The interpretation of PID_PL110_SBC_CONTROL and PID_RF_SBC_CONTROL is not implementation specific and shall not depend on the Load State. These Properties shall be evaluated as specified in 4.5.14.1 and 4.5.15.1 respectively.

 - The evaluation of the Filter Table shall return negative.

5.3.2.1.2 Usage by the MaC

The MaC shall set the LSM to *Unloaded* prior to changing any standard – or implementation specific Parameter in the Coupler Model 2.0. When the download has completed, the MaC shall set the LSM back to *Loaded*.

If the MaC needs to restart the MaS during any Configuration Procedure, then it shall firstly set the LSM to *Loaded*, prior to restarting the MaS, and possibly set the LSM back to *Loading* after the restart, if needed to continue the Configuration Procedure.

NOTE 91 This is the overall load state of the MaS, located in the Device Object. This requirement is introduced motivated by the parameters of KNXnet/IP devices, like the IP address assignment method or the IP subnet mask. If these parameters are changed during configuration and the MaS needs to be restarted during the configuration, then it would for the MaS be unclear whether to start with the “old” parameter values, or with the already set new ones. In the first case, the MaS would have to intermediately store the old parameter values and in the second case the MaS may start up with an inconsistent set of parameters. Therefore this LSM is introduced, which clearly indicates to the MaS whether the parameters are valid or not.

5.3.2.2 PID_DOWNLOAD_COUNTER (PID = 30)

- **Property name:** Download Counter
- **Property Datatype:** PDT_UNSIGNED_INT
- **Datapoint Type:** DPT_Value_2_Ucount (DPT_ID = 7.010)

The Property PID_MCB_TABLE shall exclusively relate to the contents of the Filter Table and shall not cover any Parameter (memory mapped or Property based). In order to allow for a partial download, the Property PID_DOWNLOAD_COUNTER should be integrated.

Support by the MaS (device)

Please refer to the common specification of PID_DOWNLOAD_COUNTER in 4.2.30.

Support by the MaC (ETS)

Prior to executing a Partial Download, the MaC shall verify the value of PID_DOWNLOAD_COUNTER of the part to be downloaded, as defined in [10].

If PID_DOWNLOAD_COUNTER is not available for the part to be downloaded, then the MaC shall not perform a Partial Download.

5.3.2.3 PID_SUBNET_ADDR (PID = 57)

- **Property name:** Subnetwork Address
- **Property Datatype:** PDT_UNSIGNED_CHAR
- **Datapoint Type:** None.

For the common specification of the Property *Subnetwork Address*, please refer to 4.3.8.

The Property Subnetwork Address in the Device Object of the Coupler Model 2.0 shall be the Subnetwork Address part of the own Individual Address of the Coupler which it shall respond to for Network – and Device Management.

This Property shall react to the “Configuration Procedures for Configuring the Subnetwork Address” (clause 2.4 in [10]) on the Primary Side and on the Secondary Side.

Default value

The default – and ex-factory value of the Subnetwork Address shall be FFh. The Coupler Model 2.0 shall assume this value on the following conditions.

Erase Code	Effect
01h Confirmed Restart	not influenced
02h Reset to default state	KNX default
03h Reset IA	KNX default
04h Reset AP	not influenced
05h ResetParam	not influenced
06h ResetLinks	not influenced
07h Reset to default without IA	not influenced
08h Erase persistently stored application data	not influenced
n.a. Local Reset to default state	KNX default
n.a. Power Cycle	not influenced

5.3.2.4 PID_DEVICE_ADDR (PID = 58)

- **Property name:** Device Address
- **Property Datatype:** PDT_UNSIGNED_CHAR
- **Datapoint Type:** None.

For the common specification of the Property *Device Address*, please refer to 4.3.9.

The Property Device Address in the Device Object of the Coupler Model 2.0 shall be the Device Address part of the own Individual Address of the Coupler which it shall respond to for Network – and Device Management.

Default value

The default – and value of the Device Address shall be 00h. The Coupler Model 2.0 shall assume this value on the following conditions.

Erase Code	Effect
01h Confirmed Restart	not influenced
02h Reset to default state	KNX default
03h Reset IA	KNX default
04h Reset AP	not influenced
05h ResetParam	not influenced
06h ResetLinks	not influenced
07h Reset to default without IA	not influenced
08h Erase persistently stored application data	not influenced
n.a. Local Reset to default state	KNX default
n.a. Power Cycle	not influenced

5.3.3 Router Object

5.3.3.1 PID_MCB_TABLE (PID = 27)

- **Property name:** Memory Control Table
- **Property Datatype:** PDT_GENERIC_08
- **Datapoint Type:** None
- **Access levels:** 3/x
- **Access Policies:** 3FF/0CC

Please refer to the common specification of PID_MCB_TABLE in 4.2.27.

PID_MCB_TABLE is mandatory in the Router Object of the Secondary Side.

For the implementation in the Coupler Model 2.0, PID_MCB_TABLE shall exclusively relate to the contents of the Filter Table (not to any other Parameter) and shall be read-only. Bit 0 in the contained CRC-control byte shall always be cleared: if the Load State is “Loaded”, then the CRC shall always be “valid”. The MaS shall recalculate and update the CRC if the LSM of the Filter Table transits to “Loaded”.

Usage by the MaC

The MaC shall not modify this Property. The MaC shall only read this Property to learn the CRC-value. The CRC shall always be valid.

5.3.3.2 PID_OBJECT_INDEX (PID = 29)

- **Property name:** Object Index
- **Property Datatype:** PDT_UNSIGNED_CHAR
- **Datapoint Type:** DPT_Value_1_Ucount
- **Access levels:** 3/0
- **Access Policy:** 3FF/0CC

This implementation of PID_OBJECT_INDEX shall comply with the common specification of PID_OBJECT_INDEX in 4.2.29. Additionally, it is required for this implementation that it be accessible using the A_NetworkParameter_Read-service in the procedure NM_ObjectIndex_Read, as specified in [09].

5.3.3.3 PID_PL110_DOA (PID = 105)

- **Property name:** PL110 Domain Address
- **Property Datatype:** PDT_UNSIGNED_INT
- **Datapoint Type:** DPT_Value_2_Ucount
- **Access level:** 3/2
- **Access Policy:** 3FF/0CC

For the common specification of the Property PL110 Domain Address in the Coupler Model 2.0, please refer to the specification of PID_DOMAIN_ADDRESS in 4.3.21.

This Property shall exclusively be available in the Router Object of a connection of the medium PL110.

5.3.3.3.1 Usage by the MaS (device)

5.3.3.3.1.1 Primary Side

The Coupler Model 2.0 shall use the value of this Property as PL110 Domain Address if the Primary Side is a PL110 segment.

NOTE 92 This is not yet foreseen.

5.3.3.3.1.2 Secondary Side

NOTE 93 Opposite to the Device Address and the Subnetwork Address, this Property is not mirrored with any Property in the Device Object.

In the *Router Object* of the Secondary Side, this Property shall be the Domain Address of the Coupler. The Coupler shall use this Property Value as Domain Address on its Secondary Side.

This instance of this Property shall react to the Management procedures for configuring the PL110 Domain Address (see [09]) on the Primary Side and on the Secondary Side. These are:

- NM_DomainAddress_Read
- NM_DomainAndIndividualAddress_Read
- NM_DomainAndIndividualAddress_Write
- NM_DomainAndIndividualAddress_Write2
- NM_DomainAddress_Scan

5.3.3.3.1.3 Default value and Master Reset

The default value of the PL110 Domain Address shall be 00FFh. The Coupler Model 2.0 shall assume this value on the following conditions.

Erase Code	Effect
01h Confirmed Restart	not influenced
02h Reset to default state	KNX default
03h Reset IA	KNX default
04h Reset AP	not influenced
05h ResetParam	not influenced
06h ResetLinks	not influenced
07h Reset to default without IA	not influenced
08h Erase persistently stored application data	not influenced
n.a. Local Reset to default state	KNX default
n.a. Power Cycle	not influenced

5.3.3.3.2 Usage by the MaC (ETS)

The MaC shall manage the DoA of the Secondary Side through the above listed Management Procedures. For the Primary Side, the MaC shall exclusively use Data Property services.

6 Security Resources

6.1 Factory Default Setup Key (FDSK)

6.1.1 Abstract Resource definition

The Factory Default Setup Key (FDSK) shall be a default Tool Key for the authentication and confidentiality to be used by the MaC when the KNX secure device is firstly configured fresh from factory.

The FDSK may be printed on the device's wrapping or on a paper note that is shipped with the device. Other indications that guarantee that this FDSK cannot be obtained by any unauthorized installer or MaC are possible as well.

6.1.2 Security

Please refer to clause 1.3.

6.1.3 Location and format

The FDSK shall not be accessible over the KNX medium or through any local interface to the device. There is no KNX access method (Group Object, memory mapped parameter, Property Value or other) standardised to access the FDSK.

The FDSK shall not be included in any Resource, like the Group Key Table or the Point-to-Point Key Table.

The FDSK shall have the format in function of the Security Algorithm that is used for confidentiality. As there is no storage standardised, there are no further requirements on how to store the FDSK.

6.1.4 Usage by the MaS

The MaS shall allow secure communication using the FDSK if no Tool Key is programmed.

The MaS shall make the FDSK inactive (can no longer be used) as soon as the Tool Key for confidentiality is configured by the MaC. The FDSK shall however remain in memory of the MaS.

The FDSK and the Tool Key shall have the same function, but shall not be active at the same time.

Master Reset

Erase Code		Effect on FDSK
01h	Confirmed Restart	not influenced: no change.
02h	Reset to default state	FDSK shall become active again.
05h	ResetParam	not influenced: no change.
06h	ResetLinks	not influenced: no change.
07h	Reset to default without IA	FDSK shall become active again.
n.a.	Local Reset to default state	FDSK shall become active again.

See also the specifications for DM_Restart in [09] and the clause "Reset to default" and KNX Security.

6.1.5 Usage by the MaC

The MaC **shall** activate the Security Mode (if needed for the links that are made) and prior to any other Device Management ⁹⁷⁾ operation of the MaS, using the FDSK in KNX Secure Communication, assign the Tool Key.

The MaC shall store the FDSK in a secure way in its database, so that when perform any procedure to the MaS that clears the Tool Key (e.g. via Master Reset), it will again get access to the MaS via the FDSK.

6.2 Sequence Number for Tool Access

The Sequence Number for Tool Access shall hold the SeqNr that the MaS has last accepted as SeqNr for any received message for which the flag Tool Access is set in the SCF.

Synchronisation

The *Sequence Number for Tool Access* shall be responded and possibly updated according the specification of the S-A_Sync-service ([07]) if the Tool Access flag in the SCF is set.

NOTE 94 The specification of the S-A_Sync-service allows that even the reception of an S-A_Sync_Req-PDU causes an update of this Sequence Number for Tool Access.

Master Reset

Erase Code	Effect on Sequence Number for Tool Access
01h Confirmed Restart	not influenced: no change
02h Reset to default state	KNX default: see re-initialisation below
05h ResetParam	not influenced: no change
06h ResetLinks	not influenced: no change
07h Reset to default without IA	not influenced: no change
n.a. Local Reset to default state	KNX default: see re-initialisation below

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

Re-initialisation

The Sequence Number for Tool Access shall be re-initialised with a value that is at least 20 higher than the preceding value, but lower than 00 01 00 00 00 00h.

NOTE 95 The Sequence Number for Tool Access could also be left unchanged. This would however mean that if it ever would reach its maximal value (Client bug, electronic failure or other), that there would be no way to set it back to a usable value.

NOTE 96 Only the initial configuration, when the FDSK is used, this is, mainly the synchronisation by the MaC and the Network Configuration, has to be protected from a replay attack. As soon as the MaC assigns a valid Tool Key – which shall be different from any previously assigned Tool Key, the rest of the configuration cannot be replayed anymore.

6.3 Security Interface Object

6.3.1 Introduction and overview

The Security Interface Object shall be used for managing the KNX Data Security. This concerns configuration information (e.g. encryption keys) and runtime information (e.g. Security Report and Security Failures Log).

⁹⁷⁾ This explicitly considers Device Management only. Network Management (elf. assignment of Domain Addresses and Individual Addresses) is done unsecured. See 2.6.

The Security Interface Object (Interface Object Type = 17) shall hold the Properties as listed in Table 99. (This table only gives an overview. For the mandatory and optional Properties, please refer to [17]).

The security conditions for accessing these Properties are specified in the detailed Property descriptions in clause 6.3.2 and further.

Therefore, please refer to the detailed Property definitions below for the requirements concerning access.

Table 99 – Properties in the Security Interface Object

Property Name	Property Identifier	Property Datatype
Interface Object Type	1 = PID_OBJECT_TYPE	PDT_UNSIGNED_INT
Interface Object Name	2 = PID_OBJECT_NAME	PDT_UNSIGNED_CHAR[]
Load Control	5 = PID_LOAD_STATE_CONTROL	PDT_CONTROL
Security Mode	51 = PID_SECURITY_MODE	PDT_FUNCTION
Point-to-point Key Table	52 = PID_P2P_KEY_TABLE	PDT_GENERIC_20[]
Group Key Table	53 = PID_GRP_KEY_TABLE	PDT_GENERIC_18[]
Security Individual Address Table	54 = PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE	PDT_GENERIC_08[]
Security Failures Log (SFL)	55 = PID_SECURITY_FAILURES_LOG	PDT_FUNCTION
Tool Key	56 = PID_TOOL_KEY	PDT_GENERIC_16
Security Report	57 = PID_SECURITY_REPORT	PDT_BITSET8
Security Report Control	58 = PID_SECURITY_REPORT_CONTROL	PDT_BINARY_INFORMATION
Sequence Number Sending	59 = PID_SEQUENCE_NUMBER_SENDING	PDT_GENERIC_06
Zone Key Table	60 = PID_ZONE_KEY_TABLE	PDT_GENERIC_19[]
Group Object Security Flags	61 = PID_GO_SECURITY_FLAGS	PDT_GENERIC_01[]

There are also the following, additional security Resources, which are not available as Property in the Security Interface Object.

- The Factory Default Setup Key (FDSK) see 6.1
- The Sequence Number for Tool Access see 6.2

Security Link Resources

The Security Link Resources is a term that allows to refer as a whole to the collection of the following Resources.

- The Point-to-point Key Table
- The Group Key Table
- The Security Individual Address Table
- The Tool Key.

6.3.2 PID_OBJECT_TYPE (PID: 1)

- **Property name:** Interface Object Type
- **Property Datatype:** PDT_UNSIGNED_INT
- **Datapoint Type:** DPT_PropDataType (DPT_ID = 7.010)
- **Access policy:** 3FF/0CC (see the specification of Access Policies in [08])
- **Access level:** 3/X

Please refer to clause 4.2.1 for the general requirements of PID_OBJECT_TYPE.

This Property shall contain the Object Type of the Interface Object. The Security Interface Object shall have Object Type 17.

6.3.3 PID_OBJECT_NAME (PID: 2)

- **Property name:** Interface Object Name
- **Property Datatype:** PDT_UNSIGNED_CHAR[]
- **Datapoint Type:** None.
- **Access policy:** 3FF/0CC (see the specification of Access Policies in [08])
- **Access level:** 3/X

Please refer to clause 4.2.2 for the general requirements for PID_OBJECT_NAME.

This Property shall contain the name of the Security Interface Object.

6.3.4 PID_LOAD_STATE_CONTROL (PID: 5)

- **Property name:** Load Control
- **Property Datatype:** PDT_CONTROL
- **Datapoint Type:** None.
- **Access policy:** 15F/04C (see the specification of Access Policies in [08])
- **Access level:** 2/2

This Property *Load Control* shall control the load state machine of all configuration data of the Security Interface Object. The load state machine shall comply with the “Realisation Type 1” as specified in clause 4.23.2.

The *Load Control* shall influence the evaluation of the Security Resources as specified for each Resource.

NOTE 97 This considers the following Resources.

- Point-to-point Key Table
- Group Key Table
- Security Individual Address Table
- Zone Key Table

The Tool Key and the Security Mode however shall not depend on the Load State. Please consult the formal Resource specifications for the correct and full formal requirements. This note is only informative.

The Load Control shall only be accessible using the Role “Tool” (this is the Security Key listed in the *Tool Key* - see 6.3.10 or the FDSK – see 6.1), this is, with the Tool Access flag set in the S-A_Data-service and with the security conditions (A+C) as listed above.

NOTE 98 The Load Control is designed for S-Mode Configuration. Access to the Security Resources in PB-Mode will be specified separately.

6.3.5 PID_SECURITY_MODE (PID: 51)

- **Property name:** Security Mode
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** This Property is a Function Property. The coding of the data depends on whether data is written to the function or responded by the function. No single DPT can be given.
- **Access policy:** 15F/04C (see the specification of Access Policies in [08])
- **Access level:** 2/2

This Property shall be used to enable and disable the Security Mode, as specified in [08].

This Property shall be a Function Property.

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

his Property shall only be writeable using Secure Communication, regardless of its value. This means that it shall only be possible to request the command enable and disable Security Mode using secure communication. Even if Security Mode is disabled, it shall only be possible to enable it by using secure communication.

NOTE 99 This is reflected in the Access Policies of this Property.

The Security Mode does not depend on the Load State of the Security Interface Object.

6.3.5.1 Write (A_FunctionPropertyCommand-PDU)

Basic format and common handling

octet 10	octet 11	octet 12 ... octet n
Reserved	ServiceID	ServiceInfo
	WriteServiceID	
00h	See below.	See below.

Overview Write ServiceIDs

Table 100 – Overview PID_SECURITY_MODE WriteServiceIDs

WriteServiceID	Description
00h	Write the Security Mode

WriteServiceID 00h: Write the Security Mode

octet 10	octet 11	octet 12
	ServiceID	ServiceInfo
reserved	Write Security Mode	Security Mode
00h	00h	

Figure 69 - A_FunctionProperty_Write-PDU for Write ServiceID 00h for PID_SECURITY_MODE

The MaC shall use this command to disable or enable the Security Mode.

ServiceID: 00h Command “Write Security Mode”

ServiceInfo: Security Mode 00h : Disable the Security Mode.

01h : Enable the Security Mode.

If the MaS can successfully handle the command then it shall set the Security Mode as requested and respond with an A_FunctionPropertyState_Response-PDU as in Figure 70.

If the MaS has any problem then it shall respond as specified in 6.3.5.3.

The MaS shall respond with an A_FunctionPropertyState_Response-PDU with the appropriate positive or negative Return Code as listed 6.3.5.3.

NOTE 100 This Response-PDU repeats the WriteServiceID but does not indicate the current resulting Security Mode. This shall be clear from the Return Code.

octet 10	octet 11
Return Code	ServiceID
	Write Security Mode
	00h

Figure 70 - A_FunctionPropertyState_Response-PDU for WriteServiceID 00h for PID_SECURITY_MODE

6.3.5.2 Read (A_FunctionPropertyState_Read-PDU)

Basic format and common handling

octet 10	octet 11	octet 12 ... octet n
Reserved	ReadServiceID	ServiceInfo
00h	See below.	See below.

If the MaS can respond to the command then it shall respond with an A_FunctionPropertyState_Response-PDU with a positive Return Code, the ReadServiceID and the ServiceInfo as from the request (if available) and the command result as specified below.

Overview ReadServiceIDs

Table 101 – Overview PID_SECURITY_MODE ReadServiceIDs

ReadServiceID	Description
00h	Read the current Security Mode

ReadServiceID 00h: Read PID_SECURITY_MODE

This command shall be used to read PID_SECURITY_MODE information.

octet 10	octet 11
Reserved	ReadServiceID
00h	00h

If the MaS has any problem, then it shall respond as specified in 6.3.5.3.

If the MaS can successfully respond to the request then it shall respond with an A_FunctionPropertyState_Response-PDU as in Figure 71.

octet 10	octet 11	octet 12
Return Code	ReadServiceID	ServiceInfo
00h	00h	Security Mode

Figure 71 - A_FunctionPropertyState_Response-PDU for ReadServiceID 01h for PID_SECURITY_MODE

ServiceInfo: Security Mode The encoding values shall be the same as for writing the Security Mode: see Figure 69.

6.3.5.3 Common error – and exception handling for PID_SECURITY_MODE

In case of any error or exception, the MaS shall return the specified A_FunctionPropertyState_Response-PDU, but with the field Return Code as listed in Table 102 and repeating the ServiceID – ReadServiceID or WriteServiceID - as appropriate.

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11
Return Code	ServiceID
> 7Fh	

Next to the Basic (FFh) – and Generic Negative Return Codes (FEh and F8h), the following Function Property specific Return Codes may be returned as appropriate.

Table 102 – PID_SECURITY_MODE Return Codes

Return Code	ErrorName	ErrorType
F2h	E_COMMAND_INVALID	Invalid command
	An invalid value is requested in the field <i>ServiceID</i> (ReadServiceID or WriteServiceID).	

6.3.5.4 Security Mode and Master Reset

Erase Code	Effect on Security Mode
01h Confirmed Restart	not influenced: Security Mode shall not change.
02h Reset to default state	KNX default: Security Mode shall be disabled.
05h ResetParam	not influenced: Security Mode shall not change
06h ResetLinks	not influenced: Security Mode shall not change
07h Reset to default without IA	not influenced: Security Mode shall not change
n.a. Local Reset to default state	KNX default: Security Mode shall be disabled.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.6 PID_P2P_KEY_TABLE (PID: 52)

- **Property name:** Point-to-point Key Table
- **Property Datatype:** PDT_GENERIC_20[]
- **Datapoint Type:** None
- **Access policy:** 00C/00C (see the specification of Access Policies in [08])
- **Access level:** 2/2

6.3.6.1 Abstract Resource definition

The Point-to-point Key Table shall contain the security keys that the MaS shall use for securing incoming and outgoing point-to-point – and broadcast communication.

NOTE 101 Despite its name, the Point-to-point Key Table is thus also used for secure broadcast communication. Both communication ways in and out shall be controlled by this one single Resource, meaning that a device shall use the same key to decrypt information receiving from another device as well as to encrypt information sent to that device.

The Point-to-point Key Table shall additionally for each communication partner contain the indication which of 16 possible Roles is assigned to that communication partner for incoming messages.

6.3.6.2 Format

Array index	IA_Index (2 octets)	Key (16 octets)	Roles (2 octets)
1			
2			
...			
n			

**Figure 72 – Point-to-point Key Table,
for sending and receiving point-to-point communication**

The table shall be sorted according the numerical value of IA_Index from smallest to largest.

- *IA_Index*

This shall refer to the communication partner by its index in the Security Individual Address Table (PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE).

The secure point-to-point – and broadcast communication or with one communication partner shall use exactly one key. An IA_Index shall therefore appear at maximum one time⁹⁸⁾ in the *Point-to-point Key Table*.

- *Key*

This shall be the key that shall be used for secure communication from and to the device with the IA as referred through the IA_Index.

- *Roles*

This field shall mark the Roles that shall be granted for communication from the device with the IA as referred through the IA_Index. This shall be a 16 bit field of which each bit x shall indicate whether the Role Rx is assigned to that communication partner or not.

NOTE 102 Please refer to the clause “Mandatory aspects of Roles” in [08]: the number of effectively implemented Roles is a decision of the application developer. The below Resource foresees a maximum of 16 Roles. In the implementation of Roles, the developer also has to take into account that additional Roles like “Tool” and “Unlisted” have to be supported. In the internal implementation, more Roles may thus be supported than the ones configured or supported in this Point-to-point Key Table.

	(msb)																(lsb)
bit	b ₁₅	b ₁₄	b ₁₃	b ₁₂	b ₁₁	b ₁₀	b ₉	b ₈	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀	
	R ₁₅	R ₁₄	R ₁₃	R ₁₂	R ₁₁	R ₁₀	R ₉	R ₈	R ₇	R ₆	R ₅	R ₄	R ₃	R ₂	R ₁	R ₀	

Each Role shall be encoded as follows.

Rx	Meaning
0	The sender with IA referred by IA_Index does not have the Role x.
1	The sender with IA referred by IA_Index has the Role x.

Figure 73 – Definition of Rx

⁹⁸⁾ The IA_Index of a communication partner in the Security Individual Address Table (see 6.3.8) will appear zero times if there is no secure point-to-point communication – or broadcast with that device, e.g. if there is only secure group communication with that device. It will appear exactly one time if there is (also) secure point-to-point communication with that device.

The MaS shall evaluate the Roles as part of the handling of the incoming point-to-point - and broadcast service primitives, this is, in the AIL or Management. These will only be verified if these messages are accepted and passed by the S-AL. This is, these flags will not be evaluated if there are errors in the SeqNr or the security (MAC verification, decryption).

6.3.6.3 Usage by the MaC (ETS)

The MaC shall sort the table according the numerical value of IA_Index from smallest to largest.

The MaC shall write unique Key for each IA_Index. The key shall only be assigned for point-to-point – and broadcast communication between the communication partner indicated by the IA_Index and this MaS and shall not be used for any other secure communication in the same installation.

The MaC shall set the field Rx for each Role x that is assigned to the sender with IA_Index; the field Rx shall be cleared (0) explicitly for each Role that is not assigned.

6.3.6.4 Usage by the MaS

The MaS (device) shall use this key both to decrypt secure point-to-point – and broadcast communication received from this device, as well as to encrypt secure communication sent to that device.

The MaS shall also use the contained Role information to conclude on the Roles assigned to the sender with IA_Index.

Load State dependency

If the Load State of the Security Interface Object (see 6.3.4) has any other state than “Loaded”, then the following shall apply.

- If the Point-to-point Key Table is evaluated for reception, then the received message shall be thrown away.
- If the Point-to-point Key Table is evaluated for transmission, then the request primitive shall be confirmed negatively to the P-AL. No message shall be sent to the bus. (See item 3 in “Common error handling” in the specification of the TL-service handling by the S-AL in [07].)

Master Reset

Erase Code	Effect on the Point-to-point Key Table
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: cleared or reset to default values.
05h ResetParam	not influenced: no change.
06h ResetLinks	not influenced: no change.
07h Reset to default without IA	KNX default: cleared or reset to default values.
n.a. Local Reset to default state	KNX default: cleared or reset to default values.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.7 PID_GRP_KEY_TABLE (PID: 53)

- **Property name:** Group Key Table
- **Property Datatype:** PDT_GENERIC_18[]
- **Datapoint Type:** None
- **Access policy:** 00C/00C (see the specification of Access Policies in [08])
- **Access level:** 2/2

6.3.7.1 Abstract Resource definition

The Group Key Table shall contain the security keys that the MaS shall use for securing incoming and outgoing Standard Mode group communication.

Both communication ways are controlled by this one single Resource, meaning that a device shall use the same key to decrypt information receiving on that GAs well as to encrypt information sent on that GA.

6.3.7.2 Format

Array index	GA_Index	Key
	(2 octets)	(16 octets)
1		
2		
...		
m		

**Figure 74 – Group Key Table,
for sending and receiving Standard Mode group communication**

The table shall be sorted according the numerical value of GA_Index from smallest to largest.

- *GA_Index*

The GA_Index shall refer to the Group Address by its TSAP value in the Group Address Table.

NOTE 103 In the Property based Group Address Table Realisation Types, the GA_Index and the TSAP have the same value, equal to the Index in the Array Property Value where the GA is stored; the TSAP numbering starts at 1. In the memory mapped Realisation Types of the Group Address Tables, in between “Length” and “Group Address Nr. 1”, there is the “Individual Address”; the TSAP numbering however starts at 0 with the Individual Address, so that also here, the “Group Address Nr. 1” has the same TSAP-value 1.

- *Key*

This shall be the key that shall be used for secure Standard Mode group communication from and to the device with the GA as referred through the GA_index.

6.3.7.3 Usage by the MaC (ETS)

The MaC shall sort the table according the numerical value of GA_Index from smallest to largest.

The MaC shall write unique Key for each GA_Index. The same key shall be assigned to all GA entries for the same GA in all Group Key Tables in the installation.

6.3.7.4 Usage by the MaS

The MaS (device) shall use this key both to decrypt secure Standard Mode group communication received through this GA, as well as to encrypt secure Standard Mode group communication sent through this GA.

Load State dependency

If the Load State of the Security Interface Object (see 6.3.4) has any other state than “Loaded”, then the following shall apply.

- If the Group Key Table is evaluated for reception, then the received message shall be thrown away.
- If the Group Key Table is evaluated for transmission, then the request primitive shall be confirmed negatively to the P-AL. No message shall be sent to the bus. (See item 3 “Common error handling” in the specification of the TL-service handling by the S-AL in [07].)

Master Reset

Erase Code	Effect on Group Key Table
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: cleared or reset to default values.
05h ResetParam	not influenced: no change.
06h ResetLinks	KNX default: cleared or reset to default values.
07h Reset to default without IA	KNX default: cleared or reset to default values.
n.a. Local Reset to default state	KNX default: cleared or reset to default values.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.8 PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE (PID: 54)

- **Property name:** Security Individual Address Table
- **Property Datatype:** PDT_GENERIC_08[]
- **Datapoint Type:** None
- **Access policy:** 00C/00C (see the specification of Access Policies in [08])
- **Access level:** 2/2

6.3.8.1 Abstract Resource definition

The *Security Individual Address Table* shall store the IAs of the communication partners to which the MaS has secure links. The table shall additionally for each IA store the *Last Valid SeqNr* that is accepted from any message from that IA.

6.3.8.2 Security

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

Please refer to clause 1.3 as well.

6.3.8.3 Format

The *Security Individual Address Table* shall be an array Property as shown in Figure 75. Each element shall be composed of the two octets IA and the six octets *Last Valid SeqNr* that is accepted from this IA.

The MaC shall sort the Individual Address Table from the lower indexes to the higher, according the numerical value of the Individual Address.

Array index = IA_Index	Individual Address (2 octets)	Last Valid SeqNr (6 octets)
1	IA 1	Last Valid SeqNr from IA 1
2	IA 2	Last Valid SeqNr from IA 2
...	...	...
m	IA m	Last Valid SeqNr from IA m

Figure 75 – Security Individual Address Table

6.3.8.4 Usage by the MaS

If the MaS needs an IA for reception or transmission, then it shall check the IA in the Security Individual Address Table. For point-to-point communication, it shall use the found *IA_Index* to find the key in the Point-to-point Key Table.

If the IA is not found in the *Security Individual Address Table*, then the MaS shall assume that it does not have a KNX Data Security link with the KNX device on that IA. If Roles are used, this means the Role shall be “Unlisted”.

Last Valid SeqNr

NOTE 104 The following is valid for S-A_Data-PDUs with T=0 (not using the Tool Key). The Sequence Number for Tool Access (see 6.2) is stored outside the *Security Individual Address Table*.

If the MaS receives an S-A_Data-PDU from a given SA, it shall compare the contained SeqNr with the value stored in this table (see “Handling of the Sequence Number” in the specification of the Secure Application Layer in [06]).

If the S-AL can successfully decrypt an S-A_Data-PDU and forward the contained AL-service to the P-AL, then it shall update the field *Last Valid SeqNr* for that IA in the *Security Individual Address Table* with the received value, regardless of the TSAP-type.

Synchronisation

This *Last Valid SeqNr* for a communication partner shall be responded and possibly updated according the specification of the S-A_Sync-service ([07]).

All values of the Last Valid SeqNr in all entries of the *Security Individual Address Table* shall be saved in full at power-down and be restored in full at power-up.

Load State dependency

If the Load State of the Security Interface Object (see 6.3.4) has any other state than “Loaded”, then the following shall apply.

- If the Security Individual Address Table is evaluated for reception, then the received message shall be thrown away.
- If the Security Individual Address Table is evaluated for transmission, then the request primitive shall be confirmed negatively to the P-AL. No message shall be sent to the bus. (See item 3 “Common error handling” in the specification of the TL-service handling by the S-AL in [07].)

Master Reset

Erase Code	Effect on PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: cleared or reset to default values.
05h ResetParam	not influenced: no change.
06h ResetLinks	not influenced: no change.
07h Reset to default without IA	KNX default: cleared or reset to default values.
n.a. Local Reset to default state	KNX default: cleared or reset to default values.
a	It is possible that this is not zero and that the device is delivered ex-factory with some IAs.
b	For E-Mode devices, the Resources are not standardised; the implementation may add channel information to the links and IAs and the command may then also possibly impact only the requested channel.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.8.5 Usage by the MaC (ETS)

The MaC shall during Configuration fill the Security Individual Address Table with the IAs as assigned for group- and point-to-point communication to the MaS.

The MaC shall sort the entries according the numerical values of the Individual Addresses from low to high. Consequently, if the MaC removes or changes an IA, the entire Point-to-point Key Table and Security Individual Address Table shall be rewritten.

- If the MaC knows the Last Valid SeqNr of the communication partner, then it shall write this value in the field Last Valid SeqNr.
EXAMPLE 45 If the MaC has firstly read out the Last Valid SeqNr from the communication partner, prior to configuring this MaS.
- If the MaC does not know the Last Valid SeqNr of the communication partner, then it shall write the value 000000000000h.

6.3.9 PID_SECURITY_FAILURES_LOG (PID: 55)

- **Property name:** Security Failures Log (SFL)
- **Property Datatype:** PDT_FUNCTION
- **Datapoint Type:** None
- **Access policy:** 1FF/0CC (see the specification of Access Policies in [08])
- **Access level:** 3/2

6.3.9.1 Abstract Resource definition

This Property shall be a Function Property through which the following security failure information shall be available.

- The number of security errors that have occurred in the S-AL and in the AIL per type of failure, commonly for all communication modes.
- The most recent security error(s) that have occurred.

6.3.9.2 Security

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

Location

Opposite to other Property Values, it is not required that this Property Value is protected from any access.

The SFL may be stored in non-volatile memory but shall at least be saved at power-down and restored at power-up.

In default state, the Security Failure Log shall be “empty”, that is

- all counters shall have the value 0, and
- there shall be no logged message with security failures.

Master Reset

The MaS shall reset the SFL to its default value as specified below; any other Erase Code shall not clear the SFL.

Erase Code	Effect on PID_SECURITY_FAILURES_LOG
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: cleared.
05h ResetParam	not influenced: no change.
06h ResetLinks	not influenced: no change.
07h Reset to default without IA	KNX default: cleared.
n.a. Local Reset to default state	KNX default: cleared.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.9.3 Format

6.3.9.3.1 Write (A_FunctionProperty_Command-PDU)

Basic format and common handling

octet 10	octet 11	octet 12 ... octet n
Reserved	ServiceID	ServiceInfo
	WriteServiceID	
00h	See below.	See below.

Overview Write ServiceIDs

Table 103 – Overview SFL Write ServiceIDs

WriteServiceID	Description
00h	Clear error log

WriteServiceID 00h: Clear error log

octet 10	octet 11	octet 12
reserved	ServiceID	ServiceInfo
	Clear error log	
00h	00h	See below.

Table 104 – Overview ServiceInfo for WriteServiceID 00h

ServiceInfo	Description
00h	The MaC shall use this command to clear all implemented security failure log data that can be accessed through this Property. NOTE 105 This means security failure counters (ReadServiceID 00h) and all latest security failures (ReadServiceID 01h). NOTE 106 If it would appear necessary in the future to differentiate between the security failure log information to be cleared, then this may be done through a selector in the field <i>ServiceInfo</i>. This command shall not reset the “Security Report” (6.3.11).

If the MaS has any problem then it shall respond as specified below (6.3.9.3.3).

If the MaS can successfully handle the command then it shall clear all security failure log information and respond with an A_FunctionPropertyState_Response-PDU as in Figure 76.

octet 10	octet 11
Return Code	WriteServiceID
00h	00h

Figure 76 - A_FunctionPropertyState_Response-PDU for WriteServiceID 00h

6.3.9.3.2 Read (A_FunctionPropertyState_Read-PDU)

Basic format and common handling

octet 10	octet 11	octet 12... octet n
Reserved	ServiceID	ServiceInfo
	ReadServiceID	
00h	See below.	See below.

If the MaS can respond to the command then it shall respond with an A_FunctionProperty-State_Response-PDU with the appropriate Positive Return Code, the ReadServiceID as from the request and the command result as specified below.

Overview ReadServiceIDs

Table 105 – Overview SFL ReadServiceIDs

ReadServiceID	Description
00h	Read security failure counters
01h	Read latest security failures

ReadServiceID 00h: Read security failure counters

The MaS shall count the Security Failures that occur as specified elsewhere.

If any counter of the SFL reaches its maximal value of 65 535, it shall not wrap around. There is no dedicated handling required concerning the counter overflow: the last increment of the SFL-field may cause a transmission of a *Security Report* (see 6.3.11) if this is enabled in the *Security Report Control* (see 6.3.12). There are no further requirements to the MaS. Other additional, implementation specific behaviour is allowed.

octet 10	octet 11	octet 12
Reserved	ReadServiceID	ServiceInfo
00h	00h	See below.

Table 106 – Overview ServiceInfo for ReadServiceID 00h

ServiceInfo	Description
00h	ReadCommon

If the MaS has any problem then it shall respond as specified below (6.3.9.3.3).

If the MaS can successfully respond to the request then it shall respond with an *A_FunctionPropertyState_Response-PDU* as in Figure 77.

octet 10	octet 11	octet 12
Return Code	ServiceID	ServiceInfo
	ReadServiceID	
00h	00h	as in the request

octet 13 and 14		octet 15 and 16		octet 17 and 18		octet 19 and 20	
ServiceInfo							
reserved		Counter of SeqNr errors		Cryptographic errors		Counter of Access or Roles errors	
MSB	LSB	MSB	LSB	MSB	LSB	MSB	LSB
0000h							

Figure 77 - A_FunctionPropertyState_Response-PDU for ReadServiceID 00h

ServiceInfo:	Cryptographic errors	This field shall be the counter for cryptographic errors. This shall exclusively be incremented under the following conditions. 1. A service with a fixed length (e.g. S-A_Sync) does not have the expected length. 2. the Frame is too short to be a secure Frame (e.g. only 3 octets, so not possible to contain a MAC); the minimal value would be 12 octets for S-A_Data. 3. For CCM, this shall additionally include MAC verification errors. This counter shall not be incremented on any further condition, implementation specific or other.
--------------	----------------------	--

ReadServiceID 01h: Read latest failure n

octet 10	octet 11	octet 12
Reserved	ServiceID	ServiceInfo
	ReadService-ID	Index
00h	01h	n

ServiceInfo:	Index:	The Index of the message that shall be returned.
	0:	the latest message
	1:	the lasted minus 1 message
	...	

The MaS shall store the latest messages that it received that caused a security error. The MaS shall store at least the latest message, $n = 0$; storing more messages than the latest message ($n > 0$) is optional.

If the MaS can successfully respond to the request then it shall respond with an A_FunctionPropertyState_Response-PDU as in Figure 78.

octet 10	octet 11	octet 12	octet 13 and 14	octet 15 to 16
Return Code	ServiceID	ServiceInfo		
	ReadServiceID	Index	Sender IA	not used 1
00h	01h			

octet 17	octet 18 to 23	octet 24
ServiceInfo		
not used 2	Sender SeqNr	ErrorType

Figure 78 - A_FunctionPropertyState_Response-PDU for ReadServiceID 01h

- **Sender IA**

This shall be the IA contained in the message that causes the security error.

- **not used 1**

This field is not used and may contain any value. The MaC shall not show or interpret the value of this field.

- **not used 2**

This field is not used and may contain any value. The MaC shall not show or interpret the value of this field.

- **Sender SeqNr**

If the message that causes the security error does not contain any Sender SeqNr (plain message), then this field may contain any value, else, it shall contain the SeqNr contained in the message that causes the security error.

As the Error Type 04h below also includes attempts for plain access to protected data, the MaC shall thus not interpret this field in case the Error Type is 04h.

- **Error Type**

This shall be the indication of the security error that occurs.

Error Type	Definition
00h	Reserved. Shall not be used.
01h	Reserved. Shall not be used. This field is reserved for “Invalid SCF” and may in the future be defined to register secure communication with a wrong security algorithm.
02h	<i>SeqNr error</i> This shall mean that the message contains an invalid (too small) SeqNr value; the appropriate counter (octet 15 and 16) for the ReadServiceID 00h shall also be incremented.
03h	<i>Cryptographic error</i> This shall mean that there is a cryptographic error in the message; the appropriate counter (octet 17 and 18) for the ReadServiceID 00h shall also be incremented.
04h	<i>Error against the Access and Roles</i> This shall mean that the secure message could properly be verified and decrypted, but that it attempts to access • a GO with different security settings than required in the GO Security Flags for an addressed GO (see 6.3.15), or • a Property Value or other Resource for which the access definitions refuse the access.

If the MaS cannot respond to the request because of any error, then it shall respond with the appropriate negative Return Code, a repetition of the ServiceID and without the field ServiceInfo.

6.3.9.3.3 Common error – and exception handling for SFL

In case of any error or exception, the MaS shall return the specified A_FunctionPropertyState_Response-PDU, but with the field Return Code as listed in Table 107 and repeating the ServiceID – ReadServiceID or WriteServiceID as appropriate.

Response (A_FunctionPropertyState_Response-PDU)

octet 10	octet 11
Return Code	ServiceID
> 7Fh	

Table 107 – SFL Return Codes

Return Code	ErrorName	ErrorType
FFh	E_ERROR	Error
	The service, function or command has failed without a closer indication of the problem. This Return Code shall only be used in case of an error for which no Return Code is defined or supported.	
F2h	E_COMMAND_INVALID	Invalid Command
	This Return Code shall be used in case a not defined or not supported ReadServiceID or WriteServiceID is requested.	
F8h	E_DATA_VOID	void data
	This Return Code shall be used in case any field in the request, except the ReadServiceID or WriteServiceID has an invalid or not supported value. This Return Code shall be used if octet 10 in the request does not equal 00h. This Return Code shall be used if the field ServiceInfo has an invalid or not supported value.	

6.3.10 PID_TOOL_KEY (PID: 56)

- **Property name:** Tool Key
- **Property Datatype:** PDT_GENERIC_16
- **Datapoint Type:** None
- **Access policy:** 008/008 (see the specification of Access Policies in [08])
- **Access level:** X/2

6.3.10.1 Abstract Resource definition

The Tool Key shall be used to store the security information for the central MaC in KNX S-Mode (ETS[®]) and KNX Ctrl-Mode.

NOTE 107 The Tool Key is specified as a stand-alone Resource, outside the Point-to-point Key Table, to allow a MaC to clear all Security Configuration Link data without clearing its own Tool Key and hence possibly locking itself out.

6.3.10.2 Security

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

The Tool Key shall not be readable.

Please refer to clause 1.3 as well.

6.3.10.3 Format

The Tool Key shall be a single element Property formatted as described in Figure 79.

**Figure 79 – Tool Key**

The Tool Key is stored outside the Point-to-point Key Table. This shall mean the following.

- It is not linked to any Individual Address or TSAP. This means that communication using the Tool Key shall be possible from any Source Address and in any communication mode.
- There is no Resource defined for accessing the Sequence Number for Tool Access (see 6.2). This can only be read using the S-A_Sync-service. The MaS shall store the last valid SeqNr from the MaC internally.
- There is no differentiation between incoming – and outgoing messages. The MaC and the MaS shall use the same Tool Key.

6.3.10.4 Usage by the MaS (device)

Configuration

The Tool Key shall become active immediately after it is written by the MaC. The A_PropertyValue_Response-PDU to confirm the A_PropertyValue_Write-PDU shall immediately be protected using the newly written Tool Key.

The FDSK and the Tool Key shall have the same function, but shall not be active at the same time. The MaS shall internally keep track of whether the FDSK applies or the Tool Key.

The Tool Key shall not be influenced by the Load State of the Security Interface Object! Even if the Point-to-point Key Table is unloaded, the Tool Key shall remain valid.

The Tool Key shall only be used with “authentication and confidentiality”; it shall not be used with “authentication only”.

Runtime - transmission of a Secure APDU

If a secure APDU is transmitted and encrypted according the Tool Key then the field *Tool Access* in the SCF shall be set; otherwise, it shall be cleared.

Runtime - reception of a Secure APDU

If a secure APDU is received for which the field *Tool Access* is set, then it shall be decoded using the Tool Key; otherwise, it shall be decoded using the key according the communication mode (Point-to-point Key Table, Group Key Table or Zone Key Table).

Master Reset

Erase Code	Effect on the Tool Key
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: inactive.
05h ResetParam	not influenced: no change.
06h ResetLinks	not influenced: no change.
07h Reset to default without IA	not influenced: no change
n.a. Local Reset to default state	KNX default: inactive.

NOTE 108 See also 6.1.4: after “Reset to default” (“Reset to default state”, “Reset to default without IA” and “Local Reset to default state”) the Tool Key is no longer used and the FDSK becomes active again.

The Tool Key shall become inactive according the above table. Yet, the A_Restart_Response-PDU that confirms this Master Reset *operation* shall in any case still be protected with the active Tool Key and not yet with the FDSK.

NOTE 109 This is different from the A_PropertyValue_Response-PDU that confirms the *message* of the setting of the Tool Key, which shall already be confirmed with the new Tool Key: see 6.3.10.4.

6.3.10.5 Usage by the MaC

The MaC shall use the Tool Key to authorize itself for accessing any Resource that it intends to access as the Role “Tool” (the Group Key Table, the Security Individual Address Table, the Security Failures Log, the Zone Key Table, etc. This list is not complete.)

EXAMPLE 46 The MaC may through this reset the Security Failure Counter.

If the MaS is in default state, the MaC shall set the Tool Key after it assigns the IA of the MaS and prior to managing any further Resource in the MaS. The MaC shall to this secure the communication using the Factory Default Setup Key.

If the MaS is not in default state, then the MaC shall use the already configured Tool Key for any next access to this Tool Key.

The MaS shall properly handle the change or the activation of the Tool Key: if the Tool Key is assigned (after the FDSK is used) or modified (after another Tool Key value is used), then the MaS shall confirm that modification with the newly written value of the Tool Key. The MaC shall protect the next message to the MaS with the newly assigned Tool Key. The assignment – or change of the Tool Key shall not lead to loss of communication between MaC and MaS or to error messages.

ETS shall generate a unique, random Tool Key for each separate device.

6.3.11 PID_SECURITY_REPORT (PID: 57)

- **Property name:** Security Report
- **Property Datatype:** PDT_BITSET8
- **Datapoint Type:** DPT_Security_Report (see [14])
- **Access policy:** 1FF/0CC (see the specification of Access Policies in [08])
- **Access level:** 3/2

6.3.11.1 Abstract Resource Definition

This Property shall be used to announce KNX Data Security related status - and diagnostic information.

This shall only serve for a basic security reporting. This specification does not foresee extended reporting (adding time of events, building logs...).

6.3.11.2 Security

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

Opposite to other Property Values, it is not required that this Property Value is protected from any access.

6.3.11.3 Format

The Security Report shall be encoded according DPT_Security_Report as specified in [14].

6.3.11.4 Usage by the MaS

If any security failure is detected then MaS shall set the field *Security Failure*. If the Parameter Property Security Report Control is set to “Enabled” then additionally this Property *Security Report* shall be communicated using the Management Procedure DMP_InterfaceObjectInfoReport_RCI (see [09]). This shall be done regardless of whether the field *Security Failure* is set prior to this security failure or not. This is, the MaS shall report any security failure, even if a security failure is reported before or not.

```
/* Report a KNX Data Security failure. */
DMP_InterfaceObjectInfoReport_RCI (mpp_ASAP = void, mpp_comm_mode = broadcast, mpp_hop_count_type = 6,
    mpp_object_type = Security Interface Object, mpp_PID = PID_SECURITY_REPORT, mpp_priority = urgent,
    mpp_test_info = 00h, mpp_test_result = SecurityReport)
```


The MaS shall not automatically reset the Security Failure. This shall only be possible by the MaC, in a secure communication.

In order to support also receivers that do not support the DMP_InterfaceObjectInfoReport_RCl or do not support Property Reading, the MaS may also communicate the security report via a Group Object. The DPT_Security_Report shall be used to this purpose.

Master Reset

Erase Code	Effect on PID_SECURITY_REPORT
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: cleared.
05h ResetParam	not influenced: no change.
06h ResetLinks	KNX default: cleared.
07h Reset to default without IA	KNX default: cleared.
n.a. Local Reset to default state	KNX default: cleared.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.11.5 Usage by the MaC

The MaC may receive this Property value communicated with A_NetworkParameter_InfoReport.

The MaC shall read this Property to find out if there is any security failure inside the MaS. If any security failure is indicated, then the MaC may find more detailed information in the Security Failures Log (see 6.3.9).

The MaC may write this Property to reset it, after it has properly authenticated itself using secure communication.

6.3.12 PID_SECURITY_REPORT_CONTROL (PID: 58)

- **Property name:** Security Report Control
- **Property Datatype:** PDT_BINARY_INFORMATION
- **Datapoint Type:** DPT_Enable (DPT_ID: 1.003)
- **Access policy:** 00C/00C (see the specification of Access Policies in [08])
- **Access level:** 2/2

6.3.12.1 Abstract Resource Definition

The *Security Report Control* shall be a Property Parameter that shall control the spontaneous communication of the *Security Report* through the procedure DMP_InterfaceObjectInfoReport_RCl. This parameter shall not influence the access to the *Security Report* through Data Property Services (A_PropertyValue_Read and A_PropertyValue_Write), neither the possible group communication of the *Security Report*.

6.3.12.2 Security

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

6.3.12.3 Format

This shall be a single bit, Boolean Property, encoded according DPT_Enable (DPT_ID: 1.003).

The default value shall be “Disabled”.

6.3.12.4 Usage by the MaS

If the *Security Report Control* has the value “Enabled” then the MaS shall communicate the *Security Report* with DMP_InterfaceObjectInfoReport_RCI as specified in 6.3.11.

If the *Security Report Control* has the value “Disabled” then the MaS shall not communicate the *Security Report* with DMP_InterfaceObjectInfoReport_RCI.

The *Security Report Control* shall only influence the communication to the *Security Report* through the DMP_InterfaceObjectInfoReport_RCI; it shall not influence the access to the *Security Control* through data Property services.

Master Reset

Erase Code	Effect on PID_SECURITY_REPORT_CONTROL
01h Confirmed Restart	not influenced: the value shall not change.
02h Reset to default state	KNX default: the value shall be set to “Disable”.
05h ResetParam	not influenced: no change.
06h ResetLinks	not influenced: no change.
07h Reset to default without IA	KNX default: the value shall be set to “Disable”.
n.a. Local Reset to default state	KNX default: the value shall be set to “Disable”.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.12.5 Usage by the MaC

The MaC shall clear or set this Property to disable or respectively enable the security failure reporting by the MaS.

The MaC shall use the Data Property services (A_PropertyValue_Read, A_PropertyValue_Write) to this purpose.

6.3.13 PID_SEQUENCE_NUMBER_SENDING (PID: 59)

- **Property name:** Sequence Number Sending
- **Property Datatype:** PDT_GENERIC_06
- **Datapoint Type:** None
- **Access policy:** 00C/00C (see the specification of Access Policies in [08])
- **Access level:** 2/2

6.3.13.1 Abstract Resource definition

This Property shall store the Sequence Number that shall be used by the MaS for its next outgoing secure communication.

For the common specification of the Sequence Number, please refer to the clause “Handling of the Sequence Number” in the specification of the Secure Application Layer in [06].

6.3.13.2 Security

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

6.3.13.3 Usage by the MaS (device)

The MaS shall handle the *Sequence Number Sending* as specified for the sender in “Handling of the Sequence Number” in the specification of the Secure Application Layer in [06]

If the MaC writes the SeqNr with a new value, then the MaS shall use this new value for the immediate next secure message that it sends. The MaS shall thus use the new value of the SeqNr already to protect the A_PropertyValue_Response-PDU that confirms the A_PropertyValue_Write-PDU from the MaC that changes the SeqNr.

Synchronisation

The *Sequence Number Sending* shall be responded and possibly updated according the specification of the S-A_Sync-service ([07]).

6.3.13.4 Usage by the MaC

Procedure:

1. The MaC gets its own SeqNr to use and the SeqNr it has to expect from the MaS through the synchronisation mechanism.
2. Then, it can set, using secure communication, the new value of the Sequence Number Sending in the MaS.

NOTE 110 Any new SeqNr written by the MaC applies immediately. Therefore, the MaC shall support that the confirmation of this setting is protected by a different SeqNr.

6.3.14 PID_ZONE_KEY_TABLE (PID: 60)

- **Property name:** Zone Key Table
- **Property Datatype:** PDT_GENERIC_19[]
- **Datapoint Type:** None
- **Access policy:** 00C/00C (see the specification of Access Policies in [08])
- **Access level:** 2/2

6.3.14.1 Abstract Resource definition

The Zone Key Table shall contain the security keys that the MaS shall use for securing incoming and outgoing Zone Addressing communication.

Both communication ways shall be controlled by this one single Resource, meaning that a device shall use the same key to decrypt information receiving on that Zone Address as well as to encrypt information sent on that Zone Address.

6.3.14.2 Format

Array index	TSAP			Key
	Reserved (4 bits)	EFF (4 bits)	Zone Address (2 octets)	(16 octets)
1	0000			
2	0000			
...	0000			
n	0000			

**Figure 80 – Zone Key Table,
for sending and receiving Zone Addressing communication**

The Table shall be sorted according the numerical value of the TSAP from smallest to largest.

- *TSAP*

The TSAP shall be a unique key in this table and shall be the combination of the EFF + Zone Address.

- *EFF*

The EFF (Extended Frame Format) of the Extended Control Field (see clause 2.2.4 in [04]).

- *Zone Address*

The Zone Address shall be equal to the fully qualified Zone Address, even every wildcard address shall be explicit in this table.

- *Key*

This shall be the key that shall be used for secure Zone Address communication from and to the device.

6.3.14.3 Usage by the MaC (ETS)

The MaC shall sort the table according the numerical value of the TSAP from smallest to largest.

The MaC shall write a key for each TSAP. The same key shall be assigned to all TSAP entries for the same TSAP in all Zone Key Tables in the installation.

6.3.14.4 Usage by the MaS

The MaS (device) shall use this key both to decrypt secure Zone Mode communication received through this TSAP, as well as to encrypt secure Zone Mode communication sent through this TSAP.

Load State dependency

If the Load State of the Security Interface Object (see 6.3.4) has any other state than “Loaded”, then the following shall apply.

- If the Zone Key Table is evaluated for reception, then the received message shall be thrown away.
- If the Zone Key Table is evaluated for transmission, then the request primitive shall be confirmed negatively to the P-AL. No message shall be sent to the bus. (See item 3 in “Common error handling” in the specification of the TL-service handling by the S-AL in [07].)

Master Reset

Erase Code	Effect on the Zone Key Table
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: cleared or reset to default values.
05h ResetParam	not influenced: no change.
06h ResetLinks	KNX default: cleared or reset to default values.
07h Reset to default without IA	KNX default: cleared or reset to default values.
n.a. Local Reset to default state	KNX default: cleared or reset to default values.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

6.3.15 PID_GO_SECURITY_FLAGS (PID: 61)

- **Property name:** Group Object Security Flags
- **Property Datatype:** PDT_GENERIC_01[]
- **Datapoint Type:** None
- **Access policy:** 00C/00C (see the specification of Access Policies in [08])
- **Access level:** 2/2

6.3.15.1 Abstract Resource definition

The *Group Object Security Flags* shall for each Group Object define the required security requirements. If a Group Object is accessed in a way that does not respect these *Security Config Flags*, then the Group Object server shall not give access (read or write) to this Group Object.

6.3.15.2 Security

Please refer to the above access requirements. These requirements are exclusive: other Roles, security features or services shall not have access to this Property.

Please refer to clause 1.3 as well.

Format

The Group Object Security Flags shall be an array Property of which each element shall contain the Security Config Flags for one Group Object in the Group Object Table with the same index.

The Security Config Flags of one entry, for one Group Object, shall be formatted as defined in Figure 81.

b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
0	0	0	0	0	0	conf	auth

Figure 81 – Group Object Security Flags

Each entry shall be an 8 bit value. Only the bits b₁ and b₀ are defined. The bits b₇ to b₃ are reserved for future extensions and the MaC shall always write these bit values with 0. The MaS shall ignore these reserved bits.

6.3.15.3 Usage by the MaS (device)

The MaS shall evaluate the Group Object Security Flags as part of the handling of the incoming service primitives of A_GroupValue_Read and A_GroupValue_Write, this is, in the AIL. These will only be verified if these messages are accepted and passed by the S-AL. This is, these flags will not be evaluated if there are errors in the SeqNr or the security (MAC verification, decryption).

Bits b₁b₀ shall specify the KNX Security features for the Group Object⁹⁹⁾. These flags shall be used for all service primitives of the A_GroupValue_Read– and the A_GroupValue_Write-service.

Master Reset

Erase Code	Effect on the Group Object Security Flags
01h Confirmed Restart	not influenced: no change.
02h Reset to default state	KNX default: cleared or reset to default values.
05h ResetParam	not influenced: no change.
06h ResetLinks	KNX default: cleared or reset to default values.
07h Reset to default without IA	KNX default: cleared or reset to default values.
n.a. Local Reset to default state	KNX default: cleared or reset to default values.

See also the specifications for DM_Restart in [09] and the clause “Reset to default” and KNX Security.

⁹⁹⁾ Mainly considered here are the services A_GroupValue_Read– and A_GroupValue_Write. The permissions shall however also apply for alternative accesses to the Group Object Value, like the Group Object Indirection.

Table 108 – Use of the GO Security flags for the GO service primitives

• A_GroupValue_Write.req • A_GroupValue_Read.req • A_GroupValue_Read.res	The Group Object Server shall use the flags auth and conf to correspondingly set the security service parameters par_auth and par_conf in these service primitives. NOTE 111 Specifically for the A_GroupValue_Read.res it shall be noted that this response shall use the configured security flags and does not inherit the security flags from the preceding initiating A_GroupValue_Read.ind.
• A_GroupValue_Write.ind • A_GroupValue_Read.ind • A_GroupValue_Read.Acon	The Group Object Server shall compare the used security service parameters with the configured Group Object Security flags. • If these match exactly, then the service primitive shall be accepted and processed further. • In case these flags do not match exactly then the Group Object server shall ignore the service primitive and not handle it any further; in the <i>Security Failures Log</i> (6.3.9) the “Counter of access or Roles” for ReadServiceID 00h (Read security failure counters) shall be incremented. NOTE 112 This also means that if a GO is configured to require only Authentication, but it is accessed (written or read) with Authentication + Confidentiality, that the GO Server will refuse this. NOTE 113 Such mismatch between the received security service parameters and the configured service parameters can – except for a hack attempt – only occur due to an error in the sender or the receiver or a mistake by the MaC.

6.3.15.4 Usage by the MaC (ETS)

The MaC shall set the Group Object Security Flags in each GO that is associated with a GA for which KNX Data Security is used.

Annex A

(informative)

Use cases for Group Object Diagnostics

A.1 Use case 1: set the value of a GO to diagnose its proper working

Description

The User experiences that a Group Object is not reacting to messages from the bus. KNX Data Security is used and the user wants to exclude that the communication or functional problem is due to KNX Data Security.

The User uses Group Object Diagnostics to **set** the Group Object in point-to-point communication. If the Group Object with this behaves as expected, the User may narrow down the malfunction to KNX Data Security (wrong keys, Sequence Numbers not synchronised, wrong senders IAs in receiver...). If the Group Object still behaves wrongly, then the cause may rather be searched in the configuration of the device (parameters or other).

Sequence diagram

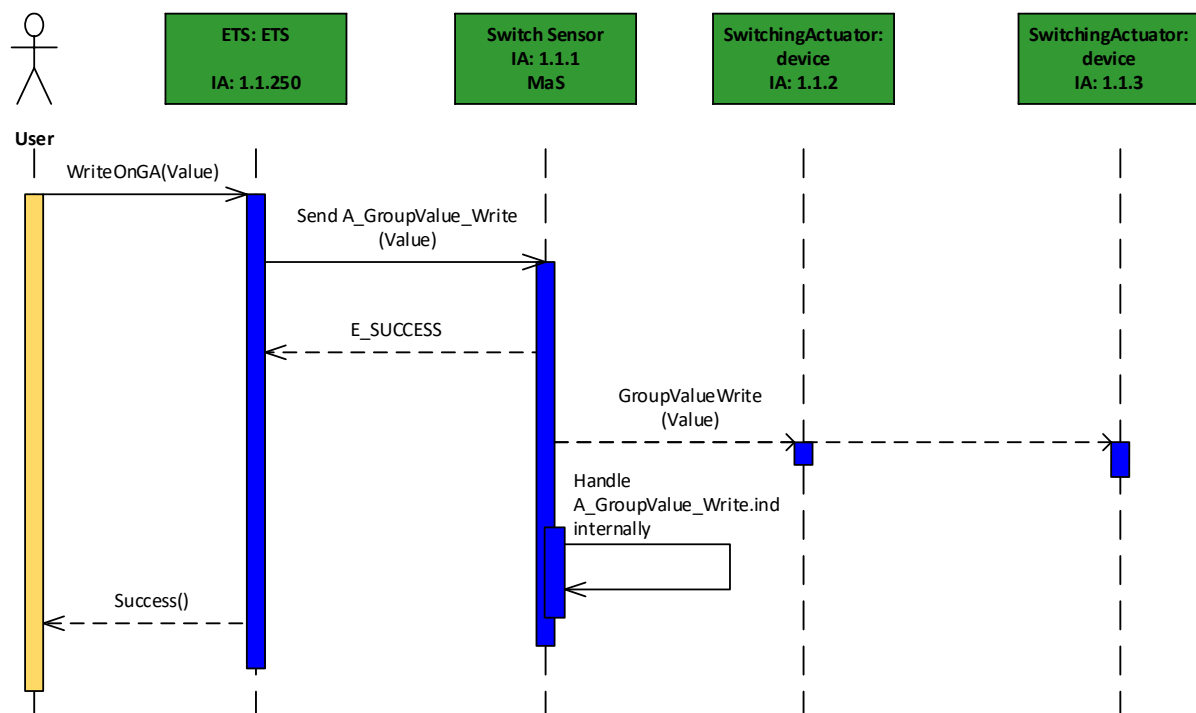


Figure 82 – Use case 1: set GO value

A.2 Use case 2: get the value of a GO to diagnose its proper working

Description

The User experiences that a Group Object is not reacting to messages from the bus. KNX Data Security is used and the User wants to exclude that the communication or functional problem is due to KNX Data Security.

The User uses Group Object Diagnostics to **read** the Group Object in point-to-point communication. If the Group Object has an expected value, the User may narrow down the malfunction to the application level (e.g. parameter settings).

Sequence diagram

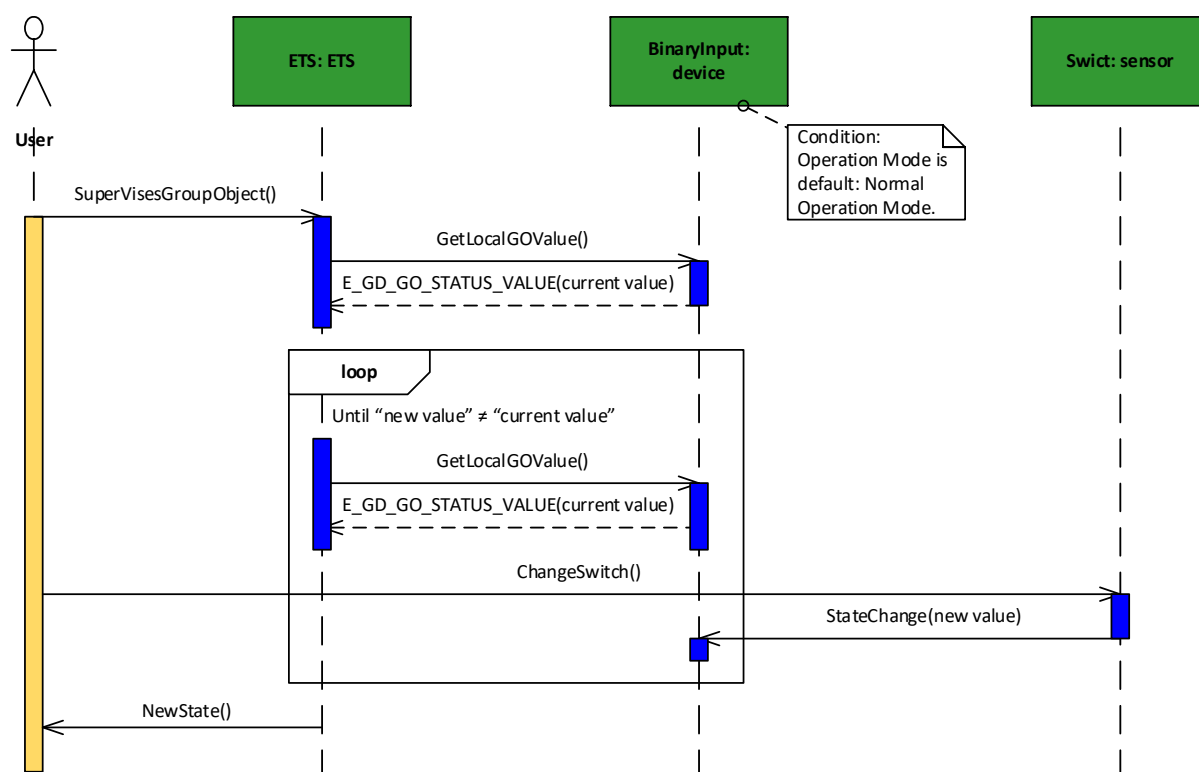


Figure 83 – Get GO value

A.3 Use case 3: path check

Description

The user experiences that the SwitchingActuator (1.1.2) appears not to react to the Telegrams of a BinaryInput (1.1.1). It is suspected that the Telegrams from the BinaryInput are not received by the SwitchingActuator. For this, the BinaryInput is forced to send a Telegram and it is checked if the SwitchingActuator effectively receives that Telegram.

The MaC limits the group communication of the SwitchingActuator to the Individual Address of the BinaryInput (1.1.1) and then forces the BinaryInput to send a group message (to the SwitchingActuator) with a certain value NewValue. If the MaC now reads the state and value of this GO in the SwitchingActuator and finds that the Update-flag is set and the new value is accepted, then the user may conclude that the communication between both devices is working well.

- The MaC shall firstly read the status and value of the GO through a first command `GetLocalGOValue`.
 - If the returned current GO-value “value” equals the value “NewValue” that the MaC intends to set next, then the MaC user should be warned that the reception of NewValue can possibly not unambiguously be detected.
 - If the returned `E_GD_GO_STATUS_VALUE` signals that the Update-flag is currently set, then the MaC shall repeat the request every 1 s until it receives a response in which this flag is cleared or until 6 s in total have passed. If after that the Update-flag is still set, the MaC shall signal to the MaC user that the application does not seem to properly handle a preceding message from the bus.
- To prevent that the SwitchingActuator clears the Update-flag before it can be read by the MaC and possibly changes the GO-value, the MaC will firstly set the Operation Mode to “Diagnostic Mode” in the SwitchingActuator.
- In order to avoid that in the SwitchingActuator in-between the `GroupValueWrite(NewValue)` from the BinaryInput and the `GetLocalValue()` from the MaC the value of the GO changes by a coincidental transmission by e.g. a Switch (1.1.3), the MaC will firstly limit the group communication in the SwitchingActuator to the IA (1.1.1) of the BinaryInput.

Sequence diagram

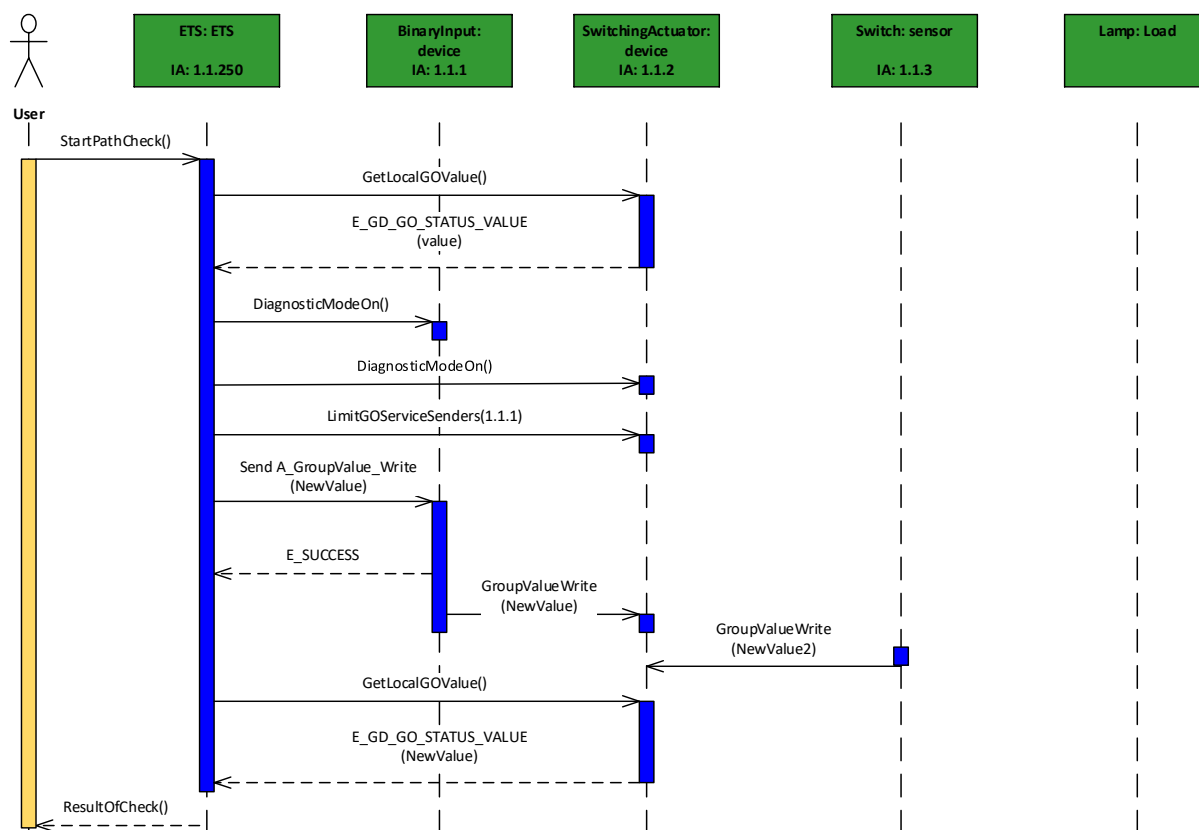


Figure 84 – Path check

Table 109 – Analysis of the conditions and resulting requirements

Object	BinaryInput	SwitchingActuator	Resulting requirement
Stack	Shall work as normal.	Shall work as normal.	None (“Normal Mode”).
Receiving GO services	(No conditions seen.)	Telegrams shall however only be received from the BinaryInput	⇒ = Limit GO service senders

Object	BinaryInput	SwitchingActuator	Resulting requirement
Application	Shall not send other Telegrams (to the SwitchingActuator), which would undo or conflict with the activities of the MaC.	Shall not clear the GO runtime flags. Shall not modify the received GO value.	⇒ The AP shall be halted or shall not have access to the GO value or GO runtime flags. = Diagnostic Mode

A.4 Use case 4: web services (and visualisation)

Description

Group Objects may not only be used for runtime process data, but also to make accessible Status – and Diagnostic Information. This requires that a GA has to be assigned to each of these, and this GA has to be passed through the Filter Tables in the Couplers on the communication path between the device and any possible Client.

The use of Group Object Diagnostics allows the basic access to a GO in point-to-point communication, even if no GA is assigned to it. The Client thus only has to know the IA of the device and the GO number.

Sequence

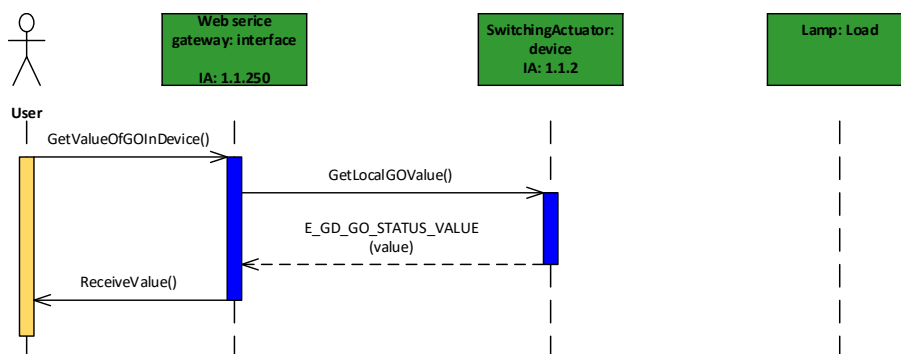


Figure 85 – Get the value of a GO (without GA)

Object	Web service gateway	SwitchingActuator	Resulting requirement
Stack	Shall work as normal.	Shall work as normal.	None ("Normal Mode").
Receiving GO services	(No conditions seen.)	Accesses a specific GO that is in this use case not accessed by other senders: no conditions.	None.
Application	(No conditions seen.)	This and the other GOs may changes as normal. This is the same situation as if the GO would be accessed over standard group Telegrams. No conditions.	None.
Security	May have to have a key for secure point-to-point communication to the SwitchingActuator.	May have to have the IA of the web service gateway. Has to have a key for the web service gateway. IA and key have to give access to PID_GO_DIAGNOSTICS.	(No requirements for GO Diagnostics.)

A.5 Use case 5: write group message from ETS

Description

The MaC (ETS) offers the functionality to send group messages (A_GroupValue_Read and A_GroupValue_Write) from the Group Monitor or from the GA-view. This will for secure GAs however not work as for plain GAs.

- The MaC (ETS) cannot send an S-A_Data-PDU encrypted with the Tool Key using that GA.
According KNX Data Security receivers shall refuse an S-A_Data-PDU received with a T_Data_Group.ind if it is protected with the Tool Key.
Additionally the MaC (ETS) assigns a different, unique Tool Key to each different MaS (device), only the MaS of which the Tool Key is used would properly receive the secure message and handle the message it; the other participants in the group will not handle the message.
- The MaC (ETS) could send an S-A_Data-PDU encrypted with the Group Key used by all MaS for that GA.
This requires that the MaC (ETS) to this purpose uses an IA that is present in the Security Individual Address Table. This is not the case.

As solution, the MaC (ETS) selects one MaS (device) that is capable of sending on this GA and make this MaS (device) send the message on behalf of the MaC (ETS).

Sequence

1. From all devices that are associated with this GA, the MaC (ETS) selects one as its MaS.
If such GO cannot be found, the MaC user shall be informed.
2. The MaC (ETS) executes the command WriteServiceID 01h - Send A_GroupValue_Write on the chosen MaS.
3. The MaS (device) confirms with the Return Code E_SUCCESS (00h).
4. The MaS sends an A_GroupValueRead_Write-PDU on the bus.

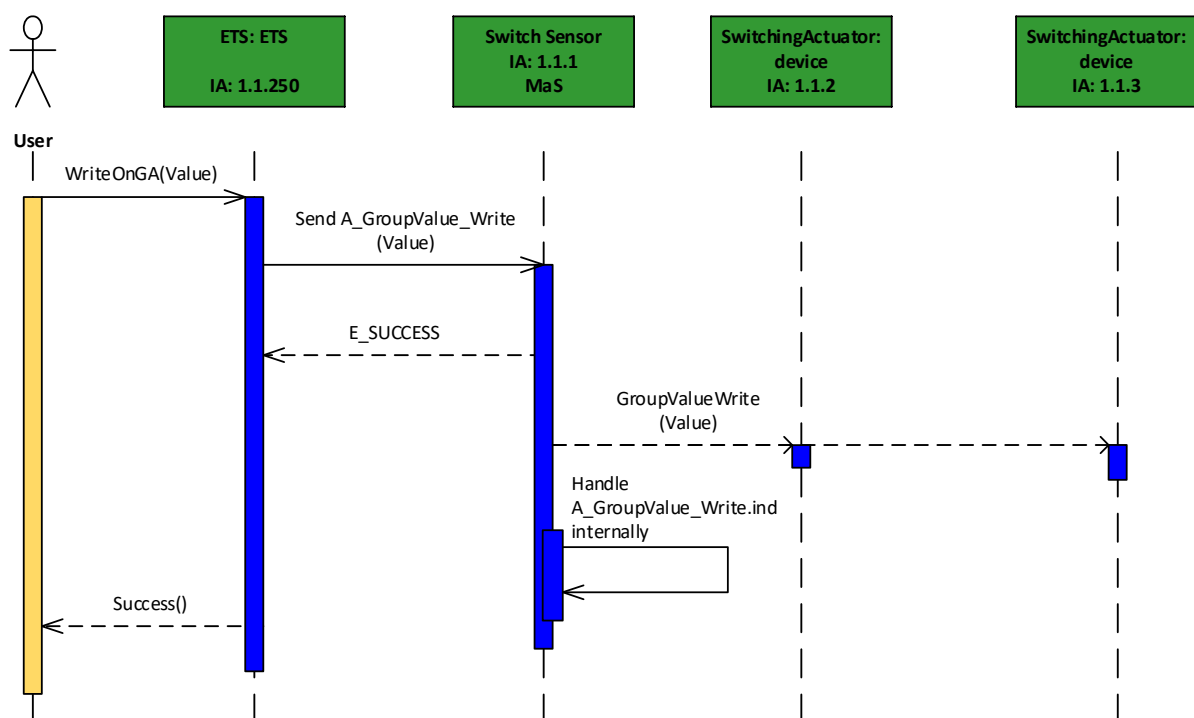


Figure 86 – Send a value on GA

A.6 Use case 6: read group message from ETS

Description

Please refer to the introduction of “Use case 5: write group message from ETS” above for the problem description.

In this use case, the MaC (ETS) user wants to “read the value of a GA” from the bus. Also in this solution, the MaC (ETS) choses a helper device that will perform the A_GroupValue_Read.

Constraints (informative)

- In case of an A_GroupValue_Read, the MaC will only be able to receive the response if it is located in the same Subnetwork as the responding device or if the Coupler Filter Tables between the MaC and the responding device are open for this Group Address. This is the same situation as today. This restriction could be bypassed if the MaC is notified by the MaS about any responses. For now, this is out of scope.

Sequence

1. From all devices that are associated with this GA, the MaC (ETS) selects one as its MaS. If such GO cannot be found, the MaC user shall be informed.
2. The MaC (ETS) executes the command WriteServiceID 03h – Send A_GroupValue_Read on the chosen MaS.
3. The MaS (device) confirms with the Return Code E_SUCCESS (00h).
4. The MaS sends an A_GroupValueRead_Read-PDU on the bus.
5. If the MaC is interested in the further result of this command then it will perform the following loop, while the GO Status is “Transmit Request” or “Transmitting” and the 6 s timeout has not elapsed.
 - ▲ The MaC executes the command ReadServiceID 01h - Get local GO value on the MaS.
 - ▲ The MaS shall confirm with the Return Code E_GD_GO_STATUS_VALUE, which contains the current *GO Status*.
 - ▲ Meanwhile, any other device may respond on the bus with an A_GroupValue_Response-PDU containing a *Value*.

The MaS may read the result of the operation either from the updated value of the used GO in the helper device, as contained in the E_GD_GO_STATUS_VALUE, or it may receive and use the A_GroupValue_Response-PDU send by any other device directly from the bus.

NOTE 114 This has the side effect that the GO value of the MaS will be changed if the U flag is set. This is considered acceptable as it is normal runtime behaviour that also occurs today if the MaC (ETS) sends a A_GroupValue_Read-PDU.

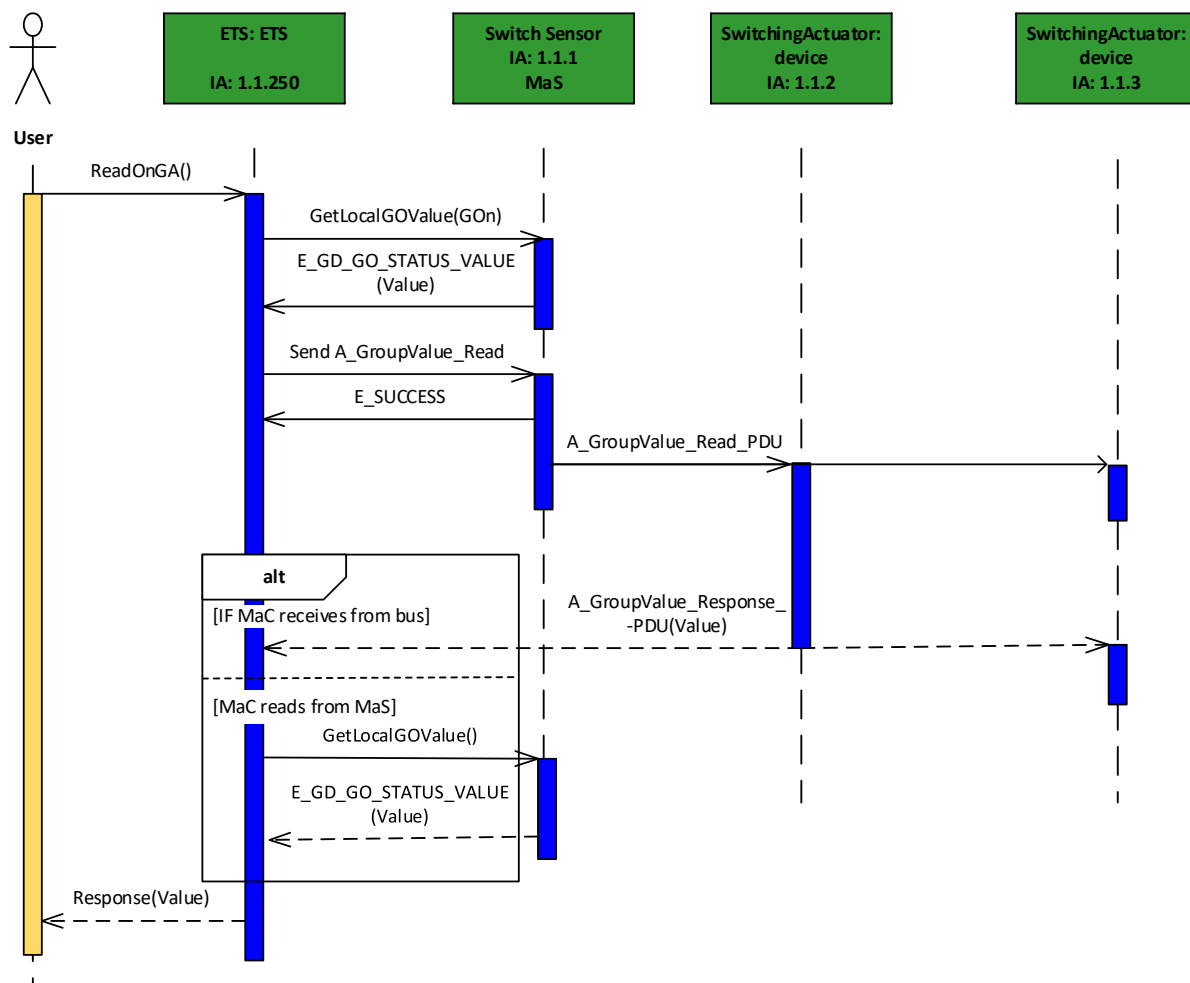


Figure 87 – Read a value from a GA

Annex B (informative)

Examples of table implementations

B.1 Introduction

This annex gives informative examples of how the Resources may be implemented on KNX RF. For KNX RF, these Resources are not standardised and not mandatory. The configuration of KNX RF devices uses dedicated services in function of the Configuration Mode, as specified in [10]. Also the S-Mode access to KNX RF devices is specified without requiring certain formats of the Resources.

B.2 Extension of KNX to enable use of Extended Addresses

B.2.1 Existing resources from KNX

B.2.1.1 Object table (Datapoint Table)

The list of Datapoints used by a device is defined in the Datapoint Table.

Datapoint#	Datapoint Table	
	Length	(1octet)
0	Datapoint description #0	(3 octets)
1	Datapoint Description #1	(3 octets)
2	Datapoint description #2	(3 octets)
3		

Each Datapoint is identified by an index called Datapoint#.

Property access:

Group object table Interface Object / PID_TABLE = 23 Group Object Table

B.2.1.2 Address Table

Address#	Address Table	
	Length	(1octet)
0	Individual Address	(2 octets)
1	Group Address = @bank + @link	(2 octets)
2	Group Address = @bank + @link	(2 octets)
3		

Each address is identified by an index called address# . There is no need to make the separation into @bank and @link.

Property access:

Address Table Interface Object / PID_TABLE = 23 Address table Property format 0

B.2.2 New resources

The list of Extended Group Address used in a device is defined by two tables:

- Serial number table, and
- an Association Table to link Group Addresses and Serial Number to build Extended Addresses.

B.2.2.1 Serial number table

serial#	Serial number table	
	Length	(1octet)
0	Serial N° 1	(6 octets)
1	Sérial N° 2	(6 octets)
2	Sérial N° 3	(6 octets)
3		

Each Serial Number is identified by a index called serial#.

B.2.2.2 New Association Table

To enable that a Datapoint can be part of multiple groups (e.g. multiple push-buttons command one lamp, one for on/off and a second for general off), an Association Table is used. This association links one Datapoint with one or more Extended Group Address (couple of Address and Serial Number).

Each association is defined by

- Address index (1octet),
- Serial Number index (1 octet),
- Datapoint index (1 octet).

Association table			
Length			(1octet)
Address #	Serial #	Datapoint#	(3 octets)

If a Datapoint is transmit only, the association Serial Number index equals FFh. No Serial Number can be associated.

Rules

The association of the sending Datapoint must be at the beginning of the Association Table.

The Serial Number Table Interface Object

The Serial Number Table Object includes Serial Numbers of devices linked to the device. It provides the management operations for downloading.

Table 110 - Serial number Interface Object

Property Name	Property ID	Type	Optional / Mandatory / Writable	
Object Type	PID_OBJECT_TYPE	PDT_UNSIGNED_INT	M	Serial Number table Object object_type 5dec
Object Name	PID_OBJECT_NAME	PDT_UNSIGNED_CHAR[]	O	Name of the Address table
Load Control	PID_LOAD_STATE_CONTROL	PDT_CONTROL	M / W	for further Information see Load / State machines
Serial Number Table	PID_TABLE(23)	PDT_UNSIGNED_INT[]	M / W	Serial number table
..				

The Serial Number Table Property contains only Serial Numbers used by the device. If the current length is 0 all Serial Numbers are accepted. To accept no Serial Number set the load control to unloaded. The current length is the number of entries in the Address Table. The Maximum Length is transferred with the load control MAX_LENGTH or is fixed in the device.

Theoretical Serial NumberTable Format :

serial#	Serial number table	
	Nb of Serial Number	(1octet)
0	Serial N° 1	(6 octets)
1	Sérial N° 2	(6 octets)
2	Sérial N° 3	(6 octets)
3		

B.3 Association Table Object

B.3.1 Existing object

The Association Table Object is used to connect Group Objects to Group Addresses. It provides the management operations for downloading.

Table 111 - Association table Interface Object

Property Name	Property ID	Type	Optional / Mandatory/ Writable	
Object Type	PID_OBJECT_TYPE	PDT_UNSIGNED_INT	M	Associationtable Object 2 dec
Object Name	PID_OBJECT_NAME	PDT_UNSIGNED_CHAR[]	O	Name of the Association table
Association table Pointer	PID_TABLE_Refence	PDT_GENERIC_03	O	Pointer to Association table and maximum entries
Load Control	PID_LOAD_STATE_CONTROL	PDT_CONTROL	M / W	for further Information see Load / State- machines
Association table	PID_TABLE (23)	PDT_GENERIC_02	M / W	
..				

The Association Table Property contains associations between the communication number and the Service Access Point (SAP). The maximum number of elements in the Property shows the maximum size of the table and current length shows the current usage. In each association the high octet is the communication number and the low octet is the SAP number.

In case of sending the first entry in the Association Table shall be used.

If the current length is set to 0 a standard a Association Table shall be used (communication number + 1 = SAP number). The Maximum Length shall be transferred with the load control MAX_LENGTH or shall be fixed in the device.

B.3.2 New Property : Association Table format 1

To enable use of Extended Group Addresses, the association must be made between a Serial Number, a Group Address and a Group Object. For this a new format is designed for the Association Table Object.

The existing one just changes its Property name from Association_Table to Association_Table_format0. The new one gets the Property name Assocaitaion_Table_Format1.

Property Name	Property ID	Type	Optional / Mandatory/ Writable	
Object Type	PID_OBJECT_TYPE	PDT_UNSIGNED_INT	M	Associationtable Object 2 dec
Object Name	PID_OBJECT_NAME	PDT_UNSIGNED_CHAR[]	O	Name of the Association table
Association table Pointer	PID_TABLE_REFERENCE	PDT_GENERIC_03	O	Pointer to Association table and maximum entries
Load Control	PID_LOAD_STATE_CONTROL	PDT_CONTROL	M / W	for further Information see Load / State-machines
Association table Format 0	PID_TABLE (23)	PDT_GENERIC_02	M / W	
Association table format 1	PID_TABLE (52)	PDT_GENERIC_03	O	Association table for extended group addresses

Table 112 - Association table Interface Object

In the Association table format 1, one entry consist of an association of a Serial Number index, a communication number and the Service Access Point (SAP). The maximum number of elements in the Property shows the maximum size of the table and current length shows the current usage. In each association, the octet 2 is Group Address index, the octet 1 is the Serial Number index and the octet 0 is the Group Object index.

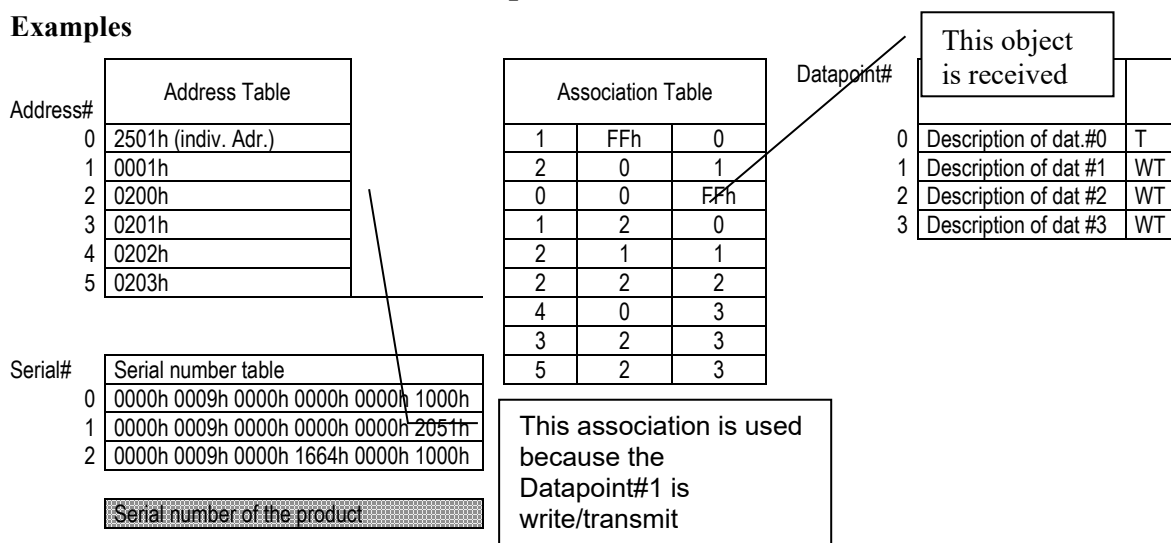
In case of sending the first entry in the association table is used.

For Association for extended Individual Addresses have the datapoint index equal to FFh.

B.4 Examples of usage

B.4.1 Use of tables in case of reception

Examples



The Serial Number of the product is never used in reception (only for specific system message physical addressing using Serial number).

If a message is received with extended Individual Address (2501h,0000h 0009h 0000h 0000h 0000h 1000h), it is accepted (Serial Number present in the table).

If a message is received with extended Individual Address (2501h,0000h 0009h 0000h 0000h 0000h 1001h), it is ignored (Serial Number not present in the table).

If a message is received with Extended Group Address (0200h,0000h 0009h 0000h 0000h 0000h 1000h) the value of data is copied in the Datapoint# 1.

If a message is received with Extended Group Address (0200h,0000h 0009h 0000h 1664h 0000h 1000h) the value of data is copied in the Datapoint# 2.

If a message is received with Extended Group Address (0001h,0000h 0009h 0000h 0000h 1000h) it is ignored.

If a message is received with Extended Group Address (0201h,0000h 0009h 0000h 0000h 1000h) it is ignored.

B.4.2 Use of tables for sending

In case of sending the Serial Number used is always the one of the product, and not the one from the Extended Address associated to the Datapoint. The column of serial index (serial#) is not used in sending. The Serial Number of the product is stored in an other place of the memory (it is not easy to implement a length variable table with a Serial Number factory written). The first association found in the table is used for the transmission.

Address #	Address Table	Association Table	Datapoint#	Datapoint Table
0	2501h (Indiv. Addr.)	1 FFh 0	0	Description of dat.#0 T
1	0001h	2 0 1	1	Description of dat #1 WT
2	0200h	0 0 FFh	2	Description of dat #2 WT
3	0201h	1 2 0	3	Description of dat #3 WT
4	0202h	2 1 1		
5	0203h	2 2 2		
		4 0 3		
		3 2 3		
		5 2 3		

Serial#	Serial Number table
0	0000h 0009h 0000h 0000h 0000h 1000h
1	0000h 0009h 0000h 0000h 0000h 2051h
2	0000h 0009h 0000h 1664h 0000h 1000h

Serial number of the product
